University Library



A gateway to Melbourne's research publications

Minerva Access is the Institutional Repository of The University of Melbourne

Author/s:
YE, PATRICK

Title:
Natural language understanding in controlled virtual environments

Date:
2009

Citation:
Ye, P. (2009). Natural language understanding in controlled virtual environments. PhD thesis, Department of Computer Science and Software Engineering, The University of Melbourne.

Publication Status:
Unpublished

Persistent Link:
https://hdl.handle.net/11343/35152

Terms and Conditions:
Terms and Conditions: Copyright in works deposited in Minerva Access is retained by the copyright owner. The work may not be altered without permission from the copyright owner. Readers may only download, print and save electronic copies of whole works for their own personal non-commercial use. Any use that exceeds these limits requires permission from the copyright owner. Attribution is essential when quoting or paraphrasing from these works.

Natural Language Understanding in Controlled Virtual Environments

A thesis presented

by

Patrick Ye

to

The Department of Computer Science and Software Engineering

in full fulfillment of the requirements

for the degree of

Doctor of Philosophy

University of Melbourne

Melbourne, Australia

June 2009

Produced on archival quality paper

©2009 - Patrick Ye

All rights reserved.

Thesis advisor(s)
Steven Bird
Timothy Baldwin

Author
Patrick Ye

Natural Language Understanding in Controlled Virtual Environments

Abstract

Generating computer animation from natural language instructions is a complex task that encompasses several key aspects of artificial intelligence including natural language understanding, computer graphics and knowledge representation. Traditionally, this task has been approached using rule based systems which were highly successful on their respective domains, but were difficult to generalise to other domains. In this thesis, I describe the key theories and principles behind a domain-independent machine learning framework for constructing natural language based animation systems, and show how this framework can be more flexible and more powerful than the prevalent rule based approach.

I begin this thesis with a thorough introduction to the goals of the research. I then review the most relevant literature to put this research into perspective. After the literature review, I provide brief descriptions to the most relevant technologies in both natural language processing and computer graphics. I then report original research in semantic role labelling and verb sense disambiguation, followed by a detailed description and analysis of the machine learning framework for natural language based animation generation.

The key contributions of this thesis are: a novel method for performing semantic role labelling of prepositional phrases, a novel method for performing verb sense disambiguation, and a novel machine learning framework for grounding linguistic information in virtual worlds and converting verb-semantic information to computer graphics commands to create computer animation.

Declaration

This is to certify that

i the thesis comprises only my original work towards the PhD,

ii due acknowledgement has been made in the text to all other material used,

iii the thesis is less than 100,000 words in length, exclusive of tables, maps, bibliographies and appendices.

Contents

Title Page i
Abstract iii
Declaration iv
Table of Contents v
List of Tables ix
List of Figures xiv
Citations to Previously Published Work xviii
Acknowledgments xix

1 Introduction **1**
1.1 Overview of the Research 4
1.2 Collecting Information about the Actions 8
1.3 Automatic Selection of Graphics Commands 12
1.3.1 Selection of a Sequence of Graphics Commands 14
1.3.2 Converting Information about the Actions to the Arguments of Graphics Commands 15
1.4 Interactions Between Information Collection and Graphics Command Selection 17
1.5 Structure of the Thesis 18

2 Literature Review **21**
2.1 The WordsEye Project 22
2.2 SHRDLU 28
2.3 The K2 Project 31
2.4 The Animation from Natural Language Project 32
2.5 The Script Visualisation (ScriptVis) Project 35
2.6 The Carsim Project 38
2.7 KRISP 41
2.8 Summary 42

3 Technology Foundations **45**
3.1 Machine Learning Algorithms 46
3.1.1 Maximum Entropy Modelling 48

3.1.2 Support Vector Machines (SVMs) . . . 49
3.2 WordNet . . . 51
3.3 Part of Speech Tagging . . . 53
3.4 Lemmatisation . . . 55
3.5 Chunking . . . 56
3.6 Statistical Parsing . . . 59
3.7 Named Entity Recognition . . . 61
3.8 Semantic Role Labelling . . . 61
3.9 Verb Sense Disambiguation . . . 63
3.9.1 Verb Sense Inventory Selection . . . 63
3.9.2 Overview of Commonly Used WSD Techniques . . . 81
3.9.3 Selectional Preference Based Word Sense Disambiguation . . . 86
3.10 Summary . . . 92

4 Brief Introduction to Animation 93
4.1 Data Structures for Storing Information about 3D Graphical Models . 93
4.1.1 3D Graphical Models . . . 94
4.1.2 Vertex Groups . . . 95
4.1.3 Armature . . . 97
4.2 Transformations for 3D Graphical Models . . . 97
4.2.1 Rigid Body Transformations . . . 97
4.2.2 Skeleton Based Transformations . . . 100
4.2.3 Scene Graph Transformations . . . 100
4.2.4 Lighting Manipulation . . . 102
4.2.5 Discussion . . . 102
4.3 Animation Creation . . . 103
4.4 Blender3D . . . 107
4.5 Summary . . . 109

5 PP Semantic Role Labelling 111
5.1 Overview of the Automatic Learning of Preposition Semantics . . . 113
5.2 Preposition Semantic Role Disambiguation for Penn Treebank . . . 114
5.2.1 System Description . . . 115
5.2.2 Results . . . 116
5.3 Preposition Sense Disambiguation with SemEval 2007 Dataset . . . 117
5.3.1 System Description . . . 118
5.3.2 Results . . . 121
5.4 Discussions on the Two Preliminary Investigations . . . 121
5.5 Preposition Semantic Role Labelling with the CoNLL 2004 Dataset . 122

5.5.1 Preposition Verb Attachment Classification 124
5.5.2 Preposition Semantic Role Disambiguation 130
5.5.3 Argument Segmentation 132
5.5.4 Combining the Output of the Subtasks 136
5.5.5 Parameter Tuning of the Maxent Based Classifiers 137
5.5.6 Merging the Output of Preposition SRL and Verb SRL 137
5.5.7 Results 137
5.6 Preposition Semantic Role Labelling with a Movie Script Dataset 140
5.7 Analysis and Discussion 142
5.8 Conclusion 144

6 Verb Sense Disambiguation 145
6.1 Frequency Distribution and Polysemy of the Verbs in the Movie Script Corpus 145
6.1.1 Choice of Verb Sense Disambiguation Strategy 146
6.2 Performing VSD using SRL 147
6.2.1 General WSD Features 148
6.2.2 Selectional Preference Based Features 159
6.2.3 System Evaluation 161
6.2.4 Evaluation of the feature selection heuristics 164
6.3 Summary 175

7 Graphics Command Generation 179
7.1 System Overview 180
7.1.1 Virtual Scene 180
7.1.2 Graphics Commands and Animation Procedure 184
7.1.3 Configuration and Data Flow 190
7.1.4 Summary 191
7.2 Determining the Timing Relationship between Verbs 191
7.2.1 Features for the Timing Relationship Classifier 195
7.2.2 Evaluation and Results 198
7.2.3 Related Work 199
7.3 Grounding Semantic Roles 200
7.4 Actions Classifier Setup 200
7.4.1 Linguistic Features 202
7.4.2 Virtual Scene Features 203
7.4.3 Summary 204
7.5 Argument Classifier Setup 206
7.5.1 Class Labels for Single Virtual Object Arguments 206
7.5.2 Class Labels for the Bone of a Virtual Object Argument . . . 206

7.5.3 Class Labels for Axis, Boolean, and Pre-recorded Skeleton Animation Arguments . . . 206
7.5.4 Class Labels for Euler, Location and Direction Arguments . . 207
7.5.5 Class Labels for Scalar arguments . . . 208
7.6 Results . . . 210
7.6.1 Variations of the Verb Generalisation Features . . . 210
7.6.2 Variations on the Semantic Role Features . . . 211
7.6.3 Variations on the Grounded Semantic Roles . . . 211
7.6.4 Partitioning of the Datasets . . . 212
7.6.5 Action Classifiers . . . 213
7.6.6 Argument Classifiers . . . 217
7.7 Conclusion . . . 228

8 Conclusion **231**
8.1 Future Work . . . 234
8.2 Summary . . . 236

A Feature Category Accuracies in VSD **248**

B Parameter tuning in VSD **269**

C Sense Tagging the Verbs in the Movie Script Corpus **285**
C.1 Graphical User Interface . . . 285
C.2 Sense Tagging . . . 287

D Base Graphics Commands **288**

E Virtual Scene Feature Functions **292**

F Argument Construction Functions **295**

List of Tables

1.1 Example of a textual version of the action information 10
1.2 Example of a grounded version of the action information 11
1.3 Example inventory of graphics commands 14
1.4 Grounded information for (1.6) 15

3.1 Summarization of CoNLL 2000 Shared Task Dataset 57
3.2 Coverage of WordNet 2.0 70
3.3 Breakdown of FrameNet Lexical Units 74
3.4 FrameNet Frame Relations 77
3.5 Senseval-2 System Performance 85

5.1 Penn Treebank semantic roles (top-9 roles) 115
5.2 Penn Treebank preposition semantic role disambiguation results . . . 117
5.3 Classification results of the other semantic roles with and without the PP-CLR 117
5.4 Example semantic-role-labelled sentence 120
5.5 VA class distribution 125
5.6 Revised VA class distribution in test set 127
5.7 Breakdown of the accuracy of the VA classifiers on the test set 129
5.8 Breakdown of the accuracy of the new MaxEnt VA classifier on the test set 130
5.9 Accuracy of the new MaxEnt VA classifier on the individual prepositions in the test set 131
5.10 CoNLL 2004 PP semantic role distribution in the CoNLL 2004 test data set (top-14 roles) 132
5.11 Accuracy of the SRD classifier on the individual prepositions in the test set 133
5.12 Preposition SRL results before merging with the holistic SRL systems, (P = precision, R = recall, F = F-score) 138
5.13 Preposition SRL combined with Hacioglu et al. (2004) (P = precision, R = recall, F = F-score; above-baseline results are underlined) 139

5.14 Preposition SRL combined with Punyakanok et al. (2004) (P = precision, R = recall, F = F-score; above-baseline results are underlined) . 139
5.15 Preposition SRL combined with Carreras and Màrquez (2004) (P = precision, R = recall, F = F-score; above-baseline results are underlined) 140
5.16 Movie script corpus preposition SRL results before merging with outputs of ASSERT (P = precision, R = recall, F = F-score; above-baseline results are underlined) 141
5.17 Preposition SRL combined with ASSERT (P = precision, R = recall, F = F-score; above-baseline results underlined) 141

6.1 Movie Script Corpus Information 146
6.2 Example of the feature type coverage algorithm 163
6.3 VSD system evaluation scheme 165
6.4 Overall results of the feature selection heuristics for Sensval-2 166
6.5 Overall results of the feature selection heuristics for MSC 167
6.6 Overall results of the feature selection heuristics for Senseval-2 without SRL based features 171
6.7 Overall results of the feature selection heuristics for MSC without SRL based features 172
6.8 SRL coverages over the target verbs in the training and test set of Senseval-2 (%) 173
6.9 Distribution of semantic roles for each verb in the test set of Senseval-2 177

7.1 Distributions of timing relationship types in the *Toy Story* data (the timing of the target verb instance was conditioned on the timing of the previous verb instance in the text) 193
7.2 Distribution of three simplified timing relationship types in the *Toy Story* data (the timing of the target verb instance was conditioned on the timing of the previous verb instance in the text) 195
7.3 The accuracy of the verb-temporal relationship classifier under ten-fold cross validation (%) 198
7.4 Inter-annotator agreement on the classification on data instance of the type two timing relationship 199
7.5 Results of the automatic grounding method on the *Toy Story* data . . 200
7.6 Grounded semantic roles for the sentence *Sid hands the Jenny doll to Hannah* 204
7.7 Features generated by `faceTowardsNoZ` for the sentence *Sid hands the Jenny doll to Hannah* 204
7.8 Features generated by `boundingCylinderPercentage` for the sentence *Sid hands the Jenny doll to Hannah* 205
7.9 Features generated by `boneStalkBoneCmdApply` for the sentence *Sid hands the Jenny doll to Hannah* 205

7.10 Class labels generated by the argument construction function `destRadiusPer` . . . 208
7.11 Precision, recall and F1-score of the automatically obtained semantic roles (%) . . . 212
7.12 Results of the Action Classifiers . . . 214
7.13 Results of the seen and unseen verbs on the Action Classifiers with the stratified verb partitioning method . . . 216
7.14 Distributions of argument types in the annotated data . . . 218
7.15 Results of the argument classifiers for the nominal arguments . . . 219
7.16 Majority class baselines for the argument classifiers for the nominal arguments . . . 220
7.17 Results of the argument classifiers for the location arguments . . . 222
7.18 Upper bound for the argument classifiers for the Location arguments 223
7.19 Majority class baselines for the argument classifiers for the Euler angle and direction arguments . . . 224
7.20 Upper bound for the argument classifiers for the Euler angle and direction arguments . . . 225

A.1 N-gram based features cross validation accuracy (and coverage) on the training set(%) . . . 249
A.2 N-gram based features cross validation accuracy (and coverage) on the training set (%) (POS tags of cooccuring words to the left of the target verb and their relative positions) . . . 250
A.3 N-gram based features cross validation accuracy (and coverage) on the training set (%) (POS tags of cooccuring words to the right of the target verb and their relative positions) . . . 251
A.4 N-gram based features cross validation accuracy (and coverage) on the training set (%) (surrounding POS tags of the target verb and their relative positions) . . . 252
A.5 N-gram based features cross validation accuracy (and coverage) on the training set (%) (cooccuring words to the left of the target verb and their relative positions) . . . 253
A.6 N-gram based features cross validation accuracy (and coverage) on the training set (%) (cooccuring words to the left of the target verb and their binary relative positions) . . . 254
A.7 N-gram based features cross validation accuracy (and coverage) on the training set (%) (cooccuring words to the right of the target verb and their relative positions) . . . 255
A.8 N-gram based features cross validation accuracy (and coverage) on the training set (%) (cooccuring words to the right of the target verb and their binary relative positions) . . . 256

A.9 N-gram based features cross validation accuracy (and coverage) on the training set (%) (surrounding words of the target verb and their relative positions) 257
A.10 N-gram based features cross validation accuracy (and coverage) on the training set (%) (surrounding words of the target verb and their relative binary positions) 258
A.11 Non-parse-tree based syntactic features (Additional chunk based features: Chunk-type-sequence after the target verb) cross validation accuracy (and coverage) on the training set (%) 259
A.12 Non-parse-tree based syntactic features (Additional chunk based features: Regular expression representation of the chunks after the target verb) cross validation accuracy (and coverage) on the training set (%) 260
A.13 Non-parse-tree based syntactic features (Additional chunk based features: the first word of each chunk after the target verb) cross validation accuracy (and coverage) on the training set (%) 261
A.14 Non-parse-tree based syntactic features (Additional chunk based features: the last word of each chunk after the target verb) cross validation accuracy (and coverage) on the training set (%) 262
A.15 Non-parse-tree based syntactic features (Additional chunk based features: the POS tag of the first word of each chunk after the target verb) cross validation accuracy (and coverage) on the training set (%) 263
A.16 Non-parse-tree based syntactic features (Additional chunk based features: the POS tag of the last word of each chunk after the target verb) cross validation accuracy (and coverage) on the training set (%) . . . 264
A.17 Non-parse-tree based syntactic features (Additional chunk based features: Chunk-type-sensitive combinations) cross validation accuracy (and coverage) on the training set (%) 265
A.18 Non-parse-tree based syntactic features (Verb voice and quotation) cross validation accuracy (and coverage) on the training set (%) . . . 266
A.19 Parse tree based features cross validation accuracy (and coverage) on the training set (%) 267
A.20 Selectional preferences based features cross validation accuracy (and coverage) on the training set (%) 268

D.1 Basic graphics commands for rigid body transformation 289
D.2 Basic graphics commands for forward and inverse kinematics based skeleton animation 290
D.3 Basic graphics commands for transform scene graph transformation and lighting 291

E.1 Spatial relationship tests for the virtual scene features 293

E.2 Tests for the properties of the virtual objects and the status of the animation 294

F.1 Argument construction functions for Euler arguments 296
F.2 Argument construction functions for Location arguments 297
F.3 Argument construction functions for Direction arguments 298

List of Figures

1.1 Virtual Scene Example 2
1.2 How the configuration of virtual environment's can affect the visualisation of the rendering 7
1.3 How grounding of the textual action information can be affected by the configuration of the virtual environment 11
1.4 Data flow diagram for the process of collecting information about the actions 13

2.1 An example of a correctly generated scene by WordsEye 22
2.2 Example of a hierarchical structure. 24
2.3 Example of a semantic representation of WordsEye. 24
2.4 WordsEye's visualisation of *The man puts the cup on the table* . . . 25
2.5 Comparisons of WordsEye's interpretation of *There is a knife next to the woman* and *There is a knife in the woman's hand* 27
2.6 A screenshot of SHRDLU 29
2.7 Example instructions to SHRDLU 30
2.8 The overall action template for the Parameterised Action Representation 34
2.9 The motion template for the Parameterised Action Representation . . 35
2.10 The template for ScriptVis' semantic representation of events. 36
2.11 Breakdown of the `Feed` action in ScriptVis 38
2.12 The simplified PAR representation for ScriptVis 39
2.13 Parse tree for the query **answer(traverse(next_to(stateid('texas'))))**, translated from the sentence *Which rivers run through the states bordering Texas?* 41

3.1 Data flow diagram for the NLP systems used in this research 45
3.2 An example of feature space and hyperplane 50
3.3 Example of Chunk Tags in IOB Format 57
3.4 WordNet Example (*bow*) 70
3.5 Hypernym-Hyponym relationships for the verb synset bow^1 71
3.6 Holonym-Meronym relationships for the noun synset bow^4 72

4.1 Example of the 2D wireframe projection of a 3D model with a single polyhedron . . . 94
4.2 Example of a 3D model with multiple polyhedrons . . . 95
4.3 Example of vertex groups . . . 96
4.4 Example of a single polyhedron skinned to an armature . . . 98
4.5 Example of a side-by-side comparison of a polyhedron and its armature 99
4.6 Example of a single polyhedron deformed by an armature . . . 101
4.7 Example of an animated walking sequence . . . 104
4.8 Example of a skeleton forearm . . . 106
4.9 Example of an armature touching a static object . . . 108

5.1 An example of the preposition semantic roles in Penn Teebank . . . 114
5.2 Parse tree examples . . . 119
5.3 Data flow between the three subtasks of the PP semantic role labeller 123
5.4 The parse tree produced by the Charniak Parser for the sentence *In the morning, I went to the market to get some apples.* . . . 128

6.1 Architecture of the VSD system for the movie script corpus data . . . 149
6.2 Basic parse tree feature example . . . 153
6.3 One level parent annotated parse tree . . . 153
6.4 Verb-argument-structure feature: ADVP example . . . 154
6.5 Verb-argument-structure feature: NP example 1 . . . 155
6.6 Verb-argument-structure feature: NP example 2 . . . 155
6.7 Verb-argument-structure feature: SBAR clause example . . . 156
6.8 Verb-argument-structure feature: non-SBAR clause example . . . 157
6.9 Verb-argument-structure feature: PP example 1 . . . 157
6.10 Verb-argument-structure feature: Verb Particle example . . . 158
6.11 Comparisons of the VSD performance with and without the SRL features on Senseval-2 dataset . . . 169
6.12 Comparisons of the VSD performance with and without the SRL features on the MSC dataset . . . 170

7.1 Example of 3D graphical models with components that have no WordNet synset counterparts . . . 181
7.2 Example of a 3D graphical model and its graphical components . . . 181
7.3 Example of the bones of an armature . . . 182
7.4 Example of multiple instantiations of the same 3D graphical model . 183
7.5 Example of the mapping from 3D graphics models in the database to virtual objects in a virtual scene . . . 184
7.6 Hierarchy of the graphics commands . . . 188
7.7 Data flow diagram of the automatic animated story boarding system 190
7.8 All types of timing relationship between two verb instances in the same action description . . . 193

7.9 Examples of the sequence of virtual scenes corresponding to *The man grabs the mug on the table* 201
7.10 Example of the different values of the percentage parameter to `leftDblTo` 207
7.11 Example of the different values of the percentage parameter to `destRadiusPer` 208
7.12 Example of parse tree emulated semantic roles 211
7.13 Heights of the 3D models for Andy and Woody used in reanimating Toy Story (the models are stored based on Blender's internal representation of dimensions, and therefore it is not necessary to have explicit unit for length) 221
7.14 Distribution of the distances between predicted locations and gold standard locations when verb synsets and Levin classes are used as the verb generalisation features under stratified-verbs partitioning 226
7.15 Distribution of the angles between predicted Euler angles and gold standard Euler angles when verb synsets and Levin classes are used as the verb generalisation features under stratified-verbs partitioning . . 227

8.1 Example timing relationship between three verb instances 235

B.1 Results of the ranking-percentage based feature selection for the verb *begin* 270
B.2 Results of the ranking-percentage based feature selection for the verb *call* 270
B.3 Results of the ranking-percentage based feature selection for the verb *carry* 271
B.4 Results of the ranking-percentage based feature selection for the verb *collaborate* 271
B.5 Results of the ranking-percentage based feature selection for the verb *develop* 272
B.6 Results of the ranking-percentage based feature selection for the verb *draw* 272
B.7 Results of the ranking-percentage based feature selection for the verb *dress* 273
B.8 Results of the ranking-percentage based feature selection for the verb *drift* 273
B.9 Results of the ranking-percentage based feature selection for the verb *drive* 274
B.10 Results of the ranking-percentage based feature selection for the verb *face* 274
B.11 Results of the ranking-percentage based feature selection for the verb *find* 275

B.12 Results of the ranking-percentage based feature selection for the verb *keep* . . . 275
B.13 Results of the ranking-percentage based feature selection for the verb *leave* . . . 276
B.14 Results of the ranking-percentage based feature selection for the verb *live* . . . 276
B.15 Results of the ranking-percentage based feature selection for the verb *match* . . . 277
B.16 Results of the ranking-percentage based feature selection for the verb *play* . . . 277
B.17 Results of the ranking-percentage based feature selection for the verb *pull* . . . 278
B.18 Results of the ranking-percentage based feature selection for the verb *replace* . . . 278
B.19 Results of the ranking-percentage based feature selection for the verb *see* . . . 279
B.20 Results of the ranking-percentage based feature selection for the verb *serve* . . . 279
B.21 Results of the ranking-percentage based feature selection for the verb *strike* . . . 280
B.22 Results of the ranking-percentage based feature selection for the verb *train* . . . 280
B.23 Results of the ranking-percentage based feature selection for the verb *treat* . . . 281
B.24 Results of the ranking-percentage based feature selection for the verb *turn* . . . 281
B.25 Results of the ranking-percentage based feature selection for the verb *use* . . . 282
B.26 Results of the ranking-percentage based feature selection for the verb *wander* . . . 282
B.27 Results of the ranking-percentage based feature selection for the verb *wash* . . . 283
B.28 Results of the ranking-percentage based feature selection for the verb *work* . . . 283

C.1 Interface of the sense tagger . . . 286

Citations to Previously Published Work

Large portions of Chapter 5 have appeared in the following papers:

> Patrick Ye and Timothy Baldwin (2006) Semantic role labeling of prepositional phrases. In *ACM Transactions on Asian Language Information Processing (TALIP)* 5,3,. pp. 228–244.
>
> Patrick Ye and Timothy Baldwin (2007) MELB-YB: Preposition Sense Disambiguation Using Rich Semantic Features. In *Proceedings of the Fourth International Workshop on Semantic Evaluations (SemEval-2007)*, Prague, Czech Republic, pp. 241–244

Large portions of Chapter 6 have appeared in the following paper:

> Patrick Ye and Timothy Baldwin (2006) Verb Sense Disambiguation Using Selectional Preferences Extracted with a State-of-the-art Semantic Role Labeler. In *Proceedings of the Australasian Language Technology Workshop*, Sydney, Australia, pp. 141–148

Large portions of Chapter 7 have appeared in the following paper:

> Patrick Ye and Timothy Baldwin (2008) Towards Automatic Animated Storyboarding. In *Proceedings of the Twenty-Third AAAI Conference on Articifical Intelligence (AAAI-08)*, Chicago, USA, pp. 578–583.

Acknowledgments

I want to thank my Mum and Dad, for their selfless and unconditional love and support. I want to thank my supervisors Steven Bird and Timothy Baldwin, for their brilliant and tireless guidance. I want to thank my colleagues Trevor Cohn, Philip Blunsom, Lars Yencken, Olivia March, Meladel Mistica, David Martinez, Aidan Furlan, Edward Ivanovic and Jeremy Nicholson for their help in data collection and programming, as well as providing valuable office entertainment. I want to thank Diana McCarthy, Tom O'Hara and Philip Resnik, for their valuable discussions during various stages of my research. Finally, I want to thank Professor Zhiqiang Liu, and Dr. Nick Barnes, for their guidance in the first year of my research.

Typesetting Conventions

Throughout this thesis, I will use the following typesetting conventions:

Category	Example Usage
Verb	The cowboy *runs* towards the lamb quickly.
Inline example sentence	Given the sentence *He dropped the ball.* ...
"Gloss"	The first sense of the verb *run* is "move fast by using one's feet"
`Graphicscommand`	Suppose we have a graphics command `move(arg1=Object,arg2=Location,arg3=Speed)` ...
`Virtual object`	The virtual human `Man.3` is closest to the virtual chair `Chair.4`.
Movie script name	The previous sentence comes from the movie script *Die Hard*.
SEMANTIC ROLE	The entity for the EATER of the verb *eat* in the previous sentence was ...
Feature for machine learning	The feature isAlpha determines whether the given word is entirely alphabetical.
PART OF SPEECH TAG	The word *cat* in the previous sentence is a NOUN.

Chapter 1

Introduction

Natural language is a vital and ubiquitous part of human intelligence and society. Accordingly, it is intuitive to believe that technologies which can automatically process or understand natural languages (NLP technologies) could facilitate the everyday lives of ordinary people. Indeed, many NLP technologies have already been applied to build end user applications which help millions of ordinary people on a daily basis. For example, automatic language translation technologies help people to access information presented in unfamiliar languages; automatic grammar analysis technologies help people avoid making grammatical mistakes during writing; and automatic summarisation technologies save people time by condensing large amounts of information into smaller snippets. However, there are still many other areas that can benefit from the application of NLP technologies. One of these areas is the generation of computer animation, and in this research, I investigate how NLP technologies can be used to give practical support to people who create computer animations.

Computer animation is a significant part of our technological society. It not only has enormous commercial value for the entertainment and advertising industries, but also the potential to become a useful illustration tool in education and communication. Currently, the creation of computer animation is mostly done manually. This manual process can be tedious, time consuming and labour intensive. For example, Figure 1.1 shows a virtual environment which contains a room, a man and some furniture, all of which could come from a particular movie scene described by a script. If the director of the movie wished to visualise the actions described by the script, in order to prepare for the actual shoot or even the construction of the physical movie set, he/she would have to somehow manipulate the entities within this environment in a way that could be understood to depict the actions described by the script. Traditionally, this manipulation is mostly performed by human animators who manually control every relevant aspect of the virtual environment according to the actions to be visualised. For instance, these actions could be described in the following passage:

(1.1) The man goes to the desk and sits down in the chair, he turns on the lamp

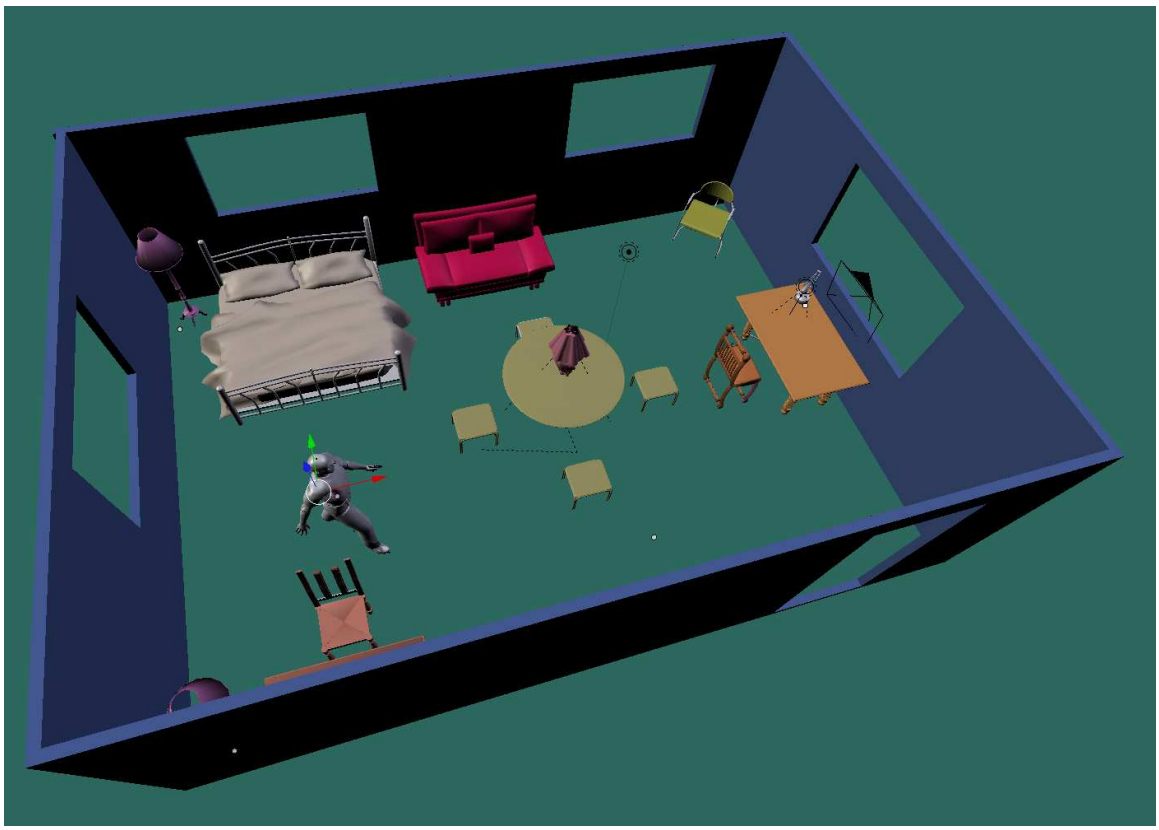


Figure 1.1: Virtual Scene Example

and stares out of the window for a while, then he stands up and runs out of the door.

If this animation was to run for 10 seconds, and the screen was to be refreshed 25 times per second,[1] then 250 images would have to be generated for this animation. In the worst case scenario where only the most rudimentary animation functionality is available, the human animators would have to manually reposition all the body parts of the virtual human and all the furniture he would affect for every single one of the 250 images. In the best case scenario where the animation system provides some predefined motion commands such as walking and sitting, the human animators would still have to decide how long each motion command would have to run, and where the virtual human and other entities have to be at what time, then they have to manually supply these details to the animation software.

It may seem that the two easiest ways to improve the process of creating animation

[1]This is the standard PAL TV refresh rate.

are constructing more predefined motion commands that can be directly used by the animators, and investigating how user interfaces of animation systems can be made more intuitive. However, since there are so many domains in which computer animation can be applied, it would be impossible for a single animation system to provide predefined motion commands for all of them. As a result, even with the most powerful animation systems, animators may still have to resort to the more rudimentary functionalities to make their animations. Furthermore, since powerful animation systems provide many customisable functionalities, the sheer amount of functionality combined with multitudes of customisable options could overwhelm most computer users even when great efforts were spent on the design of the animation system's user interfaces.

Therefore, it is fair to say that the manual creation of animation has the problem of being time consuming, labour intensive, and requires a significant amount of specialist knowledge. On the other hand, since natural language is one of the most intuitive methods of human communication, it would be desirable if the animators could bypass the complex interfaces of animation systems, and instead use natural language to create the desired animation.

In fact, some research has already been done in the area of natural language based animation generation. However, the trend in the existing research has been to provide domain-specific solutions which are based on handcrafted rules. Such solutions cannot be easily applied to domains that they were not designed for because the rules tend to be domain-specific, and as a result, a new set of rules have to be developed for every new domain. While these rules tend to be highly effective, the process of creating them can be tedious and labour intensive. Furthermore, the number of rules tends to be proportional to the complexity of the domain, and complex domains may require a large number of handcrafted rules which can be difficult to maintain.

On the other hand, advances in machine learning techniques in the last two decades have given rise to systems which can use training data to automatically induce rules similar to the handcrafted ones. These systems have achieved comparable or even better results on many NLP tasks that were traditionally solved with rule based systems, Compared with the rule based systems, the empirical systems tend to have better coverage because they often generate more rules than what can be handcrafted, and more flexibility for domain adaptation because all the rules are automatically learned from the given training data which significantly reduces the need for human intervention. Since natural language based animation generation is useful in many domains, a machine learning approach would be especially useful, since there is a large community of animators who are capable of creating the training data. However, to this day, there is little research on an empirical approach to natural language based animation generation.

Hence, this thesis is dedicated to investigating the theories and principles of how a domain-independent framework founded on machine learning techniques can be formed to use natural language based instructions to generate computer animation

with existing animation systems.

1.1 Overview of the Research

As outlined above, the goal of this research is to investigate how NLP technologies can be used to automatically drive arbitrary computer graphics systems to create animations that visualise actions described in natural language. In this section, I will give an overview of this research and provide a more precise description of its goals.

One can draw a parallel between the creation of a movie scene and the creation of computer animation. Each movie scene is created in two steps: the construction of the movie set, followed by the shooting of the scene. In the stage construction step, all the actors and properties are placed in their respective positions according to the directing crew's interpretation of the movie script. In the shooting step, the actors deliver their performances by interacting with each other and the objects on the stage according to the relevant descriptions of the movie script. Similarly, to animate an action description, the human animators first have to construct a virtual stage or environment which has all the virtual actors and props in the right places. They can then use predefined computer graphics commands to manipulate the virtual actors and objects to 'shoot' the animation. In this research, I will always assume that the virtual stage has already been constructed, and limit my efforts to investigating how the 'shooting' of the animation can be automated.

One way to start to scope this research problem is to treat the final system as a black box and only define its inputs and outputs. In this black box paradigm, the system only has two inputs and one output, namely:

Inputs	1. S, a natural language description of a list of visualisable actions 2. V, an accessible and manipulatable virtual environment in which all the actions in S is to take place. (Detailed explanation on the meanings of accessible and manipulatable will be given below.)
Outputs	Computer animation that faithfully and coherently visualises S within V.

With respect to the input action descriptions, an action is visualisable if it will cause some visible change to the environment in which it occurs. For example, consider the following two descriptions:

(1.2) a. There is a table in the room. Two coffee mugs are on the table, the big one belongs to John, the small one belongs to Mark. John walks to the table and picks up his mug.

b. John walks to the table and picks up the big mug.

In (1.2a), the first two sentences are not visualisable actions because they are instructions to set up the environment, which I assume is already accomplished by human animators. The last sentence describes two visualisable actions: *walk* and *pick up* which would cause the positions of both `John` and `the big coffee mug` to change. Therefore, given the same virtual environment, descriptions (1.2a) and (1.2b) should generate exactly the same animation because they describe the same visualisable actions.

The virtual environment is a computer generated representation of the imaginary world in which action descriptions can be visualised. It is perfectly reasonable to use the virtual environment to depict the real world or an imaginary magical world according to the particular understanding of humans. For example, a virtual environment can be created to represent part of the *Star Wars* universe, and visualise actions such as the uses of the Force, which clearly defy the known laws of physics. Every entity with a graphical representation in the virtual environment should have a counterpart in the imaginary world it represents. Furthermore, as mentioned earlier, the virtual environments must also have the properties of being totally accessible and manipulatable. Having these properties means that it should be possible to access and modify every attribute of any of the virtual entities at any time. These properties are necessary in order to approximate the background knowledge and laws of physics that exist in the real or imaginary world. Finally, it is also assumed that the virtual environment is specifically constructed such that it contains every entity that is needed for the visualisation of the relevant actions.

Continuing with the analogy between creating a movie scene and the action-visualisation, in order for the human actors to put on a convincing performance, they would need to have sufficient understanding of their individual actions as described by their script. Without this knowledge, they cannot deliver a convincing performance. Similarly, the action-visualisation system must also have the same type of understanding of the actions to be visualised.

In order to proceed further, it is necessary to first define what constitutes a sufficient understanding for an action. To depict the realization of an action, it is necessary to know exactly which real-world action is to be performed; who and what are involved and what role they play in the realization; and in what manner the realization is carried out, when it is carried out, whether it succeeds or not, and what consequences it has. This is the relevant information needed by both the human actors and the visualisation system, and obtaining them is of vital importance to this research.

The information about the real world actions cannot be completely obtained from either the action description or the virtual environment alone, because the action

descriptions almost always under-specify the virtual environment, and the realizations of the early actions will inevitably change the configuration of the virtual environment which in turn alters the virtual scene for the later actions. For example, consider the following description:

(1.3) The man turns away from the window, and walks to the round table and sits down on a stool.

If the virtual environment is set up as the top picture in Figure 1.2, one would expect the virtual actor to turn away from the window on the west wall and sit on the red stool; but if the virtual environment is set up as the bottom picture, the virtual actor would be expected to turn away from the second window on the north wall and sit down on the blue stool. This clearly illustrates that the same action description would be visualised differently given different configuration of the virtual environment. Therefore, an automatic mechanism has to be devised to use both the action description and the virtual environment to collect information about the actions which are to be visualised. This automatic collection mechanism is therefore one of the major objectives of this research.

Assuming that there was an automatic mechanism that could collect all the essential information that human actors would require, then the next question would be how this information could be used by a computer graphics system to perform the visualisation. After all, it is not possible to explicitly program a machine command for every single real-world action, and the existing graphics commands in any graphics system are not only few in number but also lack direct correspondence to specific real-world ones. This limitation of graphics commands presents a significant obstacle to this research: even with all the essential information about the real-world actions to be visualised, a computer system still may not have enough information to decide which graphics commands to use. For example, if a computer graphics system only has a pre-built command to make virtual humans `walk`, but the action description to be visualised requires the actors to `stroll`, or `jog`, or `run`, what should the system do? The easiest option would be to just refuse to do anything and force the human animators to change their action description. However, this would put a considerable limitation on the usefulness of the system, and therefore it is highly undesirable. A better option would be to directly present the animators with the list of available graphics commands, and let them decide which ones to use and how to tweak the relevant parameters to approximate what they want. In this particular example, the human animators are likely to choose the `walk` command and adjust the `speed` parameters accordingly. This option is more useful than the first, but still undesirable because of the forced human instruction. The best option then is for the system to automatically work out which available graphics commands and parameters it should use to best approximate what the animators want, thereby requiring no human involvement. Therefore, the second major objective of this research is to investigate

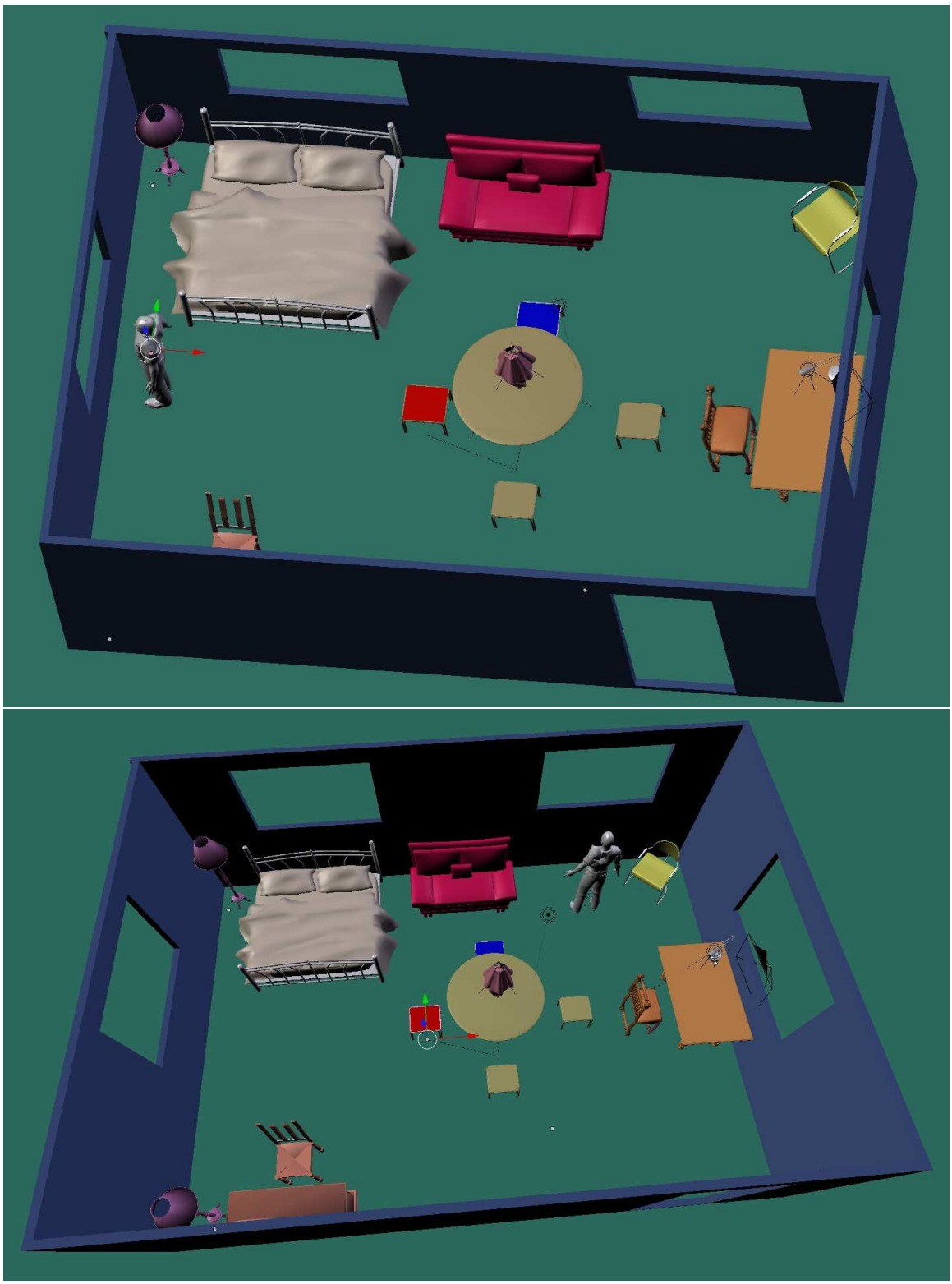


Figure 1.2: How the configuration of virtual environment's can affect the visualisation of the rendering

how to use the information about real actions so that the most appropriate graphics commands can be selected.

Lastly, it is unrealistic to expect the information collection mechanism to be perfect, and it can become expected that some information may be lost or erroneously collected. This further complicates the process of approximating real-world actions with limited graphics commands. On the other hand, it is unreasonable to treat all types of information with equal importance in all situations, as some types of information are more tolerant of errors than others. Therefore, the last major objective of this research is to investigate the influence and contribution of the various types of information on the final outcome of the approximation.

It may seem unintuitive that the automatic construction of the initial virtual scene is not an objective of this research. There are two practical reasons for omitting such a seemingly obvious task from the scope of this research. Firstly, in general, constructing the initial virtual scene from existing graphical models is far less labour intensive than visualising the actions: the initial virtual scene is static, and only needs to be constructed once before the animation is created; whereas the actions are dynamic, and need to be created for each frame of the animation. Furthermore, drag-and-drop interfaces are especially effective in constructing virtual scenes, for example, using the drag-and-drop interface developed in this research, it would only take a few seconds for the user to add a new virtual object into the virtual scene. Secondly, the current state-of-the-art for this task is sufficiently advanced that it would require far more resources than what's available to this research in order to produce better results (reviews of the relevant literature are provided in Chapter 2). Therefore, given the limited resources available, I decided to focus on the visualisation of the actions.

In summary, the goal of this research can be broken down into three major objectives: (1) to devise an automatic mechanism for collecting relevant information about the real-world actions described in natural language based action descriptions within the context of certain virtual environments; (2) to investigate how this information can be used to select computer graphics commands which can create an animation that best approximates the original action description; and (3) to investigate the dynamics between the information collection process and the graphics command selection process.

In the remainder of this chapter, I will examine each of the three main objectives in detail and discuss how they can be achieved computationally.

1.2 Collecting Information about the Actions

Since the action descriptions are written in natural language, the first step of the information collection process is to use relevant NLP technologies to analyse the descriptions and obtain a textual version of the information about the actions. Given a description of one or more actions, it is necessary to first identify the actions

themselves before collecting information about them. As the target data of this research is English text, which describes actions through verbs, the identification of actions must start by extracting verbs from the input text; this can be accomplished through the combination of Part-of-Speech (POS) tagging and lemmatisation. More specifically, POS tagging can first be used to identify the verbs in their various forms in the given text, then lemmatisation can be used to transform the inflected verbs to their canonical form. Once the verbs have been identified, it is then possible to collect specific information about them through analysing their arguments and modifiers. More specifically, the original text needs to be analysed on a verb-by-verb basis such that the individual arguments and modifiers (semantic roles) can be separated and labelled for the respective verbs. This analysis is termed semantic role labelling (SRL).

Since the verbs in English are typically polysemous, i.e. have more than one sense, simply knowing the canonical form of the verbs and their semantic roles still may not be sufficient to identify the exact action. For example, the verb *shoot* has three different meanings in the following sentences:

(1.4) a He *shoots* an arrow toward a large target.[2]
shoot = "hit with a missile from a weapon"

b Derek *shoots* Cameron a look and tries another way.[3]
shoot = "to look at"

c As Hudsucker *shoots* past the window, his draft sends the stack of papers wafting this way and that.[4]
shoot = "move quickly and violently"

Therefore, after the extraction of the verbs' canonical forms and their semantic roles, additional analysis may still need to be performed in order to identify the exact action it refers to. This analysis takes the form of verb sense disambiguation (VSD).

To illustrate the text analysis process, consider the action description in the following sentence:

(1.5) The man near the window quickly runs to the nearest stool.

First, a POS tagger can be used to identify that the word *runs* is the only verb in the description. Then, a lemmatiser can be used to transform the verb *runs*, which is in the 3rd person present tense, to its canonical form of *run*. Then, an SRL system can be used to identify that the RUNNER role of the running action is played by *the man near the window*, the locational constraint of running action is *to the nearest stool*, the manner modifier is *quickly*, and there is no other constraints. Finally, a

[2]From the script of *8MM*.
[3]From the script of *American History X*.
[4]From the script of *The Hudsucker Proxy*.

	Text Analysis	Grounding
ACTION	*move fast by using one's feet*	N/A
RUNNER	*the man near the window*	`None`
LOCATIONAL MODIFIER	*to the nearest stool*	`None`
TEMPORAL MODIFIER	*None*	N/A
MANNER MODIFIER	*quickly*	N/A

Table 1.1: Example of a textual version of the action information

VSD system can be used to determine that in the given context, the verb *run* means "move fast by using one's feet". Table 1.1 sums up this textual version of action information.

However, without a virtual environment, the textual information collected by the text analysis process is most likely to be underspecified. Continuing with Example 1.5, the NLP technologies have identified that the RUNNER of the action is *the man near the window*, and the LOCATIONAL MODIFIER of the action is *the nearest stool*. However, these are just the textual representation of the action information, and additional analysis must be performed to find their counterparts in the virtual environments. This analysis is called grounding, and it is the final step of the automatic information collection process. To illustrate the grounding process, consider the interpretations of *the man near the window* and *the nearest stool* within the virtual environment shown in Figure 1.3. There are two virtual humans in this environment, one of them is near the window on the west wall `Man.1`, and the other is near the door (`Man.2`); it should be obvious that `Man.1` is the correct counterpart for *the man near the window*. Furthermore, the virtual environment also contains four virtual stools, the blue one (`Stool.1`) is closest to `Man.2`, and the red one (`Stool.2`) is closest to `Man.1`. Thus, if the grounding process was to be consistent, then `Stool.2` would have to be chosen to represent *the nearest stool*. Since the faithfulness of the animation depends on whether the correct objects are manipulated, grounding is an important computational task with respect to this research. Furthermore, it is worth noting that the process of grounding is more or less the inverse process of generating referring expressions from knowledge bases, see Dale (1989) for more information on the latter.

It is worth pointing out that movie scripts are the target genre for the text analysis process because they tend to contain a large amount of descriptions of visually significant events, which are the ideal textual inputs of the desired visualisation system. However, they also often contain short descriptions of the movie scene, similar to the first two sentences in example 1.2(a) and irrelevant to the visualisation system. Therefore, the movie scripts used in the construction of the text analysis process will be preprocessed such that only descriptions of visually significant events are extracted.



Figure 1.3: How grounding of the textual action information can be affected by the configuration of the virtual environment

	Text Analysis	Grounding
ACTION	*move fast by using one's feet*	N/A
RUNNER	*the man near the window*	`Man.1`
LOCATIONAL MODIFIER	*to the nearest stool*	`Stool.2`
TEMPORAL MODIFIER	*None*	N/A
MANNER MODIFIER	*quickly*	N/A

Table 1.2: Example of a grounded version of the action information

In summary, as shown in the data flow diagram of Figure 1.4, the process of collecting information about the actions must incorporate the following computational components:

1. Verb identification using POS tagging and lemmatisation.
2. Verb argument and modifier extraction using semantic role labelling (SRL).
3. Verb sense disambiguation (VSD).
4. Grounding of the action information with respect to the virtual environment.

1.3 Automatic Selection of Graphics Commands

The previous section discussed how information about the actions can be collected from the input text and the virtual environment. This section will discuss how this information can be used to drive computer graphics systems to create animation.

As stated in Section 1.1, there are far more real world actions than graphics commands. This implies that on the basis of individual actions, it is not possible to match each action with a unique graphics command which means that some, if not most, actions have to be approximated. On the other hand, the same action may be visualised by different graphics commands in different scenarios, e.g. the `walk` action for a cat should be visualised differently from the `walk` action for a human. Therefore, the mapping from real world actions to graphics commands tend to be many-to-many. Furthermore, some real world actions may be so complex that an adequate approximation of them would require a *sequence* of graphics commands executed in a specific order, adding an additional layer of complexity to the selection process.

The final complication of the selection process is that the information about the real world actions may not completely match the arguments of the graphics commands. For example, suppose the action *The dog is catching up to the fleeing fox* was to be visualised, common sense would suggest that this action should not finish instantaneously when there is a considerable distance between the *the dog* and *the fox* initially. That is, the ideal graphics command would be something along the lines of `move(arg1=Object,arg2=Direction,arg3=Speed)` which can move the object specified by its first argument towards the direction specified by its second argument at the speed specified at its third argument; however, if the only available graphics command is `move(arg1=Object,arg2=Location)` which would instantaneously move the object specified by its first argument to the location specified by its second argument, then some sort of preprocessing must be done to convert the arguments of the action to the arguments of the graphics command.

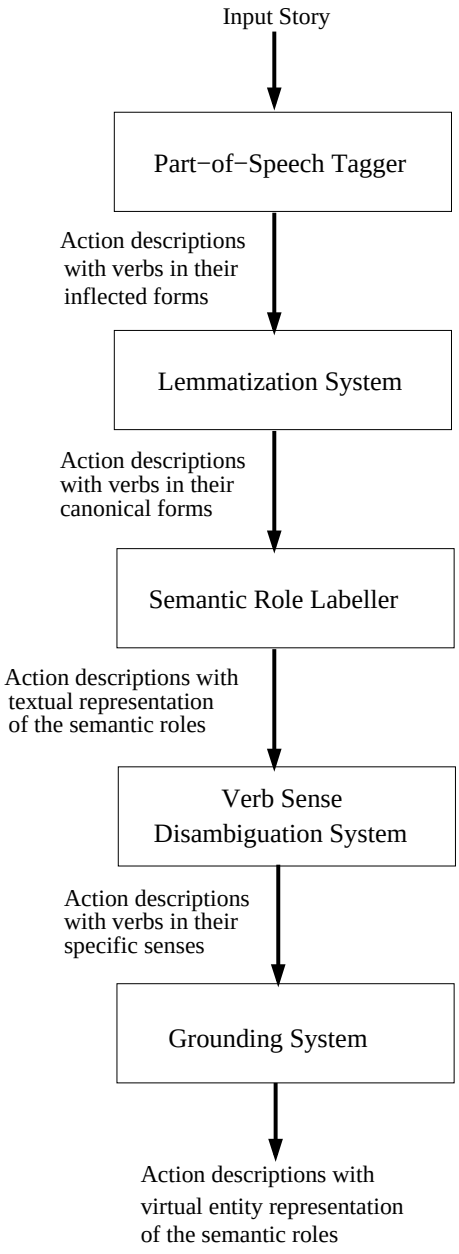


Figure 1.4: Data flow diagram for the process of collecting information about the actions

Command Name	Description
`walk_humanoid`	Translate given objects to specific locations using walking motion for humanoid entities
`walk_4-legged`	Translate given objects to specific locations using walking motion for 4-legged entities
`move`	Translate given objects to specific locations
`rotate`	Rotate objects to specific orientations
`attach_object`	Attach one object to another, so that the geometric transformation of the former mirrors that of the latter.
`detach_object`	Detach one object from another object, so that the geometric transformation of the former will be independent of the latter.

Table 1.3: Example inventory of graphics commands

Therefore, in order for the automatic selection of graphics commands to be successful, it needs to have the following functionalities:

1. Be able to select a sequence of graphics commands for a given real world action.
2. Be able to convert information about the actions to the arguments of graphics commands.

I will discuss both functionalities in more detail in the rest of this section.

1.3.1 Selection of a Sequence of Graphics Commands

The process of selecting a sequence of graphics commands to approximate a real world action is perhaps best illustrated with an example. Suppose the *fetch* action in the following sentence is to be visualised with the graphics command inventory shown in Table 1.3:

(1.6) The golden retriever *fetched* the ball.

Assuming the information collection process has derived the grounded information shown in Table 1.4 with respect to (1.6), then one acceptable sequence of graphics commands for visualising the *fetch* action may be the following:

1. `rotate`: Make `Dog.3` face the direction of `Ball.1`.
2. `walk_4-legged`: Make `Dog.3` walk to the location of `Ball.1`.

	Text Analysis	Grounding
ACTION	*fetch*	N/A
FETCHER	*The golden retriever*	`Dog.3`
FETCHEE	*the ball*	`Ball.1`

Table 1.4: Grounded information for (1.6)

3. `attach_object`: Make `Dog.3` carry `Ball.1`.
4. `rotate`: Make `Dog.3` face the direction of its original location.
5. `walk_4-legged`: Make `Dog.3` walk to its original location.
6. `detach_object`: Make `Dog.3` release `Ball.1`.

Given the vast number of real world actions that could be described in movie scripts, it is impractical to handcraft the rules for assigning sequences of graphics commands to visualise action information, and instead, an automatic method is required to create these rules. Since this research does not target any specific graphics systems, the creation of this automatic method cannot be based on any particular repository of graphics commands. In order to achieve this independence of graphics systems, a learn-by-example approach will be used. This approach is based on supervised learning methods, and uses training data created by human experts to directly learn how to map specific repositories of graphics commands to action information based on the semantics of the actions and the configuration of the virtual scene. Since the procedures of learning are independent of graphics systems, and the mapping rules are dependent only on the training data, the learn-by-example approach is still independent of any graphics systems.

1.3.2 Converting Information about the Actions to the Arguments of Graphics Commands

Once a sequence of graphics command has been chosen, the next step is to specify the values of the arguments to each command. Since the grounded action information is represented as the semantic roles to the verbs that represent the relevant actions, what is needed is an automatic mechanism for converting semantic roles to arguments of graphics commands on a verb-by-verb basis. As the argument lists of graphics commands are fixed and unambiguous, this conversion task would seem to be trivial. However, it is non-trivial because it needs to handle a vast number of verbs which not only vary greatly in terms of syntactic and semantic patterns but are also visually distinct in different situations. For example, consider the following two sentences:

(1.7) a The cowboy *walks* towards the lamb quickly.

b The cowboy *walks* towards the lamb slowly.

The two instances of the verb *walk* have the same meaning in both sentences in (1.7), however, it would be inaccurate to visualise them in the same way as they have different manner modifiers: *quickly* in the first instance and *slowly* in the second instance.

Furthermore, consider the next pair of sentences:

(1.8) a The cowboy *walks* in front of the lamb.

b The cowboy *walks* behind the lamb.

Again, the two instances of the verb *walk* have the same meaning in both sentences, but it would also be inaccurate to visualise them in the same way because this time, their destination arguments have different modifiers: *in front of* in the first instance and *behind* in the second instance.

Finally, consider the following pair of sentences:

(1.9) a The cowboy *walks* towards the lamb.

b The cowboy *runs* towards the lamb.

It is obvious that for the same virtual being, walking should be slower than running, therefore the two sentences in (1.9) should be visualised differently. In the case where the user chooses two or more specialised graphics to visualise *walk* and *run* separately, the speed at which *The cowboy* moves would be up to the individual graphics commands, and a parameter for the speed of the movement would be unnecessary. However, it is also possible that the user may use the same graphics command to visualise both *walk* and *run*, and this command can be customised with a style parameter (walking or running) and a speed parameter. In this case, the value of the speed parameter must be chosen to match the verb instance.

The above three examples illustrate that there are three factors that can influence the process of converting information about the actions to the argument of graphics commands: (1) the modifiers to the verb instances, (2)the modifiers to the arguments of the verb instances, and (3) the verb instances themselves.

Broadly speaking, arguments to graphics commands can be divided into two categories: the entity category and the real-valued category. The values of arguments that belong to the entity category should be one or more entities in the virtual environment. For example, assume the following graphics command is available:

```
move(arg1=Object,arg2=Location,arg3=Speed)
```

Suppose that `move` places the virtual entity specified by its `arg1` from its original location to the location specified by its `arg2` and the speed specified by `arg3`. It could be observed that `arg1` would belong to the entity category and its value should be a specific virtual entity; `arg2` would belong to the real-valued category, and its value should be a specific coordinate; and `arg3` would also belong to the real-valued category, and its value should be a positive scalar.

The conversion of semantic roles to entity arguments is straightforward, as it only requires the calculation of the mapping from the relevant semantic roles of each verb to the corresponding arguments of the relevant graphics commands. The conversion of semantic roles to real-valued arguments is more complicated because the domains of the real-valued arguments are continuous, and it would be impractical to map semantic roles to specific values in the continuous domains. However, since the values of the real-valued arguments are always determined by one or more of the factors listed above, it is possible to use some predefined argument constructor functions to dynamically calculate the values of the real-valued arguments on the fly. For example, to convert the argument *in front of the lamb* to a location, one could predefine a constructor function `front(arg1=Object)` which takes an entity as argument, and returns a location which can be considered as being in front of this entity. Similarly, to convert the argument *quickly* to a specific speed, one could predefine a constructor function `speedX2(arg1=Object.speed)` which takes the default speed of the target object as argument, and returns a speed that is twice as fast.[5]. With these argument constructor functions, the conversion of semantic roles to real-valued arguments can be framed as the mapping from the verb instances and their semantic roles to the most suitable argument constructor functions.

In summary, the conversion from action information to arguments of graphics commands consists of the following computational tasks:

1. Creating a mapping from semantic roles of specific verb instances to the entity arguments of graphics commands

2. Creating mappings from verb instances and their semantic roles to argument constructor functions

1.4 Interactions Between Information Collection and Graphics Command Selection

Section 1.2 has identified part-of-speech tagging, semantic role labelling (SRL), verb sense disambiguation (VSD) and grounding as the main components of the information collection process. Furthermore, complex NLP technologies such as SRL and VSD are often built on top of quite a few other less complex NLP technologies

[5]The normal speed of the entity will have to be defined as part of the target object.

such as chunking, parsing, and named entity recognition (NER). It is possible for the performances of these technologies to affect the final selection of the graphics commands and their arguments.

In order to investigate the interactions between the information collection technologies and the selection of graphics commands and their arguments, different subsets of the information collection technologies will be experimented with respect to the selection of graphics commands. The results of these experiments will then provide useful feedback on which NLP technologies are the most important or require the most improvement.

1.5 Structure of the Thesis

In summary, the goal of this thesis is to establish the theories and principles that underlie the machine learning framework for using natural language instructions to generate computer animations with arbitrary animation systems. Although the existing research on natural language based animation generation mainly relies on symbolic methods, the research reported in this thesis is not meant to invalidate the existing research. Instead, this research is meant to explore an alternative framework that is based on machine learning methods.

The remainder of this thesis will start with a review of the current literature in natural language based animation in Chapter 2. In that chapter, I will briefly analyse each existing system based on their application domain, genre of input text, how they perform semantic analysis, and how they map the results of the semantic analysis to graphics commands.

Since text analysis is the first process of the animation generation, I will devote Chapter 3 to the presentation of the machine learning algorithms, statistical NLP systems and linguistic resources that are used in this research. The machine learning algorithms to be reviewed are Support Vector Machines (SVMs) and Maximum Entropy (MaxEnt) modelling. The NLP systems to be reviewed in this chapter are POS tagging, lemmatisation, statistical parsing, chunking, named entity recognition, semantic role labelling and verb sense disambiguation. The NLP resources to be reviewed include WordNet, VerbNet and Levin's verb classes.

Furthermore, as this research is directly involved in the generation of computer animation, it is necessary to introduce the relevant technologies in computer graphics before discussing how these technologies can be controlled based on natural language inputs. These computer graphics technologies will be presented in Chapter 4. Topics in this chapter include the representations of 3D graphical models and the most commonly used geometric transformations.

Novel research in semantic analysis is presented in Chapters 5 and 6. In Chapter 5, I will present a prepositional-phrase-only (PP-only) semantic role labeller which is aimed at improving the performance of generic semantic role labellers by targeting just

the prepositional phrases. The contributions of this chapter include a novel machine learning methodology for performing semantic role labelling on prepositional phrases, and three strategies for combining the PP-only semantic roles with the outputs of generic semantic role labellers.

In Chapter 6, I will present a novel verb sense disambiguation system that combines selectional preferences extracted with generic semantic role labellers with commonly used collocation and syntactic features. This algorithm is evaluated in a standalone fashion on two datasets: the Senseval2 corpus and a movie script corpus.

The machine learning framework for generating animation from action descriptions is then presented in Chapter 7. In this chapter, I will first describe the generic data flow for this framework, then I will describe the details of the generic forms of the graphics commands and their arguments and how they fit into the overall framework. After the graphics commands, I will describe the semantic and virtual scene features that can be used for machine learning algorithms to learn how to assign graphics commands to visualise verb instances. Furthermore, I will describe how the performance of this framework is influenced by the various configurations of semantic role labelling systems developed in Chapter 5, and how NLP resources such as WordNet and Levin's verb classes may be used to generalise the effectiveness of this framework over unseen verbs, and the usefulness of verb sense disambiguation to this framework.

Chapter 8 concludes the thesis, with a review of the principal findings of this research and discussions of future prospects.

Chapter 2

Literature Review

Several other researchers have attempted to bridge the gap between NLP and computer graphics, and their findings are highly relevant to the research reported here. In this chapter, I will review the most relevant projects, eight in total, and discuss their respective merits, and how they relate to this research. These projects are WordsEye (Coyne and Sproat 2001), SHRDLU (Winograd 1972), K2 (Tokugana *et al.* 2004), Di Eugenio (1993)'s work on understanding purpose clauses, the parameterised action representation (Badler *et al.* 2000), ScriptVis (Leung 2003), Carsim (Johansson *et al.* 2004; Johansson *et al.* 2005), and KRISP (Kate and Mooney 2006).

These projects differ from each other and from the research presented in this thesis in terms of the application domain, the genre of input instructions and even the target language. WordsEye is concerned with generating static scenes using scene descriptions in the form of English third person narratives. SHRDLU deals with controlling and communicating with a virtual robot in a virtual world using conversation style instructions in English. K2 also deals with controlling and communicating with virtual robots in a virtual world, but the input instructions are in the form of spoken Japanese. Di Eugenio (1993)'s work is focused on understanding the purpose clauses often found in instruction manuals written in English. The parameterised action representation (PAR) is a formal representation of action information, and it is designed for converting instructions (in English) that contain conditional statements to formal representations that can be used in interactive simulations. ScriptVis is concerned with generating animation based on action descriptions in the form of English third person narratives. In other words, ScriptVis is identical to this research in terms of application domain, genre of input instructions and target language. However, as will be discussed later, ScriptVis took a significantly different approach to the one taken in this research. Carsim deals with recreating car crash scenes based on newspaper reports in Swedish. Finally, KRISP is a system that uses machine learning algorithms to transform natural language sentences in formal meaning representation languages.



Figure 2.1: An example of a correctly generated scene by WordsEye

2.1 The WordsEye Project

WordsEye is a project that investigates how static 3D scenes can be constructed using descriptions given in natural language (Coyne and Sproat 2001). For example, given the description in (2.1), the static scene in Figure 2.1 would be generated.[1]

(2.1) A white cat is in a black chair.

The scene generation of WordsEye is a two-stage process. The first stage is linguistic analysis in which WordsEye constructs an internal semantic representation of the scene from the natural language instruction. This stage is similar to the information collection process of the research reported in this thesis. The second stage is the

[1] All the renderings shown in this section were generated in October 2006 at the website `http://www.wordseye.com/workspace`.

scene construction, in which the semantic representation of the scene is used to place appropriate 3D objects into a virtual environment.

The linguistic analysis of WordsEye consists of two steps. In the first step, the input is POS tagged with a tagger developed by Church (1988), and then parsed with the Collins Parser (Collins 1999). In the second step, WordsEye establishes a hierarchical structure between the nouns, the verbs and the prepositions. For a noun, this structure captures the relationship between it and its modifiers. For a verb or preposition, this structure captures the relationship between it and its arguments. For example, given the sentence in (2.2), the hierarchical structure in Figure 2.2 can be created.

(2.2) John said that the cat was on the table.

The conversion from the hierarchical structure to WordsEye's internal semantic representation is performed by looking up the semantic interpretation frames associated with the head nouns, verbs and prepositions. For example, Figure 2.3 shows the semantic representation corresponding to the hierarchical structure shown in Figure 2.2. For each nominal head word, WordsEye replaces it with the 3D model that has been assigned to its WordNet entry (see Chapter 3) and its hypernym and hyponym. For example, as nodes 2 and 5 in Figure 2.3 show, *John* would be grounded with a 3D model of male humanoid in the virtual stage, and *cat* would be grounded with a 3D model for cat or other feline. For each prepositional head word, WordsEye not only includes it in the semantic representation, but also uses special semantic functions to work out the geometric relationships between its dependents. For example, node 4 in Figure 2.3 shows that for the preposition *on*, its left argument is node 5 (*the cat*) and its right argument is node 7 (*the table*). For each verbal head word, WordsEye includes the verb as part of the semantic representation, and extracts its arguments using the semantic frames given by Levin's verb classes (Levin 1993). For example, node 1 in Figure 2.3 shows that for the verb *say*, its subject is node 2 (*John*), and its direct objects[2] are nodes 4 (*on*), 5(*the cat*), and 7 (*the table*).

Once the semantic representation for the scene has been obtained. A set of depiction rules are then used to translate the semantic representation into low level descriptors of the 3D scene. During this process, the nodes for head nouns will be grounded with specific 3D models whose appearance will in turn be modified by the head nouns' dependents; the nodes for prepositions will be used to assign locations to their dependents; and the nodes for verbs will be mapped to specific graphics commands.

The WordsEye project provides valuable insights into the research reported in this thesis because it also needs to perform discourse analysis on the input text and visualisation of verbs. However, there is a significant difference: WordsEye is

[2]The use of the term "direct object" here is unconventional, however it follows the usage in WordsEye.

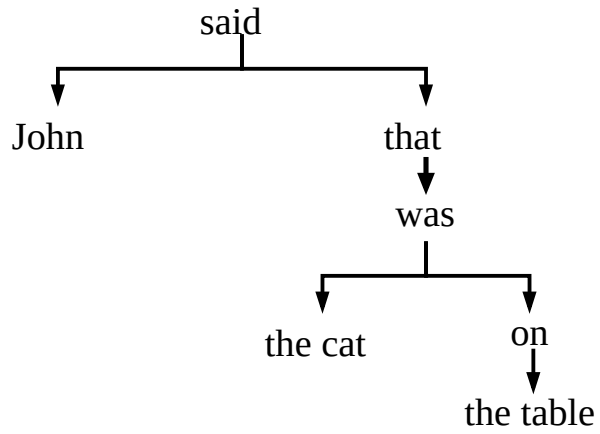


Figure 2.2: Example of a hierarchical structure.

```
((“node2”   (:ENITY :3D-OBJECTS (“mr_happy”)
            :LEXICAL-SOURCE “John” :SOURCE SELF))
(“node1”    (:ACTION “say” :SOURCE “node2”
            :DIRECT-OBJECT (“node5” “node4” “node7”) ...))
(“node5”    (:ENTITY :3D-OBJECTS (“cat-vp2842”)))
(“node4”    (:STATIVE-RELATION “on” :FIGURE “node5”
            :GROUND “node7”))
(“node7”    (:ENTITY :3D-OBJECTS
            (“table-vp14364” “nightstand-vp12374”
            “table-vp4098” “pool_table-vp8359” ...))))
```

Figure 2.3: Example of a semantic representation of WordsEye.

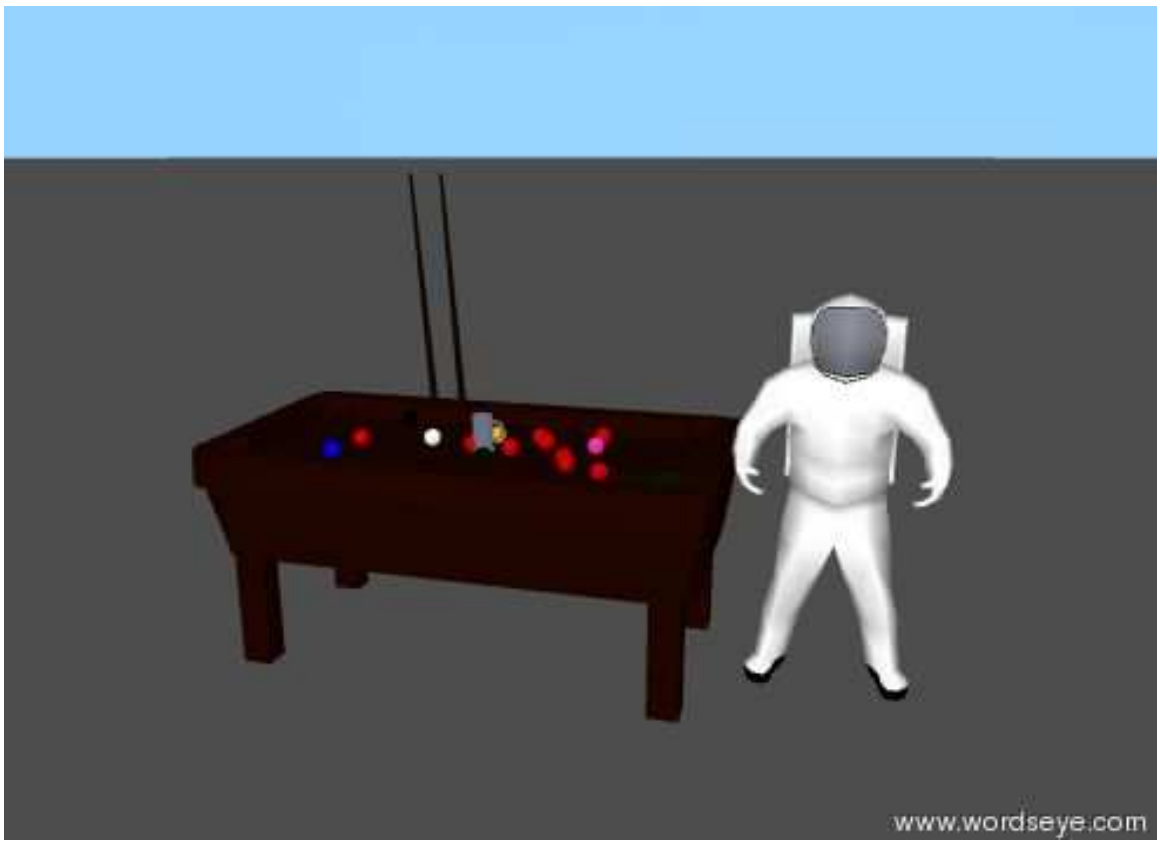


Figure 2.4: WordsEye's visualisation of *The man puts the cup on the table*

concerned with creating static 3D scenes from natural language descriptions, and this research is concerned with generating the dynamic actions of virtual objects in pre-built virtual scenes from natural language descriptions. For example, given the sentence *The man puts the cup on the table*, WordsEye will generate a static scene that consists of a virtual man, a virtual cup on a virtual table with the cup already on the table (e.g. Figure 2.4); however, this research is concerned with a generic framework for creating systems that can generate a dynamic animation that depicts the entire event described in this sentence within a pre-built virtual scene, and this animation would be different depending on the initial configuration of the virtual scene. On the other hand, it seems that the WordsEye project would be an ideal complement to this research, and that by combining the findings of these two projects, one could envisage a system which allows its users to create animations from scratch based on natural language inputs.

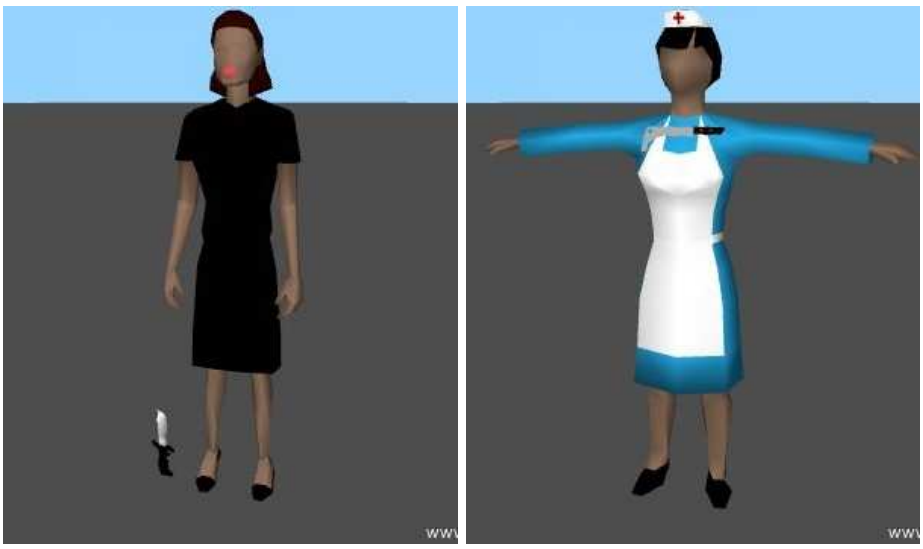
Another useful aspect of the WordsEye project is its semantic functions for interpreting prepositional phrases that denote spatial information. It was mentioned in Chapter 1 that sometimes arguments to verbs can be modified by locational modifiers, and this would affect their conversion to the arguments of graphics commands. Since most locational modifiers are prepositional phrases, WordsEye's semantic functions could provide some valuable insight into their interpretations.

Furthermore, WordsEye allows the users to use adjective phrases to modify the colour and texture information of the objects in their static scene. The method that WordsEye uses to interpret these adjective phrases can also provide some insight to the grounding process (mapping textual descriptions of objects to virtual objects), which is a necessary part of the research reported in this thesis.

The most relevant aspect of WordsEye is how it visualises verbs. WordsEye does not generate the dynamic aspects of the actions denoted by verbs. Instead, it just manipulates the relevant virtual objects in poses that resemble snapshots of the actions. However, even with this difference, WordsEye faces the same problem as the research reported in this thesis in terms of the vast number of verbs. The approach taken by WordsEye is to manually assign a visual interpretation to each verb, and ignore verbs for which no interpretation has been assigned. Coyne and Sproat (2001) claim to have created semantic entries for more than 2300 verbs, however it is not clear how well these entries generalise across different senses of the verb, or how much human labour has been spent on the creation of these entries. At the time of writing, the online version of WordsEye clearly warns the user to avoid using verbs in their scene descriptions. This is perhaps an indication of the difficulty faced by WordsEye's approach to visualising verbs.

Judging from the available resources and interfaces of WordsEye, it still faces a number of challenges that reduce its practical usefulness. One of these challenges is the selection of a 3D graphics model when two or more candidates are available. It seems that WordsEye is currently using a semi-random mechanism for this selection. For example, if the scene description mentions the existence of a table, and there are two 3D graphical models of tables available, WordsEye randomly selects one and then gives the user the option to change the 3D graphical model later. This challenge is difficult to overcome because the scene descriptions rarely contain sufficient specification for selecting the right 3D graphical models, and the 3D graphical models themselves do not normally carry linguistic information that can be used for matching descriptions in natural language. Since WordsEye cannot distinguish between the different candidate 3D graphical models, users may end up manually changing the 3D graphical models after the scene has been generated, thus reducing the level of WordsEye's automation.

Another difficult challenge faced by WordsEye is the interpretation of the parts of the objects in the scene descriptions. For example, consider the sentences in (2.3), WordsEye generally does a fair interpretation of (2.3a), but fails to correctly interpret



There is a knife next to the woman

There is a knife in the woman's hand

Figure 2.5: Comparisons of WordsEye's interpretation of *There is a knife next to the woman* and *There is a knife in the woman's hand*

(2.3b), see Figure 2.5 for an illustration.[3]

(2.3) a. There is a knife next to the woman.

b. There is a knife in the woman's hand.

In fact, at the time of writing of this thesis, WordsEye's web interface specifically warns the user to avoid mentioning the parts of objects. This is understandable because 3D graphical models rarely carry reliable linguistic information regarding their parts, and this makes it difficult to interpret textual descriptions of their parts. The same challenge is also faced by the research reported in this thesis, as will be discussed in Chapter 4.

In summary, WordsEye is a research project for generating static virtual scenes from natural language descriptions. It offers useful insights on the interpretation of location and appearance modifiers of noun phrases, as well as the method of using handcrafted rules to map verbs to predefined visual representations. It also faces some of the same challenges faced by the research reported in this thesis, namely the handling of the vast number of verbs and interpreting noun phrases that mention components of objects.

[3]The knife is just below the woman's neck in the picture for *There is a knife in the woman's hand*

2.2 SHRDLU

Terry Winograd's SHRDLU project (Winograd 1972) is one of the earliest attempts at understanding natural language within a constrained virtual environment. SHRDLU consists of a 'blocks world' which contains a robot and a number of 3D objects such as cubes and pyramids (blocks). Through natural language dialogues, human users can interact with the robot by commanding it to manipulate the blocks, informing it with additional facts about the blocks, and querying it for information about the blocks and the history of their interactions. Additionally, the robot uses natural language to provide feedback to the user, and ask for clarification when needed. Figure 2.6 shows a screenshot of the latest implementation of SHRDLU, and Figure 2.7 shows some example user instructions and SHRDLU's responses, taken from Winograd (1972).

Broadly speaking, there are many similarities between SHRDLU and the research reported in this thesis, since both are focused on using natural language instructions to generate accurate computer animation. However, there are two key differences.

The first key difference is the genre of the language. SHRDLU's genre is dialogue, and its natural language inputs are generally imperative commands addressed directly to the robot, statements about the blocks world, questions, or direction questions. As far as generating animation is concerned, only the command inputs are relevant here. On the other hand, the language genre for the research reported in this thesis is narrative descriptions of actions. There are some clear distinctions between these two genres. SHRDLU always expects its commands to be unambiguously addressed to the robot, and even though it is possible to specify multiple actions to the robot, only a single action can be carried since there is only one robot. The narrative inputs of the research reported in this thesis are not addressed to any particular entity, and multiple entities may be involved at the same time to carry out different actions. Therefore, the target genre of text for the research reported in this thesis can be expected to demand a different type of linguistic analysis than that of SHRDLU.

The second and most important difference is the research scope. SHRDLU makes a strict closed world assumption. The "blocks world" contains only blocks, i.e. cubes, boxes, spheres and pyramids of varying size and colour. The actions that can be performed in this world are limited combinations of moving the robot on top of a block, making the robot pick up a block, and making the robot drop a block down. In other words, all that is possible is simple translational movements with respect to a small fixed set of objects. The benefit of having such a small number of simple objects and actions is that it is feasible to handcraft a set of rules to interpret the user's inputs with respect to the configuration of the virtual environment. Furthermore, since the user of SHRDLU already knows the capabilities of the system, they are unlikely to give SHRDLU instructions that it is not designed to carry out. This means that it is reasonable for SHRDLU to expect its instructions to be limited to things such as picking up or dropping things in the blocks world. However, it also means that the

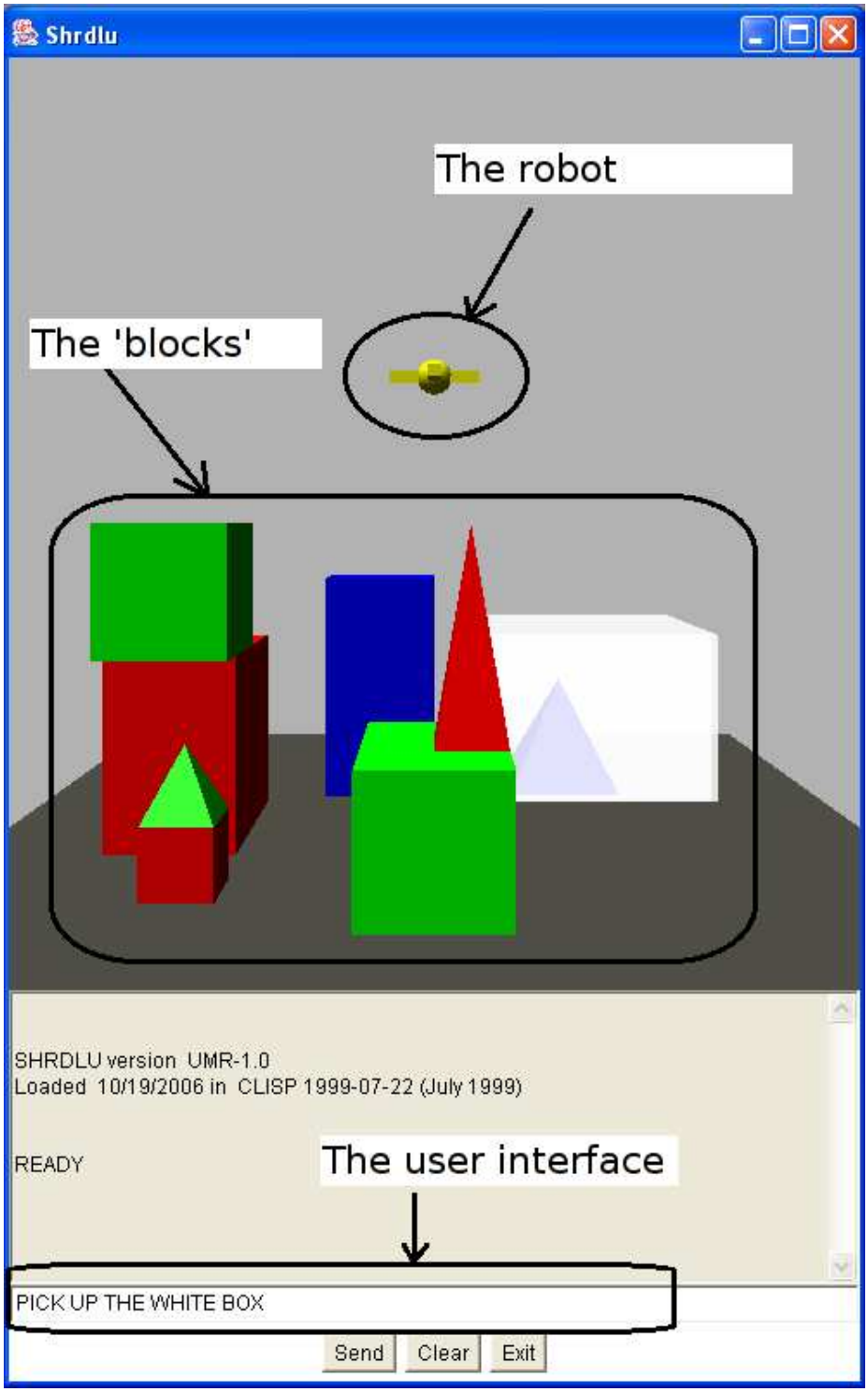


Figure 2.6: A screenshot of SHRDLU

1. **user:** pick up a big red block.

   SHRDLU: OK. (*Moves the green box to a different location, then picks up the red block that was under the green box*)

2. **user:** grasp the pyramid

   SHRDLU: I DON'T UNDERSTAND WHICH PYRAMID YOU MEAN. (*There is more than 1 pyramid, SHRDLU has to ask for clarification.*)

3. **user:** the blue pyramid is mine

   SHRDLU: I UNDERSTAND. (*SHRDLU records this information into its database.*)

4. **user:** I own blocks which are not red, but I don't own anything which supports a pyramid.

   SHRDLU: I UNDERSTAND. (*Two new theorems are created for proving things about "owning".*)

5. **user:** how many blocks are not in the box.

   SHRDLU: FOUR OF THEM.

Figure 2.7: Example instructions to SHRDLU

findings of SHRDLU are limited by its domain specificity and generalising them to other domains is rather difficult. On the other hand, the research reported in this thesis aims to come up with a generic framework for generating computer animation using natural language instructions. This framework is not domain specific in itself, but adaptable to different domains. Therefore, the scope of the research reported in this thesis is different to that of SHRDLU.

2.3 The K2 Project

K2 is a project that investigates how to control agents in virtual environments using speech-based Japanese instructions (Tokugana *et al.* 2004). Overall, K2 can be viewed as a successor of the SHRDLU project. Other than having different target languages, K2 differs from SHRDLU in several aspects.

Firstly, the input to K2 takes the form of spoken language, whereas the input to SHRDLU is written language. Spoken language input adds a layer of complexity over written language input. As a result, a significant part of K2 is focused on solving speech specific issues such as repairing ill-formed speech commands. This kind of research is irrelevant to the research reported in this thesis, and it will not be reviewed here. Similar to SHRDLU, K2 expects speech inputs that are imperative commands and questions about the virtual environment from the perspective of the individual virtual robots. However, since K2 allows more than one virtual robot, some of its imperative commands have to start by naming a specific robot. For example, commands of the form *Agent X, push the red ball* are quite common.

Secondly, the virtual environments of K2 are similar to the real world and much more complex than the blocks world of SHRDLU. They can contain complex geometric objects such as tables and chairs. The virtual robots themselves are humanoids. However, since the interactions that these robots can have with the virtual environment are defined by a lexicon of around 100 Japanese words, the capabilities of K2 are still limited by this lexicon.

According to Funakoshi *et al.* (2002), K2's linguistic analysis starts from parsing the input speech with an incremental dependency parser. The grammar of this parser consists of a set of handcrafted rules based on K2's lexicon. Furthermore, additional processing for repairing the speech inputs is also included as part of the linguistic analysis. The parsed and repaired speech command is mapped into a predefined case frame. Each case frame is then analysed in the context of the entire history of interaction in order to perform referent resolution and resolve ambiguities in the input commands. Each fully disambiguated case frame is then sequentially used to generate the animation. Judging from the available literature and personal correspondence with the project's principal investigator, K2's approach to generating animation from the linguistic analysis of input commands is rule based.

Based on the available information about K2, it can be concluded that although

K2 aims to extend SHRDLU's research findings in similar direction to this research, it has not managed to systematically overcome the limitations of SHRDLU.[4]

2.4 The Animation from Natural Language Project

The goal of the Animation from Natural Language (AnimNL) project (Badler *et al.* 1993) is to use natural language instructions to control the actions of virtual human avatars in virtual environments. AnimNL is not just a single project, but a series of related projects that use natural language to control agents in different virtual environments. According to the available literature on AnimNL, most of AnimNL modules can be considered as extensions of SHRDLU.[5] AnimNL covers a wide area of research topics including NLP, computer graphics, cognitive science and robotics. However, the most relevant AnimNL research with respect to the high level goals of the research reported in this thesis can be found in Di Eugenio (1993)'s work on executing natural language instructions in the form of purpose clauses, and Badler *et al.* (2000)'s work on Parameterised Action Representation (PAR).

Understanding Purpose Clauses

Purpose clauses generally fit the pattern of *Do* α *to do* β.[6] The frequency of these clauses is non-trivial in many genres of text, which is suggestive of the need to study them.

This section focuses on two key aspects of Di Eugenio (1993)'s work. The first is the target genre of text. The second is how the semantic analysis of text gets converted into graphics commands.

The target genre of text for Di Eugenio's work is how-to books on placing wall tiles and hanging wall covers. Di Eugenio treats the β verbs in the purpose clauses as the goals to be achieved and the α verbs as pre-steps to achieving the goals. Examples of these purpose clauses can be found in (2.4).

(2.4) a Use a sponge dipped in lukewarm water to remove excess adhesive before it dries.

b Go into the other room to get the urn of coffee.

c Unscrew the protective plate to expose the box.

[4]Two newer versions of K2 have been introduced to the literature. K3 (Funakoshi and Tokunaga 2005) is essentially the same as K2 but with much more realistic 3D models for the human avatars. K4 (Funakoshi *et al.* 2006; Funakoshi and Takenobu 2006) is an extension of K3 and offers more sophisticated algorithms for response generation and repairing the input speech. I do not review these two systems as their improvements over K2 are not relevant to this research.

[5]AnimNL's own description at `http://hms.upenn.edu/software/AnimNL/animnl.html` confirms this observation

[6]Although many sentences that fit this pattern are not purpose clauses

d Flick the switch to turn on the light.

In Di Eugenio's work, the conversion from semantic analysis of the purpose clauses to graphics commands is based on a set of handcrafted rules with respect to a predefined library of actions. This actions library defines a set of goals, and a series of steps that can be undertaken to achieve each goal. The handcrafted rules then match the β verbs to the goals, and the α verbs to one of the predefined steps.

Di Eugenio's work highlights the planning problem in natural language based animation generation. Some verbs can describe actions that are often not possible to visualise in a single step, and the configurations of the virtual scene can also influence the process of the visualisation. In order for a natural language based animation system to be successful, it needs to have the capacity to break down complex actions into multiple steps, and choose the best step according to the current configuration of the virtual scene. In Di Eugenio's work, this planning capability is realised through handcrafted rules.

The target genre of text for the research reported in this thesis is action descriptions of movie scripts, which is rather different from the instruction manual studied in Di Eugenio's work. For the action descriptions, the α and β verb in the purpose clauses of this genre mostly describe sequential events which can be regarded as being independent of each other. Furthermore, since the descriptions are given from an outside observer's point of view, it is often possible for the actions represented by the β verbs to fail. The examples in (2.5) exemplify this.

(2.5) a John took a step forward to open the door, but Michael blocked him.

b John turned around to look at Michael, but Michael was already gone.

c John reached out to take the key from Michael, but Michael gripped it tightly and refused to let go.

Even with the differences in the target genre, the findings of Di Eugenio's work still offers considerable insights to the research reported in this thesis, as it identifies some of the issues in the planning process of natural language based animation generation, which can be used to design the features for the machine learning framework investigated in this research.

Parameterised Action Representation

Parameterised Action Representation (PAR) is a formal representation for action information. The motivation for creating PAR is to use natural language instructions to control animation. The application domain of PAR is video game style simulations in which the user can observe and fully participate in the activities in the virtual scene in real time. As a result, the target genre of text for PAR is more similar to instruction manuals than action descriptions of movie scripts. For example, PAR was

used in a military training system which has training data that contains conditional instructions such as *When you draw your weapon at the driver, take cover from the driver behind the drum* and *If soldier1 draws his weapon at the driver, draw your weapon at the driver and take cover from the driver behind your drum.* These sorts of conditional instructions have negligible chance of occurring in movie script style text.

Figure 2.8 and Figure 2.9 show the action template for PAR. Through PAR, Badler *et al.* provided a valuable summary of the practical issues in natural language based animation generation. For example, they identified the fact that some times, certain applicability conditions must be satisfied before an action can be executed, and these conditions can be specified in the form of boolean functions over the virtual scene. Badler *et al.* (2000) also identified the planning issue for complex actions which cannot be completed in a single step, and allows the users of PAR to explicitly specify necessary preparatory and sequential steps. Furthermore, Badler *et al.* showed that the actions may not be independent from each other, i.e. that multiple actions could take place at the same time, and these inter-action relationships may be significant. Finally, Badler *et al.* showed that there needed to be a mechanism for actions to terminate themselves, and that such mechanism could also be specified in the form of boolean functions over the virtual scene.

There is a significant difference between the application domain of PAR and the research reported in this thesis. This difference is the concept of a smart avatar for which PAR was designed. According to Badler *et al.* (2000), a smart avatar is a software agent that primarily functions independently, but has the ability to interact with or take commands from human users and other smart avatars. On the other hand, the application domain for the research reported in this thesis is just the faithful visualisation of natural language based action descriptions. Therefore, the animation framework documented in this thesis is highly dependent on the instructions from human users.

In summary, AnimNL provides valuable insights into generating animation using natural language instructions, however, it differs from this research in terms of the genre of text and style of animation.

2.5 The Script Visualisation (ScriptVis) Project

ScriptVis is a research project that aims to create animations based on existing virtual environments and movie scripts written in natural language (Leung 2003). The research goal of ScriptVis is essentially the same as the research reported in this thesis. Therefore, the findings of ScriptVis are highly relevant. The review of ScriptVis will focus on linguistic analysis and selection of graphics commands.

The linguistic analysis of ScriptVis does not involve POS tagging or semantic role labelling. Instead, it starts with parsing the input sentence with the Apple Pie Parser

participants	**agent** *AGENT* **objects** *OBJECT* list
start	*Time*
applicability conditions	*CONDITION boolean-expression*
preparatory specifications	**condition** *CONDITION boolean-expression* **action** *PAR*
subactions	*PAR* **constraint-graph**
execution steps	primitive/complex
core semantics	
	motion *MOTION* **force** *FORCE* **path** *PATH* **purpose** *PURPOSE* **termination** *CONDITION boolean-expression* **duration** *LENGTH* **manner** *MANNER*
post-assertions	(assertions)
parent action	*PAR*
previous action	*PAR*
concurrent action	*PAR*
next action	*PAR*

Figure 2.8: The overall action template for the Parameterised Action Representation

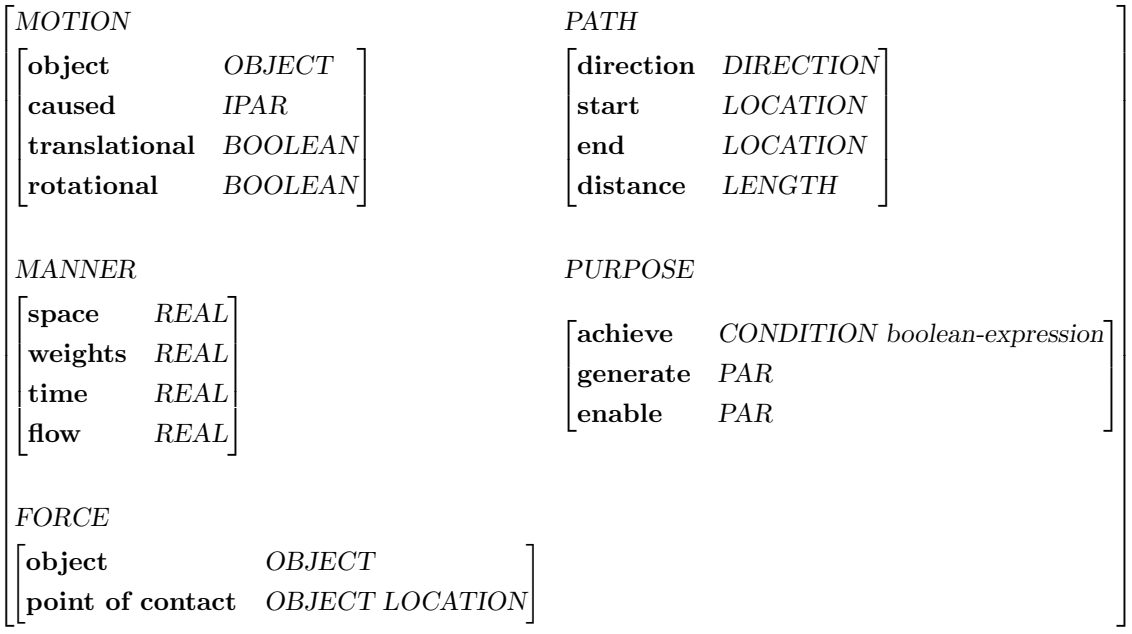


Figure 2.9: The motion template for the Parameterised Action Representation

(Sekine 1998).

After a parse tree has been produced, ScriptVis then uses a set of handcrafted rules to perform a number of additional analysis tasks, including passive-to-active voice transformation, subordinate clause separation, locating the surface subject and object of each verb, and phrase head word extraction. The final results of ScriptVis' linguistic analysis are a series of fully specified semantic templates in the form shown in Figure 2.10. Each template represents a separate event, where each event is either an action, such as *The man sits in the chair*, or a state, such as *The man is in the chair*. Sentences that contain multiple verbs will generate multiple events, one for each verb. For example, the sentence *I chased after the thief as he fled* will generate two events, one for *chased*, one for *fled*.

Two important observations can be made from Figure 2.10. Firstly, the type of the event will determine the structure of the rest of the event template. For state events, the Object, Mood, and Pace slots will be empty. For action events, only the Degree slot will be empty. Secondly, selection of graphics commands is actually part of the linguistic analysis, since the name of the event is supposed to be the 'internal name' of the event.

ScriptVis' approach to linguistic analysis has plenty of room for improvement. It is clear that the accuracy of this analysis depends entirely on the parsing accuracy of APP and the coverage of the handcrafted rules. Despite being an excellent parser,

Event Name:	*The internal name of this event*
Event Type:	*The type of the event, either STATE or Action*
Subject	
	The textual representation of the subject of this event
Object	
	The textual representation of the object of this event
Parameters	
	Degree = *10: Most; -10 Least (For STATE events)*
	Mood = *10: Happy; -10 Sad (For Action events)*
	Pace = *10: Fastest; -10 Slowest (For Action events)*

Figure 2.10: The template for ScriptVis' semantic representation of events.

the performance of APP is still not reliable enough to be the only source of linguistic information for ScriptVis. APP was trained on the training sections of the WSJ corpus of the Penn Treebank, and achieves labelled bracketing precision and recall on the WSJ test corpus of 77.45% and 75.58% respectively. This performance is significantly lower than other state-of-the-art parsers such as the Charniak parser (Charniak 2000) which has 89.5%+ precision and recall on the same test data. There is very little description of these handcrafted rules in Leung (2003), but inspection of the source code of the final ScriptVis system reveals that these handcrafted rules were designed to cover around ten verbs, and any other verbs are ignored.

Additionally, the event template of ScriptVis lacks coverage for action related information. For example, the core set of arguments of an action only includes the subject and object, and leaves out other arguments such as instrument and location. The non-core parameters for actions are mood and pace only, and not other manner parameters that can be described through adverbs such as *awkwardly*, *gingerly*, or *carelessly*. Furthermore, the mood parameter only describes a one dimensional mood (happy or sad), and leaves out other emotions such as *angry*, *surprised*, and *worried*. Overall, the event template of ScriptVis is only designed as a proof of concept, and given the vast number of verbs and verb related adverbs, it would be a non-trivial task to extend this template to model the actions of the real world.

The integration of graphics command selection into the linguistic analysis process also adds an unnecessary complexity since it binds the findings of linguistic analysis to a specific set of graphics commands, thereby reducing ScriptVis' generalisation

- Plan: FEED(actor,object)

  actor=Human,object=Baby walk, PICK_UP, ARM_MOVEMENTS_NORMAL
  actor=Human,object=Dog walk, KNEEL_DOWN, STRETCH_RIGHT_ARM_NORMAL
  actor=Human,object=Bird walk, STRETCH_RIGHT_ARM_NORMAL

Figure 2.11: Breakdown of the Feed action in ScriptVis

capability.

The selection of graphics commands in ScriptVis starts with linguistic analysis, followed by a two-stage process. The first stage is called high level planning. In this process, the target verb instance is mapped to an action or a high level graphics command. This action is then broken down into a series of sequential sub-actions based on the properties of the subject and object of the action. Leung (2003) illustrates this process with the example of the FEED action as shown in Figure 2.11. This plan mandates that when the actor and object of the FEED action are Human and Dog respectively, FEED is broken down into first walk the Human to the Dog, then make the Human KNEEL_DOWN, and finally STRETCH_RIGHT_ARM_NORMAL to make the human stretch out his right arm to the Dog's mouth.

There are three problems with this high level planning process. Firstly, it is impractical to predefine a separate plan for each combination of arguments and actions. Take the high level plan for FEED as an example, almost all animate entities can be the actor and object of FEED, and it would be a time consuming process to create rules for all of them. Furthermore, FEED is only one action, and there are thousands more actions whose arguments are just as flexible as those of FEED. It would be impractical to create high level plans for all of them. The second problem is that these plans are too rigid. For example, when the object of FEED is Dog, the high level plan requires the Human actor to KNEEL_DOWN as the second step. However, the kneeling down is only necessary when the Dog is at a lower position than the human, and it would not be necessary if the dog was sitting on the same couch as the human. The last problem is that these plans are bound to specific graphics libraries, and therefore would almost certainly have to be redone to suit different graphics libraries – an expensive and costly process.

The second stage of this two-stage process is the intermediate level reasoning. In this process, each sub-action is fully specified according to a simplified version of the Parameterised Action Representation (PAR). Figure 2.12 shows the template of this simplified version of PAR. This simplified PAR has the same problems as the full version of PAR, which were described in Section 2.4. There is no slot for timing

$$
\begin{bmatrix}
\text{participants} & \begin{bmatrix}\text{agent:} & \textit{AGENT}\\ \text{objects:} & \textit{OBJECT}\text{ list}\end{bmatrix}\\
\text{core semantics:} & \\
\begin{bmatrix}\text{motion:} & \begin{bmatrix}\text{object:} & \textit{OBJECT}\\ \text{caused:} & \textit{BOOLEAN}\\ \text{translational} & \textit{BOOLEAN}\\ \text{rotational} & \textit{BOOLEAN}\end{bmatrix}\end{bmatrix} & \\
\text{path:} & \begin{bmatrix}\text{direction:} & \textit{DIRECTION}\\ \text{start:} & \textit{LOCATION}\\ \text{end:} & \textit{LOCATION}\\ \text{distance:} & \textit{LENGTH}\end{bmatrix}\\
\text{manner:} & \textit{MANNER}\\
\text{subactions:} & \textit{PAR}\text{ constraint graph}\\
\text{previous action:} & \textit{PAR}\\
\text{next action:} & \textit{PAR}\\
\text{parent action:} & \textit{PAR}
\end{bmatrix}
$$

Figure 2.12: The simplified PAR representation for ScriptVis

information in this template, and as a result, the temporal relationships between different actions cannot be specified.

In summary, ScriptVis uses handcrafted rules to perform semantic analysis of the input text, and it also uses handcrafted rules to convert the results of its semantic analysis to graphics commands in a customised graphics system. ScriptVis can be directly extended by adding more handcrafted rules into its semantic analysis system, or adding more functionalities to its graphics system. However, these handcrafted rules do not generalise well to different genres of text or application domains.

2.6 The Carsim Project

Carsim (Johansson *et al.* 2004; Johansson *et al.* 2005) is a project that automatically creates 3D animations that depict road accidents from written Swedish texts, most of which are newspaper reports. More specifically, Carsim generates both the static scene and the animation of the crash from its input text. In this regard, Carsim provides a number of important insights into both natural language based static scene

creation, and natural language based animation generation.

The target genre of text for Carsim is newspaper articles. This means that the descriptions of road accidents are generally narrative in nature, similar to the movie scripts studied in the research. Therefore it is not surprising that the linguistic analysis process of Carsim is similar to that of this research: both use POS tagging to identify the specific events to be visualised, and both use semantic role labelling to extract the relevant information about the events. The semantic role labeller of Carsim is trained over approximately 800 manually annotated newspaper reports, and the semantic roles were designed to capture the various participants of car crashes (e.g. TIME, VICTIM, ACTOR, etc.). Carsim's use of a semantic role labeller makes an interesting contrast with the previously reviewed projects, all of which perform semantic analysis based on parse trees. Since semantic roles are deep linguistic structures, they tend to be language independent, and it is therefore not surprising that the researchers responsible for Carsim claim that the architecture of Carsim is applicable to other languages in the domain of car crashes. Parse trees are syntactic structures which are more likely to be language dependent, therefore parse tree based semantic analysis cannot not be directly applied to other languages. However, similar to the other projects reviewed in this chapter, Carsim uses handcrafted rules to convert the semantic information about car crashes to graphics commands for its customised animation system.

Carsim's target genre of text is newspaper reports which may mention the same events multiple times. As a result, Carsim has to perform unification on the events detected to avoid visualising the same event multiple times. This unification is performed by treating all the events that are of the same type and have the same participants as the same event. Since the unification problem can also occur within the movie script style text used in this thesis, it would be interesting to see the effectiveness of this strategy on different types of text.

As stated above, Carsim does not assume the existence of the virtual crash scene in its interpretation of the newspaper texts. Instead, it constructs the virtual crash scene from the descriptions of the crash scene and the descriptions of how the crash happened. This is significantly different to WordsEye which expects the input text to be just descriptions of the scene. In this regard, Carsim's approach is much more relevant to the research reported in this thesis, because action descriptions are similar to descriptions of how a car crash happened.

In summary, Carsim provides several valuable insights into natural language based animation generation. It demonstrated that semantic role labelling can be a useful and language-independent tool for performing semantic analysis in at least some natural language based animation generation domains. It provided a solution for unifying events in descriptions of dynamic actions (car crashes). Finally, it demonstrated how virtual scenes can be constructed from action descriptions.

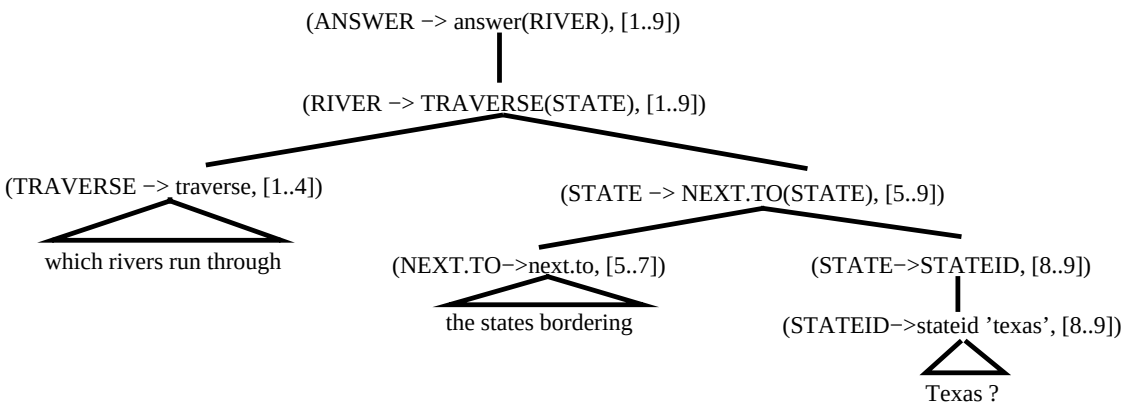


Figure 2.13: Parse tree for the query **answer(traverse(next_to(stateid('texas'))))**, translated from the sentence *Which rivers run through the states bordering Texas?*

2.7 KRISP

KRISP (Kernel-based Robust Interpretation for Semantic Parsing) is a system that uses support vector machines with string kernels to transform natural language sentences into formal languages defined in context free grammars. KRISP has been applied to two different domains: parsing natural language instructions for coaching virtual soccer teams to a formal language called CLANG (Chen *et al.* 2003); and parsing natural language queries to a formal query language for a geographical database (Zelle and Mooney 1996). Although KRISP has not been applied in a computer animation related domain, it is still highly relevant to the research reported in this thesis, because the core research problem it investigates is also how to apply machine learning algorithms to transform natural languages sentences to formal languages.

KRISP performs the semantic parsing by finding the most probable parse tree of the target formal language with respect to an input natural language sentence. Each node in the parse tree corresponds to a sequence of tokens from the input natural language sentence. The probability of a parse tree is the product of the probabilities of its child nodes, and training of KRISP involves learning the probability distribution of the non-terminals with respect to the token sequences from gold-standard data. For example, Figure 2.13[7] shows the parse tree for the query **answer(traverse(next_to(stateid('texas'))))**, which is the correct translation for the sentence *Which rivers run through the states bordering Texas?*.

It is interesting that other than tokenisation, KRISP actually does not perform any commonly used syntactic or semantic analysis on the input sentence at all (e.g. no POS tagging, parsing or chunking). The estimation of the probability is based

[7]This example is taken from (Kate and Mooney 2006).

purely on the token-wise string similarity calculated by the string kernel. Because of this, KRISP is language independent, and has been applied to languages other than English. However, it is worth noting that compared with the other system reviewed in this chapter, the two domains on which KRISP has been successfully applied are more restricted in terms of vocabulary and syntactic flexibility, and it is not clear how well KRISP can be adapted to domains with less restricted input. Additionally, Kate and Mooney also showed that KRISP's performed significantly better on domains with shorter sentences (the geographical database domain, which had an average of 7.48 tokens per sentence) than domains with longer sentences (the virtual soccer domain, which had an average of 22.52 tokens per sentence), therefore it seems that KRISP may not generalize well to domains with long sentences.

In summary, KRISP offers valuable insights to how natural language sentences can be translated into formal languages with machine learning algorithms, especially when the domain has restricted vocabulary and syntactic flexibility. It shows that the machine learning approach is indeed flexible and can replace the handcrafted rules in certain domains.

2.8 Summary

In this chapter, I have reviewed several past and ongoing research projects that attempt to generate computer animation based on natural language instructions. These projects differ in domain, scale, and linguistic analysis methodology, but all of them use handcrafted rules to translate verbs to graphics commands, and most of them are restricted to a specialised domain.

WordsEye's domain is similar to the domain of this research. It offers a number of useful insights including the interpretation of locational prepositional phrases and adjective modifiers to noun phrases, as well as the feasibility of creating handcrafted rules for large numbers of verbs. Furthermore, WordsEye complements the research reported in this thesis.

SHRDLU was one of the first projects on natural language based animation generation. The genre of its input text is written dialogue, and instructions are always directed to a virtual robot. SHRDLU's domain is a blocks world which contains simple geometric objects, and rigid body translations of these simple objects are the only type of action that can be performed over its virtual environment. Handcrafted rules are used by SHRDLU to perform both the semantic analysis of the input text, and the conversion of the results of the semantic analysis to graphics commands.

K2 has a similar domain to the project reported in this thesis, however, it currently uses a set of handcrafted rules to perform selection of graphics commands based on a lexicon of around 100 Japanese words. Since it is an ongoing project, more handcrafted rules may be added to increase the size of its lexicon.

Di Eugenio's work on purpose clauses is restricted to the domain of how-to books

on placing wall tiles and hanging wall covers. It uses a predefined action library for translating the verbs in natural language instructions to sequences of graphics commands. Naturally, the generalisation capability of Di Eugenio's work relies on the size of the action library. However, as this action library is manually defined, extending Di Eugenio's work would still require a considerable amount of human labour.

ScriptVis is in the same domain as this research. However, as a proof of concept project, it uses a set of handcrafted rules which can only translate a few verbs to a small set of predefined graphics commands. As a result, generalising ScriptVis within its original framework would require a considerable amount of human labour.

Carsim's domain is the simulation of car accidents. Its virtual environment only contains objects commonly involved in car accidents. The actions that can take place in Carsim's virtual environments consist of simple rigid body transformations such as translation and rotation which are sufficient to visualise car accidents.

KRISP's domain is the parsing of natural language sentences to formal languages in domains with restricted vocabulary and syntactic flexibility. The key feature of KRISP is that it is purely based on token-wise substring matching, and does not require any syntactic or semantic analysis other than tokenisation for the input sentences.

As far as semantic analysis is concerned, most of the projects reviewed in this chapter rely on handcrafted rules to extract semantic information from parse trees of the input text, and the only project that uses semantic role labelling to perform this task is Carsim. As far as converting verb semantics to graphics commands is concerned, with the exception of KRISP, all the projects reviewed in this chapter use handcrafted rules. In contrast with these reviewed projects, the research reported in this thesis aims to come up with a generic framework that uses machine learning based systems to perform both the semantic analysis of the input text, and the conversion from verb semantics to graphics commands.

Chapter 3

Technology Foundations

The goal of this thesis is to investigate the theories and principles underpinning a machine learning framework for creating natural language based animation systems. The first part of this framework is to use NLP technologies to extract information about the actions to be visualised. In Chapter 1, I identified Part-of-Speech (POS) tagging, lemmatisation, semantic role labelling (SRL), and verb sense disambiguation (VSD) as the main NLP technologies behind this research. I introduce these technologies in this chapter. Since the SRL and VSD systems studied in this research depend on other lower level NLP technologies, namely lemmatisation, chunking, parsing, and named entity recognition (NER), these technologies will also be briefly introduced here. Furthermore, since lexical resources such as WordNet play an important role throughout my research, they will also be introduced in this chapter. Finally, I will also briefly introduce the Maximum Entropy (MaxEnt) and Support Vector Machine (SVM) machine learning algorithms which are used as the basis of various aspects of this research. The various NLP technologies presented in this chapter are assembled together into a pipeline architecture as shown in Figure 3.1.

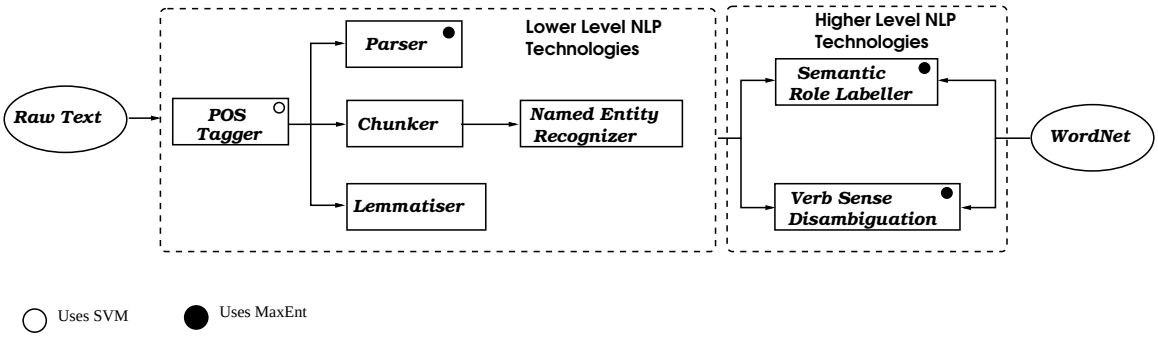


Figure 3.1: Data flow diagram for the NLP systems used in this research

This chapter is structured according to the dependencies of the technologies being presented. I will first introduce MaxEnt and SVM, as they do not depend on any NLP technologies. I will then give a general introduction to WordNet which is the most important lexical resource used in this research. Next, I will introduce the individual systems used within this research to perform POS tagging, lemmatisation, chunking, parsing, and NER. Finally, I will review the most relevant literature on SRL and VSD.

3.1 Machine Learning Algorithms

Every computational task or system involved in this research can be represented as a *supervised classification task*. The goal of such a task is to use training data to learn how to classify a particular type of data into predefined categories or classes. For example, a supervised POS tagger learns how to assign predefined POS tag to individual words.

Practically speaking, a supervised classification task can be defined as comprising the following three components: *class labels*, a *feature set* and a *model*. As stated before, the class labels represent the predefined categories for the data to be modelled by the classification task. The feature set represents all the types of evidence that can be used to perform the classification, with each feature representing a particular aspect of the data. For example, to classify the POS of a target word, a supervised POS tagger may include the following features in its feature set:

startsWithUpperCase	Whether the target word starts with an upper case letter.
isAlpha	Whether the target word consists of only alphabetical letters.
prefix	The prefix of the target word.
suffix	The suffix of the target word.

Given a specific instance of the data that the classifier is designed to model, each feature must be instantiated accordingly, and the entire set of instantiated features can be regarded as the *feature context* of the particular data instance. Continuing with the POS tagger example, given the target word *reproducing* and the above feature set, the feature context would be instantiated as the following:

startsWithUpperCase	$False$
isAlpha	$True$
prefix	re
suffix	ing

Finally, the model represents the mapping between the feature contexts and the class labels. Given a common dataset, different machine learning algorithms poten-

tially generate different models. During the training of supervised classifiers, the training data instances are each accompanied with a class label, and these pairs of feature contexts and class labels are used to train the model. Once the model has been obtained, it can then be used to classify test instances for which no class labels are available. Different machine learning algorithms have different strengths and weaknesses in their modelling capability, and therefore choosing appropriate machine learning algorithms is an important part of performing a supervised classification task.

Since the class labels and feature sets are independent of the machine learning algorithms, this section will only focus on the algorithms. Furthermore, since the main machine learning algorithms to be used are MaxEnt and SVM, they are the only ones to be reviewed. The following notations will be used in the remainder of this section:

Data instance d

Set of data instances D

Class labels $C = [c_1, c_2, ..., c_n]$ where C stands for the entire set of classification labels, and c_i stands for the ith label.

Feature set $F = [f_1, f_2, ..., f_m]$ where F stands for the entire set of features, and f_j stands for the jth feature.

Feature A feature f can be regarded as a function whose domain consists of all the possible data instances, and whose range can be either a set of predefined discrete values, or a real-valued scalar or vector. For example, the range for the feature startsWithUpperCase is the set of boolean values $\{True, False\}$, and the range for the feature suffix may be a set of predefined suffices such as $\{er, ing, nese\}$.

Feature Vector $\vec{x} = [V(f_1), V(f_2), ..., V(f_m)]$ where $V(f_n)$ is the value of the feature f_n in a particular training instance

Furthermore, given a data instance d_j and a feature function f_k, the value of f_k with respect to d_j is denoted as $f_k(d_j)$.

It should be noted that each data training instance d is a tuple of a feature vector and a class label, i.e. $d_i = (\vec{x}_i, y_i)$. Furthermore, D represents the set of all possible data instances, and $\tilde{D}$ will be used to represent the actual set of data instances available for training.

3.1.1 Maximum Entropy Modelling

Maximum entropy (MaxEnt) classifiers expect the ranges of all the feature functions to be real numbers, preferably between 0 and 1. This means that all the feature functions which range across discrete symbolic values must first be converted to binary feature functions. Take the feature function suffix, for example. Suppose that the only suffices that occurred in the training data were *ing*, *er*, and *nese*. The suffix feature function could then be converted into the following binary feature functions: isSuffix_er, isSuffix_ing and isSuffix_nese. Each of these binary functions would output 1 if the given target word had the specified suffix, and 0 otherwise. The presentation of MaxEnt in this section follows the convention of Berger *et al.* (1996).

MaxEnt classifiers perform classification by choosing the class labels which have the maximum probability with respect to the corresponding data instances. For example, suppose d_i represented the contextual information of a target word with respect to a MaxEnt POS tagger, and the set of POS tags were $[Verb, Noun, Adjective]$, the POS tagger would perform the tagging by calculating the probabilities of $p(Verb|d_i)$, $p(Noun|d_i)$, $p(Adjective|d_i)$ and choosing the POS tag with the greatest probability.

The training of MaxEnt classifiers estimates the probability distribution of the class labels conditioned upon the data instances, i.e. $p(C|D)$. MaxEnt classifiers choose the probability distributions with the highest entropy from the ones that best approximate the empirical observations of the training data (Berger *et al.* 1996). In other words, suppose P is the set of all the probability distributions for $p(C|D)$ that are most similar to the probability distribution of the training data, then the training of a MaxEnt classifier will select one distribution $p^\star(C|D)$ from P such that $p^\star(C|D)$ has the highest entropy among all the distributions in P.

In order to find $p^\star$, MaxEnt first defines a type of constraint $g(x)$ with respect to the co-occurrence of feature functions and class labels in the training data in the following form:

$$g_{(k,h)}(d_j, c_i) = \begin{cases} f_k(d_j) & \text{if } c_i = c_h \\ 0 & \text{otherwise} \end{cases} \tag{3.1}$$

Berger *et al.* (1996) show that $p^\star$ can be estimated by the following equation:

$$p^\star(c|d) = \frac{1}{Z(d)} exp\left[\sum_{i=1}^{|D|} \lambda_i \cdot g_i(d, c)\right] \tag{3.2}$$

The function Z is a normalisation function that ensures the probabilities sum to 1, and is defined in the following equation:

$$Z(d) = \sum_{c \in C} \exp\left(\sum_i \lambda_i g_i(d, c)\right) \tag{3.3}$$

Each λ_i is a weight on a corresponding constraint functions, determined by maximising the following function using methods such as Generalised Iterative Scaling (GIS) (Darroch and D. 1972), or the Limited-Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) algorithm (Liu and J. 1989):

$$\psi(\lambda) = -\sum_{(d,c)\in\tilde{D}} \tilde{p}(d)\log Z(d) + \sum_i \lambda_i \tilde{p}(g_i) \tag{3.4}$$

where $\tilde{D}$ represents the entire training set. $\tilde{p}(d)$ represents the probability distribution of the data instance d in $\tilde{D}$, and $\tilde{p}(g_i)$ is the empirical observation of the constraint function g_i, calculated according to the following equation:

$$\tilde{p}(g_i) = \sum_{(d,c)\in\tilde{D}} \tilde{p}(d,c)g_i(d,c) \tag{3.5}$$

Where $\tilde{p}(d,c)$ is calculated as: $\frac{\textit{(d,c)'s frequency in the training data}}{\textit{number of training instances}}$.

Furthermore, it is possible to smooth the resulting probability distribution with a Gaussian prior during the estimation process which has been shown to improve the empirical performance of the final statistical models (Chen and Rosenfeld 1999).

Given a new data instance d', the classification process of MaxEnt classifiers starts by calculating the probability of d' taking each of the class labels, and the class label which produces the highest probability is chosen. MaxEnt classifiers do not place any constraint on the number of class labels, and they do not make any assumption with respect to the relationships between the feature functions. Therefore, it is convenient to incorporate an arbitrary set of features in the construction of a MaxEnt classifier.

3.1.2 Support Vector Machines (SVMs)

Support Vector Machines (SVMs) are quite different from MaxEnt classifiers. To understand how SVMs work, it is necessary to first introduce the concept of *feature space*. One way to describe feature space is to regard each feature as a separate dimension, and therefore view the feature vector of a data instance as a point in the feature space. For example, a classification problem with only three features can be visualised as a 3D Cartesian system with each feature being a separate axis, and different data instances being different points in the 3D space. Since each data instance in the training data is accompanied by its label, the training of the classifier can be viewed as finding a *hyperplane* which separates the data points that belong to different classes with the largest margin. Figure 3.2 shows an example of the hyperplane. Since a hyperplane can only divide a space in a binary fashion, native SVMs can only perform binary classification, and the class labels are generally $[+,-]$ which denotes whether a data point lies below or above the hyperplane. However, it is still possible to perform multi-class classification with SVM classifiers, as will be

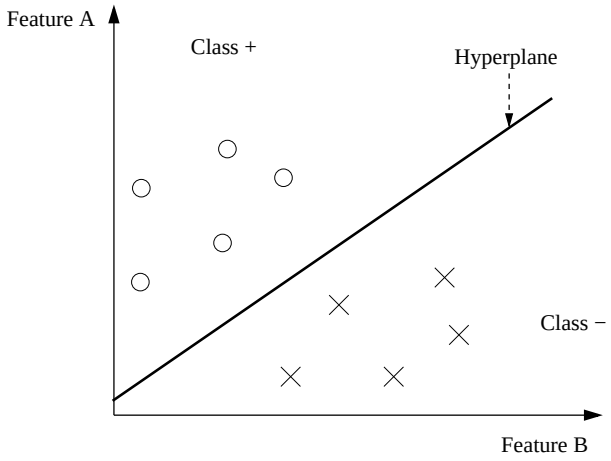


Figure 3.2: An example of feature space and hyperplane

explained later. The presentation of SVM in this section follows the conventions of Vapnik (1995) except that the class labels are denoted by c instead of y.

Formally, suppose the total number of feature functions for a given classification problem is n. The training data for SVM classifiers can be represented in the form of $[(\vec{x_1}, c_1), (\vec{x_2}, c_2), ..., (\vec{x_w}, c_w)] \in \mathcal{R}^n \times \{+, -\}$. The training of a SVM is to find the precise form of the equation for the hyperplane with the largest margin in separating the two classes of data instances in the feature space.

The general form of the equation for a hyperplane is given in the following equation:

$$\vec{w} \cdot \vec{x} + b = 0 \tag{3.6}$$

In this equation, $\vec{x}$ represents the feature context of the data instance, $\vec{w}$ is a vector that represents the weight of each feature, and b is the offset on the y axis. For a new data instance d_j, the result of Equation 3.7 determines on which side of the hyperplane it lies, and therefore what its class label ($+1$ or -1) should be.

$$c_{\vec{x}_{d_j}} = sign\ ((\vec{w} \cdot \vec{x_{d_j}}) + b) \tag{3.7}$$

The equation of the optimal hyperplane can be obtained by solving a constrained quadratic optimisation problem. The resulting w is of the form $\vec{w} = \sum_i v_i x_{d_i}$ where v_i is the scalar representing the weight of d_i, and d_i comes from a subset of the training instances which are the data points closest to the hyperplane that separate the data points from the different classes. Therefore, Equation 3.7 can be transformed into the following:

$$c_{\vec{x}_{d_j}} = sign\ (\sum_i v_i(\vec{x_{d_i}} \cdot \vec{x_{d_j}}) + b) \tag{3.8}$$

Furthermore, since the classification results only depend on the product of data instances as shown in Equation 3.8, the modeller of the SVM classifier can also map the data points into higher dimensions so that they can be separated more easily with respect to their classes. This type of mapping is performed by kernel functions. The general form of kernel functions is shown in the following equation, where ϕ is the mapping function:

$$k(x_{d_i}, c_i) = (\phi(x_{d_i}), \phi(c_i)) \tag{3.9}$$

Finally, even though the generic form of SVM classifiers can only classify data sets with binary class labels, it is possible to combine multiple SVM classifiers together to classify data sets with multiple class labels. One popular way to do this is create a series of one-vs-the-rest SVM classifiers. For example, suppose that an SVM POS tagger was to be trained on a dataset for which the valid POS tags were [*Verb*, *Noun*, *Adjective*, *Adverb*]. The instances of the training data can first be partitioned into two classes: one for words that are nouns, and one for words that are not nouns, then a classifier can be trained to distinguish nouns from non-nouns. The words that are not nouns can again be divided into two classes: one for words that are verbs, and one for words that are not verbs, based on which a second classifier can be trained to distinguish verbs from non-verbs. This process can continue until the remaining data instances only contain two types of class labels, then the normal training procedure can be applied.

3.2 WordNet

WordNet (Fellbaum 1998) is one of the most widely used lexical resources in the field of NLP. It is out of the scope of this thesis to give a comprehensive review of this lexical resource. Instead I will only review it from the perspective of how it is used in this research.

One of the most fundamental functionalities of WordNet is being a machine readable dictionary (MRD). Based on part of speech, the contents of WordNet is divided into four sections: nouns, verbs, adjectives and adverbs. The most basic unit of information in WordNet is a synonym set, or synset. Each synset corresponds to a set of word senses which are synonyms. For example, in WordNet version 2.0, the verbs *speak* and *talk* have the following senses:

$speak_1$	"express in speech"
$speak_2$	"exchange thoughts; talk with"
$speak_3$	"use language"
$speak_4$	"give a speech to"
$speak_5$	"make a characteristic or natural sound"
$talk_1$	"exchange thoughts; talk with"
$talk_2$	"express in speech"
$talk_3$	"use language"
$talk_4$	"reveal information"
$talk_5$	"divulge confidential information or secrets"
$talk_6$	"deliver a lecture or talk"

It is clear that $speak_1$ and $talk_2$ are synonyms, as are $speak_2$ and $talk_1$, and $speak_3$ and $talk_3$. As a result, $speak_1$ and $talk_2$ will belong to the same synset. Each synset in WordNet is often accompanied by one or more example sentences, for instance, one of the example sentences for $talk_2$ is *She talks a lot of nonsense*, however, sometimes not all the words in the same synset can be substituted into these example sentences. The word senses of WordNet have long been used in WSD tasks such as Senseval (Edmonds 2002; Mihalcea *et al.* 2004). The sense definitions of WordNet have stayed reasonably stable over the different versions of WordNet, and each pair of word form and sense number has an unique identifier which remains unchanged with respect to WordNet versions. This stability of sense definitions makes WordNet senses desirable for WSD systems.

WordNet also provides a number of highly useful semantic structures based on the synsets. The most widely used structures are the hypernym-hyponym and holonym-meronym relations for nouns, and the hypernym-troponym relationship for verbs.

The hypernym-hyponym structure captures the superclass-subclass relationships between objects in the real world. For example, the synset *big* cat_1 ("any of several large cats typically able to roar and living in the wild") is a direct hypernym (superclass) of $tiger_2$ ("large feline of forests in most of Asia having a tawny coat with black stripes"). Conversely, $tiger_2$ is a direct hyponym (subclass) of *big* cat_1. Since it is possible for a synset to have more than one hypernym, the hypernym-hyponym structure of WordNet noun synsets is a directed acyclic graph (DAG). The hypernym-hyponym structure is extremely useful for generalisation purposes. For example, semantic properties which are applicable to $tiger_2$ may be applicable to $lion_1$("large gregarious predatory feline of Africa and India having a tawny coat with a shaggy mane in the male") and $leopard_2$("large feline of African and Asian forests usually having a tawny coat with black spots").

The holonym-meronym structure captures the part-whole relationship between the objects in the real world. For example, the synset for gun_1 ("a weapon that

discharges a missile at high velocity") is the holonym of the synset for $trigger_1$("lever that activates the firing mechanism of a gun"), and conversely, $trigger_1$ is a meronym of gun_1. This structure of the WordNet synsets can be useful as a knowledge base for modelling the real world. For example, some complex 3D objects such as chairs are often created piece by piece just like their counterparts in the real world. Based on the holonym-meronym structure, it is possible to make virtual avatars interact with individual pieces of a virtual chair, instead of treating the chair as a single monolithic object.

The hypernym-troponym structure for verbs is similar to the hypernym-hyponym structure for nouns. For example, the synset for $propel_1$ ("cause to move forward with force") is a hypernym for $kick_1$"drive or propel with the foot". Conversely, $kick_1$ is a troponym of $propel_1$. This particular structure for verbs can be used to generalise observations about verbs. For example, in selecting graphics commands for verbs, if a verb has never been seen before, the selection process may still be able to use the verb's hypernyms to choose an appropriate graphics command to visualise it with.

The actual use of WordNet varies from task to task. Therefore, in the rest of this thesis, I will present discussions of WordNet as necessary.

3.3 Part of Speech Tagging

Parts of Speech (POS) is a word classification based the grammatical function of the target words. Most common parts of speech include noun, verb, adjective, adverb and preposition. The process of assigning POS categories to word tokens is called POS tagging.

POS tagging is a widely used NLP technology, and used most commonly as a pre-processing step. This means that the performance of POS tagging is extremely important, because whatever error it makes will propagate to downstream applications.

In general, the usefulness of POS taggers depends on two interrelated factors: the gold standard training corpus and the POS tag set.

The size and quality of the gold standard training corpus determines the quality of the POS tagger. In general, diverse corpora tend to produce models that generalise better. The domain of the training corpus also plays an important role in determining the generality of the POS tagger. This is due to the fact that in different genres, the same word tokens can often occur in different grammatical contexts and have different distributions in terms of their POS. Therefore, ideally, one should select a POS tagger whose training data is of the same genre of text as the text of the target application. However, if such a POS tagger cannot be found, then taggers trained on diverse corpora should be used because they tend to be less domain specific and generalise better than taggers trained on homogeneous data.

The POS tag sets are often determined by their training corpora. As already

stated previously, POS taggers are often used as preprocessing tools by other NLP technologies, and as a result, these technologies often assume a specific tag set and do not work with other tag sets. In general, POS taggers that use more popular tag sets are more useful.

Important Training Datasets

The Penn Treebank (Marcus *et al.* 1993) is the most widely used dataset for training and evaluating POS taggers. It contains nearly 4.9 million POS tagged word tokens, around 3 million of which come from the Wall Street Journal (WSJ) corpus. The Penn Treebank POS tagset consists of 48 tags including 12 tags for symbols and punctuations, and 36 tags for alphanumeric tokens.

The immense popularity of the Penn Treebank as a training corpus for POS taggers means that a significant number of downstream NLP tools presuppose input data that contains POS tags from the Penn Treebank POS tagset. As a result, POS taggers trained over the Penn Treebank have wide applicability.

Chosen POS Tagging System

The SVMTool POS Tagger (Giménez and Márquez 2004), version 1.2.2, was chosen as the POS tagger for this research. This tagger is trained on the training sections of the WSJ corpus of the Penn Treebank, and Giménez and Márquez (2004) report a 97.16% accuracy on the test sections of the WSJ corpus.

As its name suggests, the machine learning algorithm used by this POS tagger is SVM, the implementation of which is provided by the SVMLight package. SVMTool adopts a sliding window approach for assigning POS tags to individual words, the size of which is 5 words.

SVMTool offers two schemes for tagging new data. The greedy scheme chooses the POS tags of each word based on its surrounding context. The sentential scheme uses the Viterbi algorithm to choose the POS tag sequence that maximises the POS tag probability for the entire sentence.

SVMTool also offers several tagging strategies which trade robustness with speed. In terms of tagging direction, SVMTool can unidirectionally tag the sentences from left to right, or from right to left. SVMTool can also perform bidirectional tagging from the start and the end of the sentence simultaneously. The bidirectional tagging strategy is more robust than the unidirectional strategies, but is also slower. SVMTool has two strategies for extracting POS related features from novel data. The first strategy is to only make one pass over the sentence, i.e. a one pass tagging only using the POS tags which are unambiguous. The second strategy is to make an additional pass over the output of the already tagged sentence and use every single POS tag identified by the previous pass. The two-pass strategy is more robust than the one pass strategy, but is also slower.

The chosen tagging scheme for this research is the Sentential scheme, and the chosen tagging strategies are bidirectional tagging and two-pass tagging.

Problems with the SVMTool POS Tagger

As mentioned above, the SVMTool tagger was trained on the Penn Treebank, which consists of text from the Wall Street Journal (WSJ). The probability distributions of POS tags with respect to words in the WSJ is noticeably different from those in movie scripts. For example, words such as *jump*, *dive*, *slump* and *climb* occur mostly as nouns in WSJ, but as verbs in movie scripts. As a result, they are often mistagged by SVMTool and other POS taggers trained on the Penn Treebank. Misclassifications of this kind can cause the POS tagger's performance to drop significantly at the sentence level, which is highly undesirable.

The most reliable way to solve this problem is to retrain the POS tagger with the movie script data. However, this training requires a significant amount of gold standard data which is not available to this research. It is also possible to adapt POS taggers to other domains using machine learning methods, for example, Brill (1995) describes a method for improving POS tagger performance by repeatedly changing the model based on the errors it makes; and Daumé (2007) describes an SVM based method for domain adaptation for a number of NLP technologies including POS tagging, and shows that the adapted systems can have comparable performances to the systems trained on the target domain. However, as both Brill and Daume's methods require some gold standard data in the target domain, they were not applied in the research reported in this thesis due to time and resource limitation. Instead, I decided to manually correct the output of the semantic role labelling, because it is near the end of the semantic analysis pipeline and therefore has much more influence on the process of animation generation.

3.4 Lemmatisation

Lemmatisation is the process of finding the base lexeme of words from their inflected forms (Porter 1997), and is useful for improving the generalisation capabilities of NLP systems. For example, the verb *go* has four possible inflections: *went*, *goes*, *going*, *gone*, and without lemmatisation, they would be treated as different word tokens and modelled separately. With the help of lemmatisation, an NLP system can separate the canonical form of a word from its various inflections, and recognise a word even if it is an inflection that the system has never seen before.

It is relatively trivial to perform lemmatisation with a machine readable dictionary (MRD)that has a built-in stemmer (e.g. WordNet) and the POS tags of the words to be lemmatised. All that needs to be done is look up the inflected form of the target word in the dictionary based on its POS tag and find the appropriate canonical forms.

Since a POS tagger is already included as part of this research, and there are freely available MRDs (WordNet), this dictionary based method is chosen as the form of lemmatisation in this research. However, dictionary based lemmatisation may not be able to handle words that are not in the dictionary, and this has lead to more sophisticated lemmatisation systems, such as Morph (Minnen *et al.* 2001).

3.5 Chunking

Chunking is the process of grouping consecutive words together to form flat phrases. The phrases formed by a chunker are also called chunks. The chunk types used in this research are: noun phrase (NP), verb phrase (VP), adjective phrase (ADJP) and adverb phrase (ADVP). Each chunk can consist of one or more word tokens. Chunking captures local and phrase level syntactic information. Furthermore, since chunks are flat structures, computationally chunking is far less expensive ($O(n)$ vs. $O(n^3)$) and more accurate than parsing, which produces nested structures. Similar to POS tagging, chunking is also widely used by other NLP technologies as a preprocessing step, and can greatly affect the performance of downstream applications.

Since the set of chunk types is predefined, chunking can be considered as a classification task which assigns chunk tags to word tokens. A widely used representation for chunk tag classification is the Inside-Outside-Begin (IOB) format (Sang and Veenstra 1999). This format allows the classification of chunk tags to be performed on individual word tokens, which can be used to provide additional syntactic information about the words that they are assigned to. Figure 3.3 shows an example sentence tagged with IOB chunk tags, and (3.10) shows the chunks of same sentence in bracketing format.

Word Token	POS Tag	Chunk Tag	Chunk
He	PRP	B-NP	Noun Phrase
reckons	VBZ	B-VP	Verb Phrase
the	DT	B-NP	
current	JJ	I-NP	
account	NN	I-NP	Noun Phrase
deficit	NN	I-NP	
will	MD	B-VP	
narrow	VB	I-VP	Noun Phrase
to	TO	B-PP	Prepositional Phrase
only	RB	B-NP	
#	#	I-NP	
1.8	CD	I-NP	Noun Phrase
billion	CD	I-NP	
in	IN	B-PP	Prepositional Phrase
September	NNP	B-NP	Noun Phrase
.	.	O	*Outside*

Figure 3.3: Example of Chunk Tags in IOB Format

(3.10) $\underline{\text{He}}_{NP}$ $\underline{\text{reckons}}_{VP}$ $\underline{\text{the current account deficit}}_{NP}$ $\underline{\text{will narrow}}_{VP}$ $\underline{\text{to}}_{PP}$ $\underline{\text{only \# 1.8 billion}}_{NP}$ $\underline{\text{in}}_{PP}$ $\underline{\text{September}}_{NP}$.

There are three main chunk tag categories defined in the IOB format: the *Begin* tag, e.g. **B-NP**; the *Inside* tag, e.g. **I-NP** and the *Outside* tag: **O**. As the names of these tags suggest, the *Begin* tag is given to a word token that is the first token in its chunk; the *Inside* tag is given to a word token that belongs to a chunk but not the first word in its chunk, and the *Outside* tag is given to a word token that does not belong to any chunk at all.[1]

Relevant Datasets

One widely used dataset for training and evaluating chunking systems (chunkers) is the CoNLL 2000 Shared Task on chunking (Sang and Buchholz 2000). The sentences used in this dataset are taken from the WSJ section of the Penn Treebank. Each word token in this dataset is tokenised and accompanied with a gold standard POS tag. Table 3.1 shows the number of training sentences and tokens of this dataset.

Dataset	No. of Sentences	No. of word tokens
Training	8936	211727
Test	2012	47377

Table 3.1: Summarization of CoNLL 2000 Shared Task Dataset

[1]Most tokens with the *Outside* tags are punctuation marks.

Chosen Statistical Chunker

A statistical left-to-right chunker was used for this research.[2] This chunker was chosen because it had better performance on the CoNLL 2000 dataset than all the other freely available chunkers.

This chunker was implemented using a transformation based learner called fnTBL (Ngai and Florian 2001).[3] The training data for this chunker consists of the Brown corpus and the WSJ corpus portion of the Penn Treebank.

The first stage of this chunker's processing is to POS tag its input sentences. This POS tagging is performed with a tagger also implemented with the fnTBL package, and trained on the same training data as the chunker. Key features of this POS tagger include the following:

1. Tokenisation is based on the Penn Tokeniser.[4]
2. British-to-American spelling normalisation is performed using a lexical resourced called VarCon.[5]
3. Numbers are normalised with all cardinal numbers and all ordinal numbers respectively replaced with two symbols.
4. Case is normalised, for example, strings such as *MICROSOFT* and *MicroSoft* will be replaced as *microsoft*.
5. Hyphenation is normalised, for example, strings such as *pick-me-up* and *pick me up* will be treated as a single identical token.

The transformation templates used by the POS tagger were the default templates supplied by fnTBL. This POS tagger achieved a token level training accuracy of 97.24% on a hand tagged portion of the British National Corpus (Clear 1993).

Once the input sentences have been POS tagged, Baldwin's chunker first lemmatises the words using Morph (Minnen *et al.* 2001) (this is an example of how lemmatisation can be useful as discussed in Section 3.4), then uses the default fnTBL templates for chunking to train the statistical model of the chunker. This chunker achieves an F-score of 94.66% over the test dataset of CoNLL 2002. Compared with the published results, this performance is better than the best systems that participated in CoNLL 2002.[6] This is why it has been chosen for this research.

[2]This chunker was developed by Timothy Baldwin, but its details have not been published. The presentation of this chunker comes from the personal communication with Timothy Baldwin.

[3]`http://nlp.cs.jhu.edu/~rflorian/fntbl/`

[4]This tokeniser can be downloaded from `http://www.cis.upenn.edu/~treebank/tokenisation.html`

[5]VarCon can be accessed from `http://search.cpan.org/~adamk/Lingua-EN-VarCon-1.00/lib/Lingua/EN/VarCon.pm`

[6]Baldwin's chunker was trained on more data than those that participated in CoNLL 2002, and the additional training data is perhaps the reason for its better performance.

3.6 Statistical Parsing

Parsing is the process of deriving the recursive syntactic constituency structure from sentences according to a particular grammar. Unlike chunking, the constituents identified by a parsing system (parser) are not flat, but always in a parse tree. Statistical parsing is the application of statistical methods to learn the parsing process from existing gold standard parse tree corpus. The most widely used gold standard parse tree corpus is the Penn Treebank.

Parse trees are more expensive to produce than chunks, but they are also far more informative. As stated before, chunks are flat structures that identify the phrases of the sentence and their head words. However, they do not provide information on how the phrases attach to each other. On the other hand, parse trees are nested structures, they can specify not only which word tokens are grouped together in phrases, but also the attachment structure between the phrases and the clauses of the sentence. For example, consider the sentences in (3.11):

(3.11) a The man ate a pizza with chopsticks.

b The man ate a pizza with sausage toppings.

A chunker would assign the same sequence of chunks to both sentences, but a parser would be able to distinguish that the phrase *with chopsticks* is attached to *ate*, and the phrase *with sausage toppings* is attached to *a pizza*.

Chosen Statistical Parser

The parser used in this research is the Charniak MaxEnt parser with reranking (Charniak and Johnson 2005). This parser is based on the original Charniak MaxEnt parser (Charniak 2000). I will start the description of the reranking parser by reviewing the original parser, then I will describe the improvements of the reranking parser over the original parser. The presentation of the parser follows the convention used by Charniak (2000).

Charniak's parser was trained on the training sections of the WSJ component of the Penn Treebank, and evaluated on the test sections of the same corpus. This parser calculates the probability of a parse π according to the following equation:

$$p(\pi) = \prod_{c \in \pi} \begin{array}{l} p(t(c)|l(c), H(c)) \\ \times p(h(c)|t(c), l(c), H(c)) \\ \times p(e(c)|l(c), t(c), h(c), H(c)) \end{array} \tag{3.12}$$

where c represents a constituent in the parse π; $t(c)$ represents the pre-terminal of c;[7] $l(c)$ represents the actual label of c, e.g. NP or VP; $h(c)$ represents the

[7]There is no guarantee that there is at least one training instance in the data such that $t(c)$ is a pre-terminal of c. If no such training instances exist in the training data, then $p(h(c)|t(c), l(c), H(c))$

lexical head of c; $e(c)$ represents the expansion of c, i.e. the direct children of c and $H(c)$ represents the collection of other parsing features or histories about c. The probability distribution for $e(c)$ is modelled as a Markov grammar (Charniak 1997; Collins 1997; Magerman 1995). More specifically, Charniak (2000) models the expansion of a constituent c in the following form:

$$l \rightarrow \triangle L_m ... L_1 M R_1 ... R_n \triangle \tag{3.13}$$

Here, $\triangle$ represents the boundary of the expansion, l represents the label to be expanded, M represents the child of c from which $h(c)$ is extracted, and L_m and R_n represent the mth left and nth right siblings of M respectively. Therefore, the probability of the expansion can be represented as the product of the probabilities of the other parsing features ($H(c)$) and the siblings (Ls and Rs) of the constituent. Charniak (2000) used a second order Markov grammar for modelling the left and right siblings of the target constituent. Together with $l(c)$, $t(c)$, $h(c)$, and $H(c)$, the probability of ith left sibling is estimated as $p(L_i(c)|L_{i-1}(c), L_{i-2}(c), l(c), t(c), h(c), H(c))$, and the probability of the jth right sibling is estimated as $p(R_j(c)|R_{j-1}(c), R_{j-2}(c), l(c), t(c), h(c), H(c))$.

The probabilities of $t(c)$, $h(c)$, M, L and R are separately estimated and smoothed using a MaxEnt model over a new sentence. The Charniak parser first uses a chart parser to generate all possible parses, then it ranks each parse using the above probability distributions, and the parse with the highest probability is selected as the final parse.

The Charniak reranking parser is an improvement upon the original Charniak parser. The reranking parser works by first selecting the top 50 parses produced by the original parser, then using a set of predefined feature functions to select the best parse out of the 50 original parses.

Formally, let $Y(s) = [y_1(s), y_2(s), ..., y_m(s)]$ be the top m parses of the sentence s produced by the original parser; and suppose $F = [f_1, f_2, ..., f_n]$ is the feature set, and each feature function returns a real number as output; the reranking parser defines a score function in the following form:

$$v(y) = \sum_{i=1}^{n} \theta_i f_i(y) \tag{3.14}$$

where the θs are the weights for the respective feature functions, as calculated with a MaxEnt estimator. The feature functions are designed to capture specific aspect of the parses such as the depth of conjunct parallelism, the n-gram representation of the siblings of each constituent, and the dependencies between the heads of the constituents and the heads of their child nodes. The training and test data for the

and $p(e(c)|l(c), t(c), h(c), H(c))$ may have the value of 0, or a very small number if the corresponding probability distributions are smoothed.

reranking parser is the same as the original parser. However, since the best parse is in fact often not included in the top 50 parses, the Parseval F-score is used to select the best parse tree from the top 50 parses.

The reranking parser achieves a F-score of 91.0% on test sentences of length 100 words or less.

3.7 Named Entity Recognition

Named entity recognition (NER) is the NLP task of recognising proper nouns and time related noun phrases in written text (Sang and Meulder 2003). For example, given the sentence *George Washington died in 1799 in Mount Vernon.*, a named entity recogniser should be able to identify the phrase *George Washington* as the name of a person, the phrase *1799* as a year, and the phrase *Mount Vernon* as a location.

One of the most commonly used datasets for training and evaluating NER systems is the shared task of CoNLL-2003 (Sang and Meulder 2003). This dataset consists of 14,987 training sentences taken from news articles. Four types of named entities are present in this dataset, namely: organisation, location, person and miscellaneous.

The NER system chosen for this research is the one described in Cohn *et al.* (2005). This system is trained on the CoNLL-2003 dataset, which means that it can only recognise the four types of named entities mentioned above. The statistical model used by Cohn *et al.* (2005)'s NER system is a standardly formulated multiclass conditional random field (Lafferty *et al.* 2001). The features used in constructing the classifier include the target word's identity, the target word's prefix and suffix, the existence of digits in the target word, the case of the target word, the internal punctuation of the target word, the length of the target word, and unigram and bigram POS tags surrounding the target word.

Cohn *et al.* (2005)'s system achieves a F-score of 89.78% on the CoNLL-2003 test set.

3.8 Semantic Role Labelling

Semantic roles describe the relationships between verbs and their arguments. For example, given the sentence *The man shot the deer with a pistol on Sunday in the woods*, one can identify that with respect to the verb *shoot*, the phrase *The man* represents the SHOOTER, the phrase *the deer* represents the VICTIM, the phrase *with a pistol* represents the INSTRUMENT used by the SHOOTER, the phrase *on Sunday* represents the TIME of the shooting, and the phrase *in the woods* represents the LOCATION of the shooting. The process of automatically assigning semantic roles to word sequences is called semantic role labelling (SRL).

Intuitively, some semantic roles can be viewed as verb-specific. For example, the verb *shoot* may require a SHOOTER and a VICTIM, whereas the verb *eat* may involve

an EATER and an EATEE. However, these semantic roles can often be generalised based on their common syntactic and semantic properties. For example, both the SHOOTER and the EATER depict the agent or initiator of the action, and they also tend to be the subject of their respective verbs. This generalisation makes it easier to construct automatic semantic role labellers, since the number of semantic roles is small and independent of the verbs.

The most widely used datasets for training and evaluating SRL systems come from the shared tasks of CoNLL 2004 and 2005 (Carreras and Màrquez 2004). These datasets use the semantic roles defined by Propbank (Kingsbury and Palmer 2002). Propbank semantic roles can be divided into two categories: 5 types of argument roles (ARG0 to ARG4) and 11 types of modifier roles. The most frequent argument roles are ARG0 and ARG1 which can generally be matched to the prototypical AGENT and PATIENT roles as defined by Dowty (1991). The other argument roles tend to be less frequent and their interpretation tends to differ from verb to verb. The training sentences in both CoNLL 2004 and 2005 come from sections 02 to 21 of the WSJ sections of the Penn Treebank. The semantic roles in these two datasets are completely flat and do not overlap.

The chosen SRL system for this research is ASSERT[8] (Hacioglu *et al.* 2004). ASSERT is a statistical semantic role labeller. Its underlying statistical model is a series of one-vs-the-rest SVM with degree 2 polynomial kernel. ASSERT makes the assumption that the smallest unit of text for a single semantic role is a chunk, i.e. all the word tokens in the same chunk have the same semantic role. This is reasonable because ASSERT was trained on the CoNLL datasets where semantic roles do not overlap. Using this assumption, ASSERT only performs SRL on the last word of each chunk, and applies the same classification to all other words in the same chunk. This is called phrase-by-phrase (P-by-P) SRL, and it is much more efficient than the naive word-by-word (W-by-W) SRL.

Since SRL is a significant component of the research reported in this thesis, it is worthwhile to discuss the features used by ASSERT. The features can be divided into three categories: sentence level features, phrase level features and verb-specific features. The sentence level features include the bag of words for the entire sentence, the lemmatised verbs in the entire sentence, the bag of POS tags, the bag of named entities,[9] the IOB format chunk tag for the target word, and the clause tag for the target word.

The token level features include the relative position of the target phrase with respect to the verb, the chain of chunk types from the verb to the target phrase, the chain of clause brackets from the target phrase to the verb, the chain of clause brackets from the target phrase to the beginning or the end of the sentence, the suffixes of the head word of the target phrase,[10] the number of chunks between the

[8] Version 1.4 beta

[9] The same entities as CoNLL 2003

[10] The suffices can be 2, 3 or 4 characters long.

target chunk and the target verb, the number of VP chunks between the target chunk and the target verb, and the number of word tokens in the target chunk.

The verb-specific features include the POS tag of the target verb, the frequency of the target verb (verbs that have fewer than 3 occurrences in the CoNLL training set are ignored), the chain of 2 chunks before and after the target verb's chunk, the POS tags of the word tokens immediately surrounding the target verb, and the number of verbs in the same sentence. Furthermore, ASSERT attempts to match a target chunk to one of the 3 most frequent verb-argument syntactic frames defined by Propbank, and uses this as an additional feature.

Over the test set of CoNLL 2004, ASSERT achieved an F-score of 69.49%, which was the best result among all the SRL systems that participated. In absolute terms, this performance is not high, which is why it is necessary to investigate whether any additional mechanisms can be used to improve the performance of SRL systems. One observation that can be made with respect to ASSERT and other general purpose SRL systems is that they do not pay special attention to prepositional phrases which often act as modifier semantic roles. Therefore, in Chapter 5, I will investigate whether performing prepositional phrase based SRL can indeed improve the performance of general purpose SRL systems.

3.9 Verb Sense Disambiguation

Verb sense disambiguation (VSD) is the task of examining verbs in a given context and predicting exactly which sense of each verb is the most appropriate in that context. VSD is a part of the broader task of word sense disambiguation (WSD). Given a specific word sense inventory and a specific set of verbs, WSD is essentially a classification task (Yarowsky 2000).

The usefulness of a WSD system partially depends on which sense inventory it uses. However, there is no commonly accepted sense inventory for WSD. Therefore, this section will begin with an overview of the several popular sense inventories, and then make a case for selecting one of them as the sense inventory for this research.

The second half of this section will be dedicated to reviewing relevant WSD research. I will start with a high level overview of the techniques commonly used for WSD, and show how they may not be suitable for the task of VSD. I will then introduce the concept of selectional preferences, and show how they could be more useful than the commonly used WSD techniques for disambiguating verbs. Finally, I will review several selectional preference based WSD systems and discuss how they may be improved to provide better VSD performance.

3.9.1 Verb Sense Inventory Selection

There does not exist a set of universally accepted general purpose word senses (Wilks *et al.* 1996), and neither does there exist a set of verb senses specifically designed for visualising verbs in a virtual scene. Furthermore, given the vast number of verbs, it is infeasible to manually create a new set of verb senses for visualisation purposes. Therefore, an appropriate word sense repository must be found. The most commonly used sources of word senses are lexical resources such as machine readable dictionaries (MRDs). In this section, I will review some of the major MRDs and choose one of them as the word sense source for the verb sense disambiguation task. The purpose of this review is not to provide a detailed comparison of these lexical resources so much as a detailed comparison of their usability for word sense disambiguation and other aspects of this research. The following criteria will be used to compare the lexical resources:

1. Coverage
   (a) Number of head words
   (b) Number of word senses
2. Computational Convenience
   (a) Whether the basic unit of information of the respective lexical resource is a word or word sense.
   (b) How the basic units of information can be retrieved.
   (c) How the basic units of information are stored, and whether the storage structure contains any additional semantic information useful for WSD purposes.
   (d) Whether there is any additional abstraction over the basic units of information, and if so, how such abstraction can be used for WSD purposes.
3. Popularity
   (a) Whether there are other semantic resources which use the sense definitions of the respective lexical resources.
   (b) Whether there are other WSD or semantic analysis systems that make use of the sense definitions of the respective lexical resources.
4. Availability
   (a) Whether the respective lexical resources are free and offer programming interfaces or have an easily accessible format.

Using these criteria, the following lexical resources resources are reviewed here:

1. Longman Dictionary of Contemporary English (LDOCE) (Bullon 2006)
2. Collins Birmingham University International Language Database (COBUILD)
3. WordNet (Miller 1995)
4. FrameNet (Collin Baker 1998)
5. VerbNet (Kipper *et al.* 2000)

Longman Dictionary of Contemporary English

The LDOCE is not freely available as an offline MRD, however there is a web interface[11] through which it can be accessed for free. The LDOCE is a full-sized dictionary designed for learners of English as a second language (Wilks *et al.* 1996). LDOCE contains 41100 entries covering the full spectrum of part-of-speech categories of English. An entry in LDOCE is a collection of **senses** headed by a single word. For example, if the word *bring* is entered into the search interface of LDOCE, the MRD will bring up not only all the senses for *bring*, but also all the senses of verb-particles and phrases of which *bring* is the head verb. Such phrases include *bring a smile to someone's face*, *bring back* and *bring someone or something down* etc. On the other hand, it is not possible to input the verb particles or phrases themselves and retrieve just their separate senses.

Each sense in LDOCE contains a definition, and some senses also contain additional information such as example sentences, synonyms at the sense level, syntactic patterns, common collocations, pictures and pronunciation. The example sentences are sense specific and always show a prototypical use of the relevant senses. There are three levels of sense granularity in LDOCE. The top level distinguishes the homographs of a given word token. For example, the verb *bow* (/*bau*/, "He muttered a curse when he was bowing to the queen."), and the verb *bow* (/*bo*/, "The cellist's bowing was very sensitive.") are homographs and they are given two separate entries in LDOCE. The other two levels of granularity exist within the same entry, as illustrated in Example 1 which shows the top three LDOCE senses for *bring*.

As a dictionary for English learners, the LDOCE senses are defined based on the linguistic intuitions of the lexicographers. In addition, the entries of some words provide additional information to help language learners distinguish words that have similar meanings. Take the entry for the verb *bring* as an example. In addition to the sense related information, LDOCE provides an extra section of information that explains the differences between *bring*, *fetch*, *get* and *take*.

According to Wilks *et al.* (1996), LDOCE can function as a minimal knowledge base because it establishes 124 categories for world knowledge. Examples of such

[11] `http://pewebdic2.cw.idm.fr/`

bring

1 a) to take something or someone with you to the place where you are now, or the place you are talking about

- *"Did you bring an umbrella?"*
- *"It was the first time Joe had ever brought a girl home."*
- *"They brought news of further fighting along the border."*

bring something/somebody to something/somebody

- *"Is it OK if I bring some friends to the party?"*

bring somebody/something with you

- *"For some reason, Jesse had brought a tape recorder with him."*

b) to get something for someone and take it to them

bring somebody something

- *"Can you bring me another beer?"*
- *"Robert asked the waiter to bring him the check."*
- *"While she was in prison, friends used to bring her books."*

bring something/somebody to something/somebody

- *"He expects me to bring everything to him."*

2 a) to make a particular situation exist or cause a particular feeling:

- *"efforts to bring peace to the region"*
- *"The strikes are expected to bring chaos."*
- *"The senator's speech brought an angry response from Civil Rights groups."*

Example 1: LDOCE example for *bring* (top 3 senses only)

categories include: aeronautics, glass, golf, economics, science and zoology. However it is not clear how the category information can be accessed through LDOCE's online interface.

There does not seem to be any link between the entries of LDOCE, other than the ones already mentioned. In terms of computability, LDOCE is more of a dictionary intended for language learners than a computational resource. As a result, LDOCE is not the most frequently used MRD for word sense disambiguation. However, there are a few influential WSD systems that use LDOCE as the sense inventory, one of these systems is reported by Stevenson (2003).

Collins Birmingham University International Language Database

COBUILD is a lexical database for English similar to LDOCE, but it is not freely available.[12] The structure and indexing of COBUILD are similar to those of LDOCE. However, there is a significant difference between COBUILD and LDOCE: the word senses of COBUILD are grounded in real life texts. In other words, the word senses of COBUILD are defined to match how the relevant words are used in a corpus of more than 20 million words (the Bank of English). The indexing method of COBUILD is different to that of LDOCE. The user can query the dictionary using word tokens which include single head words, verb-particle constructions and compound nouns. Given a word w, COBUILD will return all the entries whose head word is w. The returned entries are separated based on their semantics. Each entry is accompanied by a single gloss which summarizes the semantics of the particular entry, followed by one or more specific senses. Each specific sense can be further divided into sub-senses based on part-of-speech categories or verb-particle constructions. Each sense or sub-sense may be accompanied by a more detailed gloss and one or more example sentences from the COBUILD corpus. Example 2 shows a partial query result for the word *bow*.

On top of the sense definitions and example sentences, the COBUILD senses also provide sense-specific syntactic information. COBUILD also provides information on words which are related to the queried word token. For example, the explanations section of query results for *bow* consists of words such as *archer*, *arrow*, *black tie*, *cello* and *violin*. COBUILD also provides synonym information for the queried word, although, it is not clear how the synonyms match to the specific senses of the queried word.

A major drawback of COBUILD is that its lexicon is stored in a binary format which does not seem to have an openly available specification or programming interface. As a result, access to COBUILD's lexicon can only be performed through

[12] I had access to version 3.1 of COBUILD, and all the examples of COBUILD in this section come from this particular version.

bow[1] bending or submitting

1 bow; bows; bowing; bowed

Definition: When you **bow** to someone, you briefly bend your body towards them as a formal way of greeting them or showing respect.

Example: *"They bowed low to Louis and hastened out of his way."*, *"He bowed slightly before taking her bag."*

VB

\+ bow; bows

Definition: Also a noun..

Example: *"I gave a theatrical bow and waved."*

N-COUNT

bow[3] objects

1. bow; bows

Definition: A **bow** is a know with two loops and two loose ends that is used in tying shoelaces and ribbons

Example: *"Add a length of ribbon tied in a bow."*

N-COUNT

2. bow; bows

Definition: A **bow** is a weapon for shooting arrows which consists of a long piece of curved wood with a string attached to both its ends.

Example: *"Some of the raiders were armed with bows and arrows."*

N-COUNT

3. bow; bows

Definition: The **bow** of a violin or other stringed instrument is a long thin piece of wood with fibers stretched along it, which you move across the strings of the instrument in order to play it.

N-COUNT

Example 2: COBUILD Example (*bow*)

its interface. This makes it impractical to utilise COBUILD in an automatic/semi-automatic fashion suitable for computational linguistic systems.

WordNet

As stated in Section 3.2, WordNet is a concept network for English. It is freely available for offline use. It is not a full English dictionary because it only contains information for the four open class part-of-speech categories of words: nouns, verbs, adjectives and adverbs. Each class includes not only single word tokens but also compound words such as verb-particle constructions, compound nouns and fixed verb phrases. The basic unit of information in WordNet is a synonym set (synset). A synset contains a list of word senses which WordNet considers to be synonyms. Figure 3.4 shows the verb synsets for *bow*. Here it can be observed that not all the senses in the same synset are given example sentences, and the example sentences are not always suitable for all senses in a given synset. For example, the WordNet synset for $bring_4$, get_6, $convey_7$ and $fetch_1$ ("go or come after and bring or take back") has the following example sentences:

1. Get me those books over there, please
2. Could you bring the wine?
3. The dog fetched the hat.

It is obvious that the verb *convey* would not be suitable in any of these sentences, suggesting that sense that belong in the same synset are not substitutable in all contexts.

It can also be observed that some synsets are semantically very similar; for example, the first and the third sense of *bow* are so similar that the same example sentence *"He bowed before the King"* is used to exemplify both senses. This kind of subtle difference that occurs between groups of WordNet senses may make them impossible to automatically distinguish. However, depending on the nature of the final semantic task, it may not always be necessary to distinguish between such finely defined senses. For example, the first WordNet sense of the verb *bow* means "bend one's knee or body, or lower one's head", and the third sense means "bend the head or the upper part of the body in a gesture of respect or greeting". The third sense is different from the first one because it implies respect, however this difference may not be visually significant enough to make a difference in the domain of this research.

In terms of indexing methods, users of WordNet can perform basic retrieval of WordNet information through WordNet's programming interface. Furthermore, since both the synset data and word index are freely available, it is also possible to customize WordNet for other forms of indexing.

- (*bow*[1], *bow_down*[2]): This synset states that the first sense of the verb *bow* and the second sense of the verb-particle construction *bow_down* are synonymous.

  Definition: bend one's knee or body, or lower one's head

  Examples: *"He bowed before the King", "She bowed her head in shame"*

- (*submit*[6], *bow*[2], *defer*[2], *accede*[1], *give_in*[1])

  Definition: submit or yield to another's wish or opinion

  Examples: *"The government bowed to the military pressure"*

- (*bow*[3])

  Definition: bend the head or the upper part of the body in a gesture of respect or greeting

  Examples: *"He bowed before the King"*

- (*crouch*[1], *stoop*[1], *bend*[4], *bow*[4])

  Definition: bend one's back forward from the waist on down

  Examples: *"He crouched down", "She bowed before the Queen", "The young man stooped to pick up the girl's purse"*

- (*bow*[5])

  Definition: play on a string instrument

  Examples: (None)

Figure 3.4: WordNet Example (*bow*)

Part of Speech	Number Head Words	Number of Synsets
Noun	114648	79689
Verb	11306	13508
Adjective	21436	18563
Adverb	4660	3664

Table 3.2: Coverage of WordNet 2.0

WordNet also contains entries for multiword expressions (MWE) (Sag *et al.* 2002), and these MWEs are given separate entries from their head words, which means that users of WordNet need to specify the full MWE to look them up. This is rather different from LDOCE which allows the user to retrieve MWEs with just the head word of the expression.

WordNet's coverage is reasonably broad. Table 3.2 shows the number of words and synsets in WordNet 2.0. It may seem counter-intuitive that the number of WordNet head words can outnumber the WordNet synsets, but this is just a property of the WordNet synsets which often have a one-to-many mapping with head words.

In addition to the words and senses, WordNet also provides a range of relationships between its synsets, such as hypernym-hyponym (for nouns) and hypernym-troponym (for verbs) which denote the general-specific relationship, and meronym-holonym (for nouns) which denotes the part-whole relationship. The hypernym-hyponym relationship for nouns and the hypernym-troponym relationship for verbs denote the general-specific relationships. For nouns, if synset X (e.g. $chair_1$–"a seat for one person, with a support for the back") is the hypernym of synset Y (e.g. $armchair_1$–"chair with a support on each side for arms"), then Y must also be a hyponym of X, and stand for a particular type of X. For verbs, if synset X (e.g. eat_1–"take in solid food") is the hypernym of synset Y (e.g. $slurp_1$–"eat noisily"), then Y must also be a troponym of X, and stand for a particular way of Xing.

Figure 3.5 shows the hypernyms and troponyms of the synsets containing the first sense of the verb *bow*, and Figure 3.6 shows the holonyms and meronyms of the synsets containing the fourth sense of the noun *bow*: "curved piece of resilient wood with taut cord to propel arrows".

According to Miller (1995), the senses of WordNet are inspired by psycholinguistic theories of human lexical knowledge. Therefore it is clear that WordNet was not originally created for natural language processing purposes, although it has been used in several other semantic resources such as Propbank and the Senseval datasets. Furthermore, there are various unsupervised WSD systems which use WordNet as their sense inventories, e.g SenseLearner (Mihalcea and Csomai 2005). This would be useful because VSD is also a task targeted in this research, and having the same sense

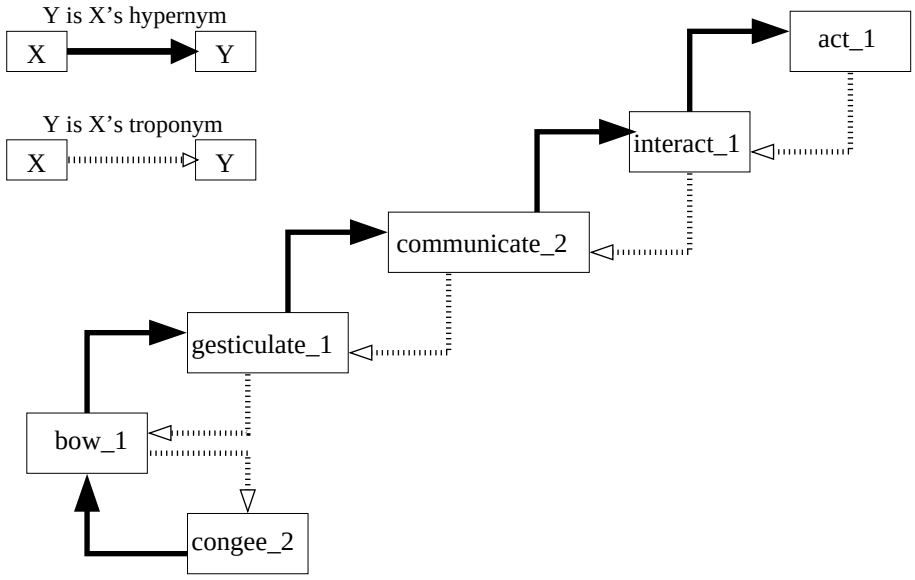


Figure 3.5: Hypernym-Hyponym relationships for the verb synset *bow*[1]

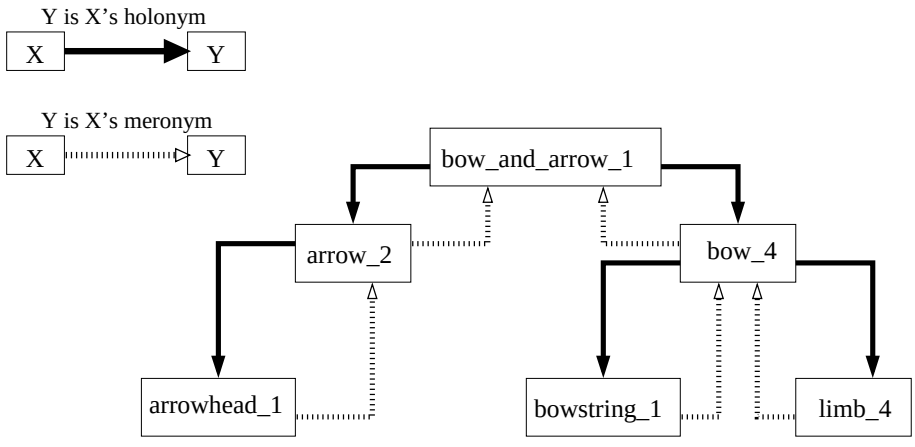


Figure 3.6: Holonym-Meronym relationships for the noun synset *bow*[4]

inventory makes it easier to perform comparisons between the various WSD systems, and even integrate them into this research.

FrameNet

FrameNet[13] is a freely available corpus-oriented computational lexicon based on frame semantics (Johnson and Fillmore 2000). In general, a frame is a schematic representation of real world information with respect to a certain scenario, situation or event. This representation not only defines the roles of each participant in the relevant scenario, situation or event, but also specifies the relationships between them. The role of each participant is called a frame element. For example, the verb *eat* may belong to an INGESTION frame which defines an INGESTOR frame element to describe the participant performing the ingestion, and an INGESTIBLE frame element to describe the participant that is ingested. Therefore, in *John is eating an apple*, *John* may be marked as the INGESTOR and *an apple* may be marked as the INGESTIBLE. Furthermore, it is also possible to define additional relationships between different frames such as the inheritance relationship.

In accordance with the concepts of frame semantics, FrameNet consists of three major components: Lexical Unit List, Frame List and Frame Relations List, all of which are stored in formats readily useable for computational purposes.

[13]This description was written based on FrameNet version 1.2.

```
<lexunit-annotation ID="1461" name="scrape.v" frame="Make_noise"
                    pos="V" incorporatedFE="Sound">
 <definition>COD: rub or cause to rub against a rough or hard surface.
 </definition>
 <subcorpus name="NoPattern">
  <annotationSet ID="1709723" status="MANUAL">
   <layers>
    <layer ID="9032959" name="FE"><labels>
      <label name="Sound_source" ID="28040219" start="0" end="10" />
      <label name="Sound" ID="31885425" start="12" end="18" />
     </labels></layer>
    <layer ID="9032960" name="GF"><labels>
      <label name="Ext" ID="28040221" start="0" end="10" />
     </labels></layer>
    <layer ID="9032961" name="PT"><labels>
      <label name="NP" ID="28040220" start="0" end="10" />
     </labels></layer>
    <layer ID="9032962" name="Sent" />
    <layer ID="9032963" name="Other" />
    <layer ID="9032964" name="Target"><labels>
      <label name="Target" ID="28040223" start="12" end="18" />
     </labels></layer>
   </layers>
   <sentence ID="1129308" aPos="62661526">
    <text>The speaker scraped and clicked a few more times ,
          but after half a minute of no reply the phone was
          rehooked
    </text></sentence></annotationSet>
 </subcorpus>
</lexunit-annotation>
```

Example 3: FrameNet Lexical Unit Example

A lexical unit (LU) is a word paired with a meaning, in other words, a LU is a specific sense of a particular word. Example 3 shows a partial LU entry for the verb *scrape*. As this example demonstrates, a LU entry in FrameNet may contain the following information:

1. The name of the LU entry. This name can be a single word, or a multiword expression. In Example 3, it is **scrape**.

2. The POS Tag of the name. In Example 3, it is a verb.

Part Of Speech	No. of LU entries (word senses)	No. of unique LU names (head words)	Average No. of Word Senses
Noun	3798	3420	1.08
Verb	3483	2513	1.39
Adjective	1548	1409	1.10
Preposition	72	63	1.14
Adverb	47	47	1.00
Number	10	10	1.0
Interjection	1	1	1.0

Table 3.3: Breakdown of FrameNet Lexical Units

3. The Frame to which it belongs. In Example 3, it is the MAKE_NOISE frame.

4. The Frame Element (FE) that it incorporates. In Example 3, it is the SOUND frame.

5. A short definition of the LU entry.

6. A set of exemplary annotations extracted from different corpora. Each annotation contains an example sentence and a list of different annotation layers with respect to this example sentence. Each annotation layer analyses the example sentence from a different perspective, such as Frame Element (FE), Grammatical Function (GF) or Phrase Type (PT). Each layer may contain specific annotation labels which provide more detailed information. In Example 3, there is only one annotation set, and it contains seven annotation layers. The FE layer specifies that the span of text between the zeroth character and the tenth character in the example sentence is the frame element of SOUND_SOURCE, and the text between the 12th and the 18th characters is the frame element of SOUND. The GF layer specifies that the same span of text for SOUND_SOURCE also serves as the the External Argument (Ext) of the target verb.[14]

Within FrameNet, there are 8869 separate LU entries which correspond to 7463 unique LU names. This indicates that some LU names are polysemous. It is possible to distinguish between the different senses of the same LU names because FrameNet provides a unique identifier for each separate LU entry. Table 3.3 shows the break down of the LU entries by their part-of-speech. The LU entries are the basic units of information in FrameNet, which are stored in a flat structure.

[14]The character-based demarkation of the labels is designed to allow labelling on the sub-token level.

The Frame List of FrameNet consists of 609 semantic frames. Example 4 shows the relevant parts of an abbreviated version of the semantic frame fragment to which the LU in Figure 3 belongs.

It can be observed from Example 4 that a typical semantic frame may hold the following information:

- The name of the frame. In the example, it's MAKE_NOISE.
- A plain text definition of the frame, which is then followed by zero or more example sentences which are Frame-Element-tagged.
- A list of frame elements which can be applied to all the lexical units covered by the semantic frame. In the example, only two of the many FEs are shown. It can also be observed that each FE comes with its own definition and tagged example sentences.
- A list of lexical units to which the semantic frame is applicable. This list only contains pointers to the actual lexical unit lists described above. It can be observed that the ID for **scrape.v** is the same as the LU example in 3.

The frames themselves are also stored in a flat structure, however, they can be viewed as an additional layer of abstraction over the LUs, and they can be used to distinguish the different senses that belong to the same LU names.

The frame relations list defines the relations between the frames. There are ten types of relations in total. Table 3.4 shows these relation types and the number of frame pairs to which they apply. A frame relation R not only defines the relevant frame pairs in R, but also the mapping between the different frame elements in the frame pairs. Example 5 shows a fragment of the frame relations list.

```
<frame ID="69" name="Make_noise" cDate="Wed Feb 07 16:12:22 PST 2001">
  <definition>A physical entity, construed as a point-Sound_source,
    emits a Sound. This includes animals and people making noise
    with their vocal tracts. Sometimes the sound itself is referred
    to with a nominal expression, in which case it is called the
    Sound. Manner expressions may also be relevant in this frame,
    if they describe properties of the sound as such.  A path-schema
    can be overlaid on the simple noise-making scene, adding a
    Location_of_source and/or a Path.
  </definition>
  <fes>
   <fe ID="323" name="Sound_source" abbrev="Src-p" coreType="Core">
     <definition>The frame element Sound_source is assigned to a
       constituent referring to a physical entity which emits a
       sound. Such an entity must have an appropriately complex
       internal structure such that it can create a sound. For
       example, a machine or some other thing with moving parts
       can create sound by itself.
                 Ex: ["Sound_source" The wind] HOWLED.
     </definition>
     <semtypes />
   </fe>
   <fe ID="326" name="Reason" abbrev="Rsn" coreType="Peripheral">
     <definition>This FE identifies the Reason that the Sound_source
       produces the sound. Generally, this is some external stimulus
       that prompts the  Sound_source to make noise.
                 Ex: The planks above him CREAKED ["Reason"  as
                     Simon hoisted himself inside]
     </definition>
     <semtypes />
   </fe>
   <lexunits>
    <lexunit ID="1461" name="scrape.v" pos="V" status="Problem"
             lemmaId="34535">
     <definition>COD: rub or cause to rub against a rough or hard
       surface
     </definition>
   <semtypes />
</frame>
```

Example 4: FrameNet Semantic Frame Example

Relation Type	No. of applicable frame pairs
Inheritance	193
Subframe	78
Using	214
See_also	33
ReFraming_Mapping	113
CoreSet	195
Excludes	124
Requires	40
Inchoative_of	4
Causative_of	12
total	1006

Table 3.4: FrameNet Frame Relations

```
<frame-relation-type ID="1" name="Inheritance" superFrameName="Parent"
                     subFrameName="Child">
 <frame-relations>
  <frame-relation ID="2" superFrameName="Event" supId="187"
                  subFrameName="Change_of_consistency" subId="341">
     <fe-relation ID="808" superFEName="Time" supId="1446"
                           subFEName="Time" subId="2921" />
     <fe-relation ID="818" superFEName="Reason" supId="1445"
                           subFEName="Reason" subId="2922" />
     <fe-relation ID="819" superFEName="Place" supId="1444"
                           subFEName="Place" subId="2920" />
     <fe-relation ID="4134" superFEName="Manner" supId="4961"
                            subFEName="Manner" subId="2987" />
 </frame-relation>
```

Example 5: FrameNet Frame Relations Example

It can be observed from Example 5 that the frame EVENT (the super-frame) is a parent of the frame CHANGE_OF_CONSISTENCY (the sub-frame), and more specifically, the TIME frame element of event should be mapped to the TIME frame element of CHANGE_OF_CONSISTENCY, the REASON frame element of EVENT should be mapped to the REASON frame element of CHANGE_OF_CONSISTENCY, and so on. The frame relations can be used to construct a semantic network/hierarchy between the frames, thereby indirectly imposing a hierarchical/graphical structure over the

lexical units. Therefore, it is also possible to access the lexical unit entries in a hierarchical/graphical manner through their frames and the frame relations.

Overall, FrameNet is a potentially valuable resource for verb sense disambiguation as it provides plenty of semantic and syntactic information. Furthermore, the frame elements of each frame define the possible semantic roles that can be expected from the verbs associated with that particular frame. These frame elements can then be coupled with suitable semantic role labellers to provide more information about the senses of the verbs (see Section 3.9.3 for a more detailed discussion). However, since FrameNet was designed to be a semantic frame repository, not a word sense inventory, its word sense coverage may be too low or too coarse to be useful for the purpose of the research reported in this thesis.

VerbNet

According to Kipper *et al.* (2000), VerbNet is a verb lexicon that contains hierarchical lexical entries which have been systematically constructed based on Levin verb classes (Levin 1993). VerbNet consists of nearly 240 top level entries, each of which has one-to-one mapping with a specific Levin verb class. Since each Levin verb class refers to a group of verbs that have similar syntactic properties, the VerbNet entries do not correspond to distinct verbs either. Each VerbNet entry is further broken down into the following 4 sections.

1. Verb Sense List
2. Thematic Role List
3. Frame List
4. Subclass List

The Verb Sense List consists of a list of verb senses which are members of the Levin verb class denoted by the corresponding top level VerbNet entry. VerbNet contains nearly 5000 such verb senses. Each verb sense is defined by a head verb together with zero or more WordNet verb synset IDs. Around 3600 unique head verbs and 3400 unique WordNet verb synsets IDs are used to form the verb senses of VerbNet.

The Thematic Role List defines the applicable thematic roles for the verbs in the corresponding Levin verb class. Examples of these thematic roles include AGENT, THEME, SOURCE and LOCATION. Each thematic role is further associated with zero or more selectional restrictions. Each selectional restriction contains one or more semantic feature. Examples of semantic features include +ANIMATE, +ORGANIZATION and BODY_PART.

The Frame List defines the applicable syntactic and semantic patterns in which the verbs in the corresponding Levin verb class can occur. Example 6 shows the partial frame for the Levin verb class of HIT-18.1.

```
<FRAME>
   <DESCRIPTION descriptionNumber="0.2" primary="Basic Transitive"
                xtag="0.2"/>
   <EXAMPLES>
       <EXAMPLE>&quot;Paula hit the ball&quot;</EXAMPLE>
   </EXAMPLES>
   <SYNTAX>
       <NP value="Agent">
           <SYNRESTRS/>
       </NP>
       <VERB/>
       <NP value="Patient">
           <SYNRESTRS/>
       </NP>
   </SYNTAX>
   <SEMANTICS>
       <PRED value="cause">
           <ARGS>
               <ARG type="ThemRole" value="Agent"/>
               <ARG type="Event" value="E"/>
           </ARGS>
       </PRED>
       <PRED value="manner">
           <ARGS>
               <ARG type="Event" value="during(E)"/>
               <ARG type="Constant" value="directedmotion"/>
               <ARG type="ThemRole" value="Agent"/>
           </ARGS>
       </PRED>
       <PRED bool="!" value="contact">
           <ARGS>
               <ARG type="Event" value="during(E)"/>
               <ARG type="ThemRole" value="Agent"/>
               <ARG type="ThemRole" value="Patient"/>
           </ARGS>
       </PRED>
   </SEMANTICS>
</FRAME>
```

Example 6: VerbNet Frame Example

The general description of this frame states that it is a basic transitive verb. This description is then illustrated with an example sentence. The syntax section is a list of syntactic constituents which form the basic syntactic pattern of the verbs in the corresponding Levin verb class. In Example 6, the syntactic pattern is *NP-Verb-NP*, with the first NP taking the AGENT semantic role and the second taking the PATIENT semantic role. The semantics section of the frame list shows a list of predefined predicates and their arguments that can apply to verbs in the corresponding Levin verb class. In the HIT-18.1 example, the first predicate is CAUSE, which can take two arguments, the first of which is an AGENT thematic role, and the second is an EVENT.

The Subclass List section contains some of the Levin verb classes which are subclasses of the corresponding top level VerbNet entries. They have the same structure with respect to the top level entries, except that they do not contain a Subclass List section.

Overall, VerbNet is not a suitable word/verb sense inventory for verb disambiguation purposes. It is clear that it was never intended for word sense disambiguation because all of its entries are structured based on Levin verb classes which classify verbs based on their syntactic behaviors. VerbNet does not have its own word sense definitions, and when such senses are needed, it uses WordNet senses. A large number of verb senses in the Members List section do not have any WordNet synset IDs, this could mean that these verb senses have no suitable WordNet candidates or that VerbNet is still incomplete.

Overall, even though VerbNet contains much valuable information about verbs which can be useful for verb sense disambiguation, it is still in development, and as a result, its coverage is limited. Therefore, VerbNet is not suitable for the purpose of verb sense disambiguation.

Machine Readable Dictionaries

Returning to the criteria introduced at the beginning of this section, in terms of availability, WordNet, FrameNet and VerbNet are better than both LDOCE and COBUILD. WordNet is the most convenient because it not only provides open access to its lexical database, but also the programming interfaces for accessing the database. LDOCE only provides free online access which is useable for computational purposes but much less convenient. In terms of the organization of the lexical data, both LDOCE and COBUILD use a table which is indexed by head word, whereas WordNet and FrameNet use a semantic network which not only supports indexing by head words, but also retrieving information based on predefined semantic relationships. However, in terms of accessibility, LDOCE and COBUILD are not suitable for this research because they are not freely available. Furthermore, in terms of coverage, VerbNet is not suitable for this research because it is still in development and has limited coverage. Finally, in terms of compatibility with other semantic analysis

systems and resources, WordNet is clearly better than the other semantic resources, as it has been used by the Senseval tasks for word sense disambiguation. Therefore, I conclude that even though some of WordNet's senses are excessively fine-grained, it is still the most suitable resource for sense disambiguation related semantic tasks, and this is why I have adopted it as the sense inventory for this research.

3.9.2 Overview of Commonly Used WSD Techniques

In the broadest sense, there are three types of WSD system: supervised, unsupervised and hybrid. A prototypical supervised WSD system must rely on sense-labelled training examples as its primary source of disambiguation information, and statistical models generated with machine learning algorithms as its disambiguation mechanism. As a result, the quality and quantity of the training data have always been the bottleneck of supervised WSD systems. In contrast, a prototypical unsupervised WSD system does not rely on sense-labelled training data but instead relies on machine readable dictionaries, thesauruses and unlabelled corpora as its main sources of disambiguation information, and heuristics as its disambiguation mechanism. For example, Wilks *et al.* describes an unsupervised WSD method that disambiguates a target word by matching its surrounding words with the words in its sense definition in a dictionary. Finally, a hybrid system can make use of labelled training data, but does not rely exclusively on it. Instead, it tends to use the supervised approach for words that have labelled training data, and the unsupervised approach for words that do not have training data.

In terms of coverage, unsupervised WSD systems tend to be able to handle much greater numbers of polysemous words than supervised systems, due to the fact that supervised systems require gold standard sense-labelled data which is expensive to produce. However, in terms of performance, supervised systems tend to perform much better than unsupervised systems on words for which training data is available. Hybrid systems tend to have both the coverage of the unsupervised systems and higher performance of supervised systems over words for which labelled training data is available.

For the purpose of this research, the choice of WSD system will depend on the nature of the movie scripts, which will be discussed in detail in Chapter 6.

General Strategies for Word Sense Disambiguation

The most common source of disambiguation features for WSD systems is the surrounding words of the ambiguous word (Yarowsky 1995; Yarowsky 1993; Gale *et al.* 1992). Using just the surrounding words, it is possible to create the following disambiguation features:

1. n-grams of the lemmatised words within a window of n word tokens to the left and right of the target polysemous word

2. n-grams of the Part of Speech (POS) tags of the words within a window of n work tokens to the left and right of the target polysemous word

To illustrate the n-grams feature, consider the ambiguous word *bank* in the sentence *I went to the bank to apply for a home loan*. If the window size is chosen to be 5 words, i.e. 5 words to the left of *bank* and 5 words to the right of *bank*, then the following n-gram features could be extracted:

unigram *("SENTENCE_START"), ("I"), ("go"), ("to"), ("the"), ("to"), ("apply"), ("for"), ("a"), ("home")*

bigram *("SENTENCE_START", "I"), ("I", "go"), ("go", "to"), ("to", "the"), ("to", "apply"), ("apply", "for"), ("for", "a"), ("a", "home")*

It is also possible to include the relative positions of the n-grams as part of the feature. For example, a unigram feature of *("apply")* could become *("apply", +)* to indicate it is to the right of the target word, or even *("apply", +2)* to indicate it is the second word token to the right of the target word.

With other commonly available natural language processing tools, it is not difficult to include higher-level features such as WordNet-based taxonomic information for the surrounding words, chunk information, parse tree information, named entity and anaphora links.

Notable WSD systems are Yarowsky (1995), Ng and Lee (1996) and Cabezas *et al.* (2001). On top of the features, two additional heuristics are often used in general purpose WSD systems to provide additional disambiguation information when the genre or topic of the sentence containing the ambiguous word is available. The first is the **one sense per collocation** (OSPC) heuristic (Yarowsky 1993), which states that if a word token c is found to occur within the context of a particular sense s_i of a polysemous word w, then it is highly likely that whenever c co-occurs with w, s_i will be w's correct sense. For example, consider the polysemous word *plant* with the following two senses:

plant$_1$: buildings for carrying on industrial labour

plant$_2$: a living organism lacking the power of locomotion

The first sense of *plant* is likely to co-occur with words such as *factory*, *worker*, *engineer* and *machinery*; whereas the second sense is likely to co-occur with words such as *flower*, *root*, and *leaf*. As a result, these words tend to have sufficient disambiguation power to determine the correct sense within given contexts.

The second heuristic is one-sense-per-discourse (OSPD) (Gale *et al.* 1992), it states that polysemous words tend to exhibit only one sense in a given discourse. For

example, consider the noun *plant* in the excerpt taken from a news article[15] shown in (3.15). The word *engineer* is a good indication that for the first occurrence of *plant* the first sense is being used, then with the OSPD heuristic, it can be correctly determined that the first sense also applies to the second occurrence of *plant*.

(3.15) As a senior engineer in the plant, he had to figure out what to do. He submitted a plan over the telephone to top plant officials holed up in the plant's bomb shelter. It worked: his studies had paid off.

One seminal WSD paper that exemplifies the use of n-gram based features, the OSPC heuristic and the OSPD heuristic is Yarowsky (1995). This paper introduces a hybrid system which learns to disambiguate polysemous words from a small number of labelled training instances and a large number of unlabelled training instances by applying the above two heuristics. Yarowsky's system starts with a few gold-standard labelled examples as the seeds of the bootstrapping process. It then trains a classifier based on these seeds, and uses it to classify the senses of the unlabelled data. Once this classification is finished, the OSPD heuristic and a probability threshold are used to select a new set of highly-ranked examples from the classified data. These new examples are then used to train the next classifier. This cycle of training-analysing-retraining keeps repeating until either all the data has been labelled or the reanalysing stage produces no more new highly-ranked examples. Yarowsky's system is elegant because it does not require large numbers of gold-standard labelled training intances but can still achieve results comparable to those that do. The features used in this system are just word collocations of the polysemous words. The evaluation data for Yarowsky's system consists of 11 polysemous nouns and 1 polysemous verb extracted from a 460 million word corpus,[16] with two senses per word. The disambiguation accuracy for all these words ranged between 93.6% to 98.8%, with an average accuracy of 96.5%. However, as all of the test words only have 2 senses, it can be argued that Yarowsky's disambiguation task may be too easy.

Mihalcea and Csomai (2005) describe a general context based WSD system called SenseLearner. Unlike Yarowsky's system, SenseLearner is a hybrid system that can handle a much broader number of polysemous words. Even though SenseLearner is designed to use as little annotated data as possible, it still requires a certain amount of gold standard training data, which comes from SEMCOR (Miller *et al.* 1993). SenseLearner makes use of a set of predefined word categories which are groups of words sharing common syntactic and semantic properties. Each verb category can handle the disambiguation of a specific group of verbs. With the categories, SenseLearner trains a contextual semantic model and a collocational semantic model which can be used independently for the disambiguation of all the SEMCOR verbs which are

[15]The title of this article is "A Worker Recalls the Chernobyl Disaster", written by ANNA MELNICHUK and published by The Associated Press, on Tuesday, April 25, 2006; 2:13 PM

[16]The Yarowsky paper does not mention the name of the corpus.

covered by the verb categories. The contextual model treats the lemmas and POS tags of the words immediately surrounding the target verb as a bag of features. The collocation model uses the same surrounding words, but treats them as bigrams, for example, given the target verb *say* in *the judge* **said** *yesterday ...*, the collocations will be *judge_said* and *said_yesterday*. For verbs which are neither in SEMCOR nor covered by the word categories, the majority sense as defined by WordNet is used. SenseLearner was evaluated on the English all-words disambiguation tasks (Edmonds and Cotton 2001) of Senseval-2 (Palmer *et al.* 2001) and beat the majority class baselines by a comfortable margin. The broad coverage of SenseLearner makes it particularly attractive to this research, and it will be evaluated against the movie script corpus in Chapter 6.

However, additional analysis performed as part of this research on SenseLearner's outputs over the English all-words task of Senseval-2 revealed that its VSD accuracy was 49.5%, which was significantly lower than its overall accuracy of 69.0%. Similar analysis performed on the other systems that took part in this task further revealed that the majority of these systems also performed significantly worse on verbs than on other words and SenseLearner's performance on verbs was in fact the highest among all the systems.

Furthermore, the supervised WSD systems that participated in the English lexical-sample task (Kilgarriff 2001) also had poor performance over verbs. Table 3.5 lists the performance of the systems that participated in this task, from which it can be observed that almost all systems performed worse on verbs than on other words. Analysis of these systems shows that they differ mainly in the machine learning systems they use, and the vast majority of their features are the general context WSD features described in Section 3.9.2.

The WSD systems reviewed so far all used general context based WSD features and almost always perform worse on verbs than other word categories. Since most of the ambiguous words are nouns, it is fair to say that the general context based WSD features are more effective over nouns than verbs. There are two possible explanations for this. Firstly, verbs are more polysemous than nouns. According to WordNet 2.0, the average number of senses per word is 2.18 for verbs, and 1.24 for nouns, but if the monosemous words are excluded, the average number of senses per word is 3.56 for verbs and 2.77 for nouns. Secondly, nouns tend to have a smaller number of collocating words than verbs, which means that nouns are more likely than verbs to have the same collocations appearing in both the training and testing data. For example, using the sense definitions of WordNet 2.0, on average, a polysemous noun in the Penn Treebank has 287 collocating open class words, but a polysemous verb in the Penn Treebank has 403 collocating open class words.[17] Hence, in order to improve the performance of verb sense disambiguation, one must employ deeper

[17]Proper nouns are excluded from the calculation, and only words in the same sentences are considered to be collocating.

System	Overall WSD Performance			Verb only WSD Performance		
	Precision	Recall	F-Score	Precision	Recall	F-Score
UNED - LS-U (Fernandez-Amoros *et al.* 2001)	0.4	0.4	0.4	0.29	0.29	0.29
UNED - LS-T	0.5	0.5	0.5	0.39	0.39	0.39
WASPS-Workbench (Tugwell and Kilgarriff 2001)	0.58	0.32	0.41	0.0	0.0	0.0
UMD - SST (Cabezas *et al.* 2001)	0.57	0.57	0.57	0.49	0.49	0.49
DIMAP (Litkowski 2001)	0.29	0.29	0.29	0.22	0.22	0.22
IIT 1 (Haynes 2001)	0.22	0.22	0.22	0.19	0.19	0.19
IIT 2	0.23	0.23	0.23	0.20	0.20	0.20
JHU-English (Yarowsky *et al.* 2001)	0.57	0.57	0.57	0.49	0.49	0.49
JHU-Final	0.64	0.64	0.64	0.57	0.57	0.57
SMUls (Mihalcea and Moldovan 2001)	0.64	0.64	0.64	0.56	0.56	0.56
KUNLP (Seo *et al.* 2001)	0.63	0.63	0.63	0.58	0.58	0.58
CS224N (Klein *et al.* 2001)	0.62	0.62	0.62	0.52	0.52	0.52
Sinequa-LIA - SCT (Crestan *et al.* 2001)	0.61	0.61	0.61	0.54	0.54	0.54
TALP (Escudero *et al.* 2001)	0.59	0.59	0.59	0.51	0.51	0.51
Alicante	0.42	0.41	0.41	0.49	0.48	0.48
IRST (Magnini *et al.* 2001)	0.67	0.25	0.36	0.66	0.14	0.23
EHU-dlist-all (Agirre and Martinez 2001)	0.57	0.56	0.56	0.51	0.49	0.50
EHU-dlist-best (Agirre and Martinez 2001)	0.83	0.23	0.36	0.85	0.23	0.36
DuluthA (Pedersen 2001)	0.52	0.52	0.52	0.44	0.44	0.44
DuluthB	0.51	0.51	0.51	0.40	0.40	0.40
DuluthC	0.55	0.55	0.55	0.45	0.45	0.45
Duluth1	0.53	0.53	0.53	0.44	0.44	0.44
Duluth2	0.54	0.54	0.54	0.44	0.44	0.44
Duluth3	0.57	0.57	0.57	0.47	0.47	0.47
Duluth4	0.54	0.54	0.54	0.45	0.45	0.45
Duluth5	0.55	0.55	0.55	0.45	0.45	0.45
Baseline Lesk Corpus	0.51	0.51	0.51	0.45	0.45	0.45
Baseline Commonest	0.48	0.48	0.48	0.40	0.40	0.40
Baseline Grouping Lesk Corpus	0.44	0.44	0.44	0.36	0.36	0.36
Baseline Grouping Commonest	0.43	0.43	0.43	0.34	0.34	0.34
Baseline Grouping Lesk	0.27	0.27	0.27	0.22	0.22	0.22
Baseline Grouping Lesk Def	0.23	0.23	0.23	0.14	0.14	0.14
Baseline Lesk	0.23	0.23	0.23	0.18	0.18	0.18
Baseline Grouping Random	0.18	0.18	0.18	0.09	0.09	0.09
Baseline Lesk Def	0.16	0.16	0.16	0.09	0.09	0.09
Baseline Random	0.14	0.14	0.14	0.09	0.09	0.09

Table 3.5: Senseval-2 System Performance

semantic analysis than just the collocation based features used for nouns. One such type of deeper semantic analysis is selectional preferences.

3.9.3 Selectional Preference Based Word Sense Disambiguation

In general, selectional preferences describe the affinity for predicates to take arguments of particular types in a given argument position. For example, the nouns which can play the **eater** role for the verb *eat* ("take in solid food"), tends to be filled by an element of the ANIMATED_THING CLASS, and the nouns which can play the EATEE role would tend to belong to the EDIBLE class.[18] Furthermore, for the purpose of this thesis, I will extend the general definition of selectional preferences to include nouns which function as adjuncts to predicating words. For example, the verb *left* in the sentence *I left my cat in my mother's house* stands for "go and leave behind, either intentionally or by neglect or forgetfulness", but in the sentence *I left my cat in my mother's care*, it stands for "put into the care or protection of someone". The two instances of *left* have the same values for their LEAVER and LEAVEE semantic roles,[19], but have different senses because the head nouns of their adjunct prepositional phrases are different.

More specifically, a selectional preference is a function mapping semantic-roles to noun **classes**. Selectional preferences between predicating words (verbs and adjectives) and their arguments (nouns) have previously been combined with statistical methods to perform word sense disambiguation (Resnik 1997; McCarthy and Carroll 2003). The basic assumption made by all selectional preference based WSD systems is that the different senses of the same predicating word have different selectional preferences. Using the *kick* example, the sentences exemplifying each sense may generate the following selectional preferences:

[18]Unless specified, all semantic roles used in this chapter will use platform-independent neutral terms such as EATER, EATEE, KICKER and KICKEE.

[19]In order to simplify the terminology and avoid making premature association with other syntactic/semantic analysis systems, theories or platforms, I will use the term "semantic role" to refer to both the arguments and adjuncts of verbs in the context of selectional preference based VSD, irrespective of whether such case slots are derived from syntactic analysis such as parsing (subject, object etc) or semantic analysis such as semantic role labelling (agent, patient etc).

(3.16) a. He put a foot on the back of the car and kicked it off the cliff.
kicker: male human
kickee: the car
direction: off the cliff

b. The lamb fell to the ground, its hinder legs kicked weakly then it died.
kicker: leg
manner: a few times

c. The boy kicked the ball out of the door.
kicker: boy
kickee: ball
direction: out of the door

d. The gun kicked back into the shooter's shoulder.
kicker: gun
direction: back into the shooter's shoulder

With semantic resources such as WordNet, it is possible to further generalise the selectional preferences for each semantic role to greatly increase the coverage of selectional preferences. For example, using the hypernyms of WordNet, the KICKER in example (3.16a) can be generalised to the set *person*, *being*, *animated thing*, *causal agent*; and the *car* **kickee** can be generalised to the set *motor vehicle*, *self-propelled*, *wheeled vehicle*, *vehicle* and *transport*. Such generalisation may then be used to identify the correct sense of *kick* in similar sentences such as *The woman placed her boot on the motorcycle and kicked it off the bridge*.

There are two major practical difficulties that can greatly reduce the effectiveness of selectional preferences: (1) the identification of the semantic roles in the text, i.e. semantic role labelling; and (2) finding an appropriate set of noun classes for the semantic roles. Semantic role labelling is a difficult task that will be discussed separately in Chapter 5. The difficulty in establishing an appropriate set of noun classes is due to the fact that the same noun may act as different roles to different verbs, and the noun classes derived for one verb may not be suitable for another, thus making it necessary to assign the same noun to different noun classes with respect to different verbs. This combination of a large number of verbs and verb-dependent variation makes it impossible to come up with a universal noun taxonomy that can be applied to every single verb. If the noun taxonomy is too general, it diminishes the disambiguation power of the selectional preferences. If the noun taxonomy is too specialised, it reduces the coverage of the selectional preferences. Therefore, it is necessary to create noun classes on a verb-by-verb basis, and find the right balance for the specificity of noun types. If the noun taxonomy with respect to a particular verb were to be created manually, it would require massive amounts of human labour

to survey sufficient amounts of natural text to ensure the coverage of the taxonomy. Such manual annotation is not feasible within the scope of this research, and a more scalable alternative must be found which can automatically decide how to form the noun classes on a verb-by-verb basis. More detailed discussion of this issue will be provided in Section 6.2. The rest of this section will be devoted to reviews of the existing literature on selectional preference based WSD.

One seminal paper on using selectional preferences to perform unsupervised WSD is (Resnik 1993). For verbs, Resnik's system uses the subject and object as the set of semantic roles, and each synonym set (synset) in WordNet as a separate noun class. Given a non-class-labelled training corpus, Resnik's system first uses maximum likelihood estimation to calculate how often a noun class c occurs with a particular semantic role R relativec to a given verb. This estimation produces $P_R(c)$ which is the prior probability distribution of c occurring as semantic role R. It then estimates how often c acts as the semantic role R when its predicating word is p. This estimation produces $P_R(c|p)$, which is the posterior probability distribution of c given the predicating word p.

According to the definition of conditional probability, $P_R(c|p)$ is calculated as $\frac{P_R(p,c)}{P_R(p)}$, and therefore in order to calculate the posterior probability from a non-class-labelled corpus, it is necessary to first estimate the frequency $freq(p, c)$. Resnik's system uses the following definition for this estimate:

$$\text{freq}_R(p, c) = \sum_{w \ in \ c} \frac{\text{count}_R(p, w)}{\text{classes}(w)}$$

In this definition, w represents a word in one or more noun classes. For example, in *the boy kicked the ball*, the object of *kick* is the word *ball* which will be represented by w. The operator *classes* calculates the number of noun classes that w belongs to. The function $count_R(p, w)$ is the number of times word w was observed as the argument of p with respect to the semantic role R.

Once the prior and posterior probabilities have been calculated, Resnik's system uses relative entropy to calculate an additional metric called selectional preference strength to judge the disambiguation power of the predicating words based on all the noun classes that were seen in the training data as arguments of the target predicating word. The metric is calculated as:

$$S_R(p) = D(P_R(c|p)||P(c)) = \sum_c P(c|p) \log \frac{P_R(c|p)}{P(c)}$$

Resnik's system performs disambiguation of the arguments of the predicating words, not the predicating words themselves. Since the noun classes of the arguments are WordNet senses, the disambiguation process essentially becomes a process of determining the right noun class for the arguments. In order to measure the ability for a particular predicating word p and semantic role R to disambiguate the senses of

the arguments, Resnik's system defines another metric called selectional preference association in the following way:

$$A_R = \frac{1}{S_R(p)} P_R(c|p) \log \frac{P_R(c|p)}{P(c)}$$

Given an ambiguous word w, its predicating word p and its semantic role R, the disambiguation process in Resnik's system starts by collecting all the senses of w: $[s_1, s_2, ..., s_k]$. For each s_i, it collects all of s_i's hypernyms from WordNet and store them in a set: $C_i = \{c | c \textit{ is an ancestor of } s_i]\}$. Finally, for each C_i, it calculates the maximum selectional preference strength $a_i = \max_{c \in C_i} A_R(p, c)$. The sense s_{max} that generates the maximum selectional preference strength is then chosen as the correct sense.

Resnik's system was trained on approximately 80% of the SEMCOR corpus and tested on the remainder. The accuracy of the system was between 36% and 51% for disambiguating nouns which were direct objects of their respective verbs, and between 36% and 44% for disambiguating nouns which were subjects of their respective verbs.

Although Resnik's system was not designed to disambiguate verbs, it still illustrates the disambiguation power that exists in the relationship between verbs (or predicating words) and their arguments. In fact, in the disambiguation process of Resnik's system, instead of treating the predicating word p as having only one sense, one can calculate the s_{max} for all the possible senses of p and select the sense that produces the highest s_{max} as the correct sense.

One interesting observation about Resnik's system is that the selectional preferences constructed from the verb-object relationship perform much better than the ones constructed from the verb-subject relationship. This indicates that the objects of verbs have greater disambiguation power than their subjects.

Furthermore, in terms of verb-argument relationships, Resnik's system does not use more than one semantic role in constructing the selectional preferences, as it only uses either the verb-subject relationship, or the verb-object relationship. In theory, using multiple semantic roles in constructing selectional preferences should be more effective then using just one, as each semantic role can act as a filter to eliminate incorrect senses. Therefore Resnik's system has not fully utilised the disambiguation power of selectional preferences. Another feature of Resnik's system is that it does not use any general context based WSD features. This may cause a data sparseness problem and reduce the coverage of Resnik's system. It would be interesting to investigate how a WSD system that uses multi-semantic-role based selectional preferences combined with general context based features could perform.

McCarthy and Carroll (2003) describe another important unsupervised selectional preference based WSD system. Their system automatically acquires selectional preferences for nouns, verbs, and adjectives from the Senseval-2 English-all-words task with respect to WordNet senses, then uses these selectional preferences to train a

probabilistic classifier to perform word sense disambiguation.[20] As stated above, the two practical problems for selectional preference based WSD systems are: (1) the set of semantic roles, and (2) the set of noun classes. In terms of semantic roles used, similar to Resnik's system, McCarthy and Carroll's system also used the subjects and direct objects for the selectional preferences of verbs. However, compared with Resnik's system which used the Tgrep utility over the Penn Treebank to perform extraction from gold-standard parse trees, McCarthy and Carroll's system extracted the verb subjects and objects from the parse trees produced by a wide-coverage, unification based shallow parser. This difference is significant for two reasons, firstly, as McCarthy and Carroll's system use an automatic parser instead of gold standard data, it may be argued that the results produced by their system are more realistic. Secondly, McCarthy and Carroll's parser produces "deep subjects" and "deep objects", correctly identifying the underlining subject and object in passive constructions, e.g. in *The cat was caught by the animal handler*, *The cat* is the deep object, and *the animal handler* is the deep object. Selectional preferences acquired this way would be more accurate than just using the syntactic parse tree provided in the Penn Treebank.

In terms of set of noun classes, McCarthy and Carroll's system used a tree-cut model (TCM) constructed from WordNet word senses and synsets using a similar method to Li and Abe (1998). The construction works in the following way: firstly, a noun class is defined as a synset in WordNet, and a noun sense is defined as a WordNet word sense, from this, a TCM is constructed in the form of a set of noun classes that partitions the set of all noun senses. Since TCMs constructed this way are not unique, the TCM is further conditioned on the verb class (which is a WordNet verb synset and denoted as vc) and semantic role (R) in the following way:

$$\sum_{c \in \Gamma} P(c|vc, R) = 1$$

The best TCM is then selected according to the Minimal Description Length principle (Rissanen 1978).

Once the TCMs are constructed, the verb disambiguation process of McCarthy and Carroll's system consists of simply finding the combination of nc and vc that maximises the probability $P(vc|c, R) = P(c|vc, R) \times \frac{P(vc|R)}{P(c|R)}$. The estimation of $P(c|vc, R)$ is also generalised to include the hypernyms of the noun synsets corresponding to the nouns that occurred in the training data.

McCarthy and Carroll's system was evaluated on the English all-words task of Senseval-2. For polysemous verbs, its overall precision is 38.1%, with 47.3% precision for selectional preferences constructed with only the direct objects of the verbs, and 33.8% precision with only the subjects of the verbs. For polysemous nouns, its overall

[20] The method of McCarthy and Carroll (2003) is an updated version of the one that participated in Senseval-2.

precision is 36.8%, with 33.7% precision for selectional preferences constructed with only the direct objects of the verbs, and 26.8% precision with only the subjects of the verbs.[21] Furthermore, with the incorporation of the one-sense-per-document (OSPDOC) heuristic, McCarthy and Carroll's system was able to gain roughly 3% recall at the cost of roughly 1% precision. However, when only the OSPDOC heuristic is used, McCarthy and Carroll's system had a precision of 49.0% on polysemous nouns, and a precision of 29.8% on polysemous verbs.

Two interesting observations can be made from the results of McCarthy and Carroll's system. Firstly, the performance over polysemous verbs seems to be better than polysemous nouns. This contrasts starkly with the general Senseval-2 English lexical-sample task results presented in Table 3.5 which showed that almost all the WSD systems performed better on nouns than on verbs. Secondly, the OSPDOC heuristic performed much better on nouns than on verbs. This is consistent with system described in Yarowsky (1995) and Gale *et al.* (1992) in which the OSPDOC heuristic was used to enhance WSD systems evaluated on mostly noun-based datasets. These two observations indicate that selectional preference methods may indeed be more effective for verbs than for nouns. Similar to Resnik's system, McCarthy and Carroll's system also made little use of the n-gram features, and this might have reduced the coverage of their system.

The two selectional preference based WSD systems reviewed so far are both unsupervised and do not use any general context features. In order to adequately access the disambiguation capability of selectional preferences, it is useful to investigate how a supervised WSD system which uses both selectional preference and general context features performs. Chen and Palmer (2005) describes such a system. Chen and Palmer's WSD system was a supervised verb-specific system trained and evaluated on the Senseval-2 English-lexical-sample data. This system trains a MaxEnt classifier on the following features:

1. Voice of the target verbs: passive or active
2. Head noun of the subject of the target verb
3. Head noun of the object of the target verb
4. Whether the target verb has a sentential complement
5. Whether the target verb has a prepositional phrase (PP) adjunct, and if so, the head noun of the NP argument of the PP
6. Named Entity (NE) tags and common nouns around the target verb

[21] The overall precision for polysemous nouns is obtained when both direct subjects and direct objects are considered in the disambiguation process, therefore it is higher than the precision obtained by using only the direct subject or the direct object.

7. Word collocations within a half-sentence-length window of the target verb

The extraction of verb subjects and verb objects is performed by a set of hand-crafted rules from the parse trees of the sentences. This is similar to Resnik's system. However, the inclusion of the voice of the verb may help decrease the erroneous extraction of subjects and objects in passive constructions.

Chen and Palmer's system achieved an F-score of 64.6% on the fine-grained verb senses from the Senseval-2 English lexical-samples task which has an inter-annotator-agreement of 71.3%. This result is significantly higher than any of the systems that participated in the original Senseval-2, and it is also claimed to be the best result reported in the intervening period. More detailed analysis of the results will be provided in Section 6 in comparison with the VSD system developed in this research.

Chen and Palmer's work suggests that a supervised WSD system that uses both general context features and selectional preferences features may have the greatest potential in achieving the highest VSD accuracy. Since the selectional preference features used by Chen and Palmer's system only include the surface subject and direct object of the verbs, one possible way to improve their system is to use a richer set of semantic roles to create the selectional preference features. In Chapter 6, I will present such a VSD system.

3.10 Summary

In this chapter, I have briefly introduced all the NLP technologies and resources that will be used in this research, i.e. part of speech tagging, semantic role labelling, word sense disambiguation and WordNet. It is noticeable that none of these technologies have near perfect performance, and it is one of the goals of this research to investigate the influence of their imperfections on the performance of the final system.

Chapter 4

Brief Introduction to Computer Animation

As discussed in Chapter 1, the goal of this research is to investigate how NLP technologies and machine learning algorithms can be used to generate computer animation in a domain independent fashion. In order to fully describe the contribution of this research, it is necessary to first introduce the relevant concepts and technologies in both statistical NLP and computer animation.

Since I have introduced the relevant concepts and technologies in NLP and machine learning in Chapter 3, the purpose of this chapter is then to introduce the essential background information on computer animation. This chapter is not meant to be a comprehensive introduction to computer graphics and animation. Readers unfamiliar with this field may consult (Foley *et al.* 1990) for a comprehensive introduction.

I will start this chapter with a brief introduction to the data structures used to store information about the 3D graphical models used in this research. I will then discuss the various types of geometric transformations that can be applied to these 3D graphical models. Finally, I will discuss how these transformations are used in this research to generate computer animation.

4.1 Data Structures for Storing Information about 3D Graphical Models

This section describes the data structures that were used in this research to store information about 3D graphical models. These data structures are generic in the sense that there are available in all the major 3D modelling and animation systems.

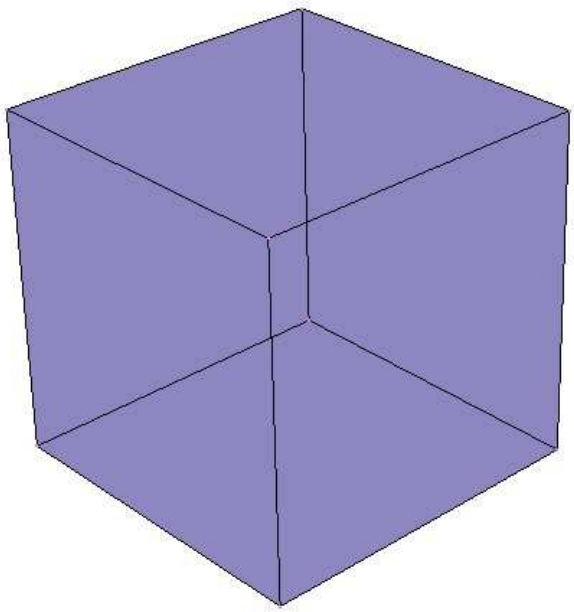


Figure 4.1: Example of the 2D wireframe projection of a 3D model with a single polyhedron

4.1.1 3D Graphical Models

In this research, polyhedrons are the only data structure used for storing the geometric information of 3D graphical models. A single 3D graphical model often consists of one or more polyhedrons.

Each polyhedron is defined by a list of vertices and a list of polygons. Each vertex (x, y, z) represents a point in the 3D Cartesian space. Each polygon represents a face of the polyhedron and is specified by the list of vertices that define the boundaries of the face. Figure 4.1 shows a 2D perspective drawing of the 3D graphical model of a cube that contains a single polyhedron which has eight vertices and six polygons. Figure 4.2 shows a 2D perspective drawing of the 3D graphical model of a table that contains 5 polyhedrons.

It is also worth noting that there is no theoretical limit on how complex a polyhe-

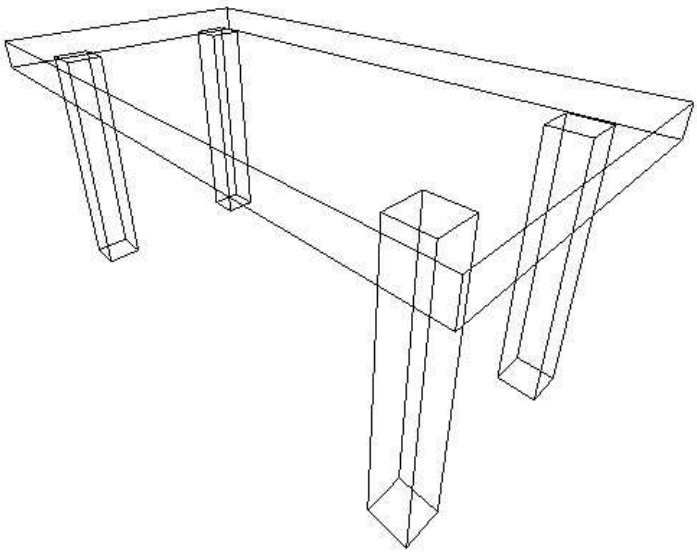


Figure 4.2: Example of a 3D model with multiple polyhedrons

dron can be. A single polyhedron can be as simple as the cube shown in Figure 4.1, or as complex as the model of a humanoid shown in Figure 4.5.

4.1.2 Vertex Groups

Some graphical operations are designed to manipulate part of a polyhedron. To do so, they need to be able to reference a specific region of the polyhedron using the vertices which define that region. A convenient way to perform this referencing is to use a data structure called a vertex group. A vertex group is simply a collection of vertices that belong to the same polyhedron. It is possible for a vertex to belong to more than one group, each of which must have a unique identifier. Figure 4.3 shows three vertex groups for the same polyhedron. An application of vertex groups will be introduced in the next section.

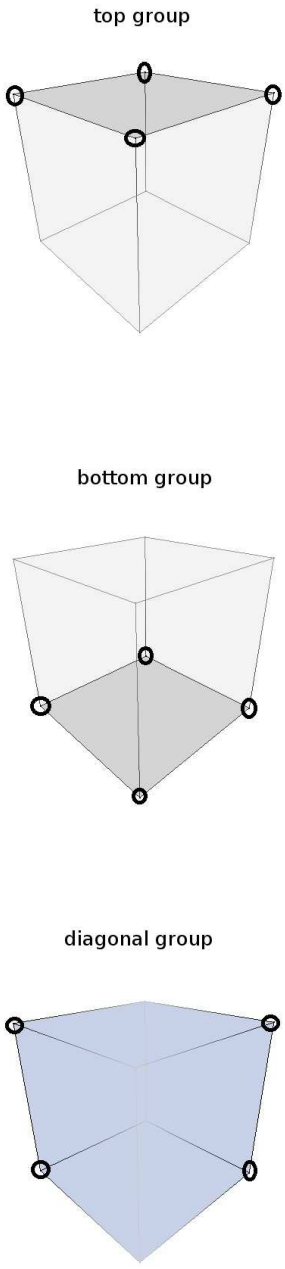


Figure 4.3: Example of vertex groups

4.1.3 Armature

The real world contains many objects which can be partially bent, twisted and stretched. These objects cannot be modelled using just polyhedrons because polyhedrons only define rigid and fixed bodies. A common method to model these non-rigid bodies is to use a data structure called an armature.

An armature defines a skeleton model with respect to one or more polyhedrons. However, it is rare for an instance of an armature to be defined with respect to more than one polyhedron.

Similar to the skeleton structures of real world objects such as humans, animals and robotic limbs, an armature consists of a tree-like hierarchy of bones which connect to each other through joints. Each bone in the same armature must have a unique name, and there must be a unique top bone in an armature.

Since an armature only represents a skeleton structure, it has no direct visual representation. Instead, it must be paired with a polyhedron to be renderable. The process of pairing an armature with a polyhedron is sometimes called 'skinning', and is carried out by creating a separate vertex group from the target polyhedron for each bone in the target armature. Skinning is usually manually performed by 3D artists as it requires artistic interpretation.

Figure 4.4 shows a two-boned armature before and after it was 'skinned' to a cube polyhedron. `Bone Top` is a child of `Bone Bottom`. The top four vertices of the cube are skinned exclusively to `Bone Top`, the bottom four vertices of the cube are skinned exclusively to `Bone Bottom`, and the middle four vertices of the cube are skinned to both bones. Figure 4.5 shows a side-by-side comparison of a 3D graphical model of a male human and its armature.

After a polyhedron is skinned to an armature, its regions (as defined by the various vertex groups) will move with their respective bones, thus achieving deformation of the original rigid polyhedron. Armature based transformations used in this research will be described in detail in Section 4.2.2.

4.2 Transformations for 3D Graphical Models

Objects in real life have physical interactions with each other and their environments in many ways, most, if not all of which can be animated. Furthermore, the majority of these interactions can be classified as either a rigid body transformation (Rogers and Adams 1989), a skeleton based transformation (Ratner 1998), a scene graph based transformation (Wernecke 1994), or a lighting manipulation (Rogers 1997).

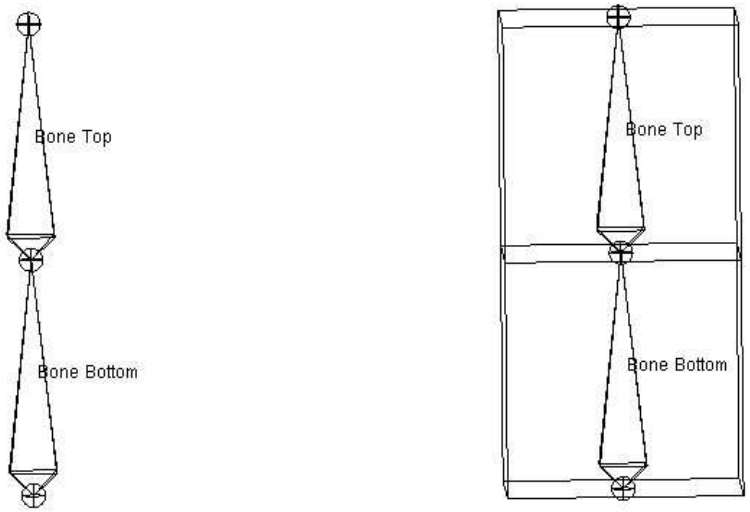


Figure 4.4: Example of a single polyhedron skinned to an armature

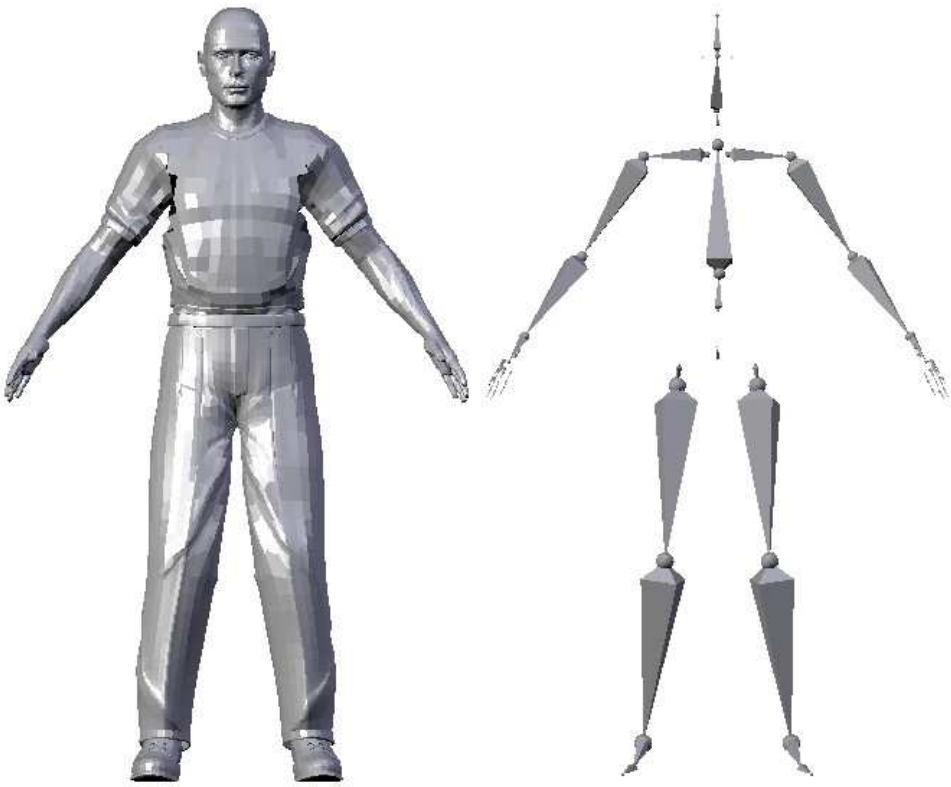


Figure 4.5: Example of a side-by-side comparison of a polyhedron and its armature

4.2.1 Rigid Body Transformations

Rigid body transformations treat a 3D graphical model as a single rigid entity, and cannot change the original geometry of the 3D graphical model that they are applied to. There are three types of rigid body transformations: translation, rotation and resizing.

In the context of this research, translation refers to a change in the location of the 3D graphical model, rotation refers to a change in the orientation of the 3D graphical model, and resizing refers to a change in the size of the 3D graphical model. Furthermore, all rigid body rotations in this research are performed such that the centre of rotation is always the centre of the 3D graphical model.

For simple objects such as stones and bricks, rigid body transformations of the polyhedrons may be sufficient. However, for complex objects such as humans, animals or anything that contains limb-like structures connected with flexible joints, rigid body transformations are generally not sufficient because the movements of the limbs will change the topology of the original polyhedron. In order to overcome the limitations of rigid body transformations, skeleton based transformations are often used.

4.2.2 Skeleton Based Transformations

Skeleton based transformations treat a 3D graphical model as a structured soft body which can be partially bent, rotated and stretched. As mentioned in Section 4.1.3, armatures (i.e. skeleton models) are essential for the skeleton based transformations.

The movement of a bone in a skeleton structure can be a complex process. Since the bones are joined together through a hierarchical structure, when a bone moves or rotates in a certain direction, all of its child bones will also move or rotate in the same direction. The movements of the bones will cause the vertex groups skinned to them to move as well, thereby deforming the original polyhedron to give it a soft or limb-like appearance. Furthermore, the bone hierarchies of skeleton models are not necessarily fixed, and can often be changed at run time.

Figure 4.6 shows what a box looks like before and after deformation by an armature. There are three bones in this armature, and `Bone Middle` and `Bone Top` are rotated to deform the original box.

4.2.3 Scene Graph Transformations

Rigid body and skeleton based transformation transform 3D graphical models by directly modifying their geometric properties. However, there are situations where it is desirable to use the transformation of one 3D graphical model to drive the transformation of another 3D graphical model. For example, in real life, if a person

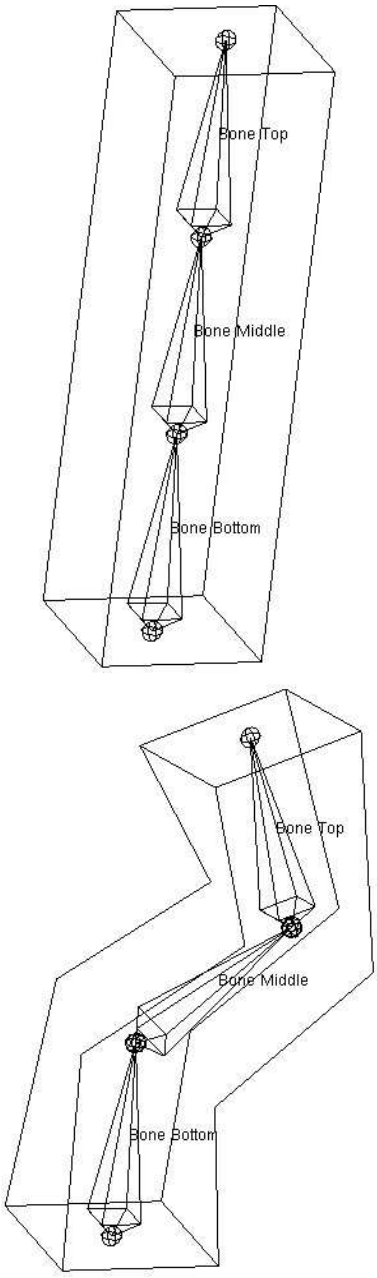


Figure 4.6: Example of a single polyhedron deformed by an armature

puts a hat on his head and then moves about, the hat should translate and rotate with the head. If this person keeps his head still, the hat should also remain still. On the other hand, if a person removes his hat from his head and puts the hat on a table, then the hat's movements should become independent of the movements of its previous wearer's head. These type of situations are generally handled by scene graph transformations.

There are two basic types of scene graph transformation: parenting and un-parenting. Parenting establishes an asymmetrical relationship between a parent 3D graphical model and a child 3D graphical model such that the transformations applied to the parent will also be applied to the child, but the transformations applied to the child will not be applied to the parent. A child can only have only a single parent, but a parent can have multiple children. Un-parenting a 3D graphical model from one of its children will allow both 3D graphical models to move independently of one another.

4.2.4 Lighting Manipulation

Manipulation of light sources is common in real life. For example, when people walk into a dark room, they will often turn on the light (if there is one). The intensity and colour of light sources have significant influence on the appearance of objects in an environment. As a result, manipulation of the intensity and colour of light sources is an important aspect of computer animation.[1]

Lighting manipulations refer to operations that modify the intensity and colour of light sources. The position and orientation of the light sources can be manipulated with rigid body or skeleton based transformation just like other objects.

The intensity of a light source refers to how bright it is. Changing the intensity of a light source changes its energy level, not its wavelength, i.e. colour.

4.2.5 Discussion

In this section, I have introduced three categories of commonly used transformations that can be applied to 3D graphical models in computer graphics. It is important to note that the effect of these transformations is instantaneous on models that they are applied to. For example, applying a translation to a cube will change its location right away, and applying a rotation to a bone of an armature will change its orientation and deform its polyhedron instantly. Furthermore, it is a common practice in computer graphics to combine multiple transformations together to achieve more complex transformations. For example, rigid body rotation and translation are often combined to rotate and move the same model at the same time.

[1] Translation and rotation of light sources are covered under rigid body transformations.

Since transformations are applied instantaneously, they need to be applied continuously to create animation. In the next section, I will discuss how these transformations are generally applied to create animation.

4.3 Animation Creation

An animation can be considered to be a series of still images (frames) of the 3D graphical models as they go through various positions, orientations and poses. Each image in the series is taken sequentially at a fixed time interval. Therefore, creating an animation equates to applying the appropriate transformations to alter the position, orientation and pose of the relevant models at the right time.

Most physical actions that can be animated are complex enough to require multiple transformations to be applied simultaneously. For example, Figure 4.7 shows three snapshots of an animation that depicts a man walking from a chair to a bookshelf. It can be observed that the position, orientation and pose of the man are different in all three snapshots, indicating that three separate transformations were continuously applied to the man in each of the three snapshots.

In order to animate rigid body translational movements of a model, it is necessary to know when and where to place it. There are a wide variety of mathematical models that can be used to calculate the location of a target model with respect to time, and it is up to the animators to choose the most appropriate one at run time. For example, when the destination of the target model is explicitly defined (e.g., a man walking from point A to point B), the animator could choose to move the target 3D graphical model towards its destination along a curve defined by a second degree polynomial, and at a constant speed; or when the direction of the translational movement of the target model is explicitly defined (e.g., a stone free falling from a cliff), the animator could choose to move the target model in a straight line along the given direction, and at speeds defined by a second degree polynomial.

Similarly, in order to animate rigid body rotational movements of a model, it is necessary to know how fast it rotates and in which direction it rotates. There is also a wide variety of mathematical models that can be used to calculate the orientation of the target model with respect to time. For example, when the final orientation of the target model is explicitly defined (e.g. a man tuning towards a window), the animator could choose to turn the target model counterclockwise around the z-axis at speeds defined by a cosine function; or when the direction of the rotation is explicitly defined, the animator could choose to rotate the target object in the specified direction at a constant speed.

Animations that require skeleton based transformation can be divided into two categories: forward kinematics (FK) based animation and inverse kinematics (IK) based animation, I will discuss these next.

As stated in Section 4.2.2, skeleton based transformations work by rotating or

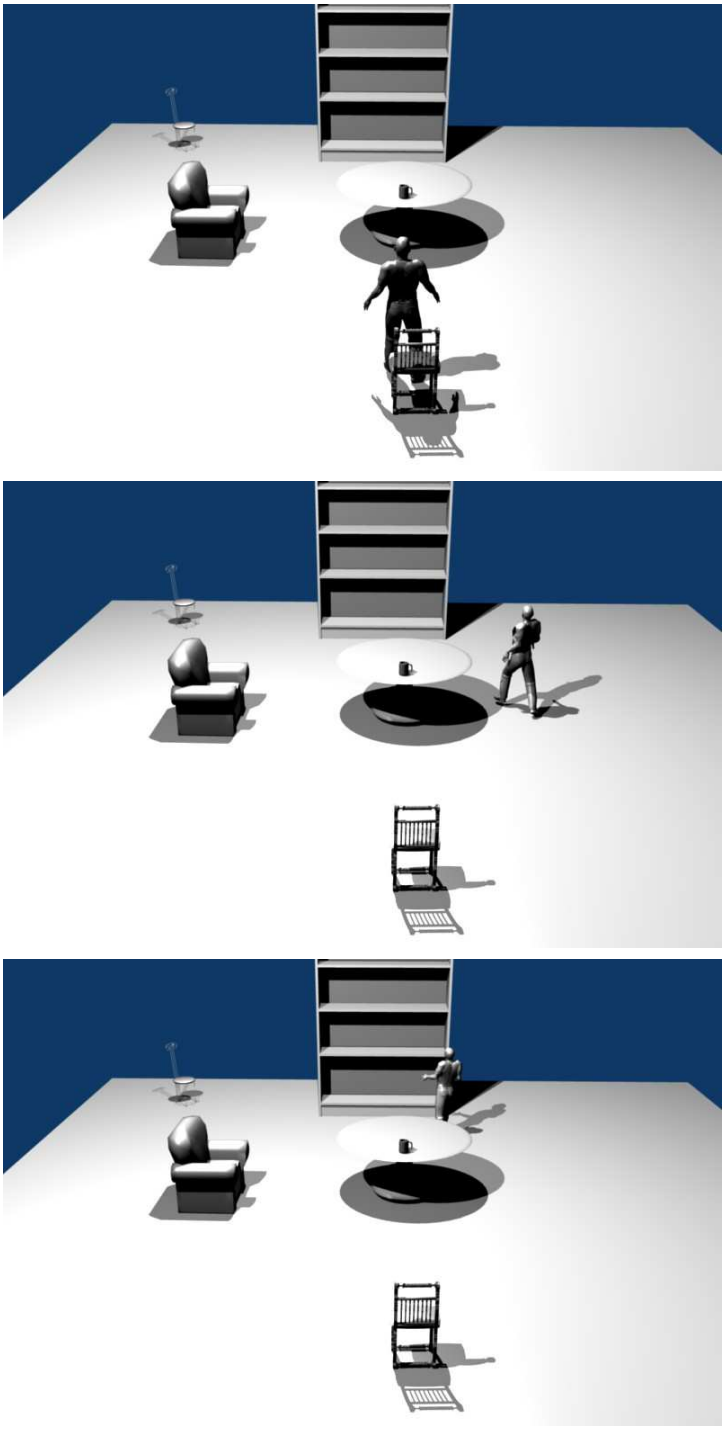


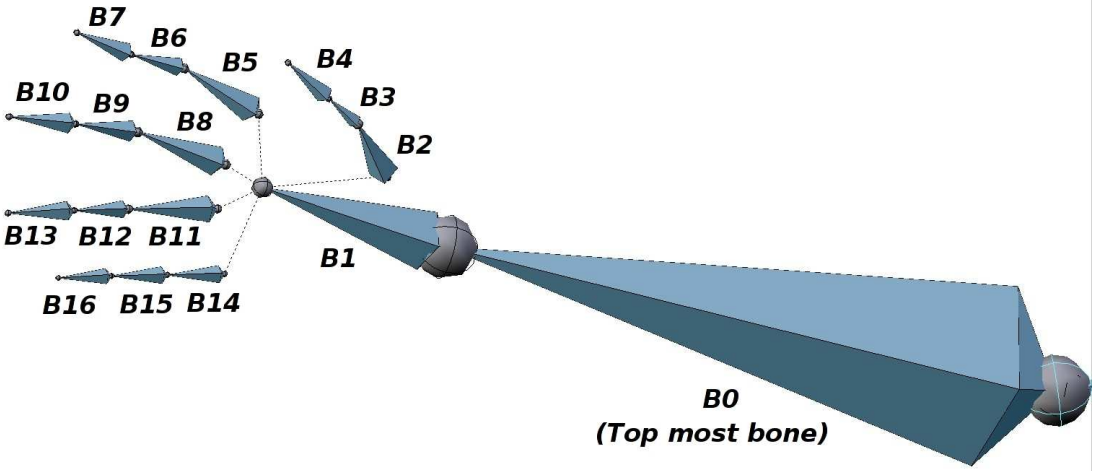
Figure 4.7: Example of an animated walking sequence

translating the bones of the armature. Forward kinematics is concerned with posing the skeleton structure in the exact way required by the user. In other words, the final angle or displacement of each relevant bone must be provided externally, and the animation only involves setting the rotation or location of each relevant bone to the prescribed value at the right time. In this research, forward kinematics based animation is restricted to playbacks of pre-recorded motion capture file (mocap file). Mocap files contain recordings of skeleton movements. Examples of recorded motions include walking, sitting down, chatting and even duelling. Each mocap file contains the following information:

- A skeleton hierarchy definition which contains the geometric and hierarchical information of the bones in the skeleton (armature)
- The number of frames of captured data, and duration of each frame (typically 1/25 of a second)
- The angles of each bone for all the frames

The playback of a mocap file is a two-step process. First, the skeleton hierarchy defined by the mocap file is matched with the skeleton hierarchy of the target model. Then, the orientations of the bones in the model are set to the value specified by the mocap file at the time specified by the mocap file.

Inverse kinematics (IK) based skeleton animation is concerned with posing the skeleton so that a particular bone is in the pose specified by the user. In other words, the user only has to specify the final location and orientation of a single bone in the skeleton structure, and the IK based animation system will work out how to pose some of the other bones to satisfy the user's specification. Since the user only specifies the final pose of a single bone, IK based skeleton animation only rotates all the bones on a specified IK chain. An IK chain is a sequence of bones $[b_1, ..., b_n]$ such that b_{i-1} is the direct parent of b_i. IK chains are typically specified by the head bone and the length of the chain. The head bone of an IK chain is the bone that is furthest from the topmost bone in the skeleton hierarchy. The tail bone of an IK chain is the one that is closest to the topmost bone of the skeleton hierarchy. The length of the IK chain is the number of bones in the chain. The topmost bone in the skeleton hierarchy does not have to be included in the IK chain, however if it is included in the IK chain, it must be the tail bone. Recall that a skeleton is a tree-like hierarchical data structure, and each bone (except for the top most bone) has a single parent. Therefore, each IK chain defines a unique set of bones. For instance, consider the armature model of a forearm shown in Figure 4.8, and suppose that the topmost bone in this armature is $b0$. Examples of valid IK chains would include $[b0]$, $[b0, b1]$, $[b1]$, $[b1, b2]$, and $[b0, b1, b2, b3, b4]$; examples of invalid IK chains would include $[b11, b8]$ and $[b11, b1, b8]$.



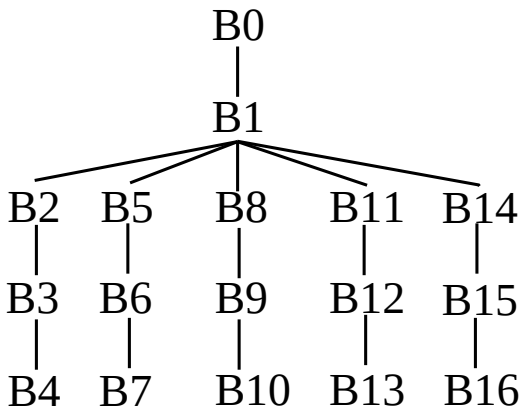


Figure 4.8: Example of a skeleton forearm

While FK based animations are used exclusively to replay recorded skeleton motions in this research, IK based animations are used exclusively to visualise real time skeleton motions such as making the head bone of an IK chain point in a certain direction or touch another object. Unlike FK based animations for which the rotations of all the relevant bones must be specified in advance, an IK based animation only needs to know the final location of the tip of the head bone and the length of the IK chain, and an IK solver algorithm is used to dynamically generate all the rotations for all the bones involved in the IK chain. Figure 4.9 shows snapshots of an armature touching a static cube under different configurations.

Similar to rigid body transformation based animations, various mathematical models can also be used to specify the speed and path of the head bone in an IK chain.

4.4 Blender3D

The animation aspect of this research is implemented in Blender3D. Blender3D is a powerful and constantly evolving open source 3D modelling and animation system that offers functionalities which are comparable with or even better than those offered by commercial animation systems.[2]. A subset of Blender3D's functionalities can be accessed through its Python programming language interface, which is the primary interface between the animated storyboarding system developed in this research and Blender3D. As a result, the automatically animated storyboards are immediately renderable by Blender3D.

In terms of animation functionalities, Blender3D's Python interface allows its users to perform rigid body transformations, skeleton based transformations (both FK and IK based), scene graph based transformations, lighting manipulations, and material manipulations (e.g. changing the colours and texture of renderable objects). Advanced animation technologies offered by Blender3D, but inaccessible through its Python interface, include particle simulation (e.g., for smoke and fire), soft body simulations (e.g., springs, ropes and soft clothes), real time physics simulation (gravity, collision detection and response) and fluid dynamics. However, it is entirely possible for these functionalities to be included into Blender3D's Python interface in the future.

Furthermore, given the open source nature of Blender3D, there is a large community of active contributors who regularly add new functionalities to Blender3D. For example, at the time of writing, there is an active project for simulating large crowds of (fighting) people,[3] and another active project for simulating demolitions such as

[2]Blender3D is downloadable from `http://www.blender.org`, comparisons between Blender3D and other animation systems may be found at `http://wiki.cgsociety.org/index.php/Comparison_of_3d_tools` and `http://www.tdt3d.be/articles_viewer.php?art_id=99`

[3]See the Blender People Project at `http://www.harkyman.com/bp.html`

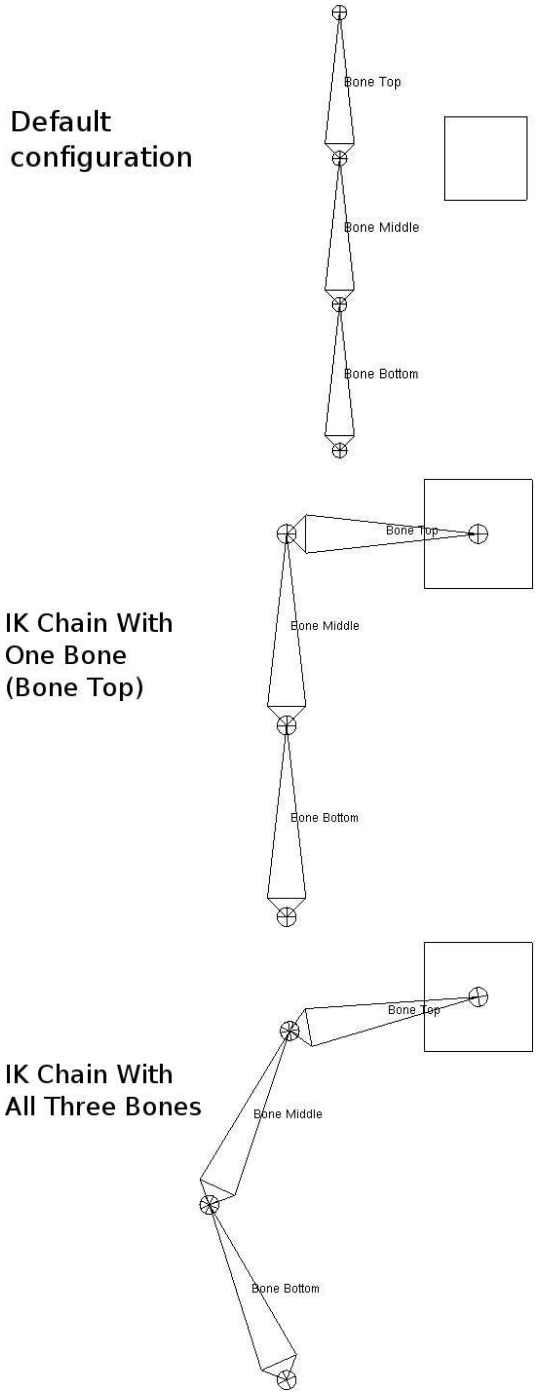


Figure 4.9: Example of an armature touching a static object

car crashes.[4] These projects can all be incorporated transparently into this research.

Finally, the computer graphics aspect of this research is implemented in such a way that Blender3D is only used as the final actuator of the animation. Therefore, given the similarities between Blender3D and other animation systems, the graphics aspect of the automated storyboarding system can be trivially converted to other animation systems provided that they offer the same level of access to their functionalities as Blender3D does.

4.5 Summary

In this chapter, I have briefly introduced the relevant concepts and technologies in computer animation that will be used in this research. The main data structures for storing information about 3D graphics models of real world objects are polyhedrons, vertex groups, and armatures (skeleton models).

Four types of 3D graphical model transformation are implemented in this study: rigid body transformation, skeleton based transformation, scene graph based transformation and lighting manipulation. Since these transformations change the properties of the affected 3D graphical models instantaneously, the creation of animation requires an additional level of control of these transformations.

Finally, it is important to note that the findings of this research are not restricted to the computer animation technologies that were experimented with, or the animation platform on which it was implemented. As will be shown in Chapter 7, the strength of this work is that it is meant to work with any available set of computer animation technologies.

[4]See Blender Demolition Project `http://www.kaikostack.no-ip.com/pc_de.htm`

Chapter 5

Semantic Role Labelling of Prepositional Phrases

In order to faithfully transform the actions described by a movie script into computer animation, it is first necessary to identify the verbs that describe these actions, and extract the semantic information of these verbs. In this research, the task of extracting verb-semantic information is performed with a semantic role labeller. It is obvious that the accuracy of the chosen semantic role labeller has great influence on the success of the final system, and consequently, it is desirable to have the most accurate semantic role labeller possible.

Chapter 3 stated that ASSERT, the chosen semantic role labelling system in this research, did not treat prepositional phrase based semantic roles differently from other types of semantic roles. Since a significant fraction of semantic roles are prepositional phrases (PPs),[1] it is possible that a PP-specific semantic role labeller can handle PP-based semantic roles more accurately than a generic semantic role labeller. Furthermore, PP semantic roles are especially important in the context of this research: most PP semantic roles function as locational, temporal, or manner modifiers to their head verbs, and such modifiers generally have significant influence over the visualisation of their head verbs.

It has been suggested that semantics of prepositions is either too rigid or too diverse to warrant a proper investigation, however, Baldwin (2006) has shown that prepositions do exhibit strong context-dependent semantic similarities. Since it was unclear whether the semantics of prepositions can be determined by their context, we began the research of preposition semantics with two preliminary investigations.

O'Hara and Wiebe (2003) showed that it was possible to use machine learning algorithms to perform semantic role disambiguation on the prepositions in the Penn Treebank with an accuracy of around 86%. Therefore, our first preliminary investigation was to reproduce their disambiguation system and investigate whether there

[1] We found that PPs accounted for about 20% of the semantic roles in the dataset of the CoNLL 2004 semantic role labelling task (to be introduced later).

was any room for improvement. The experiments we performed not only confirmed O'Hara and Wiebe (2003)'s findings, but also showed that comparable disambiguation accuracy can be achieved with a much smaller number of features than those used in O'Hara and Wiebe (2003)'s system. This investigation was a confidence booster for our research on preposition semantics, as it clearly showed that the semantics of prepositions can indeed be learned based on their context.

The second preliminary investigation was based on the preposition sense disambiguation task of SemEval 2007 (Litkowski and Hargraves 2007). A different preposition semantic role disambiguation system was trained on the SemEval data, and achieved an overall accuracy of roughly 70% over 34 prepositions.

The two preliminary investigations on the preposition sense disambiguation clearly showed that the semantics of prepositions can be determined from their contexts reliably. Therefore, we decided to expand our investigation on the semantics of prepositions to building a complete PP semantic role labeller. Compared with the preposition disambiguation systems that we built in our preliminary investigations, the PP semantic role labeller is much more complicated because it not only has to perform the same sense/semantic role disambiguation task, but also has to label the verb-preposition attachment (Collins and Brooks 1995), and determine the boundaries of the preposition-headed semantic roles. In other words, the PP semantic role labeller consists of three subtasks: (1) identifying the prepositions that are attached to verbs, (2) performing sense disambiguation on the target prepositions, and (3) determining the boundaries of the semantic roles which are headed by the target prepositions.

The PP semantic role labeller that we developed was evaluated on two different datasets: the CoNLL 2004 dataset and a corpus of movie scripts. However, as the movie script corpus is small, the PP semantic role labeller was trained only on the CoNLL 2004 dataset. Since the PP semantic role labeller only targets PPs, it would be difficult to evaluate its effectiveness against the holistic semantic role labellers. Therefore, we combined the outputs of the PP semantic role labeller with the outputs of the top three semantic role labellers that participated in CoNLL 2004, and the combined outputs in all three cases had marginally better F-score than just the original outputs of the three holistic semantic role labellers. This indicates that the PP semantic role labeller can indeed offer some small benefits. For the movie script corpus, only the output of ASSERT was used in the evaluation, and in this case, the combined outputs from ASSERT and the PP-semantic role labeller had slightly worse F-score than just the outputs of ASSERT.

This chapter will start with a brief overview of the automatic learning of preposition semantics in the context of semantic role labelling. The overview will be followed by the presentation of the preliminary experiments on preposition sense disambiguation using the Penn Treebank data and the SemEval data. Then, I will report the work on the construction of a preposition semantic role labeller, and the results of combining it with existing generic semantic role labellers. Finally, I will present the overall semantic role labelling strategy used in this research.

The majority of the research reported in this chapter was jointly published with Timothy Baldwin, and has already been presented in Ye and Baldwin (2006a) and Ye and Baldwin (2007).

5.1 Overview of the Automatic Learning of Preposition Semantics

Prepositions in English exhibit a wide range of semantic properties with respect to the verbs that they are attached to (Baldwin 2006). Take (5.1) for example: in the first sentence, *in* is part of the locational complement for the verb *sat*; in the second sentence, *in* is part of the temporal modifier for the verb *ate*; and in the third sentence, *in* is part of the manner modifier for the verb *fired*. It is obvious that the semantics of the verbs in these sentences cannot be fully understood without first knowing the meaning of the preposition *in*.

(5.1) a The man sat **in** the chair.

b The man ate his lunch **in** 5 minutes.

c The man fired his gun 3 times **in** quick succession.

In practice, the task of PP semantic role labelling is much more complicated than just assigning semantic roles to prepositions. It in fact consists of three subtasks: verb-preposition attachment classification, preposition semantic role disambiguation and PP semantic role segmentation. Since it is possible for a PP to attach to either a noun phrase (NP) or a verb phrase (PP) and multiple VPs can occur in the same sentence, it is necessary to first identify whether the head preposition of a given PP is attached to a verb, and if so, which verb it is attached to. Once the verb-PP attachment has been established, the process of assigning semantic roles to the head preposition can begin. Finally, since semantic roles have variable lexical span, it is necessary to have an automatic method that can reliably determine the extent of the semantic roles. To illustrate the process of PP semantic role labelling, consider the preposition *after* in (5.2). Since there are two verbs in this sentence, *leaves* and *peeks*, the PP semantic role labeller must first determine whether *after* is attached to either of the two verbs (verb-preposition attachment classification), then once it has decided that *after* is attached to *peeks*, it then must work out that *after*'s semantic role is TEMPORAL (preposition semantic role disambiguation), and finally, it must determine the entire sub clause *after the driver leaves the truck* is the semantic role headed by *after* (PP semantic role segmentation).

(5.2) After the driver leaves the truck, Buzz peeks out from the passenger window.

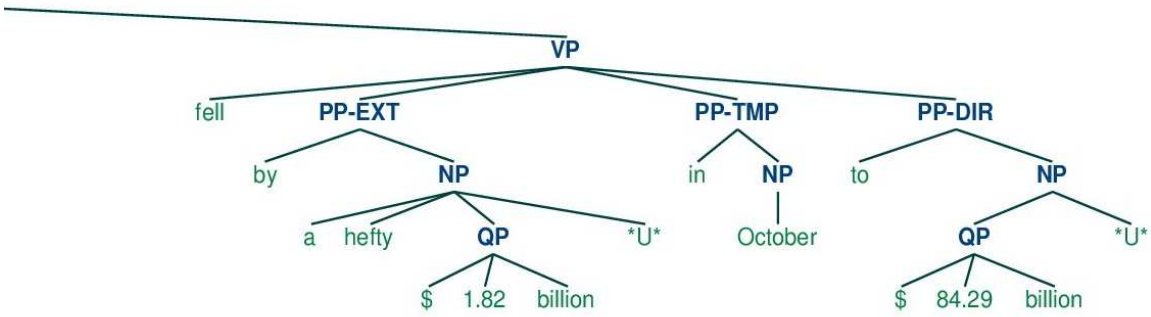


Figure 5.1: An example of the preposition semantic roles in Penn Teebank

As stated at the beginning of this chapter, we first performed two feasibility studies on the semantic role disambiguation of prepositions to investigate whether it is a feasible task. In the first study, we used the Penn Treebank dataset (Marcus *et al.* 1993), and in the second study, we used the SemEval dataset (Litkowski and Hargraves 2007). We then constructed a PP semantic role labeller and investigated all three subtasks as well as strategies for combining the outputs of the PP semantic role labeller with the outputs of holistic semantic role labellers.

5.2 Preposition Semantic Role Disambiguation for Penn Treebank

Significant numbers of prepositional phrases (PPs) in the Penn Treebank (Marcus *et al.* 1993) are tagged with their semantic role relative to the governing verb. For example, Figure 5.1 shows a fragment of the parse tree for the sentence *[Japan's reserves of gold, convertible foreign currencies, and special drawing rights] fell by a hefty $1.82 billion in October to $84.29 billion [the Finance Ministry said]*, in which the three PPs governed by the verb *fell* are tagged as, respectively: `PP-EXT` "extent", meaning how much of the reserve fell; `PP-TMP` "temporal", meaning when the reserve fell; and `PP-DIR` "direction", meaning the direction of the fall.

According to our analysis, there are 143 preposition semantic roles in the treebank. However, many of these semantic roles are very similar to one another; for example, the following semantic roles were found in the treebank: `PP-LOC`, `PP-LOC-1`, `PP-LOC-2`, `PP-LOC-3`, `PP-LOC-4`, `PP-LOC-5`, `PP-LOC-CLR`, `PP-LOC-CLR-2`, `PP-LOC-CLR-TPC-1`. Inspection of the data revealed no systematic semantic differences between these PP types. Indeed, for most PPs, it was impossible to distinguish the subtypes of a given superclass (e.g. `PP-LOC` in our example). We therefore decided to collapse the PP semantic roles based on their first semantic feature. For example, all semantic roles

Semantic Role	Count	Frequency	Meaning
PP-LOC	21106	38.2	Locative
PP-TMP	12561	22.7	Temporal
PP-CLR	11729	21.2	"Closely related" (somewhere between an argument and an adjunct)
PP-DIR	3546	6.4	Direction (*from/to* X)
PP-MNR	1839	3.3	Manner (incl. instrumentals)
PP-PRD	1819	3.3	Predicate (non-VP)
PP-PRP	1182	2.1	Purpose or reason
PP-CD	654	1.2	Cardinal (numeric adjunct)
PP-PUT	296	0.5	Locative complement of *put*

Table 5.1: Penn Treebank semantic roles (top-9 roles)

that start with `PP-LOC` are collapsed to the single class `PP-LOC`. Table 5.1 shows the distribution of the collapsed preposition semantic roles. The total number of collapsed semantic roles is 29.

The collapsing the PP semantic roles was also implemented in O'Hara and Wiebe's system, which we replicated as part of our study.

5.2.1 System Description

O'Hara and Wiebe (2003) describe a system[2] for disambiguating the semantic roles of prepositions in the Penn Treebank according to 7 basic semantic classes. In their system, O'Hara and Wiebe used a decision tree classifier, and the following types of features:

POS tags of surrounding tokens The POS tags of the tokens before and after the target preposition within a predefined window size. In O'Hara and Wiebe's work, this window size is five words ($[-2, 2]$).

POS tag of the target preposition

The target preposition

Word collocation All the words in the same sentence as the target preposition; each word is treated as a binary feature

[2] This system was trained with WEKA's J48 decision tree implementation.

Hypernym collocation The WordNet hypernyms of the open class words before and after the target preposition within a predefined window size (set to 5 words); each hypernym is treated as a binary feature

O'Hara and Wiebe's system also performs the following pre-classification filtering on the collocation features:

- Frequency constraint: $f(coll) > 1$, where *coll* is either a word from the word collocation or a hypernym from the hypernym collocation
- Conditional independence threshold: $\frac{p(c|coll)-p(c)}{p(c)} \geq 0.2$, where c is a particular semantic role and *coll* is from the word collocation or a hypernym from the hypernym collocation

We began our research by replicating O'Hara and Wiebe's method and seeking ways to improve it. Our initial investigation revealed that there were around 44000 word and hypernym collocation features even after the frequency constraint filter and the conditional independence filter have been applied. We did not believe all these collocation features were necessary, and deployed an additional frequency-threshold-based filtering mechanism over the collocation features to only select collocation features which occur in the top N frequency bins. This frequency-threshold-based filtering mechanism allows us to select collocation feature sets of differing size, and in doing so not only improve the training and tagging speed of the preposition semantic role labelling, but also observe how the number of collocation features affects the performance of the PP semantic role labeller and which collocation features are more important.

5.2.2 Results

Since some of the preposition semantic roles in the treebank have extremely low frequencies, we decided to build our first classifier using only the top nice semantic roles, as detailed in Table 5.1. We also noticed that the semantic roles `PP-CLR, PP-CD and PP-PUT` were excluded from O'Hara and Wiebe's system which only used `PP-BNF, PP-EXT, PP-MNR, PP-TMP, PP-DIR, PP-LOC and PP-PRP`. As a result, we built a second classifier using only the semantic roles used by O'Hara's system.[3] The two classifiers were built with a maximum entropy learner (Berger *et al.* 1996).[4]

Table 5.2 shows the results of our classifier under stratified 10-fold cross validation[5] using different parameters for the ranking-based filter. We also list the accuracy reported by O'Hara and Wiebe for comparison.

[3] `PP-BNF` with only 47 counts was not used by the second classifier.

[4] `http://homepages.inf.ed.ac.uk/s0450736/maxent_toolkit.html`

[5] O'Hara's system was also evaluated using stratified ten-fold cross validation.

Top N most Frequent Features	Accuracy (%)	
	Classifier 1	Classifier 2
10	74.75	81.28
20	76.53	83.52
50	79.21	86.34
100	80.13	87.02
300	81.32	87.62
1000	82.34	87.71
all	82.76	87.45
O'Hara & Wiebe	N/A	85.8

Table 5.2: Penn Treebank preposition semantic role disambiguation results

Semantic Roles	With `PP-CLR`	Without `PP-CLR`
PP-PUT	74.16%	83.22%
PP-MNR	46.86%	64.44%
PP-PRD	53.95%	61.47%
PP-TMP	85.03%	88.10%
PP-CD	38.44%	53.02%
PP-LOC	88.17%	91.33%
PP-PRP	55.57%	77.79%
PP-DIR	82.36%	91.30%

Table 5.3: Classification results of the other semantic roles with and without the `PP-CLR`

The results show that the performance of the classifier increases as we add more collocation features. However, this increase is not linear, and the improvement of performance is only marginal when the number of collocation features is greater than 100. It also can be observed that there is a consistent performance difference between classifiers 1 and 2, which suggests that `PP-CLR` may be harder to distinguish from the other semantic roles. This is not totally surprising given the relatively vague definition of the semantics of `PP-CLR`. Table 5.3 shows the classification accuracies of the other nine semantic roles with and without the `PP-CLR` role, and it can be observed that when `PP-CLR` was included, the classification accuracies of all the other semantic roles dropped significantly. These results will be analysed in greater depth in Section 5.4.

5.3 Preposition Sense Disambiguation with SemEval 2007 Dataset

The SemEval-2007 task of preposition sense disambiguation provided us an excellent set of data for further investigating the semantics of prepositions. The SemEval data consisted of over 25000 training instances for a total of 34 prepositions, and each target preposition in the training data is tagged with a sense ID from its entry in the Oxford Dictionary of English (ODE) (Litkowski and Hargraves 2007). The average number of senses of these prepositions is 9.76, and the overall majority class baseline for the prepositions is 39.6%.

Unlike the Penn Treebank data which only consists of prepositions attached to verbs, the SemEval task includes prepositions attached to both nouns and verbs. Therefore, the SemEval task is more about sense disambiguation than verb-semantic role classification.

5.3.1 System Description

We used three categories of features to train the sense disambiguation system for the data of SemEval-2007: collocation features, syntactic features and semantic role based features. The details of these features are listed below.

Collocation Features

The collocation features were designed to capture the correlation between the senses of the target prepositions and their contexts. The following features were investigated:

Bag of open class words The combination of the part-of-speech (POS) tags and lemmas of all the open class words that occur in the same sentence as the target preposition.

Bag of WordNet synsets The WordNet (Miller 1995) synonym sets and their hypernyms of all the open class words that occur in the same sentence as the target preposition.

Bag of named entities Each named entity in the same sentence as the target preposition is treated as a separate feature.

Surrounding words The combinations of the lemma, POS tag and relative position of the words surrounding the target preposition within a window of seven words.

Surrounding super senses The combinations of super-sense tag POS tag and relative position of the words surrounding the target preposition within a window of seven words. The super-sense tags were produced by the system described in Ciaramita and Altun (2006). The super-senses are WordNet lexicographer classes.

Syntactic Features

The syntactic features were designed to capture both the flat and recursive syntactic properties of the target preposition. The flat syntactic features were derived from the surrounding POS tags and chunk tags of the target preposition; the recursive syntactic features were derived from the parse trees produced by the Charniak parser (Charniak and Johnson 2005). The details of these feature are given below.

Surrounding POS tags The combination of POS tag and relative position of the words surrounding the target preposition within a window of seven words.

Surrounding chunk tags The combination of IOB style chunk tag and relative position of the words surrounding the target preposition within a window of five words.

Surrounding chunk types Instead of using only the chunk tags themselves, we also extracted the actual chunk types (NP, VP, ADJP, etc) of the words surrounding the target preposition within a window of five words. Each chunk type is also combined with its relative position to the target preposition as a separate feature.

Parse tree features Given the position of the target preposition p in the parse tree, the basic form of the corresponding parse tree feature is just the list of nodes of p's siblings in the tree (the POS tags are treated as part of the terminal). For example, suppose the original parse tree for the sentence *I live in Melbourne* is the left tree in Figure 5.2, for the target preposition *in*, the basic form of the parse tree feature would be (1, NP). In order to gain more syntactic information, we further annotated each non-terminal of the parse tree with its parent node, and used the new non-terminals as our features. The right tree in Figure 5.2 shows the result of applying this annotation once to the original parse tree. Two levels of additional annotation were performed on the original parse trees in our feature extraction.

Semantic-Role Based Features

Finally, since the senses of prepositions attached to verbs are often related to their semantic roles with respect to their head verbs, we designed some semantic-role-based features to specifically capture this type of verb-preposition semantic information. The details of these features are as follows:

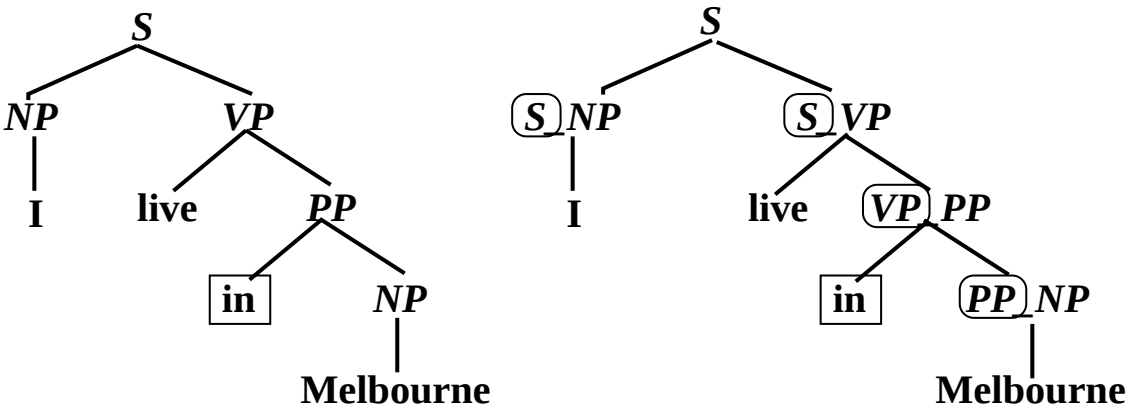


Figure 5.2: Parse tree examples

Surrounding semantic role tags The semantic role tags of the words surrounding the target preposition within a window of five words are combined with their relative positions to the target preposition and treated as separate features. For example, consider the preposition *on* in the sentence *The man who stole my car on Sunday has apologised to me*, the semantic roles for the two verbs (*stole* and *apologised*) are shown in Table 5.4. The semantic roles for *stole* would generate the following features: (-5, I-A0), (-4, R-A0), (-3, TARGET), (-2, B-A1), (-1, I-A1), (0, B-AM-TMP), (1, I-AM-TMP), (2, O), (3, O), (4, O) and (5, O).

Attached verbs This feature was designed to capture the verb-particle and verb-preposition-attachment relationships between verbs and prepositions. There are two situations in which a preposition p is deemed to be attached to a verb v: (1) p has a semantic role tag relative to v and this tag is a 'B' tag, (2) p has no semantic role tag relative to v, but the first token to the right of p has a 'B' tag relative to v. In the sentence shown in Table 5.4, *stole* would be considered as the governor of *on*.

Verb's relative position The lemma of each verb in the same sentence as the target preposition is combined with its relative position to the target preposition and treated as a separate feature. For example, the sentence shown in Table 5.4 would generate the two features: (-1, steal) and (1, apologise).

5.3.2 Results

Using the features described above, we trained separate MaxEnt classifiers for each of the prepositions with the training set of SemEval-2007. We then performed feature selection for each classifier, and tuned the values of their Gaussian smoothing

	stole	apologised
The	B-A0	B-A0
man	I-A0	I-A0
who	R-A0	I-A0
stole	TARGET	I-A0
my	B-A1	I-A0
car	I-A1	I-A0
on	B-AM-TMP	I-A0
Sunday	I-AM-TMP	I-A0
has	O	O
apologised	O	TARGET
to	O	B-A2
me	O	I-A2

Table 5.4: Example semantic-role-labelled sentence

parameter using the trial set of SemEval-2007.[6] The weighted average accuracy of these classifiers is 69.3%.

5.4 Discussions on the Two Preliminary Investigations

The results of the two preliminary investigations on the semantics of prepositions are encouraging, and demonstrate that the semantics of prepositions can be determined from context. However, they are only comparable to the subtask of semantic role disambiguation in the overall task of PP semantic role labelling, and therefore the results of these preliminary investigations may not directly translate to the overall task.

Since the semantic role disambiguation task with the Penn Treebank data only involves prepositions that are attached to a verb, it is more closely related to the overall task than the SemEval task. However, it is also more artificial since the data is in the form of gold standard parse tree which is generally not available to new data. The gold standard parse tree can always be relied upon to determine not just whether a given preposition is attached a verb, but also which verb this preposition is attached to. As a result, prepositions attached to nouns or irrelevant verbs are never included in the training data, and this reduces the noise in both the training and the test data, thereby making the resulting classifiers more effective.

[6]The details of the feature selection process are described in Section 6.2.3

The SemEval task is essentially a word sense disambiguation task focusing on prepositions. Unlike the Penn Treebank data, the prepositions in the SemEval data are not exclusively attached to verbs. As a result, their syntactic patterns are more diverse and realistic than the prepositions in the Penn Treebank task. Semantically, the prepositions in the SemEval task are also more diverse than the prepositions in the Penn Treebank task. In the Penn Treebank data, there are nine fixed semantic roles for the target prepositions, but in the SemEval task, each preposition has its own set of senses, and it is not always possible to map the senses of one preposition to the senses of another preposition. Furthermore, the average number of senses of the prepositions in the SemEval task is also greater than the number of semantic roles of the prepositions in the Penn Treebank data. Some of the more frequent prepositions in the SemEval data have more than 20 senses (*by* has 22 senses, *of* has 20 senses, and *on* has 25 senses). Therefore, it is not surprising that the accuracy of the disambiguation system is nearly 20% lower on the SemEval task than the Penn Treebank task.

5.5 Preposition Semantic Role Labelling with the CoNLL 2004 Dataset

Having built two classifiers which have reasonable performance on the task of treebank preposition semantic role disambiguation and the SemEval-2007 task of preposition sense disambiguation, we decided to investigate whether we could use a similar set of features to perform PP semantic role labelling over alternate systems of PP classification. We chose the 2004 CoNLL Semantic Role Labelling (SRL) data set (Carreras and Màrquez 2004) because it contained a wide range of semantic classes of PPs, in part analogous to the Penn Treebank data, and also because we wished to couple our method with a holistic SRL system to demonstrate the ability of PP semantic role labelling to enhance overall system performance.

Since the focus of the CoNLL data is on SRL relative to a set of pre-determined verbs for each sentence input,[7] our primary objective is to investigate whether the performance of SRL systems in general can be improved in any way by an independent preposition SRL system. We achieve this by embedding our PP classification method within an existing holistic SRL system—that is a system which attempts to tag all semantic role types in the CoNLL 2004 data—through the following three steps:

1. Perform SRL on each preposition in the CoNLL data set;

[7] Note that the CoNLL 2004 data identifies certain verbs as having argument structure, and that the semantic role annotation is relative to these verbs only. This is often not the sum total of all verbs in a given sentence: the verbs in relative clauses, e.g., tend not to be identified as having argument structure.

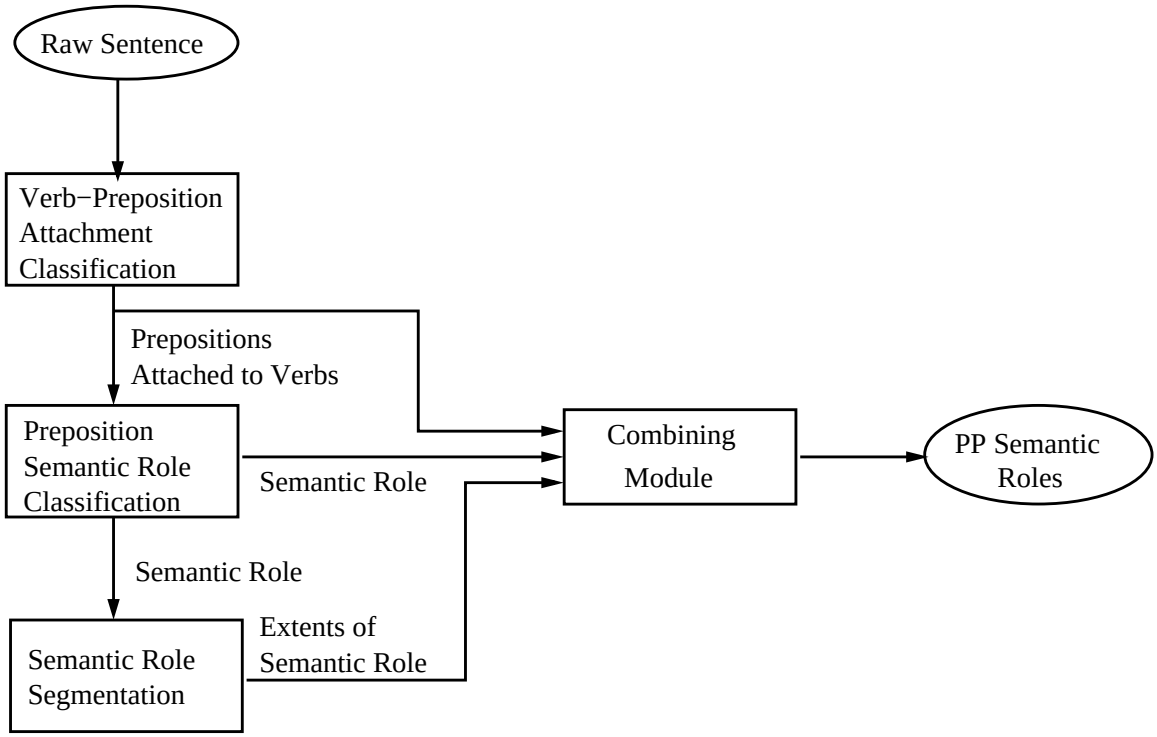


Figure 5.3: Data flow between the three subtasks of the PP semantic role labeller

2. Merge the output of the preposition SRL with the output of a given verb SRL system over the same data set;

3. Perform standard CoNLL SRL evaluation over the merged output.

As mentioned in Section 5.1 of this chapter, the task of PP semantic role labelling can be broken down to the following three subtasks: verb-preposition attachment classification, preposition semantic role disambiguation and PP semantic role segmentation. The verb-preposition attachment classifier identifiers all the prepositions that are attached to verbs, then the preposition semantic role disambiguation classifier assigns semantic roles to the chosen prepositions, and finally the PP semantic role segmentation module identifies the extent of each PP semantic role. Once the outputs from all three subtasks have been obtained, they can be combined together to form the final PP semantic roles. Figure 5.3 illustrates the data flow between the three subtasks.

The three subtasks are not totally independent of each other, as we demonstrate in Section 5.5.7, and improved performance over one of the subtasks does not necessarily correlate with an improvement in the final results.

The details of the solution to each subtask, and how to combine the outputs of the PP-specific semantic role labeller with the outputs of a holistic SRL system, are discussed below.

5.5.1 Preposition Verb Attachment Classification

Preposition-Verb attachment (VA) classification is the first step of preposition semantic role labelling and involves determining the verb attachment site for a given preposition, i.e. which of the pre-identified verbs in the sentence the preposition is governed by.

Verb Attachment Classification Using a Maximum Entropy Classifier

This classifier uses the following features, all of which are derived from information provided in the CoNLL data:

POS tags of surrounding tokens The POS tags of the tokens before and after the target preposition within a window size of five tokens ($[-2, 2]$).

POS tag of the target preposition

The target preposition

Verbs and their relative position (VerbRelPos) All the (pre-identified) verbs in the same sentence as the target preposition and their relative positions to the preposition are extracted as features. Each (verb, relative position) tuple is treated as a binary feature. The relative positions are determined in such a way that the first verb before the preposition will be given the position -1, the second verb before the preposition will be given the position -2, and so on.

The type of the clause containing the target preposition

Neighbouring chunk type The types (NP, PP, VP, etc.) of chunks before and after the target preposition within a window of three chunks.

Word collocation (WordColl) All the open class words in the chunks before and after the target preposition within a predefined window of three chunks.

VA	Count	Frequency
None	3005	60.71
-1	1454	29.37
1	411	8.30
-2	40	0.81
2	29	0.59
3	8	0.16
-3	2	0.04
-6	1	0.02

Table 5.5: VA class distribution

Hypernym collocation (HyperColl) All the WordNet hypernyms from all the senses of the open class words in the phrases before and after the target preposition within a predefined window of three chunks.

Named Entity collocation NEColl All the named entity information from the phrases before and after the target preposition within a predefined window of three chunks.

Chunk-based N-Gram features A series of n-gram features were used to capture the more abstract syntactic and contextual features around the relevant preposition. In this study, the first five chunks after the relevant preposition were used to derive these features. These features are:

- Regular expression representation of the chunk types: This feature is created by merging consecutive identical chunk types into a single symbol. For example, the chunk sequence VP NP PP NP NP will be represented as VP_NP_PP_NP+.
- The first word of each chunk
- The last word of each chunk
- The first part of speech tag of each chunk
- The last part of speech tag of each chunk

The VA classifier outputs the relative position of the governing verb to the target preposition, or `None` if the preposition does not have a semantic role. Such prepositions include those which are attached to noun phrases and those which are attached to verb phrases but are not semantically labelled by CoNLL 2004.

We trained the VA classifier over the CoNLL 2004 training set, and tested it on the testing set. Table 5.5 shows the distribution of the classes in the testing set.

The same maximum entropy learner used in the treebank SRL task was used to train the VA classifier. The accuracy of this classifier on the CoNLL 2004 testing set is 80.14%.

Verb Attachment Classification Using the Charniak Parser

We also experimented with the Charniak parser (Charniak 2000) in the verb-preposition attachment classification. Since this parser was trained on the Penn Treebank data, which was also the source for the CoNLL 2004 data, we expect its accuracy to be reasonably high.

In this experiment, the parser was used to identify which verb a given preposition was attached to, or whether the given preposition was attached to a verb at all. However, it must be noted that since the parse trees produced by the Charniak parser do not contain any semantic role information, it is not possible to distinguish between prepositions which have semantic roles and prepositions which do not have semantic roles. Algorithm 1 shows the process of verb-preposition attachment extraction in pseudo code.

Using the Charniak parser, the accuracy of the verb-preposition attachment classification is 78.13%.

Verb Attachment Classification Error Analysis

Table 5.5 shows that more than half of the prepositions classified by the verb attachment classifier actually did not have any semantic roles. In other words, most of the prepositions in the VA classification will not play a direct role in determining the performance of the entire preposition SRL system. Therefore, these prepositions are not as important as the ones that do have semantic roles. However, this factor was not taken into account by the naive accuracy metric used to measure the performance of the VA classifier, and as a result, the naive accuracy metric may not accurately reflect the true difficulty of the VA classification task.

To address this issue, we re-analysed the performance of the VA classifier by only looking at its accuracy on prepositions which have semantic roles. Table 5.6 shows the distribution of verb-preposition attachments without the `None` classification. Table 5.7 shows the breakdown of the verb-preposition attachment classification accuracy on the test data set of both the MaxEnt classifier and the Charniak parser based classifier.

One interesting observation that can be made from Table 5.7 is that the parse tree based classifier performed extremely poorly when the preposition was attached to the first verb after it. An examination of the data showed that in these particular cases, the Charniak parser tends to attach the preposition to its governing sentential

Let w be the preposition for which we wish to find the parent
Let p_w be the position of w in the parse tree
Let p_{pp} be the parse tree position of the **prepositional phrase** that w is a direct child of
Let p_x be the direct parent of p_{pp}
while p_x is not a VP **do**
    if p_x is an NP or SBAR **then**
        terminate and return None as w's verb attachment
    else
        re-assign p_x to be its direct parent
    end if
end while
Let C be the set of p_x's terminal children ordered according to their positions in the original sentence from left to right
Let $v = None$
for c in C **do**
    if c is w or to the right of w and $v \neq None$ **then**
        terminate and return v as w's verb attachment
    else if c is a verb **then**
        $v = c$
    end if
end for
Return $None$ as w's verb attachment

Algorithm 1: Verb attachment using parse trees produced by the Charniak parser

VA	Count	Frequency %
-1	1454	74.76
1	411	21.13
-2	40	2.06
2	29	1.49
3	8	0.41
-3	2	0.10
-6	1	0.05

Table 5.6: Revised VA class distribution in test set

VA	MaxEnt Classifier Accuracy %	Parse Tree Classifier Accuracy %
None	88.65	71.19
-1	73.87	78.95
1	55.47	0.97
-2	2.50	60.00
2	0.00	0.00
-3	0.00	0.00
3	0.00	0.00
-6	0.00	0.00
overall with *None*	80.14	78.13
overall without *None*	66.99	60.46

Table 5.7: Breakdown of the accuracy of the VA classifiers on the test set

non-terminal, instead of its governing verb (see Figure 5.4 for an example). The Charniak parser based classifier performed poorly in these cases because this particular idiosyncrasy of the Charniak parser was not taken into account in Algorithm 1.

Recall that the `None` classification is assigned to prepositions not attached to any verbs, which therefore have no direct impact on the preposition SRL task as a whole. However, since these prepositions account for the majority of the data, if they are included in the classification, the accuracy would appear to be higher. Hence, we are more interested in the classification accuracy of prepositions which are actually attached to verbs. As Table 5.7 shows, the classification accuracy of these prepositions is rather poor, and as a result, in the best case scenario, the overall preposition SRL system can only achieve an accuracy of 66.99%. It would be highly desirable to significantly improve this upper bound.

Another interesting observation about Table 5.7 is that the two VA classifiers performed quite differently with respect to the different verb-preposition attachments.

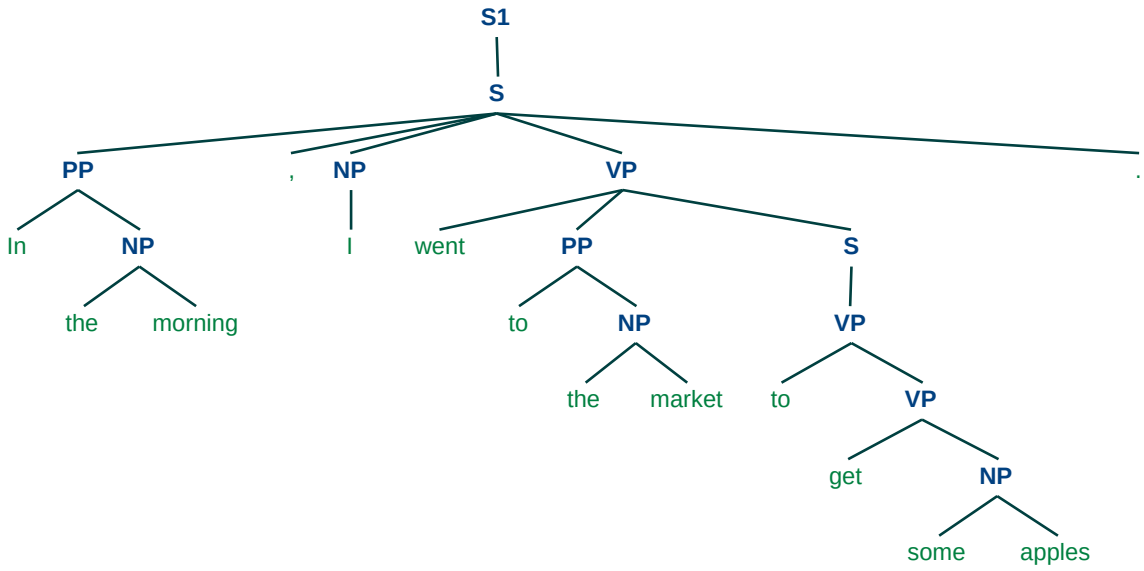


Figure 5.4: The parse tree produced by the Charniak Parser for the sentence *In the morning, I went to the market to get some apples.*

VA	Accuracy %
None	92.18
-1	87.14
1	54.26
-2	42.50
2	0.00
-3	0.00
3	0.00
-6	0.00
overall with *None*	86.40
overall without *None*	77.48

Table 5.8: Breakdown of the accuracy of the new MaxEnt VA classifier on the test set

This means that there is a difference in the capabilities of these two classifiers. Therefore, even though the parse tree classifier performs noticeably poorer than the MaxEnt classifier, it is quite possible that a significant portion of the mistakes made by the two classifiers are actually made on different test instances. A further analysis of the mistakes made by the two classifiers confirmed this: for the test set, if the accuracy was calculated in a way such that an example is considered correctly classified when one of the two classifiers produces the right classification, then the overall accuracies with and without the `None` classification would respectively become 92.16% and 83.24%. This is a much more reasonable upper bound for the accuracy of the overall preposition SRL system.

A New Maxent Classifier Incorporating the Parse Tree Classifier for Verb Attachment

In order to take advantage of the different strengths of the two existing VA classifiers, we constructed a new MaxEnt classifier using all the features of the first MaxEnt classifier, and one additional feature: the classification result of the parse tree VA classifier. Table 5.8 shows the breakdown of the accuracies of this new MaxEnt classifier. It is obvious that this new MaxEnt classifier performs much better than both the old MaxEnt classifier and the parse tree classifier, and it was therefore used in the final preposition SRL system.

Table 5.9 shows the breakdown of the VA classifier's accuracy based on the prepositions. It can be observed that overall, prepositions with over 50 instances in the test data had better accuracies than prepositions with less than 50 instances. For most of the prepositions that have than 10 instances, the accuracy of the classifier is either

equal to or below the majority class baseline. This indicates that there is not enough training data to properly train the classifier.

5.5.2 Preposition Semantic Role Disambiguation

For the task of preposition semantic role disambiguation (SRD), we constructed a classifier using the same features as the VA classifier, with the following differences and additional features:

1. The window size for the POS tags of surrounding tokens is five tokens.
2. The window sizes for the WordColl, the HyperColl and the NEColl features are set to include the entire sentence.
3. The chunk tags (in the IOB format (Sang and Veenstra 1999)) of the words within a window of five.

We trained the SRD classifier once again on the CoNLL 2004 training set, and tested it on the testing set. Table 5.10 shows the distribution of the classes among the PP semantic roles in the test set.

We used the same maximum entropy learner as for the VA classifier to train the SRD classifier. The accuracy of the SRD classifier on the CoNLL 2004 testing set is 72.77%.

Table 5.11 shows the breakdown of the SRD classifier's accuracy based on the prepositions: Similar to the VA classifier. It can be observed that the SRD classifier is largely ineffective against prepositions for which very training instances are available. Furthermore, even among prepositions that are attached to verbs, the SRD classifier often performed below the majority class. This indicates that the features used were not sufficient in capturing the semantics of the prepositions.

5.5.3 Argument Segmentation

Once the semantic role and verb attachment of a preposition has been determined, it is then necessary to determine the extent of the semantic role, i.e. argument segmentation. For this task, we experiment with both a simple regular expression based method and a more complex statistical parser approach. The details are given below.

Argument Segmentation Using a Regular Expression

This method determines the extent of each NP selected for by a given preposition (i.e. the span of words contained in the NP), and is based on a simple regular expression (RE) over the chunk parser analysis of the sentence provided in the CoNLL 2004 data, namely: `PP NP`$^{+}$. The details of this algorithm are shown in Algorithm 2.

Preposition	Accuracy	Counts	Majority Class	Majority Class without the None class
about	76.29	97	None(56.70)	-1(71.43)
above	100.00	5	None(60.00)	-1(100.00)
across	33.33	3	None(66.67)	-1(100.00)
after	68.89	45	-1(37.78)	-1(47.22)
against	100.00	10	None(50.00)	-1(100.00)
ago	0.00	1	1(100.00)	1(100.00)
along	80.00	5	-1(60.00)	-1(100.00)
although	81.82	11	1(81.82)	1(90.00)
amid	80.00	5	-1(80.00)	-1(80.00)
among	82.35	17	None(52.94)	-1(50.00)
around	100.00	15	None(73.33)	-1(100.00)
as	79.08	153	None(49.02)	-1(70.51)
at	83.40	235	None(45.96)	-1(81.10)
because	57.14	42	-1(54.76)	-1(65.71)
before	70.37	27	-1(40.74)	-1(57.89)
behind	100.00	4	None(50.00)	-1(100.00)
below	60.00	10	None(40.00)	-1(66.67)
beside	100.00	1	-1(100.00)	-1(100.00)
besides	0.00	1	None(100.00)	None(0.00)
between	62.50	8	None(62.50)	-1(66.67)
beyond	75.00	4	None(75.00)	-1(100.00)
by	85.37	164	-1(57.32)	-1(86.24)
despite	41.67	12	None(41.67)	1(42.86)
down	75.00	4	None(75.00)	-1(100.00)
during	88.24	17	-1(64.71)	-1(78.57)
except	100.00	1	None(100.00)	None(0.00)
for	83.61	360	None(53.06)	-1(76.33)
from	89.70	165	None(52.12)	-1(87.34)
in	81.90	685	None(52.70)	-1(76.54)
into	89.36	47	-1(70.21)	-1(97.06)
like	85.71	28	None(60.71)	-1(54.55)
near	66.67	3	-1(66.67)	-1(100.00)
next	0.00	1	-1(100.00)	-1(100.00)
of	92.96	923	None(90.79)	1(62.35)
off	100.00	2	None(50.00)	-1(100.00)
on	84.09	220	None(48.64)	-1(86.73)
onto	0.00	1	-1(100.00)	-1(100.00)
out	65.22	23	None(91.30)	-1(100.00)
outside	71.43	7	-1(57.14)	-1(100.00)
over	82.76	29	None(48.28)	-1(86.67)
per	100.00	5	None(100.00)	None(0.00)
since	78.57	28	None(50.00)	-1(50.00)
so	75.00	8	1(87.50)	1(87.50)
than	83.05	59	None(88.14)	1(100.00)
though	81.82	11	None(81.82)	-2(50.00)
through	69.23	26	-1(61.54)	-1(94.12)
throughout	100.00	1	None(100.00)	None(0.00)
till	50.00	2	-1(100.00)	-1(100.00)
to	95.15	886	None(81.94)	-1(93.75)
toward	100.00	5	None(80.00)	-1(100.00)
towards	50.00	2	None(50.00)	-1(100.00)
under	87.50	24	-1(45.83)	-1(68.75)
unless	50.00	4	1(50.00)	1(66.67)
unlike	80.00	5	None(80.00)	1(100.00)
until	76.92	13	-1(53.85)	-1(63.64)
up	62.50	8	None(50.00)	-1(100.00)
upon	100.00	1	-1(100.00)	-1(100.00)
versus	100.00	1	None(100.00)	None(0.00)
via	40.00	5	-1(80.00)	-1(100.00)
whether	53.85	13	-1(46.15)	-1(50.00)
while	60.00	25	-1(40.00)	-1(47.62)
with	86.32	212	None(45.28)	-1(82.76)
within	87.50	8	-1(50.00)	-1(66.67)
without	100.00	11	-1(54.55)	-1(85.71)

Table 5.9: Accuracy of the new MaxEnt VA classifier on the individual prepositions in the test set

Semantic Role	Count	Frequency	Meaning
A1	424	21.79	Argument 1
A2	355	18.24	Argument 2
AM-TMP	299	15.36	Temporal adjunct
AM-LOC	188	9.66	Locative adjunct
A0	183	9.40	Argument 0
AM-MNR	125	6.42	Manner adjunct
A3	106	5.45	Argument 3
AM-ADV	71	3.65	General-purpose adjunct
A4	44	2.26	Argument 4
AM-CAU	40	2.06	Causal adjunct
AM-PNC	32	1.64	Purpose adjunct
AM-DIS	32	1.64	Discourse marker
AM-DIR	19	0.97	Directional adjunct
AM-EXT	7	0.36	Extent adjunct

Table 5.10: CoNLL 2004 PP semantic role distribution in the CoNLL 2004 test data set (top-14 roles)

The performance of RE argument segmentation cannot be independently evaluated. This is because the segmentation convention used by the CoNLL 2004 data differentiates between argument type semantic roles (such as A0, A1) and modifier type semantic roles (such as AM-LOC, AM-TMP). In the case of an argument, the semantic role boundary will start from the first word after the preposition, but in the case of a modifier, the semantic role boundary will start from the preposition. For example, consider the verb *vote* in (5.3): its A1 is PP with *for* as its head, and its second AM-TMP is a PP with *during* as its head. For A1, the head preposition *for* is not included as part of the semantic role, but for AM-TMP, the head preposition *during* is included.

Preposition	Accuracy	Counts	Accuracy without None	Counts without None	Majority Class	Majority Class without None
about	69.07	97	52.38	42	None(56.70)	A1(45.24)
above	60.00	5	0.00	2	None(60.00)	A3(50.00)
across	66.67	3	0.00	1	None(66.67)	A2(100.00)
after	51.11	45	50.00	36	AM-TMP(60.00)	AM-TMP(75.00)
against	70.00	10	40.00	5	None(50.00)	A2(40.00)
ago	0.00	1	0.00	1	A1(100.00)	A1(100.00)
along	20.00	5	0.00	3	None(40.00)	AM-ADV(33.33)
although	54.55	11	60.00	10	A1(36.36)	A0(40.00)
amid	0.00	5	0.00	5	AM-LOC(60.00)	AM-LOC(60.00)
among	58.82	17	37.50	8	None(52.94)	A2(25.00)
around	80.00	15	25.00	4	None(73.33)	AM-LOC(50.00)
as	58.17	153	33.33	78	None(49.02)	A2(33.33)
at	65.11	235	46.46	127	None(45.96)	AM-LOC(25.20)
because	64.29	42	60.00	35	AM-CAU(59.52)	AM-CAU(71.43)
before	48.15	27	31.58	19	AM-TMP(44.44)	AM-TMP(63.16)
behind	25.00	4	50.00	2	A1(50.00)	A1(100.00)
below	60.00	10	33.33	6	None(40.00)	A1(33.33)
beside	0.00	1	0.00	1	A2(100.00)	A2(100.00)
besides	0.00	1	0.00	0	None(100.00)	- (0.00)
between	62.50	8	33.33	3	None(62.50)	A5(33.33)
beyond	50.00	4	0.00	1	None(75.00)	A3(100.00)
by	57.32	164	60.55	109	A0(37.80)	A0(56.88)
despite	58.33	12	71.43	7	AM-ADV(50.00)	AM-ADV(85.71)
down	75.00	4	0.00	1	None(75.00)	AM-ADV(100.00)
during	58.82	17	57.14	14	AM-TMP(82.35)	AM-TMP(100.00)
except	100.00	1	0.00	0	None(100.00)	- (0.00)
for	63.61	360	36.69	169	None(53.06)	A1(23.08)
from	66.06	165	45.57	79	None(52.12)	A2(35.44)
in	66.86	685	45.37	324	None(52.70)	AM-LOC(35.49)
into	42.55	47	38.24	34	A2(42.55)	A2(58.82)
like	64.29	28	16.67	12	None(57.14)	A1(25.00)
near	33.33	3	0.00	2	A2(66.67)	A2(100.00)
next	0.00	1	0.00	1	AM-TMP(100.00)	AM-TMP(100.00)
of	90.90	923	9.41	85	None(90.79)	A1(42.35)
off	50.00	2	0.00	1	A1(50.00)	A1(100.00)
on	60.91	220	38.94	113	None(48.64)	A1(27.43)
onto	0.00	1	0.00	1	AM-DIR(100.00)	AM-DIR(100.00)
out	86.96	23	0.00	2	None(91.30)	AM-DIR(50.00)
outside	57.14	7	25.00	4	AM-LOC(57.14)	AM-LOC(100.00)
over	62.07	29	40.00	15	None(48.28)	AM-TMP(40.00)
per	100.00	5	0.00	0	None(100.00)	-(0.00)
since	60.71	28	42.86	14	None(50.00)	AM-TMP(92.86)
so	62.50	8	62.50	8	A0(62.50)	A0(62.50)
than	91.53	59	57.14	7	None(88.14)	A0(71.43)
though	90.91	11	100.00	2	None(81.82)	A0(50.00)
through	30.77	26	11.76	17	None(34.62)	AM-MNR(41.18)
throughout	100.00	1	0.00	0	None(100.00)	-(0.00)
till	50.00	2	50.00	2	AM-TMP(100.00)	AM-TMP(100.00)
to	89.05	886	53.75	160	None(81.94)	A2(55.00)
toward	80.00	5	100.00	1	None(80.00)	A1(100.00)
towards	50.00	2	0.00	1	AM-DIR(50.00)	AM-DIR(100.00)
under	45.83	24	25.00	16	AM-LOC(37.50)	AM-LOC(56.25)
unless	50.00	4	66.67	3	A1(25.00)	A1(33.33)
unlike	60.00	5	0.00	1	None(80.00)	A1(100.00)
until	46.15	13	36.36	11	AM-TMP(84.62)	AM-TMP(100.00)
up	50.00	8	0.00	4	None(50.00)	A1(75.00)
upon	0.00	1	0.00	1	AM-MNR(100.00)	AM-MNR(100.00)
versus	100.00	1	0.00	0	None(100.00)	-(0.00)
via	20.00	5	0.00	4	AM-MNR(80.00)	AM-MNR(100.00)
whether	69.23	13	75.00	12	A1(76.92)	A1(83.33)
while	48.00	25	47.62	21	AM-ADV(40.00)	AM-ADV(47.62)
with	50.00	212	25.86	116	None(45.28)	A2(30.17)
within	75.00	8	66.67	6	AM-TMP(75.00)	AM-TMP(100.00)
without	45.45	11	28.57	7	AM-MNR(63.64)	AM-MNR(100.00)

Table 5.11: Accuracy of the SRD classifier on the individual prepositions in the test set

```
Let s be the index of the start of the preposition chunk
Let e be the index of the end of the preposition chunk
Let C be the set of chunks that follow the preposition chunk
for Each chunk c in C do
    if c is a preposition attached to a verb then
        Terminate
    else if c is not an NP chunk then
        Terminate
    else
        Assign the end index of c to be e
    end if
end for
```

Algorithm 2: Regular Expression Based Segmentation Algorithm

(5.3)

Couch-potato	B-A0
jocks	I-A0
watching	I-A0
ABC	I-A0
's	I-A0
“	I-A0
Monday	I-A0
Night	I-A0
Football	I-A0
”	I-A0
can	B-AM-MOD
now	B-AM-TMP
vote	TARGET
during	B-AM-TMP
halftime	I-AM-TMP
for	O
the	B-A1
greatest	I-A1
play	I-A1
in	I-A1
20	I-A1
years	I-A1
from	I-A1
among	I-A1
four	I-A1
or	I-A1
five	I-A1
filmed	I-A1
replays	I-A1
.	O

During the boundary extraction process, the segmentation module has no access to the semantic role information, so it is not possible to determine where exactly the argument boundary should start. As a result, the first word after the preposition of interest is always assigned to be the start of the argument boundary. In the process of

Let w be the preposition of interest
Let t be a sibling of w in the parser tree, where t is immediately to the right of w
if t is a nonterminal **then**
    The boundary of the argument starts from the first terminal child of t and ends at the last terminal child of t
else
    The boundary of the argument starts from and ends at t
end if

Algorithm 3: Charniak Parser Based Segmentation Algorithm

combining the final outputs of all the subtasks, we use the semantic role information produced by the preposition SRD module to finally determine where exactly the relevant argument should start.

Since the inclusion of the head preposition of a PP-based semantic role is determined by whether the semantic role is an argument or a modifier, incorrect semantic roles can artificially lower the performance of the segmentation module. In order to get accurate evaluation of the segmentation module, we decided to use gold standard semantic roles for the head prepositions in the evaluation of the segmentation module.

To evaluate the RE segmentation algorithm, we applied it to all the head prepositions of all the PP-based semantic roles in the test set of CoNLL 2004. We then compared the extents produced by the RE segmentation algorithm against the original extents of the semantic roles. The accuracy of the RE segmentation algorithm is 53.08%.

Argument Segmentation Using Statistical Parsers

The RE segmentation method is brittle as it is only capable of extracting arguments which are noun phrases. Therefore we decided to experiment with the Charniak parser and the RASP parser (Briscoe and Carroll 2002) to see if better segmentation results could be achieved.

Similar to the RE method, we assume that each argument starts from the first word after the preposition of interest. Algorithm 3 shows how the parse trees are used to perform the task of argument segmentation.

The evaluation of the parser based segmentation method was performed in the same way as the RE segmentation method. The Charniak parser based classifier achieved an accuracy of 78.00%, and the RASP based classifier achieved an accuracy of 50.05%. Since the Charniak parser based classifier worked significantly better than the other two methods, it was used in the final preposition SRL system.

We were not surprised by the significant gap between the performances of the

Charniak parser and RASP. As stated before, the Charniak parser was trained over a superset of the CoNLL 2004 data, whereas RASP was trained on independent data.

5.5.4 Combining the Output of the Subtasks

Having identified the association between verbs and prepositions, and disambiguated the semantic roles of the prepositions, we can begin the process of creating the final output of the preposition semantic role labelling system. This takes the form of identifying the data column corresponding to the verb governing each classified PP in the CoNLL data format (as determined by the VA classifier), and recording the semantic role of that PP (as determined by the SRD classifier) over the full extent of the PP (as determined by the segmentation classifier).

5.5.5 Parameter Tuning of the Maxent Based Classifiers

Since the MaxEnt based machine learning package can be tuned based on the number of iterations i and the Gaussian prior smoothing parameter g, we decided to train all the MaxEnt based classifiers on the training set of the CoNLL 2004 data with a wide range of combinations of the two parameters. We then applied each combination on the development set, and chose the best one to apply to the test set.

5.5.6 Merging the Output of Preposition SRL and Verb SRL

Once we have generated the output of the preposition SRL system, we can proceed to the final stage where the semantic roles of the prepositions are merged with the semantic roles of an existing holistic SRL system.

It is likely that the semantic roles produced by the two systems will conflict in terms of overlap in the extent of labelled constituents and/or the semantic role labelling of constituents. To address any such conflicts, we designed three merging strategies to identify the right balance between the outputs of the two component systems:

S1 When a conflict is encountered, only use the semantic role information from the holistic SRL system.

S2 When a conflict is encountered, if the start positions of the semantic role are the same for both SRL systems, then replace the semantic role of the holistic SRL system with that of the preposition SRL system, but keep the holistic SRL system's end position.

S3 When a conflict is encountered, only use the semantic role information from the preposition SRL system.

5.5.7 Results

As stated in the beginning of this chapter, our goal is to investigate whether a PP specific semantic role labeller can more accurately capture PP based semantic roles than holistic semantic role labellers. However, since a PP semantic role labeller does not label non-PP semantic roles, the evaluation of its effectiveness has to be performed in conjunction with other holistic semantic role labellers.

To evaluate the performance of our preposition SRL system, we combined its outputs with the three top-performing holistic SRL systems from the CoNLL 2004 SRL shared task (Hacioglu *et al.* 2004; Punyakanok *et al.* 2004; Carreras *et al.* 2004).[8] If the PP semantic role labeller is more effective at capturing PP-based semantic roles, than the combined outputs would have higher F-Score than the original outputs of the holistic SRL systems. Furthermore, in order to establish the upper bound of the improvement of preposition SRL on verb SRL, and investigate how the three subtasks interact with each other and what their respective limitations are, we also used oracled outputs from each subtask in combining the final outputs of the preposition SRL system. The oracled outputs are what would be produced by perfect classifiers, and are emulated by inspection of the gold-standard annotations for the test data.

Table 5.12 shows the results of the preposition SRL systems before they are merged with the verb SRL systems. These results show that the coverage of our preposition SRL system is quite low relative to the total number of arguments in the testing data, even when oracled outputs from all three subsystems are used (recall = 18.15%). However, this is not surprising because we expect the majority of semantic roles to be noun phrases.

In Tables 5.13, 5.14 and 5.15, we show how our preposition SRL system performs when merged with the top 3 systems under the 3 merging strategies introduced in Section 5.5.6. In each table, ORIG refers to the base system without preposition SRL merging.

	SRD_{AUTO}						SRD_{ORACLE}					
	SEG_{AUTO}			SEG_{ORACLE}			SEG_{AUTO}			SEG_{ORACLE}		
	P	R	F	P	R	F	P	R	F	P	R	F
VA_{AUTO}	48.71	7.65	13.22	62.80	10.33	17.73	74.50	11.69	20.21	94.54	15.51	26.65
VA_{ORACLE}	46.82	8.84	14.87	63.21	12.69	21.14	73.91	13.93	23.44	99.38	19.91	33.17

Table 5.12: Preposition SRL results before merging with the holistic SRL systems, (P = precision, R = recall, F = F-score)

We can make a few observations from the results of the merged systems. First, out of verb attachment, SRD and segmentation, the SRD module is both: (a) the

[8] Using the test data outputs of the three systems made available at `http://www.lsi.upc.edu/~srlconll/st04/st04.html`.

	SRD_{AUTO}						SRD_{ORACLE}					
	SEG_{AUTO}			SEG_{ORACLE}			SEG_{AUTO}			SEG_{ORACLE}		
	P	R	F	P	R	F	P	R	F	P	R	F
ORIG	72.43	66.77	69.49	72.43	66.77	69.49	72.43	66.77	69.49	72.43	66.77	69.49
S1												
VA_{AUTO}	72.20	66.98	69.50	72.12	67.01	69.48	72.36	67.13	69.65	72.41	67.27	69.75
VA_{ORACLE}	72.15	67.21	69.59	72.05	67.45	69.67	72.54	67.58	69.97	72.83	68.18	70.43
S2												
VA_{AUTO}	71.00	65.88	68.34	70.68	65.69	68.10	73.47	68.16	70.72	73.68	68.45	70.97
VA_{ORACLE}	70.68	65.85	68.18	70.17	65.70	67.86	73.95	68.87	71.32	74.43	69.66	71.97
S3												
VA_{AUTO}	70.96	64.85	67.77	73.43	68.23	70.74	75.42	68.90	72.01	79.07	73.43	76.15
VA_{ORACLE}	70.59	64.42	67.36	74.00	69.67	71.77	76.22	69.52	72.72	81.73	76.90	79.24

Table 5.13: Preposition SRL combined with Hacioglu et al. (2004) (P = precision, R = recall, F = F-score; above-baseline results are underlined)

	SRD_{AUTO}						SRD_{ORACLE}					
	SEG_{AUTO}			SEG_{ORACLE}			SEG_{AUTO}			SEG_{ORACLE}		
	P	R	F	P	R	F	P	R	F	P	R	F
ORIG	70.07	63.07	66.39	70.07	63.07	66.39	70.07	63.07	66.39	70.07	63.07	66.39
S1												
VA_{AUTO}	69.21	64.89	66.98	69.34	65.25	67.24	70.58	66.16	68.30	71.04	66.83	68.87
VA_{ORACLE}	68.93	65.21	67.02	69.16	65.93	67.51	70.84	66.99	68.86	71.70	68.33	69.97
S2												
VA_{AUTO}	68.50	64.24	66.30	68.49	64.46	66.41	71.95	67.45	69.63	72.57	68.27	70.36
VA_{ORACLE}	68.05	64.39	66.17	67.95	64.78	66.33	72.55	68.61	70.52	73.62	70.15	71.84
S3												
VA_{AUTO}	67.79	62.36	64.96	70.17	65.72	67.87	72.25	66.43	69.22	75.79	70.94	73.29
VA_{ORACLE}	67.01	61.74	64.27	70.49	67.08	68.74	72.61	66.87	69.62	78.17	74.33	76.20

Table 5.14: Preposition SRL combined with Punyakanok et al. (2004) (P = precision, R = recall, F = F-score; above-baseline results are underlined)

component with the greatest impact on overall performance, and (b) the component with the greatest differential between the oracle performance and classifier (AUTO) performance. This would thus appear to be the area in which future efforts should be concentrated in order to boost the performance of holistic SRLs through preposition

	SRD_{AUTO}						SRD_{ORACLE}					
	SEG_{AUTO}			SEG_{ORACLE}			SEG_{AUTO}			SEG_{ORACLE}		
	P	R	F	P	R	F	P	R	F	P	R	F
ORIG	71.81	61.11	66.03	71.81	61.11	66.03	71.81	61.11	66.03	71.81	61.11	66.03
S1												
VA_{AUTO}	70.89	62.90	66.66	70.85	63.17	66.79	72.41	64.23	68.08	72.74	64.85	68.57
VA_{ORACLE}	70.55	63.43	66.80	70.61	64.07	67.18	72.66	65.31	68.79	73.51	66.68	69.93
S2												
VA_{AUTO}	70.33	62.40	66.13	70.13	62.53	66.12	73.52	65.21	69.12	74.05	66.00	69.80
VA_{ORACLE}	69.73	62.69	66.02	69.46	63.02	66.08	73.94	66.44	69.99	74.99	68.01	71.33
S3												
VA_{AUTO}	69.79	60.84	65.00	72.38	64.34	68.12	74.47	64.90	69.36	78.29	69.55	73.66
VA_{ORACLE}	68.99	60.50	64.47	72.64	65.89	69.10	74.86	65.62	69.94	80.68	73.13	76.72

Table 5.15: Preposition SRL combined with Carreras and Màrquez (2004) (P = precision, R = recall, F = F-score; above-baseline results are underlined)

SRL.

Second, the results show that in most cases, the recall of the merged system is higher than that of the original SRL system. This is not surprising given that we are generally relabelling or adding information to the argument structure of each verb, although with the more aggressive merging strategies (namely S2 and S3) it sometimes happens that recall drops, by virtue of the extent of an argument being adversely affected by relabelling. It does seem to point to a complementarity between verb-driven SRL and preposition-specific SRL, however.

There are a few anomalies in the results. Sometimes, an all-auto method achieved better results than when one of the subtasks was oracled. For example, in Table 5.13, when merge scheme 1 was used, the all-auto combination yielded a precision of 72.20%, and an F-score of 69.50%, whereas when the segmentation was substituted to oracled results, the precision dropped to 72.12% and the F-score dropped to 69.48%. This behaviour is caused by the poor accuracy of the SRD classifier and the merging strategy. The perfect segmentation results would reduce the number of argument boundary conflicts between the preposition SRL system and the original system, thereby increasing the recall of the combined system. However, due to the poor performance of the preposition SRD subsystem, these additional arguments were not correctly classified, and this was why the precision dropped while the recall improved.

Finally, it was somewhat disappointing to see that in no instance did a fully-automated method significantly surpass the base system in precision or F-score. Hav-

ing said this, we were encouraged by the size of the margin between the base systems and the fully oracle-based systems, as it supports our base hypothesis that preposition SRL has the potential to boost the performance of holistic SRL systems, up to a margin of 10% in F-score for S3.

5.6 Preposition Semantic Role Labelling with a Movie Script Dataset

A corpus of 567 sentences from movie scripts was also used to evaluate the PP semantic role labeller developed in this study. These sentences were first automatically labelled with ASSERT, then hand corrected.

For the verb-preposition attachment subtask, the MaxEnt classifier with the parse tree features achieved an accuracy of 74.97% with the `None` classification, and 73.23% accuracy without the `None` classification. For the preposition SRD subtask, the MaxEnt classifier achieved an accuracy of 72.77%. Finally, for the argument segmentation task, the Charniak parser based method achieved an accuracy of 78.00%.

Table 5.16 and 5.17 show the SRL results before and after preposition specific semantic roles were merged with the output of ASSERT. The same evaluation schemes used for the CoNLL 2004 dataset were also used here.

	SRD_{AUTO}						SRD_{ORACLE}					
	SEG_{AUTO}			SEG_{ORACLE}			SEG_{AUTO}			SEG_{ORACLE}		
	P	R	F	P	R	F	P	R	F	P	R	F
VA_{AUTO}	40.23	4.70	8.42	50.62	5.48	9.88	82.31	12.20	21.25	<u>96.65</u>	14.55	25.29
VA_{ORACLE}	42.45	5.01	8.96	49.45	6.01	10.72	80.14	15.05	25.35	<u>100.00</u>	19.86	33.14

Table 5.16: Movie script corpus preposition SRL results before merging with outputs of ASSERT (P = precision, R = recall, F = F-score; above-baseline results are <u>underlined</u>)

	SRD_{AUTO}						SRD_{ORACLE}					
	SEG_{AUTO}			SEG_{ORACLE}			SEG_{AUTO}			SEG_{ORACLE}		
	P	R	F	P	R	F	P	R	F	P	R	F
ORIG	71.87	60.01	65.41	71.87	60.01	65.41	71.87	60.01	65.41	71.87	60.01	65.41
S1												
VA_{AUTO}	71.01	60.08	65.09	71.43	60.15	65.30	71.93	60.95	65.99	72.22	61.22	66.27
VA_{ORACLE}	71.33	60.11	65.24	71.41	60.18	65.32	71.93	61.39	66.24	72.51	61.96	66.82
S2												
VA_{AUTO}	70.45	59.61	64.58	69.59	58.60	63.63	73.39	62.20	67.33	73.68	62.47	67.61
VA_{ORACLE}	70.81	59.68	64.77	69.42	58.50	63.49	73.98	63.14	68.13	74.64	63.78	68.78
S3												
VA_{AUTO}	68.89	57.90	62.92	70.48	59.04	64.25	75.27	63.41	68.83	78.07	65.93	71.49
VA_{ORACLE}	69.38	57.86	63.10	70.56	59.21	64.39	75.70	63.84	69.27	80.13	68.31	73.75

Table 5.17: Preposition SRL combined with ASSERT (P = precision, R = recall, F = F-score; above-baseline results underlined)

It can be observed that the stand-alone performance of the PP semantic role labeller on the movie script corpus follows a similar pattern to its performance on the CoNLL 2004 data. Unfortunately, the combined outputs of the PP semantic role labeller and ASSERT failed to outperform the original outputs of ASSERT. More discussions will be provided in Section 5.7

5.7 Analysis and Discussion

In the previous sections, we presented the methodologies and results of three systems that perform statistical analysis on the semantics of prepositions, each using a different data set. The performance of the these systems was very different. The SRD system trained on the Penn Treebank produced highly creditable results, the disambiguation system trained on the SemEval-2007 dataset produced reasonably credible results, but the SRL system trained on CoNLL 2004 SRL data set produced somewhat negative results. In the remainder of this section, we will analyse these results and discuss their significance.

There is a substantial difference between the results obtained by the preliminary studies and that obtained by the CoNLL SRL classifier. In fact, even with a very small number of collocation features, the classifier trained on the Penn Treebank still outperformed the CoNLL SRL classifier. The difference in performances suggests that the semantic tagging of prepositions is somewhat artificial. The artificial nature of the Penn Treebank data is evident in three ways. First, the proportion of prepositional phrases tagged with semantic roles is small – around 57,000 PPs out of the

million-word treebank corpus. This small proportion suggests that the preposition semantic roles were tagged only in certain prototypical situations. Second, we were able to achieve reasonably high results even when we used a collocation feature set with fewer than 200 features. This further suggests that the semantic roles were tagged for only a small number of verbs in relatively fixed situations. Third, the preposition SRD system for the CoNLL data set used a very similar feature set to the treebank system, but was not able to produce anywhere near comparable results. Since the CoNLL data set is aimed at holistic SRL across all argument types, it incorporates a much larger set of verbs and tagging scenarios; as a result, the semantic role labelling of PPs is far more heterogeneous and realistic than is the case in the treebank. Therefore, we conclude that the results of our treebank preposition SRD system are not very meaningful in terms of predicting the success of the method at identifying and semantically labelling PPs in open text.

The results of the SemEval task are more comparable with the results of the CoNLL task. More specifically, the accuracy of the preposition semantic role disambiguation system of the PP semantic role labeller is roughly in the same range as the accuracy of the disambiguation system developed in the SemEval task, and both are significantly lower than the accuracy of the preposition semantic role disambiguation system over the Penn Treebank task. This difference in performance is caused by two factors which made the Penn Treebank task easier: (1) the prepositions in the Penn Treebank data are less diverse both syntactically and semantically than the prepositions in the SemEval data and the CoNLL data, because the former are all attached to only verbs, and the latter are attached to both verbs and nouns; (2) there is significantly less noise in the test set of the Penn Treebank data than the test sets of the SemEval data and the CoNLL data, because the test set of the Penn Treebank data is created from gold standard parse trees, and the test set of the SemEval task and the CoNLL task are created with automatic methods.

Furthermore, the verb-preposition attachment subtask of the PP semantic role labeller has an overall accuracy of 86.4% on the test set of the CoNLL data. If we take this into account, then the overall accuracy on the preposition semantic role disambiguation part of the PP semantic role labeller is even lower (62.9%). This is because the semantic role disambiguation module only assigns semantic roles to the prepositions which are deemed to be attached to verbs by the verb-preposition attachment classifier, and therefore, incorrect verb-preposition attachment classification reduces the overall accuracy of the preposition semantic role disambiguation. On the other hand, the sense disambiguation of the prepositions in the SemEval task does not require any attachment classification for the prepositions, and therefore is not directly affected by the ambiguities of the preposition attachment problem. Hence, it is not surprising that the results for the SemEval task are noticeably better than the results for the CoNLL task.

A few interesting facts came out of the results over the CoNLL data set. The most important one is that by using an independent preposition SRL system, the results of

a general verb SRL system can be significantly boosted. This is evident because when the oracled results of all three subtasks were used, the merged results were around 10% higher than those for the original systems, in all three cases. Unfortunately, it was also evident from the results that we were not successful in automating preposition SRL. Due to the strictness of the CoNLL evaluation, it was not always possible to achieve better overall performance by improving just one of the three subsystems. For example, in some cases, worse results were achieved by using the oracled results for VA and the results produced by SRD classifier, than using the VA classifier and the SRD classifiers in conjunction. The reason for the worse results is that in our experiments, the oracled VA always identifies more prepositions attached to verbs than the VA classifier. Therefore more prepositions will be given semantic roles by the SRD classifier, thus increasing the recall of the final system. However, since the performance of the SRD classifier is not high, and the segmentation subsystem does not always produce the same semantic role boundaries as the CoNLL data set, most of these additional prepositions would either be given a wrong semantic role or wrong phrasal extent (or both), thereby causing the overall performance to fall.

It is also evident that the merging strategy also plays an important role in determining the performance of the merged preposition SRL and verb SRL systems: when the performance of the preposition SRL system is high, a more preposition-oriented merging scheme would produce better overall results, and vice versa.

Finally, it is interesting to observe that with respect to almost all the evaluation schemes, the PP semantic role labeller performed in a similar way on the MSC dataset as it did on the CoNLL 2004 dataset.

However, all three sub-classifiers performed worse on the MSC dataset than they did on the CoNLL dataset, especially the VA classifier and the SRD classifier. This is not surprising for two reasons: firstly, these two classifiers were trained on the CoNLL dataset which is from a completely different domain to the MSC dataset. A similar drop in performance can be observed for ASSERT, which achieved an F-score of 69.49% on the CoNLL test set, but only 65.41% on the MSC dataset. Secondly, the quality of the preprocessing of the CoNLL dataset was much higher than that of the MSC dataset. The CoNLL dataset came from the Penn Treebank, and therefore its POS tags, chunk tags and named entities were either gold standard or near gold standard. On the other hand, the preprocessing of the MSC dataset was performed using automatic systems which were trained on the Penn Treebank, and the difference in genre between the two datasets again resulted in more errors.

Two interesting observations were also made during the process of manually correcting ASSERT's labelling of the MSC dataset. Firstly, it seemed that the POS tagger often mis-tagged some nouns, especially proper names, as verbs, and also regularly mis-tagged some verbs as nouns. This type of mis-tagging translated into incorrect chunk tags and parse trees. Secondly, the average sentence length of the MSC dataset (13.79 words per sentence) is much lower than that of the CoNLL test set (23.96 words per sentence). The labelling of ASSERT tended to be reasonably

accurate on shorter sentences, but very poor on the long sentences.

5.8 Conclusion

In this chapter, we presented the methodologies and results of a novel preposition-phrase-only semantic role labeller designed to enhance the performance of holistic semantic role labellers. The experiments performed in this study showed that this PP semantic role labeller was able to consistently provide marginal enhancements to the performance of holistic semantic role labellers on similar type of texts with gold standard preprocessing, but fell short on movie script style data processed with automatic NLP systems. On the other hand, the semantic role information provides just a subset of the features used in the overall task of visualising movie scripts. As a result, the stand-alone performance of the PP semantic role labeller may not have a direct correspondence with its performance in the context of the overall task. Therefore, although the PP semantic role labeller does not seem to significantly impact on the performance of ASSERT over the movie script data, it will still be interesting to see whether it will make a difference in the subsequent experiments. Furthermore, the hand corrected data obtained in the process of this study can also be used to measure the significance of ASSERT's accuracy in the final system. This issue will be discussed in more detail in Chapter 7.

Chapter 6

Verb Sense Disambiguation

In Chapter 3, I suggested that context based WSD is not well suited to the disambiguation of verbs, and that the selectional preferences of verbs may be able to provide additional disambiguation information. The purpose of this chapter is to examine the verbs in the corpus of movie scripts, and devise a suitable strategy incorporating selectional preferences to perform verb sense disambiguation.

This chapter will start with a description of the movie script corpus, and provide a corpus-specific analysis of the distribution of verbs and their senses. This analysis will show that the best strategy for performing VSD with the movie script corpus is with a combination of a broad coverage semi-supervised system, SenseLearner, and a small coverage, supervised system which was developed as part of this research. The analysis of the movie script corpus will be followed by a detailed description of this supervised VSD system.

The majority of the research reported in this chapter was jointly published with Timothy Baldwin, and has already been presented in Ye and Baldwin (2006b).

6.1 Frequency Distribution and Polysemy of the Verbs in the Movie Script Corpus

There are 268 scripts in the movie script corpus, containing more than three million text tokens of action description. The total number of unique verbs in this corpus is 4174 and the average number of WordNet senses of the verbs is 3.28.[1] It was observed that a small number of verbs in this corpus have very high frequencies, and most verbs have very low frequencies. Furthermore, it was also observed that in general, highly frequent verbs also tended to be more polysemous, i.e. have more senses.

[1]These statistics were collected based on the output of the SVMTool POS tagger, and therefore may not be entirely accurate.

Verb	Freq. in Tagged Corpus	Freq. in Entire Corpus	No. of Tagged Senses	Majority Class Ratio	Inter-Annotator Agreement	SenseLearner Accuracy
hit	32	1770	6	43.75%	82.81%	9.36%
lift	31	1051	4	54.84%	80.65%	54.84%
look	164	19480	5	88.35%	96.32%	47.24%
pull	79	5734	7	41.03%	74.36%	5.13%
stop	43	3025	4	52.38%	91.67%	28.57%
take	54	8277	14	37.04%	67.90%	0.00%
throw	29	1940	6	37.93%	72.41%	6.90%
turn	106	8982	10	73.58%	95.28%	70.75%

Table 6.1: Movie Script Corpus Information

However, although some verbs have many senses according to WordNet, these senses may not all occur in the movie script corpus. The only reliable way to determine the sense distributions of the verbs in this corpus is to manually sense-tag the verb instances. Since sense-tagging is a time consuming effort, only eight of the high frequency verbs selected from three scripts were tagged to produce the sense-tagged movie script corpus. The sentences containing the eight selected verbs were sense-tagged by two independent annotators and the final sense tags were determined by an adjudicator. The details of this tagging process are summarised in Table 6.1 which also shows the accuracy of SenseLearner on the chosen verbs. It can be observed from this table that even within a small subset of the movie corpus, the high frequency verbs are already highly polysemous. It can also be observed that the majority class baseline for some of these verbs is rather high, indicating that the sense distribution of some verbs in the movie script is predictably skewed. Furthermore, it can be observed that SenseLearner performed below the majority class baseline for all the verbs, indicating that it was not suited for the highly polysemous verbs of the movie script corpus.

6.1.1 Choice of Verb Sense Disambiguation Strategy

In the previous section, I showed that in the movie script corpus, most of the verbs have low frequency and low polysemy, but there is also a relatively small number of verbs which have both high frequency and high polysemy. Additionally, I showed that SenseLearner, which is one of the top broad coverage WSD systems in the Senseval-2 all words task, performed poorly on the highly polysemous verbs in the movie script corpus. Furthermore, in my review in Chapter 3, I showed that existing supervised WSD systems were not particularly suited to verbs, and they were also not trained

for movie script style data. Therefore, it seems that the best approach is to adopt a hybrid WSD strategy which uses a low coverage, supervised system to disambiguate the small number of highly frequent verbs that have no single, dominant senses in the corpus, and a broad coverage based system (which may employ some unsupervised methods) to disambiguate the rest of the verbs. As discussed in Chapter 3, this hybrid strategy has already been adopted by some existing hybrid systems. In fact, SenseLearner, the chosen broad coverage system for this research, is one such hybrid system. The choice of the supervised system is a selectional preferences based system independently developed as part of this research, which I present in the next section.

6.2 Selectional Preferences Based Verb Sense Disambiguation Using Semantic Role Labelling

The selectional preference based WSD systems reviewed in Chapter 3 have one common limitation: they only use the subject and object of verbs as the set of semantic roles. This is not surprising, because until recently, the only way to reliably extract semantic roles was to use heuristics. Since the current state-of-the-art SRL systems can extract a broad set of verb semantical roles with relative success, it is reasonable to use the output of SRL systems in modelling selectional preferences.

The main contribution of this VSD work is to show that together with general WSD features, it is possible to use selectional preferences to achieve comparable results with current state-of-the-art VSD systems by using semantic roles extracted by an automatic method such as SRL. Since this VSD system is developed as part of a broader framework of visualising action descriptions of movie scripts, it is trained and evaluated on the sense tagged movie script corpus. Furthermore, for comparison purposes, this VSD system has also been trained and evaluated on the Senseval-2 data.

Figure 6.1 shows the data flow and major modules of the VSD system for the movie script corpus. The input to the system is raw text. The raw text is first separated into individual sentences.[2] The individual sentences then undergo a series of pre-processing steps which include part-speech-tagging, parsing, chunking, named-entity-recognition, lemmatisation, and semantic-role-labelling. This is performed with the tools introduced in Chapter 3. After the pre-processing, some limited anaphora resolution is performed on the head nouns of the semantic roles identified in the sentences. The final head noun of each semantic role is then generalised with its WordNet noun synsets, and combined with its semantic role to form selectional preferences. The selectional preferences are then analysed to produce selectional preference features. The selectional preference features are combined with the general WSD features in

[2]The sentence separation is performed using a script which is downloadable from `http://l2r.cs.uiuc.edu/~cogcomp/atool.php?tkey=SS`.

the feature vector construction module to produce the final feature vectors which are used to train a MaxEnt based disambiguation model. Details of these modules will be presented in the remainder of this section.

It is worth noting that the data provided in the Senseval-2 task was already separated into sentences and POS tagged, and the training architecture described in Figure 6.1 is applied only after the POS tagging stage.

6.2.1 General WSD Features

There are three broad types of general (i.e. non-selectional-preference) WSD features: n-gram based features of surrounding words and WordNet noun synsets, parse-tree-based syntactic features, and non-parse-tree based syntactic features.

N-gram based features

In order to illustrate the features with concrete examples, the following pre-processed sentence containing the target verb *call* will be used as an examples to illustrate the features.

(6.1)

Rel. Position	Word	Lemma	POS Tag	Chunk Tag	NER Tag
-6(left)	The	the	DT	B-NP	O
-5(left)	slender	slender	JJ	I-NP	O
-4(left)	,	,	,	I-NP	O
-3(left)	handsome	handsome	JJ	I-NP	O
-2(left)	fellow	fellow	NN	I-NP	O
-1(left)	was	be	VBD	B-VP	O
	called	call	VBN	I-VP	O
1(right)	Dandy	dandy	NNP	B-NP	I-PER
2(right)	Brandon	brandon	NNP	I-NP	I-PER
3(right)	by	by	IN	B-PP	O
4(right)	the	the	DT	B-NP	O
5(right)	other	other	JJ	I-NP	O
6(right)	slaves	slave	NNS	I-NP	O
7(right)	.	.	.	O	O

The following n-gram based features have been investigated:

Bag of Words: Lemmatised open class words in the entire sentence of the target verb. Using the *call* example, each open class word (*slender*, *handsome*, *fellow*, *other*, *slaves*) that occurs in the **Lemma** column will be treated as a separate binary feature. Words that occur multiple times are only counted once.

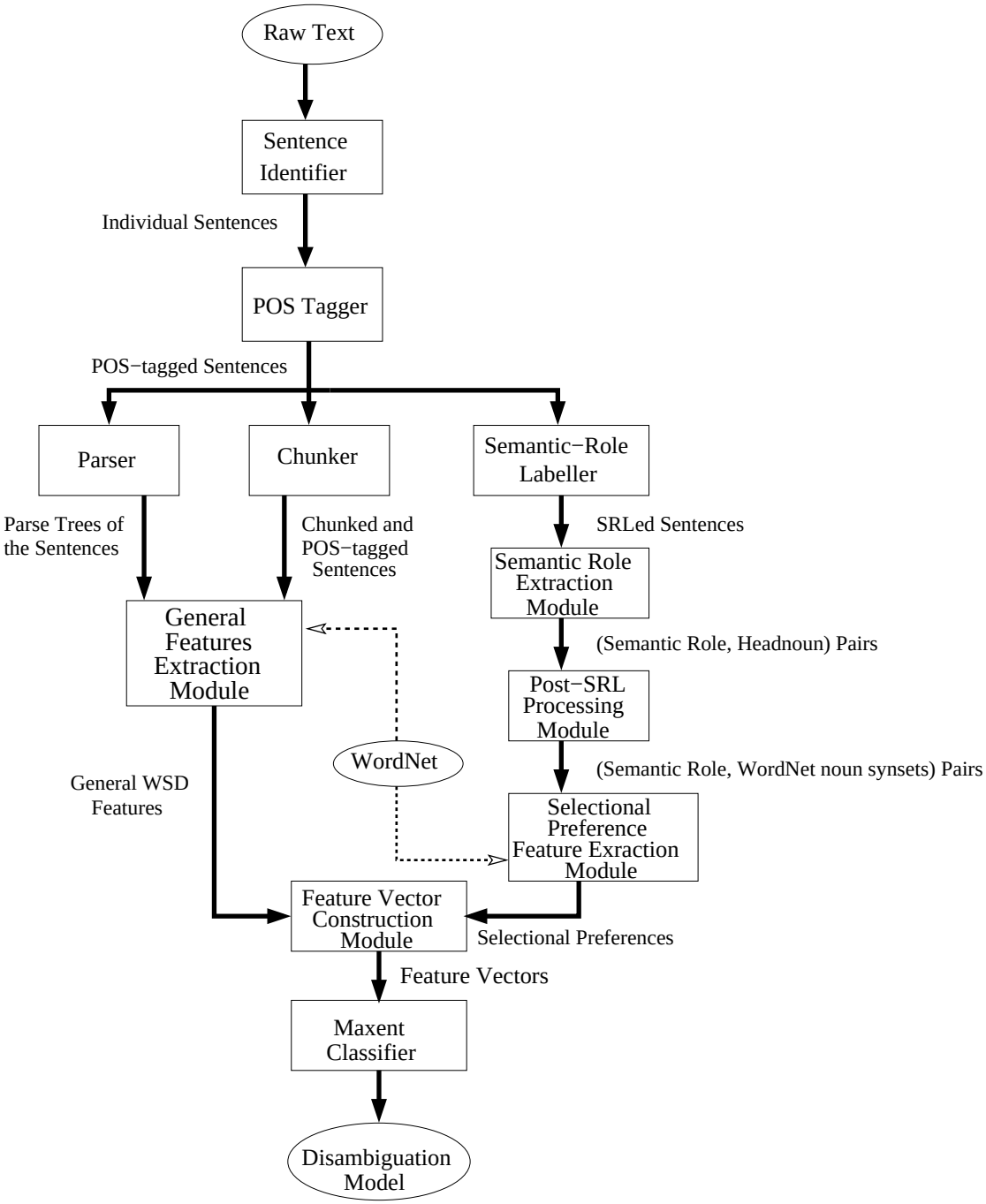


Figure 6.1: Architecture of the VSD system for the movie script corpus data

Bag of Synsets: The WordNet synsets for all the open class words in the entire sentence. Using the *call* example, all the WordNet synsets of the adjectives *slender* and *handsome*, the nouns *fellow* and *slaves* and the verb *call* will used as separate binary features.

Bag of POS Tags: The POS tags surrounding and including the target verb within a window of eleven words. Using the *call* example, each POS tag in the **POS Tag** column between relative positions -5 and 5 will be treated as a separate binary feature indexed by its position.

Bag of Chunk Tags: The chunk tags (in IOB format (Sang and Veenstra 1999)) surrounding and including the target verb within a window of eleven words. Using the *call* example, each chunk tag in the chunk tag column between relative positions -5 and 5 along with its relative position, will be treated as a separate binary feature. Chunk tags that occur multiple times are only counted once.

Bag of Chunk Types: The chunk types (e.g. NP, VP) surrounding and including the target verb within a window of eleven words. The following chunk types and their spans can be found in the *call* example: (**NP**, [-6, -2]), (**VP**, [-1, 0]), (**NP**, [1, 2]), (**PP**, [3, 3]), and (**NP**, [4, 6])). Since the spans of all these chunk types fall in the window of [-5, 5], all of them will be treated as separate binary features.

Bag of Named Entities: The named entities found with the system developed in Cohn *et al.* (2005) in the sentence context of the target verb. Using the *call* example, all the NER tags in the **NER Tag** column will be treated as separate binary features.

Left Words: The lemmatised words and their relative positions to the left of the target verb within a predefined window of three words. Using the *call* example, given a window size of three, the tuples of (-1, *be*), (-2, *fellow*) and (-3, *handsome*) will be each treated as a separate binary feature.

Left Words with Binary Relative Position: The lemmatised words and their binary positions to the left of the target verb within a predefined window of three words. Using the *call* example, given a window size of three, the tuples of (left, *be*), (left, *fellow*) and (left, *handsome*) will be each treated as a separate binary feature.

Right Words: The lemmatised words and their relative positions to the right of the target verb within a window size of three words.

Right Words with Binary Relative Position: The lemmatised words and their binary positions to the right of the target verb within a window size of three words.

Surrounding Words: Lemmatised words with their relative positions to the target verb within a window size of seven words. Using the *call* example, each word that occurs between positions −3 and 3 in the **Lemma** column, along with its relative position, will be treated as a separate binary feature.

Surrounding Words with Binary Relative Position: Lemmatised words with their relative positions to the target verb within a window size of seven words. Using the *call* example, given a window size of seven, each word that occurs between positions −3 and 3 in the **Lemma** column, along with its binary relative position ("left" if the relative position is negative and "right" if the relative position is positive), will be treated as a separate binary feature.

Left POS-tags: POS tags of the words and their relative positions to the left of the target verb within a window size of three words. Using the *call* example, given a window size of three, the tuples (-1, **VBD**), (-2, **NN**) and (-3, **JJ**) will each be treated as a separate binary feature.

Right POS-tags: POS tags of the words and their relative positions to the right of the target verb within a window size of three words. Using the *call* example, given a window size of three, the tuples (1, **NNP**), (2, **NNP**) and (3, **IN**) will each be treated as a separate binary feature.

Surrounding POS Tags: POS tags of the words surrounding the target verb within a window size of seven words, also with their relative positions to the target verb. Using the *call* example, given a window size of seven, each POS tag in the **POS Tag** column between relative positions -3 and 3, along with its relative position, will be treated as a separate binary feature.

It may seem redundant that for the same window size, the Surrounding-Words (POS) features are a superset of both the Left-Words (POS) features and the Right-Words (POS) features. However, this overlapping of features makes it convenient to investigate the disambiguation effectiveness of the word collocations before and after the target verb, as well as the syntactic pattern before and after the target verb. Furthermore, different window sizes for the Surrounding-Words (POS), Left-Words (POS) and Right-Words (POS) are investigated separately to determine the most appropriate window size. In the ranking based evaluation method described below, only the Surrounding-Words (POS) feature types with the largest window size are used.

Parse tree based features

The parse tree based syntactic features are inspired by research on verb subcategorization acquisition (e.g. Korhonen and Preiss (2003)), and it is hoped that these

features can capture the differences in syntactic patterns of the different senses of the same verb. Given the position of the target verb v in the parse tree, the basic form of the corresponding parse tree feature is just the list of nodes of v's siblings in the tree. Figure 6.2 shows the parse tree for the sentence in the *call* example. Given the position of the target verb *called* in the parse tree, the basic form of the features that can be created will be (NP, PP). However, there are 3 additional types of variations that have to be applied to the basic features. The first variation is to include the relative positions of the sibling node types as part of the feature. This variation will change the basic feature for *call* to $\{(1, NP), (2, PP)\}$. The second variation is to include the binary relative direction of the siblings to the target verb as part of the feature, i.e. is the sibling to the left or right of the target verb. This variation will change the basic feature for *call* to $\{(r, NP), (r, PP)\}$. The third variation is to include the parent node type as part of the sibling node type to add more information on the syntactic pattern. Figure 6.3 shows what the original parse tree looks like when every nonterminal is additionally annotated with its parent type. Since the third variation is compatible with the first two variations, I decided to combine them to create the following parse tree based features:

Type 1: Original sibling node types (zero level of parent node annotated) with relative positions. Using the *call* example, this feature will be $\{(1, NP), (2, PP)\}$.

Type 2: Original sibling node types with relative binary directions. Using the *call* example, this feature will be $\{(right, NP), (right, PP)\}$.

Type 3: One level of parent node annotated sibling node types with relative positions. Using the *call* example, this feature will be
$\{(1,$ *VP-NP*$), (2,$ *VP-PP*$)\}$.

Type 4: One level of parent node annotated sibling node types with relative binary directions. Using the *call* example, this feature will be
$\{(right,$ *VP-NP*$), (right,$ *VP-PP*$)\}$.

Type 5: Two levels of parent node annotated sibling node types with relative positions. Using the *call* example, this feature will be
$\{(1,$ *VP-VP-NP*$), (2,$ *VP-VP-PP*$)\}$.

Type 6: Two levels of parent node annotated sibling node types with relative binary directions. Using the *call* example, this feature will be
$\{(right,$ *VP-VP-NP*$), (right,$ *VP-VP-PP*$)\}$.

On top of these six types of purely syntactic based parse tree features, there is an additional type of parse tree based feature designed to capture the **verb-argument**

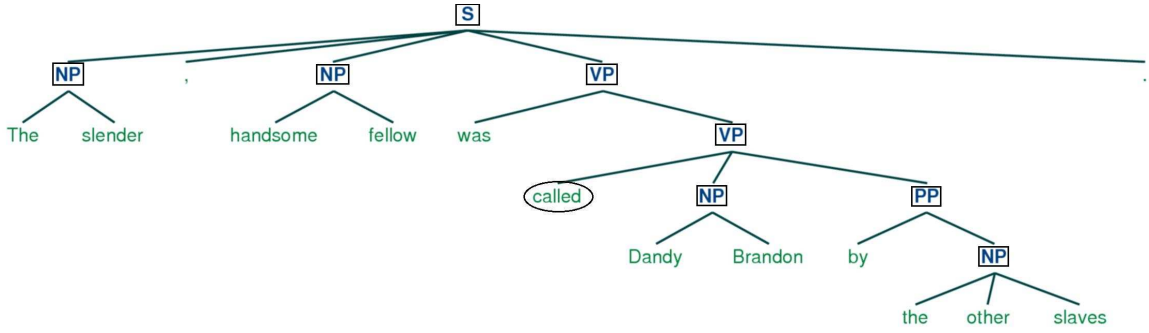


Figure 6.2: Basic parse tree feature example

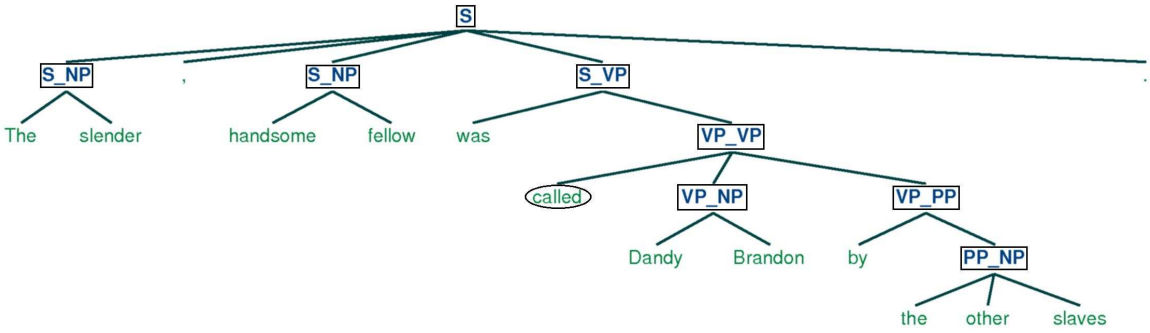


Figure 6.3: One level parent annotated parse tree

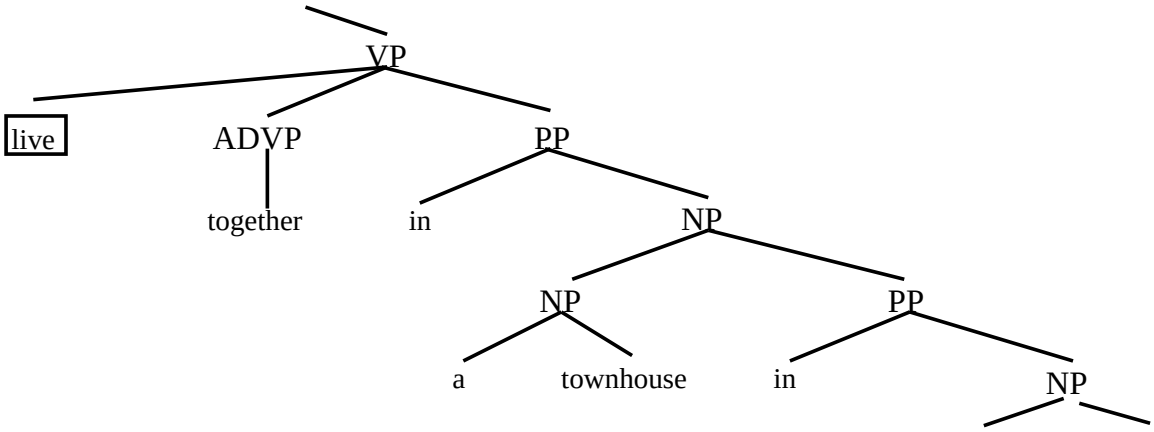


Figure 6.4: Verb-argument-structure feature: ADVP example

structure of the target verb. The basic intuition behind this type of features is that verbs tend to exhibit stronger selectional preferences with words to their immediate right, and these preferences may be represented by the syntactic and semantic properties of the first argument to the right of the verbs. The verb-argument structure features only cover five particular types of parse tree nodes which are immediately after the pre-terminal node of the target verb. The five types of parse tree nodes are: ADVP, NP, PP, particle nodes and clausal nodes.

For an ADVP node, its head adverb *H_ADV* will be extracted, and the tuple of (**ADVP**, *H_ADV*) will be treated as a separate binary feature. For example, given the parse tree in Figure 6.4, this feature will be (**ADVP**, *together*).

For an NP node, such as the one shown in Figure 6.5, first its head noun will be extracted, then the head noun will be replaced with its WordNet synsets and the hypernyms of these synsets; each of these synsets will be treated as a separate binary feature.[3] In order to cover the cases in which the head noun of a NP node is a quantity noun, e.g. *a glass of water*, the head nouns of PPs attached to the NP nodes are also included as head nouns. Figure 6.6 is an example of the NP-PP type of parse trees. Furthermore, head nouns which are named entities identified by the system described in Cohn *et al.* (2005) are replaced by appropriate WordNet synsets.

For a clausal node which is an SBAR, we extract the list of node types of its children and arrange them in their original order, and treat this list as a single binary feature. If a child node is a relative pronoun, then its parse tree node type will be replaced by the relative pronoun itself. If a child node is a terminal, then its node type will be replaced by its POS tag. For example, given the parse tree in Figure 6.7,

[3]If the correct sense of the head noun is known, then only the synsets of the correct senses is included.

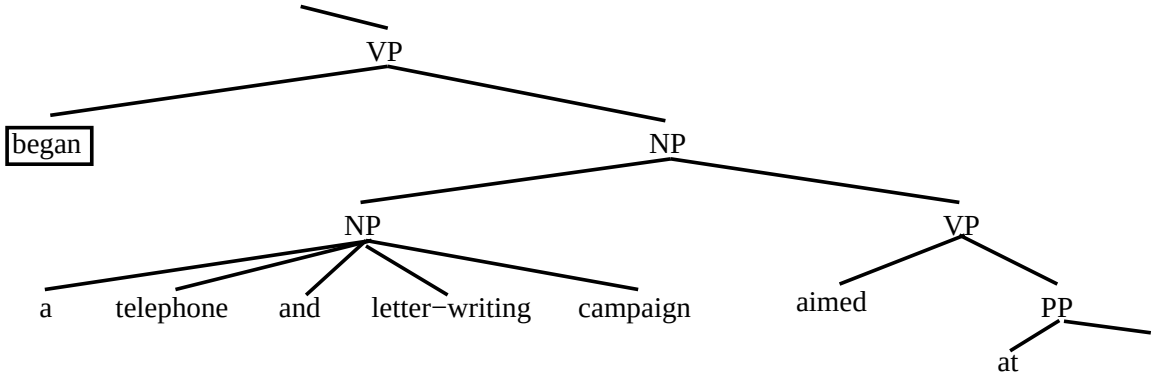


Figure 6.5: Verb-argument-structure feature: NP example 1

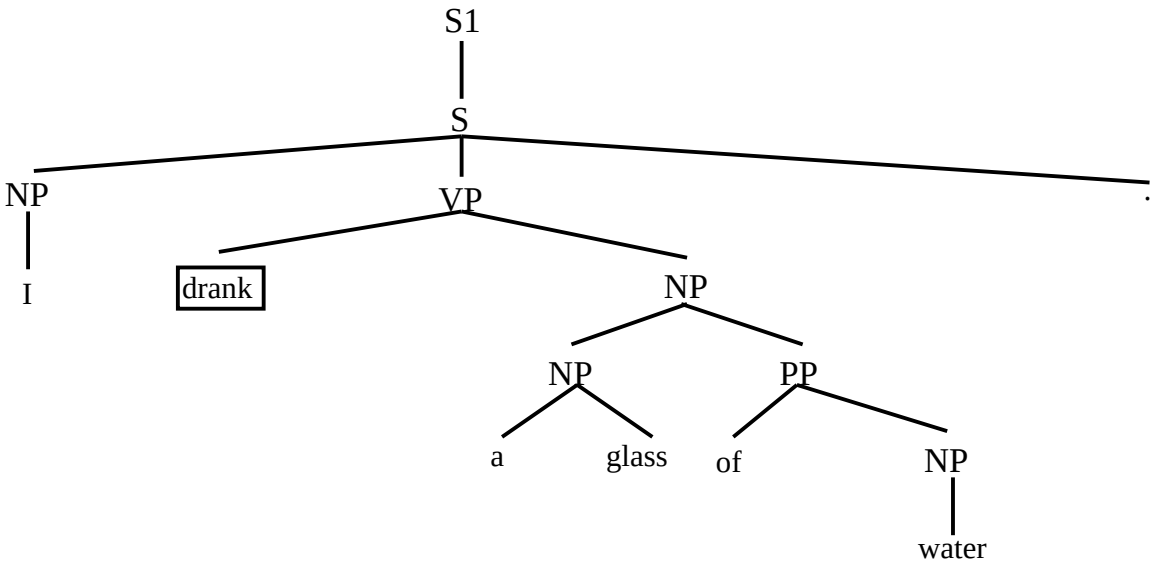


Figure 6.6: Verb-argument-structure feature: NP example 2

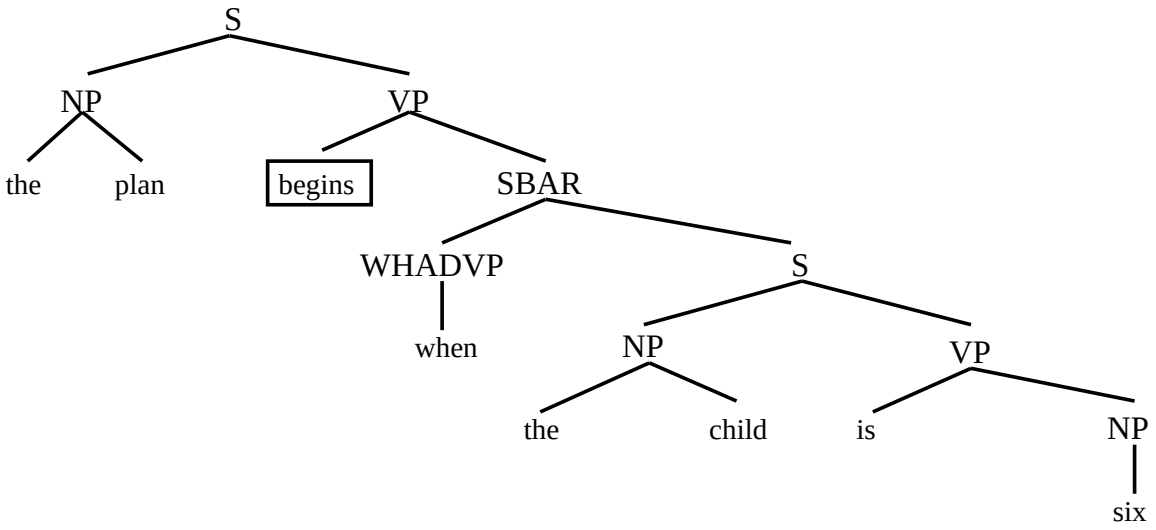


Figure 6.7: Verb-argument-structure feature: SBAR clause example

this feature will be (**SBAR**, (*when*, *S*)).

For a clausal node which is not an SBAR, but has a single non-terminal child node, we first extract the type of this child node, then we extract the list of node types for the children of this child node. The tuple of (child-node-type, list-of-grandchildren-node-types) is then treated as a separate binary feature. If a grandchild node is a preposition nonterminal, then its parse tree node type will be the preposition itself; if a grandchild node is a non-preposition terminal, then its parse tree node type will be the terminal's POS tag . For example, given the parse tree in Figure 6.8, this feature will be (**S**, (*VP*, (*to*, *VP*))).

For a PP node, first its head preposition will be extracted, then the PP's head noun's WordNet synsets will be extracted in the same way as the NP node, and finally each synset will be combined with the head preposition to form a separate binary feature. For example, Figure 6.9 shows a parse tree in which the PP-attachment of the target verb *lived* contains a head preposition *in* and an NP which has a single head noun *Britain*. Since *Britain* has a `LOC` NE tag, it will be replaced as a set of WordNet noun synsets which are the noun synset location_1 and its hypernyms. Each of these WordNet synsets will then be combined with *in* to form a separate binary feature. If the PP has a clausal child instead of an NP, the NP features will be extracted according to the rules explained above and combined with the head preposition to form a single binary feature.

For a particle node (PRT), if its immediate post-sibling is one of the above four types of nodes, such as the one shown in Figure 6.10, then the above rules would be applied to extract the appropriate features from this sibling node, and each of

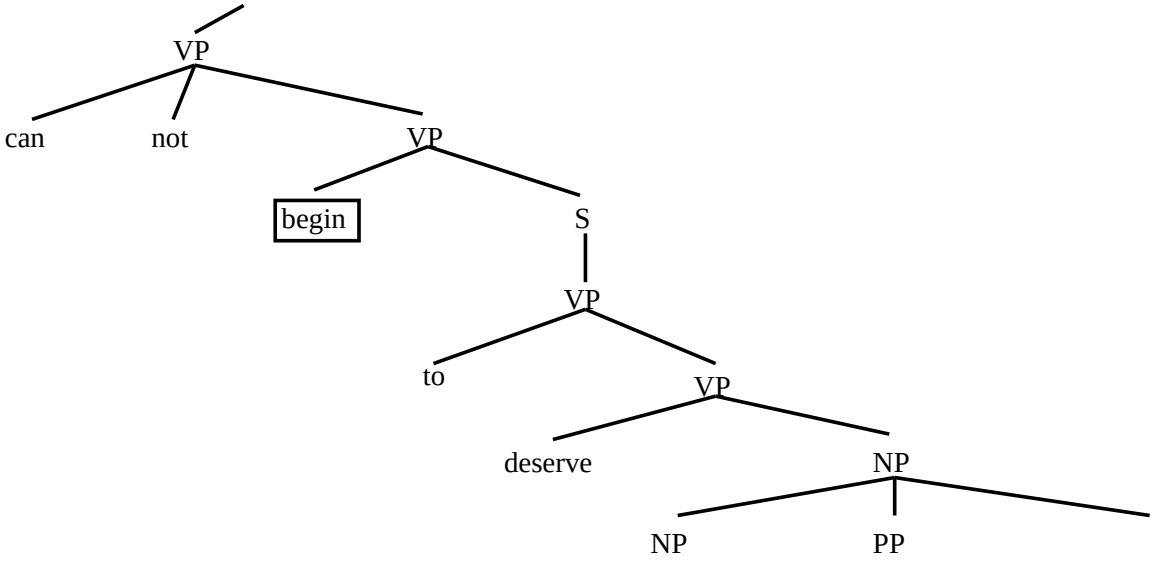


Figure 6.8: Verb-argument-structure feature: non-SBAR clause example

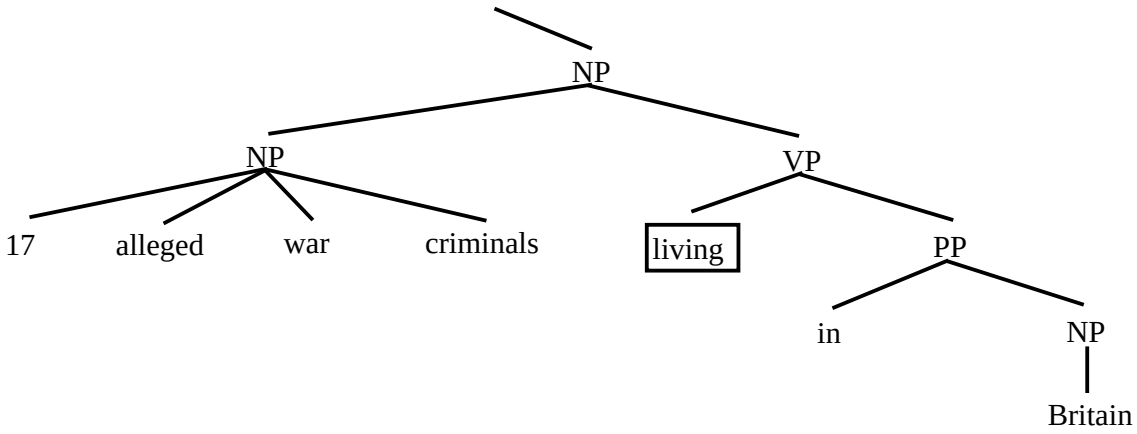


Figure 6.9: Verb-argument-structure feature: PP example 1

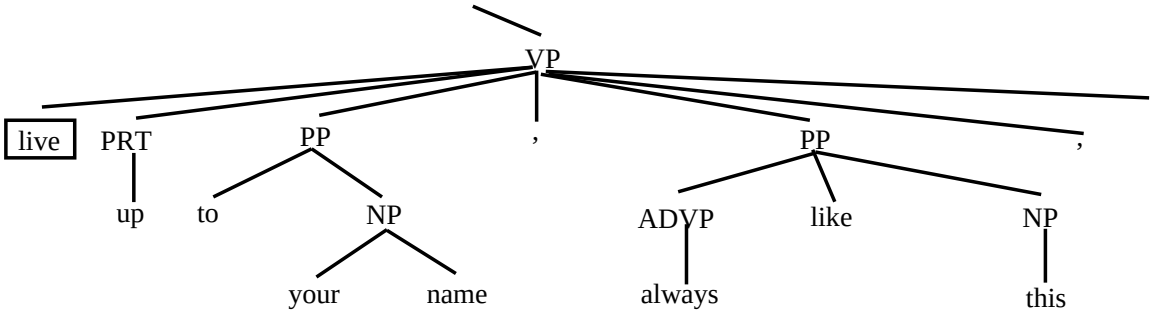


Figure 6.10: Verb-argument-structure feature: Verb Particle example

these features will be combined with the word form of the particle. Otherwise, the word-form of the particle itself will be a separate binary feature.

Morphosyntactic features

There are 3 types of non-parse-tree based syntactic features:

Voice of the verb: The voice of the verb determines the relative position between the surface subject and object, so this feature may provide additional disambiguation information.

Whether the verb is direct quotation of human speech: Verbs that appear in directly quoted speech can sometimes occur in imperative voice and without an overt subject, e.g *Call the police!*. Also, not every sense of a given verb is equally likely to be used in imperative, so this feature may provide additional word sense disambiguation clues.

Additional Chunk based features: A number of additional chunk based features are also used to capture phrase level localised syntactic and collocation patterns in the context to the right of the target verb within a window of between three and ten chunks. For example, using a window of seven, for the verb *kick* in the sentence: *[I/PRP]$_{NP}$ [***kicked***/VBD]$_{VP}$ [him/PRP]$_{NP}$ [out/IN of/IN]$_{PP}$ [the/DT door/NN]$_{NP}$ [through/IN]$_{PP}$ [which/WDT]$_{NP}$ [he/PRP]$_{NP}$ [came/VBD in/IN]$_{VP}$*, the first seven chunks after the chunk that contains *kick* will be used to for feature extraction. These additional chunk based features are:

Chunk-type-sequence: This type of feature is formed by just taking the concatenation of all the relevant chunk types. For example, using the above *kick* example,

this feature will be NP_PP_NP_PP_NP_NP_VP.

Regular expression (RE) representation of the chunk types: This feature is created by merging consecutive identical chunk types in the Chunk-type-sequence feature into a single symbol. For example, the chunk-type-sequence of NP_PP_NP_PP_NP_NP_VP will be represented as NP_PP_NP_PP_NP+_VP.

The first and last word of each chunk with the chunk type

The first and last part of speech tag of each chunk with the chunk type

Chunk-type-sensitive combinations: This feature is created by merging the chunk types and some additional information associated with the chunks. If a chunk is a PP, then the head preposition will also be part of the feature; if a chunk is a NP, and the head word is a question word, then the head word itself will be part of the feature, but if the head word is not a question word, its POS tag will be part of the feature; if a chunk is a VP, then the POS tag of the head verb will be part of the feature. Using the above *kick* example, this feature will be: (NP, PRP)_(PP, out)_(NP, NN)_(PP, through)_(NP, which)_(NP, PRP)_(VP, VBD).

6.2.2 Selectional Preference Based Features

In order to illustrate the features with concrete examples, the following pre-processed sentence containing the target verb *faces* will be used as a source of examples for the features.

(6.2)

Rel. Position	Word	Lemma	NE-tag	Semantic Role Tag
-11(left)	There	there	O	O
-10(left)	's	be	O	O
-9(left)	a	a	O	O
-8(left)	certain	certain	O	O
-7(left)	irony	irony	O	O
-6(left)	in	in	O	O
-5(left)	the	the	O	O
-4(left)	fact	fact	O	O
-3(left)	that	that	O	O
-2(left)	Federal	federal	I-ORG	A0
-1(left)	Express	express	I-ORG	A0
	faces	face	O	TARGET
1(right)	its	its	O	A1
2(right)	first	first	O	A1
3(right)	union	union	O	A1
4(right)	problems	problem	O	A1
5(right)	as	as	O	AM-CAU
6(right)	a	a	O	AM-CAU
7(right)	result	result	O	AM-CAU
8(right)	of	of	O	AM-CAU
9(right)	its	its	O	AM-CAU
10(right)	Tiger	tiger	I-ORG	AM-CAU
11(right)	purchase	purchase	O	AM-CAU
12(right)	.	.	O	O

The following selectional preference based features have been experimented with:

WordNet synsets for the head nouns of each semantic role Since each semantic role is associated with one or more WordNet synsets, each synset is combined with the semantic role to form a separate feature. Given the example sentence, there are three semantic roles: A0, A1 and AM-CAU, each with the head noun of *express*, *problem* and *result* respectively. Suppose $express_1$ and $express_2$ were the WordNet synsets for *express*, $problem_1$ and $problem_2$ were the WordNet synsets for *problem*, and $result_1$ and $result_2$ were the WordNet synsets for *result*, then each tuple of (A0, $express_i$), (A1, $problem_j$) and (AM-CAU, $result_k$) will become a binary feature. The same process of extracting WordNet synsets from the head nouns described as part of the Verb-Argument-Structure feature is used here.

Relative positions of the semantic roles to the target verb These features are designed to capture the syntactic patterns of the target verb and its semantic roles. The relative position is set up such that the first semantic role to the left of the target verb will be given the position of -1, and the first one to the right will be given the position of $+1$. The example sentence will produce the following binary features: (**A0**, -1), (**A1**, $+1$), and (**AM-CAU**, $+2$).

Lemmatised head noun of each semantic role Similar to the synset features, each semantic role is also paired with its head noun to form a separate feature. The example sentence will produce the following binary features: (**A0**, *organization*), (**A1**, *problem*), and (**AM-CAU**, *result*).

Lemmatised and anaphora referenced head noun of semantic roles Similar to the synset features, each semantic role is also paired with its head noun to form a separate feature. If the head noun is a pronoun, it will first undergo anaphora resolution.

Preposition of the adjunct semantic role If there is an adjunct semantic role which is also a prepositional phrase, the preposition is also paired with the semantic role to form a separate feature. The example sentence will produce the binary feature of (**AM-CAU**, *as*).

6.2.3 System Evaluation

This section provides an analysis of the features and shows their strengths and weaknesses. The evaluation is performed on two datasets: the movie script corpus and Senseval-2. The aim of this section is to investigate the effectiveness of the features explained in the previous section for disambiguating the verbs in movie scripts. However, as the size of the movie script corpus may be too small to be convincing, Senseval-2 dataset was also used for evaluation so that the disambiguation system developed in this research can be compared with other systems.

During the experiments, we observed that combining certain features together can lead to a decrease in classification accuracy, despite the longer training and evaluation time. Hence a feature selection process is deemed necessary. Due to the high numbers of individual binary features and feature categories, it would be impractical to generate all possible combinations of the single features, or categories of features, and therefore, an automatic method must be devised to perform this feature selection task. Three automatic feature selection heuristics have been investigated: individual feature ranking, top performing feature categories, and feature type coverage. The rest of this section will explain these heuristics in detail.

The selection of features is not the only factor that can affect the performance of the VSD system. Since the final WSD system is trained and evaluated using a Maximum Entropy (Berger *et al.* 1996) classifier,[4] the effectiveness of the disambiguation models are partially determined by the value of their Gaussian prior smoothing parameter g, which specifies how much probability mass is assigned to data unseen during training. Since Senseval-2 does not provide a separate development data, the

[4] `http://homepages.inf.ed.ac.uk/s0450736/maxent_toolkit.html`

final smoothing parameter is tuned over the training set using ten-fold cross validation.

Individual Feature Ranking Heuristic

This heuristic works by first calculating the following 3 scores for each binary feature:

1. Information gain (IG)
2. Gain ratio (GR)
3. Chi-square (Chi) [5]

Once these statistics have been calculated for each feature, the features are then separately ranked according to each score function. The ranking is performed such that the greater the statistic, the higher the rank. Features which have the same value for a particular score are given the same rank, however, the number of ranks is the same as the number of total features. For example, suppose there are three features A, B and C, where both A and B have the same information gain of 0.4, and C has an information gain of 0.1. Here the information-gain-based rank for A, B and C would be the following:

Feature	Info. Gain	Rank
A	0.4	1
B	0.4	1
C	0.1	3

Once individual ranks have been determined for each feature, the ranks are summed and used as a new score which is then used to re-rank all the features. This final ranking is used to perform the feature selection. Continuing with the above example, suppose the gain ratio values of A, B and C are 3.1, 2.0 and 2.5 respectively, and the Chi-square values of A, B and C are 2.1, 1.0 and 2.3 respectively, then the final ranking of A, B and C would be:

Feature	IG Rank	GR Rank	Chi Rank	Rank-sum	Final Rank
A	1	1	2	4	1
C	3	2	1	6	2
B	1	3	3	7	3

[5]There is no need to calculate the shared variance statistic for the features, as all the features are binary.

Data Instance ID	Bag of words	Right words window 3	Parse tree feature type 1
1	1	0	0
2	0	0	1
3	0	1	1
4	0	1	1
5	0	1	1

Table 6.2: Example of the feature type coverage algorithm

Once the final ranking of the features has been generated, the selection process starts. First, the top N ranked features are used to generate separate datasets from the training set, where N ranges from 0% to 100% with a 5% increment. Then each separate training dataset is evaluated using ten-fold stratified cross validation. Each fold is then separately trained and evaluated using a MaxEnt classifier with the smoothing parameter g from the set $G = [0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0, 500.0]$. The combination of (top-N-percent, g_i) that produces the best cross-validation result is then chosen to train the final MaxEnt model used to classify the instances in the test set. If the best cross-validation result is produced by multiple tuples of (top-N-percent, g_i), then the tie is broken by first selecting the tuple with the largest N (for more feature coverage), then selecting the tuple with the smallest g_i (for more faithfulness to the training data). It should be pointed out that when $N = 0$, no features are used, and the resulting classifiers will classify every instance with the majority class of the training set.

Feature Type Coverage Heuristic

This heuristic is also based on the observation that overly many features may lead to a decrease in disambiguation performance. The aim of this heuristic is to use the minimal number of feature types to generate the best performance on the development dataset. Figure 6.2 shows an example of how the information used by this heuristic is collected. Suppose there are three types of features: bag of words, right words with window size of three, and the type 1 parse tree feature, and that the right words features outperforms the bag of words features, and the combination of bag of words and type 1 parse tree features outperforms any other combinations. Here, this pairing would be chosen by the feature type coverage heuristic.

The application of the feature coverage heuristic is straightforward when a development dataset is available. However, in the absence of a development set, it is still possible to carry out cross validation within the training set with this heuristic.

1: Let I be the set of IDs of the training instances
2: Let $Coverage_1$ and $Coverage_2$ be the two coverages to be merged
3: Let *NewCoverage* be the combined coverage of $Coverage_1$ and $Coverage_2$
4: **for** $i \in I$ **do**
5: $\quad NewCoverage[i] = \max(Coverage_1[i], Coverage_2[i])$
6: **end for**
7: Return *NewCoverage*

Algorithm 4: The **Combine** Operator that combines two separate coverages

Given a dataset, N-fold cross validation splits it into N pairs of training and held-out data. The held-out data can then act as the development set. Since the union of the held-out sets equals the original dataset, if a unique ID is associated with each data instance, it can be traced into the held-out sets, thus making it possible to keep track of how well each feature type performs with respect to each data instance.

The feature type coverage can be defined as a hash table which maps the IDs of the training instances to an integer (0/1) which represents whether it was correctly classified in the held-out sets during cross validation. On top of the coverage, the percentage of the training instances for which at least one feature from the particular feature type can be extracted is also calculated; this percentage represents the applicability of the particular feature type. The final phase of feature selection is then a process of incrementally combining the coverages of feature types together (the combine operator is described in Algorithm 4) to achieve a local maximum of total *coverage*. This process is described in Algorithm 5.

6.2.4 Evaluation of the feature selection heuristics

Figures B.1 to B.28 (in the Appendix A) present the individual verb disambiguation results for the Individual Feature Ranking (IFR) heuristic on the Senseval-2 dataset. The following three curves are shown in each graph:

- Cross validation accuracy on the training set using the best MaxEnt smoothing parameter.
- Test set accuracy using the same MaxEnt Gaussian prior tuned by the training set cross validation.
- Test set accuracy using MaxEnt Gaussian prior that was tuned on the test set.

The most prominent feature of these graphs is that, for most verbs, the optimal features and the optimal MaxEnt smoothing parameters tuned with the training

Let I be the set of IDs of the training instances
$CurrentCoverage = \{(i \rightarrow 0.0) | i \in I\}$
Let F be the set of feature types
while True **do**
    $NCs \leftarrow \{\}$ ▷ Initialize a temporary list to hold the new combined coverages
    for $f_i \in F$ **do**
        $NewCoverage_i \leftarrow Combine(CurrentCoverage, Coverage_{f_i})$
        Add $NewCoverage_i$ to NCs
    end for
    Let $NewCoverage^\star \in NCs$ be the one with highest overall coverage. ▷ Break tie with the lowest applicability.
    if $CurrentCoverage$ has the same overall coverage as $NewCoverage^\star$ **then**
        Break
    end if
    $CurrentCoverage \leftarrow NewCoverage^\star$
end while

Algorithm 5: The Incremental Process of Determining the Final Set of Feature Types

Gaussian prior tuning metrics	Feature selection metrics		
	Oracled	Auto.	None
Oracled	Fully Oracled	MaxEnt Oracled	N/A
Auto	Feature Oracled	Fully Auto	All Features

Table 6.3: VSD system evaluation scheme

set are in fact suboptimal with respect to the test set. Furthermore, it can also be observed that the performance differences caused by the feature mismatch are significantly greater than those caused by the MaxEnt smoothing parameter mismatch.

Since the disambiguation accuracy is determined by both the feature selection algorithm and the tuning of the MaxEnt Gaussian prior, it is useful to investigate the upper bounds of their impact on the final VSD system. These upper bounds can be established through a combination of the following two metrics:

- Oracled: Optimised feature selection of Gaussian prior over the test set
- Auto.: Optimised feature selection of Gaussian prior over the training set

Table 6.3 shows all the combinations of the above two metrics used in this study.

Tables 6.4 and 6.5 show that the performance increment as a result of oracled feature selection is around twice the increment caused by the oracled MaxEnt smoothing

parameter, confirming the hypothesis that feature selection mismatch has greater impact on the system performance than MaxEnt parameter mismatch. It can also be observed that the feature ranking heuristic does not work as well as the feature type coverage heuristic, or even having no feature selection process at all.

Verb	Heuristics							Chen and Palmer
	Individual Feature Ranking					Feature Coverage		
	Fully Oracled	Feature Oracled	MaxEnt Oracled	Fully Auto.	All Features	MaxEnt Oracled	Fully Auto.	
begin	90.00	89.64	87.50	87.14	85.36	89.29	89.29	91.20
call	45.45	43.94	37.88	37.88	43.94	53.03	53.03	56.80
carry	45.45	43.94	33.33	31.82	34.85	40.91	40.91	44.70
collaborate	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00
develop	56.52	55.07	39.13	34.78	55.07	50.72	50.72	49.30
draw	31.71	31.71	31.71	26.83	29.27	34.15	34.15	41.50
dress	64.41	62.71	61.02	55.93	57.63	59.32	59.32	64.40
drift	46.88	46.88	46.88	46.88	37.50	40.63	37.50	67.20
drive	57.14	57.14	57.14	57.14	54.76	50.00	50.00	60.70
face	86.02	86.02	84.95	84.95	86.02	83.87	83.87	81.20
find	38.24	33.82	36.76	33.82	32.35	30.88	30.88	41.20
keep	59.70	59.70	25.37	25.37	43.28	47.76	47.76	64.20
leave	54.55	53.03	51.52	48.48	51.52	60.61	57.58	57.60
live	62.69	59.70	58.21	58.21	55.22	55.22	53.73	69.40
match	59.52	57.14	47.62	47.62	54.76	54.76	54.76	59.50
play	57.58	57.58	53.03	53.03	50.00	53.03	48.48	62.10
pull	40.00	40.00	26.67	23.33	26.67	30.00	30.00	58.30
replace	60.00	60.00	57.78	53.33	51.11	60.00	55.56	61.10
see	43.48	43.48	37.68	36.23	42.03	42.03	42.03	44.20
serve	70.59	70.59	60.78	60.78	70.59	66.67	66.67	68.60
strike	42.59	40.74	24.07	22.22	40.74	42.59	42.59	51.90
train	69.84	69.84	58.73	58.73	66.67	58.73	57.14	60.30
treat	70.45	70.45	70.45	70.45	63.64	52.27	52.27	50.00
turn	40.30	40.30	29.85	25.37	29.85	34.33	34.33	48.50
use	71.05	71.05	71.05	69.74	68.42	72.37	72.37	69.70
wander	82.00	82.00	82.00	82.00	82.00	80.00	80.00	82.00
wash	58.33	58.33	33.33	33.33	50.00	50.00	50.00	75.00
work	43.33	40.00	35.00	28.33	38.33	40.00	40.00	53.30
average	62.33	61.50	55.57	54.02	57.45	58.84	58.28	**64.59**

Table 6.4: Overall results of the feature selection heuristics for Sensval-2

Verb	Heuristics							Majority Class Baseline
	Individual Feature Ranking					Feature Coverage		
	Fully Oracled	Feature Oracled	MaxEnt Oracled	Fully Auto.	All Features	MaxEnt Oracled	Fully Auto.	
hit	72.86	70.00	66.67	60.95	67.14	55.24	52.38	43.8
lift	73.81	64.76	45.71	40.00	67.62	61.43	52.86	54.8
look	92.03	91.42	55.06	55.06	90.19	92.65	92.05	88.4
pull	63.68	56.32	57.50	53.82	45.07	56.40	51.47	41.0
stop	76.11	73.61	61.94	57.50	59.17	56.94	56.94	52.4
take	60.61	56.97	55.61	48.33	40.91	50.15	33.79	37.0
throw	64.76	58.10	48.57	45.24	55.24	57.62	47.62	37.9
turn	81.26	80.35	77.62	77.62	78.53	78.53	78.53	73.6
Overall	77.66	74.60	60.27	57.80	69.29	71.51	67.76	61.7

Table 6.5: Overall results of the feature selection heuristics for MSC

Usefulness of Multi-Semantic-Role Based Selectional Preferences Features

The key assumption of this VSD research is that multi-semantic-role based selectional preferences can contribute positively to disambiguation performance. The results shown in Table 6.4 indicate that even with the fully oracled features and MaxEnt Gaussian prior values, the final VSD system developed in this study still performs significantly poorer than Chen and Palmer's system (Chen and Palmer 2005). However, efforts made to partially replicate Chen and Palmer's system were not able to reproduce any comparable results. Chen and Palmer's system reported an F-score of 58.0% on the test set of Senseval-2 using only the bag of words features and the surrounding words features, but in this study, the same features only produced an F-score of 53.0%. This indicates that Chen and Palmer's system may be an inappropriate baseline for evaluating the VSD system developed in this research.

One way to assess the influence of SRL based features is to look at how much disambiguation information they carry. Since the individual feature ranking heuristic ranks all the features based on each feature's disambiguation information, a convincing positive argument could be made for SRL features if most of them occurred among the top ranked features. An analysis of the rankings of the features shows that for both the Senseval-2 and the movie script datasets, the distributions of the SRL features are such that their representation in the top 10% ranked features is approximately 5% than their overall proportion among all the features, and their representation in the bottom 10% ranked features is approximately 7% lower than their overall proportion. This shows that the original assumption about the SRL features' disambiguation power is correct.

A more comprehensive way to examine the effect of multi-semantic-role features on the performance of the VSD system is to take them out of the entire feature set, and observe the revised results. Figure 6.11 and 6.12 show side-by-side comparisons of how the VSD system performed with and without the SRL features on the Senseval-2 dataset and the MSC dataset respectively. Tables 6.6 and 6.7 show how the VSD system performed without the SRL features on the Senseval-2 dataset and the MSC dataset respectively, as well as the differences between with the SRL features and without the SRL features. Furthermore, in these two tables, verbs that have overall lower performance without the SRL features are marked with $^{-}$, verbs that have overall better performance with the SRL features are marked with $^{+}$, and verbs that are not affected by SRL features are unmarked.

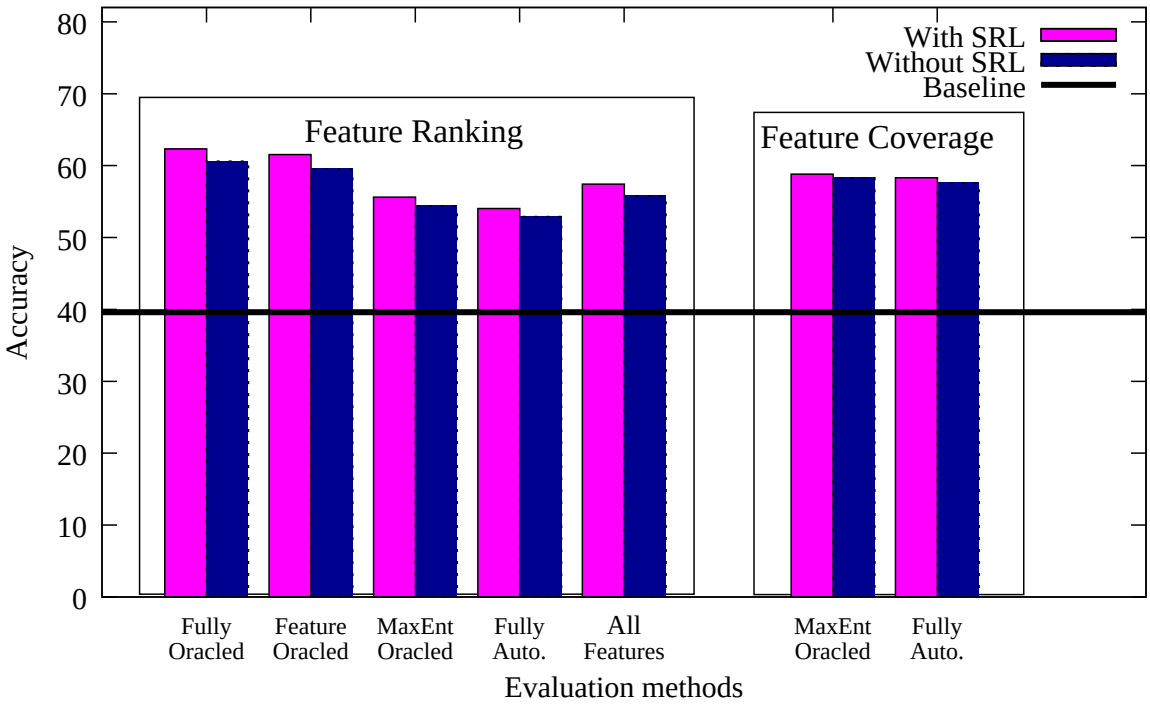


Figure 6.11: Comparisons of the VSD performance with and without the SRL features on Senseval-2 dataset

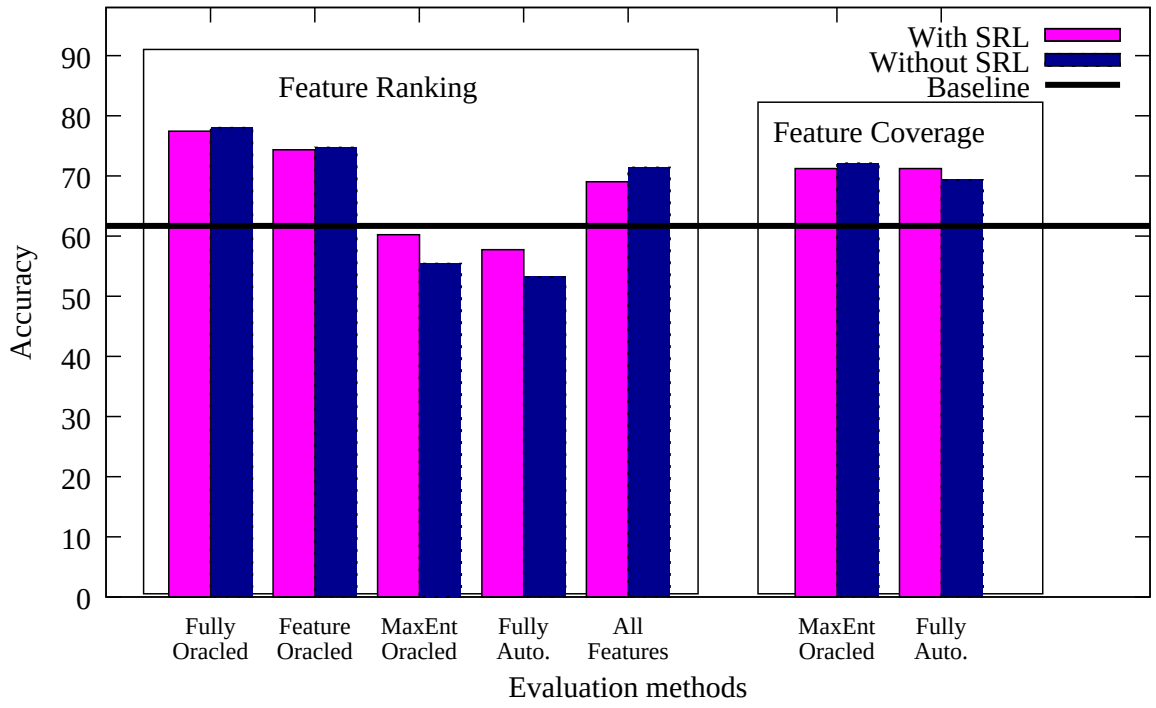


Figure 6.12: Comparisons of the VSD performance with and without the SRL features on the MSC dataset

Verb	Heuristics							Chen and Palmer
	Individual Feature Ranking					Feature Coverage		
	Fully Oracled	FS Oracled	MaxEnt Oracled	Fully Auto.	Most Features	MaxEnt Oracled	Fully Auto.	
begin$^-$	88.21(-1.79)	88.21(-1.43)	85.00(-2.50)	84.64(-2.50)	84.64(-0.72)	90.00(0.71)	89.64(0.35)	**91.20**
call$^+$	46.97(1.52)	45.45(1.51)	37.88(0.00)	37.88(0.00)	40.91(-3.03)	**57.58**(4.55)	56.06(3.03)	56.80
carry	42.42(-3.03)	39.39(-4.55)	37.88(4.55)	34.85(3.03)	39.39(4.54)	40.91(0.00)	40.91(0.00)	**44.70**
collaborate	**90.00**(0.00)	**90.00**(0.00)	**90.00**(0.00)	**90.00**(0.00)	**90.00**(0.00)	**90.00**(0.00)	**90.00**(0.00)	**90.00**
develop$^-$	**50.72**(-5.80)	49.28(-5.79)	40.58(1.45)	40.58(5.80)	46.38(-8.69)	47.83(-2.89)	46.38(-4.34)	49.30
draw$^-$	31.71(0.00)	31.71(0.00)	21.95(-9.76)	21.95(-4.88)	31.71(2.44)	31.71(-2.44)	31.71(-2.44)	**41.50**
dress$^+$	62.71(-1.70)	62.71(0.00)	61.02(0.00)	59.32(3.39)	61.02(3.39)	61.02(1.70)	61.02(1.70)	64.40
drift$^+$	46.88(0.00)	43.75(-3.13)	46.88(0.00)	43.75(-3.13)	40.63(3.13)	46.88(6.25)	46.88(9.38)	**67.20**
drive$^-$	54.76(-2.38)	52.38(-4.76)	52.38(-4.76)	52.38(-4.76)	47.62(-7.14)	50.00(0.00)	47.62(-2.38)	**60.70**
face	**86.02**(0.00)	**86.02**(0.00)	**86.02**(1.07)	84.95(0.00)	84.95(-1.07)	83.87(0.00)	83.87(0.00)	81.20
find$^-$	36.76(-1.48)	30.88(-2.94)	33.82(-2.94)	30.88(-2.94)	29.41(-2.94)	29.41(-1.47)	29.41(-1.47)	**41.20**
keep$^+$	61.19(1.49)	55.22(-4.48)	61.19(35.82)	55.22(29.85)	44.78(1.50)	47.76(0.00)	46.27(-1.49)	**64.20**
leave$^-$	53.03(-1.52)	53.03(0.00)	50.00(-1.52)	48.48(0.00)	50.00(-1.52)	56.06(-4.55)	53.03(-4.55)	**57.60**
live$^-$	56.72(-5.97)	56.72(-2.98)	31.34(-26.87)	25.37(-32.84)	53.73(-1.49)	52.24(-2.98)	50.75(-2.98)	**69.40**
match$^+$	**59.52**(0.00)	**59.52**(2.38)	52.38(4.76)	50.00(2.38)	57.14(2.38)	57.14(2.38)	57.14(2.38)	59.50
play$^-$	56.06(-1.52)	56.06(-1.52)	30.30(-22.73)	30.30(-22.73)	48.48(-1.52)	48.48(-4.55)	48.48(0.00)	**62.10**
pull$^-$	38.33(-1.67)	36.67(-3.33)	26.67(0.00)	23.33(0.00)	20.00(-6.67)	21.67(-8.33)	21.67(-8.33)	**58.30**
replace	57.78(-2.22)	57.78(-2.22)	53.33(-4.45)	51.11(-2.22)	55.56(4.45)	**64.44**(4.44)	60.00(4.44)	61.10
see$^+$	42.03(-1.45)	42.03(-1.45)	37.68(0.00)	37.68(1.45)	40.58(-1.45)	43.48(1.45)	43.48(1.45)	**44.20**
serve	64.71(-5.88)	64.71(-5.88)	58.82(-1.96)	56.86(-3.92)	62.75(-7.84)	60.78(-5.89)	60.78(-5.89)	**68.60**
strike$^-$	38.89(-3.70)	38.89(-1.85)	27.78(3.71)	22.22(0.00)	38.89(-1.85)	38.89(-3.70)	38.89(-3.70)	**51.90**
train	**68.25**(-1.59)	**68.25**(-1.59)	58.73(0.00)	58.73(0.00)	66.67(0.00)	60.32(1.59)	58.73(1.59)	60.30
treat$^-$	**68.18**(-2.27)	**68.18**(-2.27)	65.91(-4.54)	65.91(-4.54)	63.64(0.00)	52.27(0.00)	50.00(-2.27)	50.00
turn$^-$	37.31(-2.99)	34.33(-5.97)	26.87(-2.98)	25.37(0.00)	23.88(-5.97)	34.33(0.00)	32.84(-1.49)	**48.50**
use$^-$	69.74(-1.31)	67.11(-3.94)	68.42(-2.63)	67.11(-2.63)	65.79(-2.63)	**71.05**(-1.32)	**71.05**(-1.32)	69.70
wander	82.00(0.00)	82.00(0.00)	82.00(0.00)	82.00(0.00)	82.00(0.00)	82.00(2.00)	82.00(2.00)	82.00
wash	58.33(0.00)	50.00(-8.33)	41.67(8.34)	41.67(8.34)	50.00(0.00)	41.67(-8.33)	41.67(-8.33)	**75.00**
work$^+$	43.33(0.00)	43.33(3.33)	38.33(3.33)	38.33(10.00)	36.67(-1.66)	41.67(1.67)	40.00(0.00)	**53.30**
average	60.61(-1.72)	59.50(-2.00)	54.35(-1.22)	52.85(-1.17)	55.84(-1.61)	58.34(-0.50)	57.56(-0.72)	**64.59**

Table 6.6: Overall results of the feature selection heuristics for Senseval-2 without SRL based features

Verb	Heuristics							Majority Class Baseline
	Individual Feature Ranking					Feature Coverage		
	Fully Oracled	FS Oracled	MaxEnt Oracled	Fully Auto.	Most Features	MaxEnt Oracled	Fully Auto.	
hit^{-}	70.00(-2.86)	58.10(-11.90)	66.67(0.00)	55.24(-5.71)	55.24(-11.90)	52.38(-2.86)	49.52(-2.86)	43.8(0.00)
lift^{+}	70.48(-3.33)	67.14(2.38)	43.33(-2.38)	40.48(0.48)	67.62(0.00)	64.29(2.86)	61.43(8.57)	54.8(0.00)
look^{-}	92.03(0.00)	91.42(0.00)	39.39(-15.67)	38.79(-16.27)	91.42(1.23)	92.65(0.00)	92.05(0.00)	88.4(0.00)
pull^{+}	64.85(1.17)	61.10(4.78)	55.07(-2.43)	51.47(-2.35)	58.75(13.68)	57.57(1.17)	52.65(1.18)	41.0(0.00)
stop	78.33(2.22)	71.11(-2.50)	59.44(-2.50)	57.22(-0.28)	70.83(11.66)	59.44(2.50)	56.94(0.00)	52.4(0.00)
take^{+}	64.24(3.63)	60.91(3.94)	56.97(1.36)	53.33(5.00)	46.21(5.30)	50.15(0.00)	42.88(9.09)	37.0(0.00)
throw	68.10(3.34)	58.57(0.47)	49.05(0.48)	49.05(3.81)	52.38(-2.86)	57.62(0.00)	47.62(0.00)	37.9(0.00)
turn^{+}	81.34(0.08)	80.43(0.08)	79.48(1.86)	79.48(1.86)	75.76(-2.77)	81.30(2.77)	81.30(2.77)	73.6(0.00)
Overall	78.21(0.55)	74.97(0.37)	55.33(-4.94)	53.23(-4.57)	71.73(2.44)	72.43(0.92)	69.71(1.95)	61.7(0.00)

Table 6.7: Overall results of the feature selection heuristics for MSC without SRL based features

Verb	Training Set	Test Set
begin	100.0	99.3
call	99.2	100.0
carry	100.0	100.0
collaborate	100.0	100.0
develop	97.0	97.1
draw	100.0	100.0
dress	95.0	94.9
drift	100.0	100.0
drive	98.8	100.0
face	99.5	100.0
find	100.0	100.0
keep	100.0	100.0
leave	98.5	100.0
live	99.2248	98.5
match	97.7	100.0
play	98.4	100.0
pull	99.2	100.0
replace	100.0	100.0
see	100.0	100.0
serve	100.0	100.0
strike	95.2	94.4
train	96.8	95.2
treat	100.0	100.0
turn	99.2	100.0
use	100.0	98.7
wander	100.0	98.0
wash	100.0	100.0
work	99.2	100.0
average	99.1031	99.1136

Table 6.8: SRL coverages over the target verbs in the training and test set of Senseval-2 (%)

It can be observed that without the SRL based features, the general performance of the feature selection heuristics is about 2% lower. This difference in performance is statistically significant (paired t-test, $p < 0.05$). This result shows that while multi-semantic-role based SRL features can indeed improve the performance of verb disambiguation, the improvement is rather limited. Furthermore, Table 6.8 shows the coverage of the semantic role labelling system on the training and testing data sets, and it can be seen that the coverage is very high. As such, the limited disambiguation improvement is not caused by an insufficient volume of semantic-role-labelled examples.

A closer examination of both the oracled and fully automatic disambiguation

performance on the individual verbs shows that the SRL based features tend to have an above average positive effect on verbs which exhibit strong selectional preferences with respect to their arguments. Examples of such verbs include *develop*, *drive*, and *serve*. Given the basic assumption of selectional preference based WSD, this observation is expected. However, there are other verbs for which the SRL based features did not make a statistically significant difference, one such verb being *dress*. Table 6.9 shows the distributions of semantic roles with respect to each verb in the test set. In this table, verbs that did better without SRL features are marked with $^+$, and verbs that did worse without SRL features are marked with $^-$. It can be observed that out of the thirteen verbs for which the semantic role features had a positive influence, eight of them had more A1 roles than A0 roles; out of the seven verbs for which the semantic role features had negative influence, four of them had more A0 roles than A1 roles; and out of the eight verbs for which the semantic roles features did not seem to have any consistent influence, two of them had the same proportion of A0 and A1 roles, and five of them had more A0 roles than A1 roles. This suggests that in general, the A1 role has more disambiguation power than the A0 role, and verbs that can take an A1 role are most suited to the disambiguation method described in this chapter.

Furthermore, after manually checking the SRL-tagged sentences for the training data of *dress*, it was observed that the following factors greatly reduced the effectiveness of the SRL based features:

- A large number of sentences only have one argument type semantic roles; 31.7% of the sentences do not have the **A0** semantic role, 57.5% of the sentences do not have the **A1** semantic role, and only 23.3% sentences have both the **A0** and **A1** semantic roles.
- For the majority of the example sentences, the semantic role arguments are either pronouns or proper nouns, or both. This means the semantic role arguments provide very little disambiguation information with respect to the senses, even when they are present.
- The SRL accuracy is not particularly high. Only 76.0% of the sentences do not contain any incorrect labellings, and a significant number of these sentences miss at least one semantic role.

Analysis of the Results

The analysis of the results will focus on two aspects: the poor performance of the individual features ranking heuristic in general, and the poor performance of the SRL features on the MSC dataset.

On both the Senseval-2 dataset and the MSC dataset, the IFR heuristic performed worse than having no feature selection at all. Since the IFR heuristic ranks the features by the amount of disambiguation information they carry, there are two possible

explanations for the poor performance. The first one is that the training data is so different from the test data that the probability distribution of the features with respect to the senses is totally different from that of the test data. This explanation is unlikely to be true since in both datasets, the training data and the test data come from the same genre of text. The second explanation is that a significant portion of the features from the training data do not exist in the test set and the lack of features caused the poor performance. This explanation is more plausible given the small size of the datasets. In fact, an investigation into the coverage of the features revealed that for the Sensval-2 dataset, only 32.6% of the features from the training set were found in the test set; for the much smaller MSC dataset, the coverage is even less at 20.0%. This indicates that the small sizes of the datasets was mainly responsible the poor performance of the IFR heuristic.

The analysis of the SRL features' poor performance on the MSC dataset required a manual examination of the semantic-role-labelled sentences. This examination revealed three major problems that contributed to the poor performance of the SRL features. Firstly, a large number of sentences only have one argument type semantic role; 31.7% of the sentences do not have the A0 semantic role, 57.5% of the sentences do not have the A1 semantic role, and only 23.3% sentences have both the A0 and A1 semantic roles. A0 tends to be the surface subject of the verbs and appears to the left of the verb. However, previous experiments have shown that the most useful words for VSD tend to occur to the right of the verb. As a result, the lack of non-A0 semantic roles greatly reduced the effectiveness of the SRL features. Secondly, for the majority of the sentences that were manually examined, the head nouns of a significant portion of the semantic roles were third person pronouns which could not be automatically resolved to WordNet synsets. Examples of such pronouns include *it*, *its*, *they* and *them*. These pronouns cannot be mapped to any word class, therefore providing almost no disambiguation information. Finally, the SRL accuracy was not sufficiently high. Only 76.0% of the sentences examined did not contain any mislabelled roles, and a significant number of these sentences miss at least one semantic role, thus reducing the effectiveness of the semantic role features. Since the dataset was already small, the noise introduced by incorrectly labelled semantic roles further exacerbated the performance of the VSD system.

6.3 Summary

The purpose of this chapter has been to show the necessity of performing verb sense disambiguation as part of the controlled natural language visualisation task, and to investigate whether multi-semantic-role based selectional preferences extracted with current state-of-art SRL systems can make any significant improvements to supervised verb disambiguation.

The discussion of the movie script dataset in Section 6.1 has demonstrated that on

the one hand, even within the small movie script corpus, there is already a non-trivial number of verbs whose frequencies and degree of polysemy may pose a problem for later stages of the final system, if they are not disambiguated. On the other hand, the majority of the verbs in the movie script either have very low frequencies or are not very polysemous. Given this particular property of the movie script corpus, and the fact that there does not exist any WSD system or training dataset which has both high verb coverage and high disambiguation accuracy, the most suitable strategy would be to employ a VSD system that can accurately disambiguate a small number of difficult verbs, and at the same time competently disambiguate a large number of less polysemous verbs. Following the high level discussion on WSD systems in general, such a VSD system can only be constructed as a hybrid system comprising a supervised system for the more difficult verbs, and an unsupervised (or semi-supervised) system for the rest of the verbs.

Section 6.2 has provided a detailed description of a wide range of contextual, collocational, syntactic and semantic features that can be used for VSD, with an emphasis on multi-semantic-role based selectional preference features. It has shown that the even though multi-semantic-role based selectional preference features are not able to dramatically improve the VSD performances, there are small but consistent and statistically significant improvements. However, it also seems that SRL features can only improve the performance of the VSD system when there is a sufficient amount of gold standard training data, as demonstrated by their negative influence on the MSC dataset.

Since VSD is only one of the early semantic analysis components of the natural language visualisation task, and it is intuitively useful, the final judgement of its usefulness can only be made when the entire system is evaluated. This will be discussed in Chapters 7 and 8.

Verb	A0	A1	A1-FOR	A1-WITH	A2	A2-DIR	A2-EXT	A2-PNC	A3	A4	AM-ADV	AM-CAU	AM-DIR	AM-DIS	AM-LOC	AM-MNR	AM-MOD	AM-NEG	AM-PNC	AM-TMP	C-ARG1	R-ARG0	R-ARG1	R-ARG3-FOR	Total No. of roles
begin^{-}	17.80	35.60	0.0	0.0	1.30	0.0	0.0	0.0	0.0	0.0	4.05	0.0	0.0	2.46	2.32	2.03	3.18	0.29	0.58	19.68	8.10	0.58	1.30	0.0	14
call^{+}	30.82	28.30	1.89	0.0	16.98	0.0	0.0	0.0	0.0	0.0	1.26	1.26	0.63	1.89	0.63	0.63	3.14	0.0	1.26	7.55	0.0	2.52	1.26	0.0	15
carry	34.97	34.97	0.0	0.0	0.61	0.0	0.0	0.0	1.23	0.61	1.84	0.0	1.84	1.84	1.84	2.45	6.13	0.61	1.23	4.91	0.0	3.07	1.23	0.0	16
collaborate	29.41	10.29	0.0	0.0	27.94	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	7.35	11.76	0.0	1.47	7.35	0.0	4.41	0.0	0.0	8
develop^{-}	26.14	42.48	0.0	0.0	1.96	0.0	0.0	0.0	0.65	0.65	3.27	0.0	0.0	1.31	2.61	5.88	1.31	1.31	0.65	7.84	0.0	1.31	1.96	0.0	15
draw^{-}	29.13	30.10	0.0	0.0	4.85	0.0	0.0	0.0	0.0	0.0	4.85	0.0	11.65	1.94	1.94	4.85	3.88	0.97	0.97	3.88	0.0	0.0	0.0	0.0	12
dress^{+}	35.58	20.19	0.0	0.0	6.73	0.0	0.0	0.0	0.0	0.0	1.92	0.0	0.0	0.96	0.0	14.42	0.96	4.81	3.85	7.69	0.0	0.96	0.0	0.0	11
drift^{+}	8.86	34.18	0.0	0.0	3.80	0.0	1.27	0.0	0.0	0.0	7.59	0.0	18.99	1.27	5.06	1.27	0.0	1.27	0.0	12.66	0.0	1.27	2.53	0.0	13
drive^{-}	29.91	26.17	0.0	0.0	3.74	8.41	0.0	0.0	0.0	0.0	4.67	0.0	4.67	0.0	2.80	5.61	1.87	0.0	0.0	10.28	0.0	1.87	0.0	0.0	11
face	34.73	34.73	0.0	0.0	0.76	0.0	0.0	0.0	0.0	0.0	4.58	1.15	0.0	3.44	1.91	0.0	5.34	1.15	0.0	10.31	0.0	1.53	0.38	0.0	12
find^{-}	34.30	35.47	0.0	0.0	1.16	0.0	0.0	0.0	0.58	0.0	1.74	0.0	0.0	4.07	4.07	1.16	4.65	1.16	0.0	6.98	1.16	1.74	0.58	0.0	14
keep^{+}	28.86	33.56	0.0	0.0	8.72	0.0	0.0	0.0	0.0	0.0	5.37	0.0	0.0	0.67	2.68	2.68	7.38	2.68	0.67	4.03	0.67	2.01	0.0	0.0	13
leave^{-}	32.89	25.66	0.0	0.0	10.53	0.0	0.0	0.0	0.0	0.0	1.32	1.32	0.0	1.32	1.97	1.32	6.58	1.97	0.66	9.87	0.0	3.29	1.32	0.0	14
live^{-}	37.65	8.64	0.0	0.62	0.62	0.0	0.62	0.0	0.0	0.0	1.23	1.23	0.0	0.62	20.99	7.41	3.09	2.47	0.0	8.02	0.0	4.32	0.62	0.0	15
match^{+}	14.94	36.78	0.0	0.0	2.30	0.0	0.0	0.0	0.0	0.0	4.60	0.0	0.0	0.0	0.0	10.34	3.45	3.45	0.0	5.75	10.34	6.90	1.15	0.0	11
play^{-}	31.55	30.36	0.0	0.0	1.19	0.0	0.0	0.0	0.0	0.0	4.17	0.60	0.0	2.98	10.12	7.74	1.79	1.19	0.60	4.76	0.0	1.19	0.60	0.0	14
pull^{-}	28.30	28.93	0.0	0.0	5.66	9.43	0.0	0.0	0.0	0.0	2.52	0.63	1.26	1.26	2.52	2.52	2.52	1.26	1.89	8.81	0.0	2.52	0.0	0.0	15
replace	12.82	36.75	0.0	0.0	23.93	0.0	0.0	0.0	0.0	0.0	3.42	0.0	0.0	0.85	3.42	0.0	5.98	0.0	0.0	5.98	0.0	2.56	0.0	0.0	9
see^{+}	37.21	36.05	0.0	0.0	0.58	0.0	0.0	0.0	0.0	0.0	3.49	0.0	0.0	1.74	5.23	0.58	4.65	3.49	0.0	6.98	0.0	0.0	0.0	0.0	10
serve	31.11	20.00	0.0	0.0	13.33	0.0	0.0	0.0	0.0	0.0	4.44	0.0	0.0	1.48	6.67	2.96	4.44	1.48	0.0	6.67	0.0	5.93	0.74	0.0	12
strike^{-}	27.61	27.61	0.0	0.0	6.72	0.0	0.0	0.0	0.0	0.0	5.22	0.75	0.0	3.73	4.48	4.48	2.99	0.0	0.75	10.45	0.0	2.99	0.75	0.75	14
train	30.67	25.33	0.0	0.0	15.33	0.0	0.0	0.0	0.0	0.0	1.33	0.67	0.0	1.33	1.33	5.33	3.33	2.00	0.0	8.67	0.0	4.67	0.0	0.0	12
treat^{-}	21.71	30.23	0.0	0.0	16.28	0.0	0.0	0.0	1.55	0.0	2.33	0.0	0.0	1.55	0.0	10.85	4.65	0.78	0.78	3.88	0.0	4.65	0.78	0.0	13
turn^{-}	22.88	31.37	0.0	0.0	7.19	0.0	0.0	0.0	0.0	0.0	2.61	0.0	13.07	4.58	0.0	0.65	2.61	0.0	2.61	7.19	1.31	1.96	1.31	0.0	13
use^{-}	28.95	32.11	0.0	0.0	5.79	0.0	0.0	7.89	0.0	0.0	3.16	0.53	0.0	0.53	1.05	1.05	5.79	2.63	1.05	5.26	0.0	2.63	1.58	0.0	15
wander	35.25	24.59	0.0	0.0	1.64	0.0	0.0	0.0	0.0	0.0	3.28	0.0	3.28	1.64	3.28	4.92	6.56	0.0	1.64	9.02	0.0	1.64	3.28	0.0	13
wash	28.12	25.00	0.0	0.0	6.25	0.0	0.0	0.0	12.50	0.0	6.25	0.0	0.0	0.0	3.12	0.0	0.0	0.0	0.0	15.62	0.0	0.0	0.0	0.0	7
work^{+}	35.00	17.50	0.0	0.0	5.00	0.0	0.0	0.0	4.38	0.0	3.12	0.0	0.0	1.88	3.75	9.38	5.00	4.38	0.0	8.12	0.62	1.25	0.0	0.0	13

Table 6.9: Distribution of semantic roles for each verb in the test set of Senseval-2

Chapter 7

Graphics Command Generation

Chapters 3, 5 and 6 discussed the linguistic analysis performed on the action descriptions of the input scripts. The results of the linguistic analysis include the identification of the target verb instances, their semantic roles and their senses in context.

This chapter discusses how the linguistic analysis of the acting instructions of movie scripts can influence the process of generating computer graphics commands to visualise these instructions in pre-built virtual scenes. I will start this chapter with an overview which describes the properties of the virtual scenes and the graphics commands, as well as how the generation of the graphics commands fits into the overall animation process. Following the overview, I will describe a machine learning method for determining the timing relationships between verb instances in the same action description, and an automatic method for grounding textual semantic roles to objects in the virtual scene. I will then explain how machine learning algorithms can be applied to the generation of graphics commands. Finally, I will report the results of some experiments conducted to evaluate the output of the graphics commands generation process.

The key contribution of this chapter is a new approach for generating graphics commands based on machine learning methods. This approach is significantly different from the rule-based approaches that dominate the existing literature on using NLP techniques to drive the generation of animation. Other contributions of this chapter include evaluation of the usefulness of the various types of linguistic and virtual scene information with respect to the generation of the graphics commands.

This chapter is organised as follows: Section 7.1 gives an overview of this chapter; Section 7.2 describes an empirical method for determining the timing relationships between the verb instances in the same script; Section 7.3 explains how textual semantic roles are grounded to virtual objects in a given virtual scene; Sections 7.4 and 7.5 describe how a machine learning algorithm is used to generate the graphics commands; and finally, Section 7.6 presents the results of the evaluation.

Part of this research has been jointly published with Timothy Baldwin, and pre-

sented in Ye and Baldwin (2008).

7.1 System Overview

I will start this section with a description of the virtual scenes used in this research, then I will introduce the graphics commands in the context of the animation cycle, and finally I will describe how the linguistic information is combined with the virtual scene information for the training of statistical models of graphics commands.

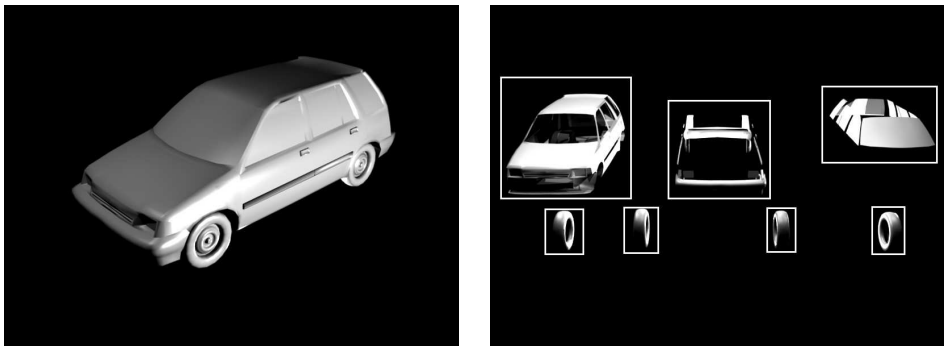
7.1.1 Virtual Scene

The building blocks of the virtual scenes are individual 3D graphical models of real world objects, such as houses, people, lights and furniture. Each model is hand-assigned a single WordNet noun synset and a unique serial number in a database. These tuples of noun synset and serial number act as the only index of the models in the database. The database is connected to the graphics system via a drag-and-drop interface, which is used to construct the virtual scenes used in this research. WordNet synsets are chosen for the annotation of models because WordNet provides good coverage of real world objects, and its hypernym-hyponym (general-specific) structure allows a certain degree of generalisation for the models. For example, models annotated with the synset *arm chair*$_1$ ("chair with a support on each side for arms"), *barber chair*$_1$ ("a large fixed adjustable chair in which barbers seat their customers"), and *folding chair*$_1$ ("a chair that can be folded flat for storage") could all be collectively referred to as *chair*.

Furthermore, in the process of collecting the models,[1] it was observed that many of the models consisted of multiple separate polyhedrons. For most of these multi-polyhedron 3D models, the division into components was rather arbitrary, and often did not resemble the structure of the real world counterpart. For example, consider the model of a car shown in Figure 7.1: Figure 7.1a shows the whole car, and Figure 7.1b shows the individual polyhedrons of the car. While the polyhedrons for the wheels of the car are separated from the rest of the car just like their real world counterparts, for the other polyhedrons, it is quite difficult to pinpoint what their real life counterparts should be. However, there is a small number of models that are separated into multiple polyhedrons in the same way that their real life counterparts could be separated. Figure 7.2 shows a model of a workbench as a whole and in separate polyhedrons. It can be observed that all of these polyhedrons have exact counterparts in real life.

As far as the technology for creating 3D graphical models is concerned, there is no difficulty in creating 3D graphical models that exactly resemble real life objects in both appearance and structure, but doing so may require substantially more time and

[1] With the exception of three 3D graphical models, all the 3D graphical models were obtained from free sources on the Internet.



a b

Figure 7.1: Example of 3D graphical models with components that have no WordNet synset counterparts

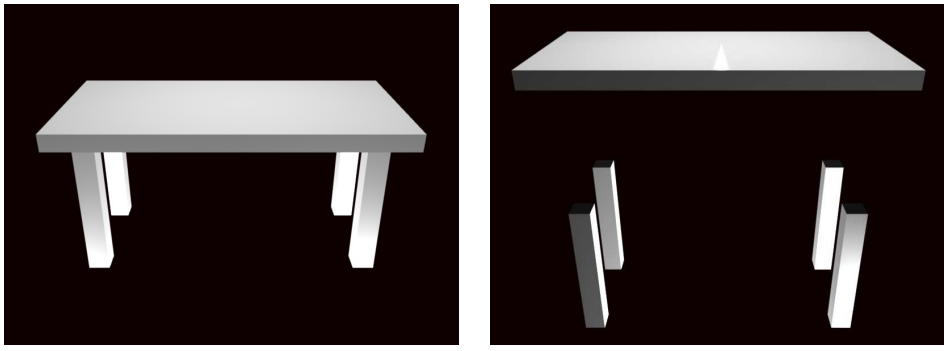


Figure 7.2: Example of a 3D graphical model and its graphical components

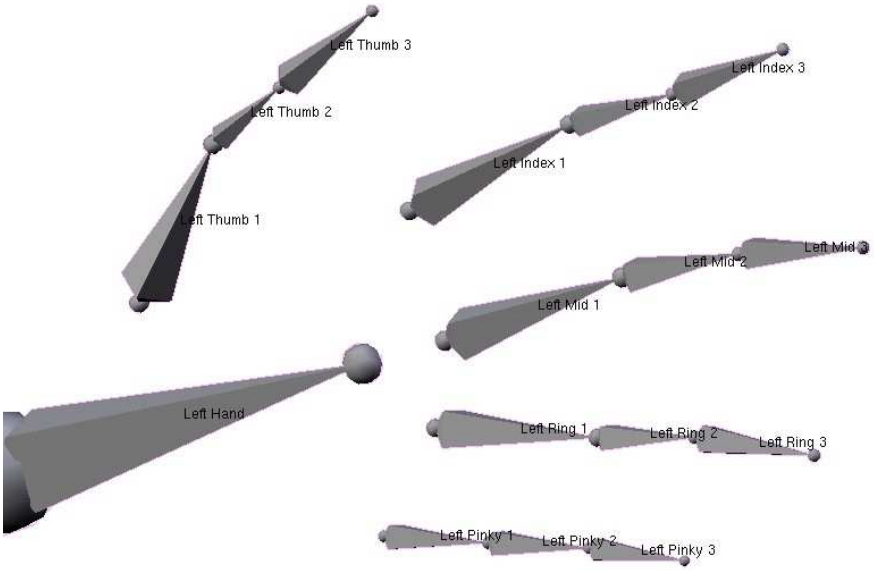


Figure 7.3: Example of the bones of an armature

human labour. However, if the models could be created with the same structure as their real world counterparts, it would be possible to further annotate each separate polyhedron with an appropriate WordNet synset according to WordNet's meronym-holonym (part-whole) structure, thereby potentially increasing the accuracy of the visualisation. For example, consider the sentence *Andy sits in the chair*. Since a chair (typically) consists of a seat, legs, and a back, it would be useful to differentiate the individual components of the model and avoid unnatural visualisation of the sentence, such as Andy sitting on the back of the chair instead of the seat. Furthermore, the WordNet meronym-holonym (part-whole) structure could also provide additional generalisation power to the visualisation.[2]

Additionally, as introduced in Chapter 4, some 3D graphical models contain an armature component for skeleton based animation. It it possible to annotate each bone with a unique WordNet synset. However, since the armatures are designed for animation purposes, it is common for the bones to not have a unique corresponding WordNet synset. For example, Figure 7.3 shows the names of bones on the left hand of the model. It can be observed that each finger consists of 3 bones. The most appropriate WordNet synset for these bones is $phalanx_1$ ("any of the bones of the fingers or toes") which does not contain sufficient detail to distinguish all the bones

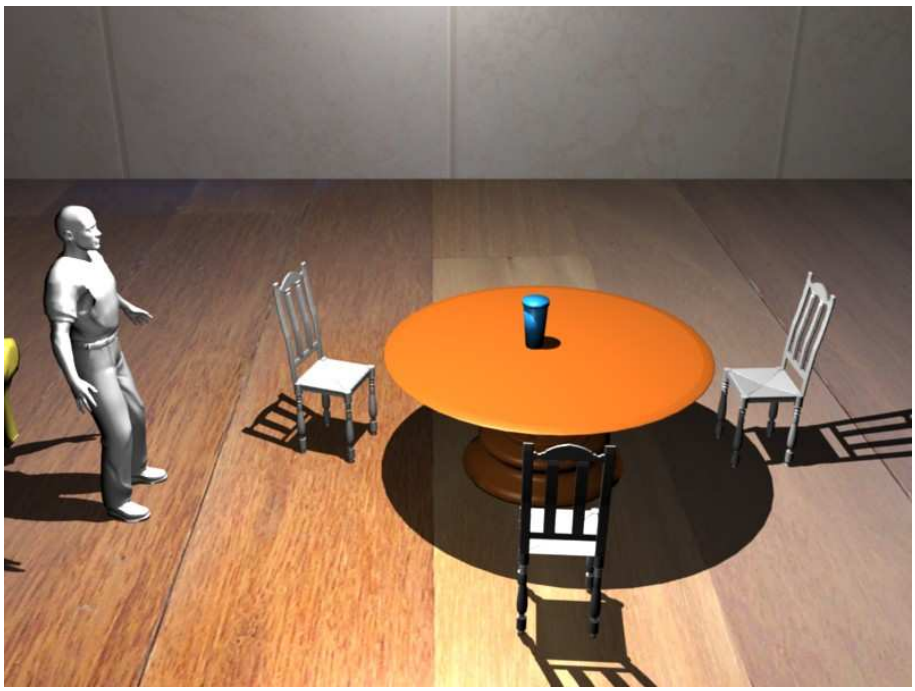


Figure 7.4: Example of multiple instantiations of the same 3D graphical model

that could be considered as phalanges.

A virtual scene normally contains one or more virtual objects. A virtual object is an instantiation of a 3D graphical model. It is common to instantiate a model multiple times in the same virtual scene to create multiple virtual objects, especially when the model is a piece of furniture or cutlery. For example, Figure 7.4 shows a virtual scene that contains three virtual chairs that were all instantiated from the same 3D graphical model. All virtual objects are automatically assigned the WordNet synset of the models that they are instantiated with.

Each virtual object can be annotated with additional WordNet synsets, which is useful for finding entities that have multiple roles in a movie. For example, the character Woody in *Toy Story* is both a toy_1 ("an artifact designed to be played with"), and a man_1 ("an adult male person"). Furthermore, each virtual object can also be given one or more names, which is useful for finding entities with more than one title. For example, in the script of Toy Story, the character of Mrs. Davis is often referred to as either *Mrs. Davis* or *Andy's mum*. Figure 7.5 exemplifies the relationship between 3D graphical models in the database and virtual objects in a virtual scene.

[2]The substance meronyms may not be very useful in this case.

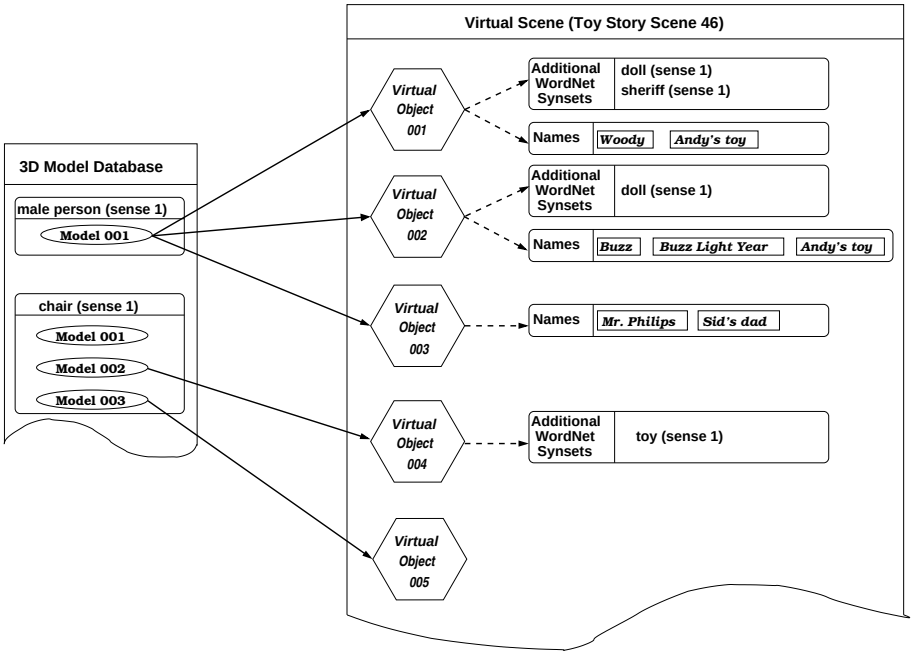


Figure 7.5: Example of the mapping from 3D graphics models in the database to virtual objects in a virtual scene

Finally, an important feature of the virtual scenes is that the status (position, orientation, etc) of any virtual object can be queried at any time. This feature makes it possible to extract extra-linguistic information about the virtual objects during the animation process. For example, virtual objects that are closer to or facing the virtual camera tend to have greater salience in the discourse than virtual objects further from or not facing the virtual camera; and virtual objects associated with ongoing commands also tend to have greater salience than idle virtual objects. The salience information can be useful in tasks such as anaphora resolution.

7.1.2 Graphics Commands and Animation Procedure

The graphics commands used in this research are designed to work with commonly used frame-based animation systems. In a frame-based animation system, an animation controller creates the animation frame by frame, and the status of the virtual objects are calculated and updated at each frame by the relevant graphics commands before they are actuated. More specifically, in the context of creating animated story boards, the role of the animation controller is to choose which verbs to visualise next, which graphics commands to use for animating the chosen verbs,

and which ongoing graphics command to terminate. When the animation is created manually (e.g. during the creation of the training data[3]), the animation controller is a human animator, and when the animation is created automatically, the animation controller is a software module.

Basic Graphics Commands

As discussed in Chapter 4, most of the physical actions in the real world can be animated with rigid body transformations, skeleton based transformations, scene graph based transformations and lighting manipulations. These four types of transformations can animate all the visualisable verb instances in *Toy Story* with the exception of four verb instances which required soft body simulation and explosion simulation, neither of which could be accessed through Blender3D's programming interfaces when the training data was created. In order to perform these three types of transformations, 21 basic graphics commands have been implemented in this research.[4] These basic graphics commands were designed to be as generic as possible to ensure that they could be reproduced by other 3D animation systems. As a result of this genericness, they are rather simplistic individually. However, these basic graphics commands were designed so that at run time, they could be combined sequentially or in parallel to produce much more complex animations than they could individually. As the rest of this chapter will demonstrate, the basic graphics commands are in fact rarely used individually. In general, their functionalities can be divided into the following categories:

- Rigid body transformation, e.g. translation and rotation.
- Inverse kinematics based skeleton operation (Welman 1993), e.g. positioning the hand of a virtual human on a virtual mug.
- Reenactment of pre-recorded skeleton animation stored in motion captured files, e.g. making a virtual human go through a pre-recorded walking cycle.
- Transform scene graph operation, i.e. making a virtual object the transform parent of another virtual object so that when the parent virtual object moves, the child virtual object will also move accordingly. For example, if a virtual hat is placed on top of the head of a virtual human, and the head of the virtual human is then made to be the parent of a virtual hat, then when the virtual human walks or bends down, the hat will move with the head as if the virtual human is wearing it; if the head is not made to be the parent of the virtual

[3]The training data is created by annotating the target verb instances in the action descriptions with the appropriate graphics commands, and then executing these graphics commands to create the animation.

[4]The details of these commands are listed in Appendix D.

hat, the virtual hat will be stationary no matter how the head moves, and this would cause the animation to look unrealistic.

- Lighting operations, e.g. changing the intensity of a light source.

The basic graphics commands are similar to procedures in programming languages, and have the following general form:

$$command(arg0, arg1, ..., argN)$$

The behaviour of each basic graphics command determines how it terminates. Most of the graphics commands have built-in tests for termination, and these tests are run after each frame to determine whether the execution of the graphics command has finished. We will refer to these commands as "active commands". An example of an active command is the `translateObjectToPosition` which moves a given virtual object to a specified destination at a specified speed. Its built-in termination test calculates the distance between the position of target virtual object and the destination after every frame, and when the distance is less then a predefined threshold, it will terminate itself. On the other hand, some graphics commands do not have built-in tests and will keep executing until they are terminated by the animation controller. We will refer to these commands as "passive commands". An example of the passive commands is the `armatureStalkObject` command which performs a scene graph based transformation that makes a bone of the armature of a virtual object track another virtual object. This command has no termination test and the only way to stop the tracking is to terminate this command explicitly.

Termination Commands

Sometimes, an ongoing basic graphics command has to be explicitly terminated by the animation controller as part of the process of visualising a verb instance. This explicit termination is necessary for all the passive commands which do not have built-in termination tests. However, for the active graphics commands, the explicit termination will override their built-in termination tests and force their execution to stop immediately. For the human animation controller, this explicit termination is performed directly through the annotation interface. For the automatic animation controller, this explicit termination is performed through a special class of termination commands.

In this research, it is assumed that a termination command is always used to stop an ongoing basic command from manipulating one or more grounded semantic roles of the target verb instance. Therefore, in order for the automatic animation controller to terminate an ongoing basic graphics command for a given target verb instance, it needs to know the name of this basic graphics command, which semantic roles of the target verb instance are being modified, and how they are being modified,

i.e. the mapping between the semantic roles and the arguments of the basic graphics command.

Each termination command is associated with a single target verb instance, and has the following general form:

$$(\text{command}_j, [(\text{srole}_i, \text{arg}_i), ..., (\text{srole}_n, \text{arg}_n)])$$

Here, command_j is the name of the ongoing basic graphics command to be terminated, srole_i refers to the label of a semantic role of the target verb instance, (e.g. A0 or AM-LOC), and arg_i is the name of an argument of command_j. Both srole_i and arg_i act as pointers to virtual objects, and they must refer to the same virtual object for a termination command to be valid. For example, consider the verbs *walk* and *grab* in (7.1).

(7.1) Woody starts to walk toward the podium, but a hook grabs his neck and pulls him back.

It should be obvious that the act of *grabbing* takes place during the act of *walking*, and the visualisation of *grab* would require the visualisation of *walk* to terminate immediately. Suppose that the virtual object for *Woody* is the grounded A1 for *grab*, and the basic command `NavigateObjectToObject` is used to visualise *walk*, with the virtual object for *Woody* as its `srcObj` argument, then in order for the automatic animation controller to terminate this `NavigateObjectToObject` command for the verb *grab*, it would have to issue the termination command of (`NavigateObjectToObject`, [(A1, `srcObj`)]). This termination command would then find all the ongoing `NavigateObjectToObject` commands whose `srcObj` argument is the same virtual object as the grounded A1 of *grab*, and terminate them.

It should be noted that unlike the basic commands, all the termination commands start and finish execution at the same frame. It should not be assumed that verbs such as *stop* and *terminate* can always be visualised by a single termination command. For example, consider the verb *stop* in the following sentences:

(7.2) a. The bus starts to pull out of the bus stop, then suddenly stops when a dog runs in front of it.

b. The bus slowly stops in front of the bus stop.

c. The bus drives down the street, and stops slowly at the intersection.

Suppose that each sentence in (7.2) is the very first sentence in the input text. In (7.2a), *stop* may indeed correspond to a single termination command, because a slowly moving bus can be expected to stop immediately. In (7.2b), *stop* is interchangeable with the verb *arrive*, because visually, it just depicts the bus moving in front of the bus stop slowly, and therefore may not require a termination command at all. In

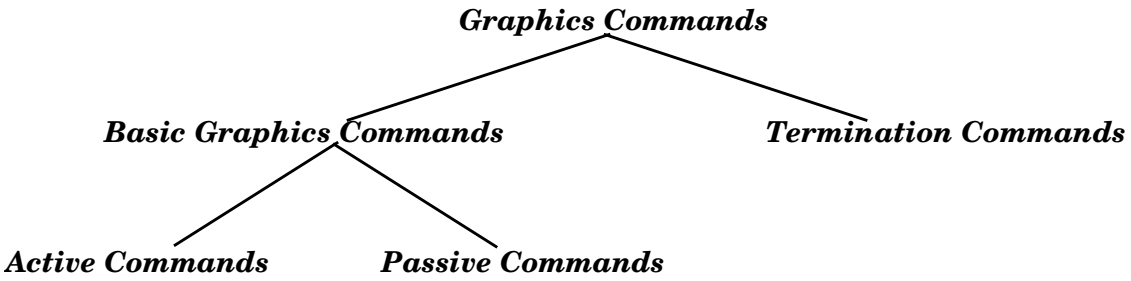


Figure 7.6: Hierarchy of the graphics commands

(7.2c), *stop* would require a termination command to first terminate the ongoing basic command used to visualise the verb *drives*, then another basic command to make sure that the *bus* arrives slowly at the *intersection*.

The termination commands are not explicitly programmed like the basic commands. Instead, the annotation interface provides a menu which lists all the ongoing basic commands, and the human animator can then choose the ones that they wish to terminate. Whenever the human animator terminates an ongoing command during the visualisation of a target verb instance, the grounded semantic roles of this verb instance are matched with the appropriate arguments of the terminated basic graphics command to form a new instance of a termination command.

Brief Recap of Graphics Commands

Two types of graphics command have been introduced so far: basic graphics commands and termination commands. The basic graphics commands are explicitly programmed individually, and the termination commands are extracted from the training data. In the rest of this thesis, the term "graphics command" refers to both basic graphics commands and termination commands. Figure 7.6 shows the hierarchical relationships between all the types of graphics command introduced so far.

Actions

As stated above, it is possible to apply more than one graphics command to the same virtual object at the same time. In fact, it is often necessary to combine several of them together to perform a partial visualisation of a verb instance. In the rest of this thesis, such combinations of graphics commands will be referred to as 'actions'.

More specifically, an 'action' is a sequence of graphics commands. All the graphics commands in the same action will start execution at the same frame, and in the order that they are added into the list. However, they do not have to finish execution at

the same time. An action only finishes when all of its active graphics commands have finished. Each action is verb instance specific, and associated with a unique verb instance. The visualisation of a verb instance can require a sequence of actions. Each new action is chosen by the animation controller after the previous action has finished.

Since actions are simply combinations of graphics commands, there is no way for an automatic animation controller to know when a verb instance has been completely visualised. Therefore, an additional `end-of-sequence` action has been included to signal the completion of the visualisation of a verb instance. This action does not consist of any graphics command. When the automatic animation controller receives this action, it will stop choosing new actions for the target verb instance.

Finally, it is important to note that the sequences of actions are always selected to be as direct as possible. In other words, the animator always tries to visualise a verb instance with as few actions as possible without reducing the faithfulness of the visualisation.

Arguments to Basic Graphics Commands

The arguments to the basic graphics commands are semantically typed just like their counterparts in programming languages. The following nine types of argument are accepted by the basic graphics commands:

- Single virtual object
- Bone of a virtual object
- Axis (possible values are `x-axis`, `y-axis` and `z-axis`)
- Boolean (used to fine tune the behaviours of certain basic graphics commands)
- Scalar (used for speed, angle, etc.)
- Location (a vector of the form (x, y, z), denoting a location in Cartesian space)
- Euler angle (a tuple of the form (x, y, z), denoting rotations around the x, y, and z axes)
- Direction (a vector of the form (x, y, z), denoting a direction in Cartesian space)
- Motion captured file (Mocap): Pre-recorded skeleton animation stored in separate files

All the arguments of a basic graphics command must be fully specified before the command can be executed by the graphics system. Therefore, the task of graphics command classification actually consists of two subtasks: determining which graphics

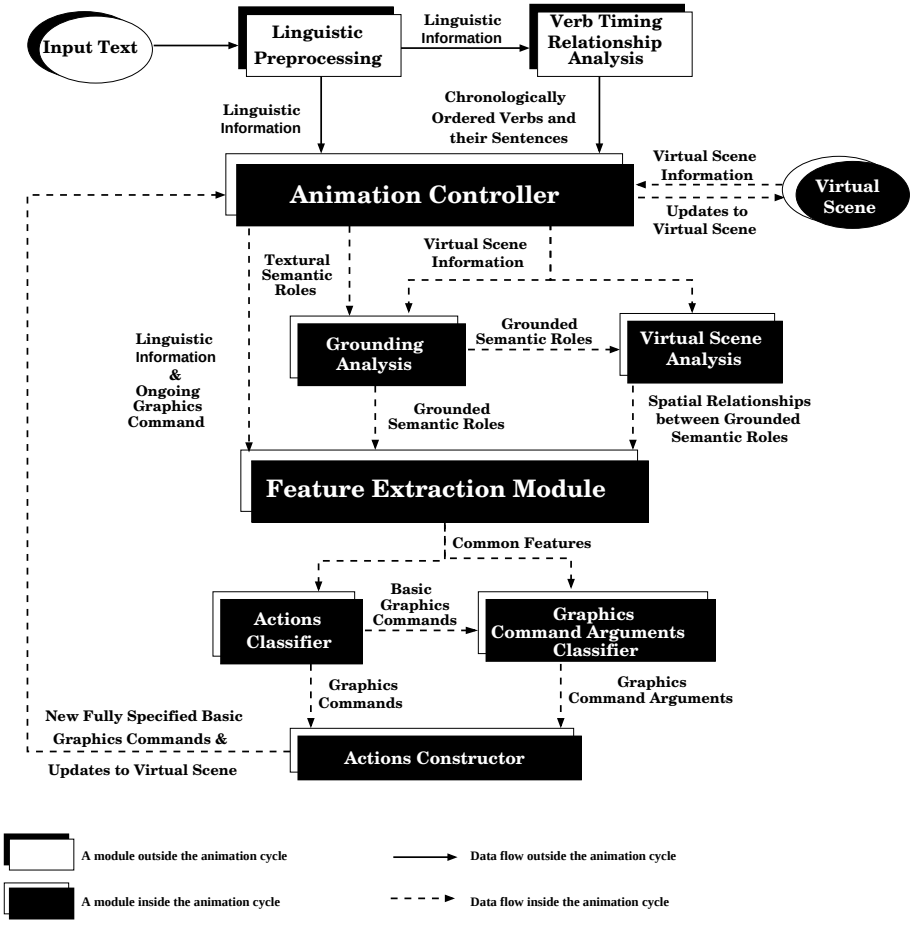


Figure 7.7: Data flow diagram of the automatic animated story boarding system

command to use, and determining the values of all the arguments to the chosen commands. The 21 basic graphics commands and their arguments are listed in Tables D.1, D.2 and D.3 of Appendix D.

7.1.3 Configuration and Data Flow

Figure 7.7 shows the data flow between the various modules of the automatic animated story boarding system. As this figure demonstrates, there is an animation cycle in the automated animated story boarding system developed in this research, and some of the modules are outside this cycle.

The linguistic preprocessing of the input script and the analysis of the verb timing relationships are performed outside the animation cycle because they are independent

of the virtual scene. The grounding analysis, virtual scene analysis and feature extraction all take place inside the animation cycle because they are all virtual scene dependent.

The animation controller controls the animation cycle by visualising verb instances from the preprocessed input script in the order determined by the verb timing relationship analysis. For a new verb instance, the animation controller will first pass the textual semantic roles to the grounding analysis module which will identify the virtual objects corresponding to the semantic roles of the chosen verb instances. The grounding analysis only happens once for each new verb instance before the first action is chosen.

For each verb instance that is not currently being visualised by an action, the animation controller will pass its grounded semantic roles to the virtual scene analysing module which will extract the latest spatial relationships between them. The grounded semantic roles and their spatial relationships are then passed to the feature extraction module along with other linguistic information about the target verb instances. The feature extraction module then extracts the common features used by both the actions classifier and the argument classifier. Using these common features, the actions classifier will choose the next action to be performed for the target verb instance. Then, the basic graphics commands in the new action are passed to the argument classifier to have their arguments constructed. Finally, the fully specified new action will be passed to the graphics system and executed for one frame along with any existing ongoing graphics commands. The execution of the ongoing graphics commands automatically updates the virtual scene. The newly specified basic graphics commands will also be fed back to the animation controller module so that they can be used as virtual scene features.

7.1.4 Summary

In this section, I have discussed the properties of the graphics commands and the animation process that were used in this research. However, there are two important missing pieces in this process: how to determine the timing relationships between verb instances in the same action description, and how the textual version of the semantic roles is grounded to virtual objects in the virtual scene. These will be the topics of the next two sections.

7.2 Determining the Timing Relationship between Verbs

The action descriptions of movie scripts often contain multiple verb instances, and it is necessary to determine the correct timing relationship between them in order to produce a faithful visualisation of the entire action description. While in most cases,

a faithful visualisation could be produced by simply visualising the verb instances one after another in the order of their appearance in the text, there is still a non-trivial number of situations where such simplification would not work.

In absolute terms, the timing of a single verb instance is defined by the starting and ending time of its visualisation, e.g. starting from frame 20 and ending at frame 90. Since the visualisation of verb instances is a dynamic process that depends heavily on the specific configuration of the virtual scene, it is not practical to pre-calculate the absolute timing for every verb instance in a given action description. Instead, a more flexible method would be to condition the timing of every verb instance on the timing of the other verb instances. For example, consider the *underlined* verb instances in the following sentences:

(7.3) A bowling ball *topples* out, *smashing* Woody square on the head. Woody *gets* up and *creeps* down the hallway, *dragging* the Christmas lights with him.

If the timing of the current verb instance was conditioned solely on the timing of the previous verb instance in the text, then the following would be a reasonable interpretation of the timing relationships between the verb instances in (7.3):

- *topples* can start immediately without waiting for any other verb instances to start or end.
- *smashing* can start immediately after *topples* ends.
- *gets* can start immediately after *topples* ends.
- *creeps* can start immediately after *gets* ends.
- *dragging* can start at the same time as *creeps*, and must also end when *creeps* ends.

The task of determining the timing relationships between two verb instances can be framed as a classification problem, with the class labels denoting the types of the timing relationships. There are thirteen types of timing relationship between two verb instances (Allen 1983), as illustrated in Figure 7.8. The distribution of these types of timing relationship are far from uniform in the *Toy Story* data. Table 7.1 shows the frequencies of the different timing relationships between pairs of consecutive verbs in the *Toy Story* data. It can be observed that, in most situations, verb instances in the action descriptions should be visualised sequentially in the order that they appear in the text.

In some situations, the timing of a verb instance cannot be determined just by its previous verb in the action description. For example, consider the underlined verb instances in following sentences:

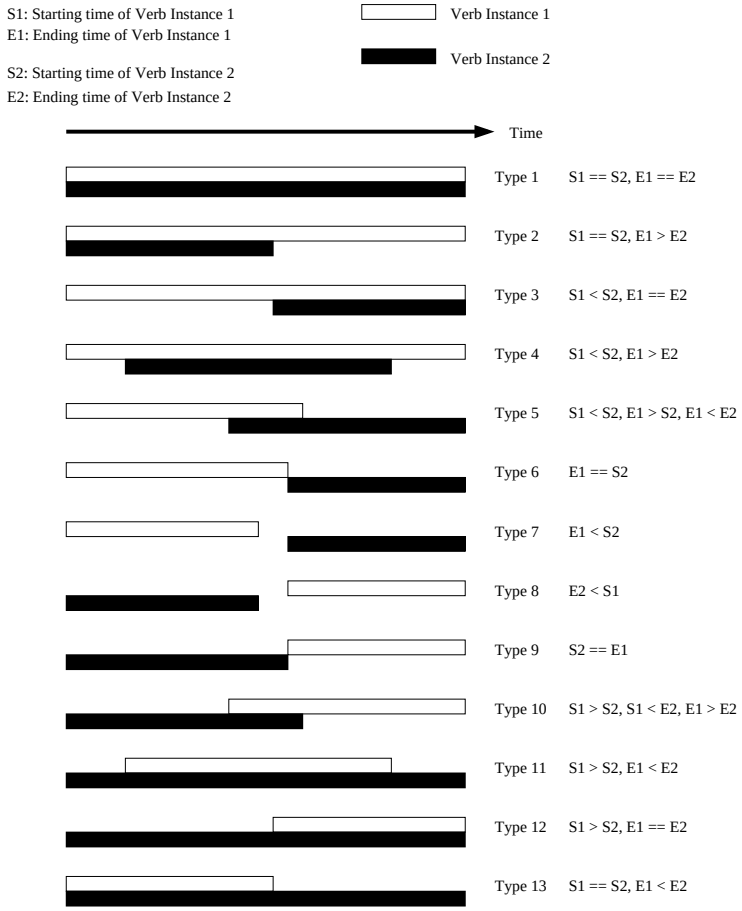


Figure 7.8: All types of timing relationship between two verb instances in the same action description

	Type											
	7	4	6	2	8	5	3	13	11	1	9	12
Frequency (%)	76.62	6.84	5.24	1.96	1.69	1.69	1.69	1.60	1.51	0.98	0.09	0.09

Table 7.1: Distributions of timing relationship types in the *Toy Story* data (the timing of the target verb instance was conditioned on the timing of the previous verb instance in the text)

(7.4) Suddenly, the entire milk crate begins to shake. Woody looks up to see Buzz. He is trying to push the tool box off.

The shaking of the milk crate is caused by *Buzz*'s pushing, and therefore, *shake* should start after *push* starts. However, the shaking of the milk crate is the event that causes *Woody* to *look up*, and therefore, *shake* should start before *looks* starts. If the timing condition of the target verb instance was conditioned only on its previous verb instance, then the timing relationships between these three verb instances cannot be captured correctly using the thirteen types of timing relationships described above. because there is no way to specify that *push* should start before *shake*. However, if the timing of the target verb instance is conditioned on its two previous verb instances, then the timing relationships of these three verb instances can be correctly captured by making *push* start before *shake* (type 12).

Theoretically, it is possible to condition the timing of the target verb instance on all of its previous verb instances using the thirteen types of timing relationships to avoid visualising the verb instances in the wrong order. However, as far as the *Toy Story* data is concerned, it would be sufficient to simply condition the timing of the target verb instance on its immediate predecessor.

These thirteen types of timing relationships are not without problems. One problem is that some of them are not precise enough for the animation controller at run time to uniquely determine the absolute values of the timing of verb instances. Again, taking type 7 as an example, this type of timing relationship mandates a "waiting period" between the end of the first verb instance and the start of the second verb instance. However, since the actual duration of this waiting period is not specified, the animation controller will have to employ additional measures to determine when to start the second verb instance. Some of these measures will be discussed later.

Another problem is that some of the thirteen timing relationship types require a certain level of synchronisation between pairs of verb instances. For example, type 1 mandates that both verb instances in this relationship should start and end at the same time; and type 2 mandates that the first verb instance should end after the second verb instance ends. Synchronising the visualisation of multiple verb instances is a non-trivial task, especially considering that a single verb instance may be visualised by multiple actions, each of which is generated only after the previous one is finished. The synchronisation task is further complicated by the fact that the execution of an action may be affected by other concurrent actions, and therefore it may not be possible to accurately estimate the duration of an action when it first starts. For example, an action with a single path planning based basic command[5] may update its path at every frame to ensure the path is obstacle free, and as a result, if multiple virtual objects are moving at the same time, the initial path may be significantly different from the final path.

[5]Path planning is a process of creating an obstacle free path between two objects. This obstacle free path may need to be updated every single frame to account for the changing virtual scene.

Type	Frequency (%)
Type 1	89.51
Type 2	7.11
Type 3	3.38

Table 7.2: Distribution of three simplified timing relationship types in the *Toy Story* data (the timing of the target verb instance was conditioned on the timing of the previous verb instance in the text)

Given the problems with the thirteen types of timing relationship, and the fact that the available data shows that it is sufficient to simply visualise verb instances in their textual order in most cases, I decided to simplify the original timing relationships to the following three:

Type 1 The second verb instance starts immediately after the first verb instance ends.

Type 2 Both verb instances start at the same time.

Type 3 The first verb instance starts immediately after the second verb instance ends.

Table 7.2 shows the frequency distribution of the three simplified types of timing relationship in the *Toy Story* data. The timing of the target verb instance is conditioned on just its predecessor.

7.2.1 Features for the Timing Relationship Classifier

The timing relationship between a pair of verb instances is influenced by many factors, some of which are identified in this research. Judging by the most frequent type of relationship, the most important factor seems to be the order in which the two verb instances appear in the text. Other factors that can influence the timing relationship include the presence of temporal adverbs, the connectives between the two verb instances, the overlapping of semantic roles between the two verb instances, and the semantic relationship between the verbs.

Temporal adverbs can sometimes influence the timing relationship between two verb instances that are in the same sentence. For example, in sentence (7.5a), the verb instances in the phrase *gets up and slams the front door shut* appear after the verb instance in the phrase *get back inside*. However, given the presence of the temporal adverb *before*, a reasonable interpretation of this sentence would be that *get back inside* should take place after *gets up and slams the front door shut*. Similarly,

in sentence (7.5b), the verb instance *dives* appears before the verb instance *jumps*, however, given the presence of the temporal adverb *while*, it would be reasonable to start the visualisations of *dives* and *jumps* at the same time.

(7.5) a Scud suddenly realizes he's been duped, but before he can *get* back inside, Hannah *gets* up and *slams* the front door shut.

b Buzz *dives* behind the door of the living room while Woody *jumps* into a closet.

Connectives between two verb instances in the same sentence sometimes can also influence the interpretation of their timing relationship. However, the influence of connectives is not as clear cut as for temporal adverbs. For example, consider the four sentences in (7.6). All the sentences have two verb phrases connected together by *and*. In the first two sentences, it would be reasonable to interpret the two verb phrases as happening at the same time, but in the last two sentences, the only reasonable interpretation would be for the two verb phrases to happen sequentially.

(7.6) a. Slinky *walks* happily and *laughs* hard.

b. Buzz *dives* off to the side and Woody *collapses* right where he stands on the pavement.

c. Andy *picks* up Woody from the couch and *runs* upstairs.

d. The traffic light *changes* red again and the wall of cross traffic *resumes*.

The sentences in (7.6) illustrate that syntactic patterns alone cannot always be relied upon to correctly distinguish the temporal relationships between verb instances connected by the same connectives. Additional clues as to the temporal relationship between two verb instances connected by a connective can come from the overlap of their semantic roles and the verb instances themselves. For example, it was observed in the *Toy Story* data that when verb instances share the same A0 role, they are more likely to be sequential. However, sentence (7.6a) clearly shows that this is not always true, and certain verbs describe actions that can be performed simultaneously by the same agent, e.g. walking and laughing.

There are also situations in which one verb instance is the head word of the semantic role of another verb instance. In these situations, the verb instances themselves can provide vital information on their relationships. For example, consider (7.7); in sentence (7.7a), the verb instance *wearing* is the head word of the AM-ADV role of the verb instance *pops*, and the *wearing* event must happen at the same time or prior to the *pops up* event for the sentence to make sense; on the other hand, in (7.7b), the verb instance *sailing* is also the head word of the AM-ADV role of the verb instance *separates*, but in this case, the event of *sailing* must happen after the event of *separates* in order for the sentence to make sense.

(7.7) a. The rubber dolphin *pops* up *wearing* Woody's cowboy hat.

b. Finally Buzz *separates* from the plane, *sailing* him across the room towards the bed.

The following features have been designed to capture the various factors that influence the timing relationship between two verb instances:

Textual order of the verb instances This feature captures the textual order of the two verb instances, and is in the form of a tuple (sd, td), where sd is the sentence distance between the two verb instances, and td is the token distance between the two verb instances. For example, suppose that v_1 is the ith token of the jth sentence, and v_2 is the kth token of the lth sentence, then sd would be calculated as $j - l$. If sd is zero, i.e., when both verbs are in the same sentence, then td would be calculated as in Equation 7.8, otherwise, td would be set to zero.

$$td = \begin{cases} i-k & : \text{ if } |i-k| <= 3 \\ -4 & : \text{ if } i-k < -4 \\ 4 & : \text{ if } i-k > 4 \end{cases} \tag{7.8}$$

Temporal adverb This feature only applies when both verb instances are in the same sentence. The value of this feature follows the pattern of $(adverb, position)$, where *adverb* is one of *before*, *after*, *during*, *when*, *while*, *as*, *and*, *till*, and *until*, and *position* is assigned according to Equation 7.9. All temporal adverbs in the same sentence are considered by this feature.

$$position = \begin{cases} -1 & : \text{ if found before the first verb instance} \\ 0 & : \text{ if found between the two verb instances} \end{cases} \tag{7.9}$$

Temporal adverbs + Levin class This feature is an expansion of the previous temporal adverb feature. The value of this feature follows the pattern (*adverb*, *position*, *Levin class of verb 1*, *Levin class of verb 2*).[6] For example, the value of the feature for *dives* and *collapses* in sentence (7.5) would be (*while*, 0, 35.2, 45.4), where 35.2 is the Levin class number for *dive*, and 45.4 is the Levin class number for *collapse*.[7]

[6]Since each Levin verb class contains a group of verbs that are both syntactically and semantically similar, the class ID of each verb can be used to as a rough approximation of the verb's syntactic and semantic properties, thus providing some generalisation capability to the classifier.

[7]For this task, if a verb belongs to multiple Levin classes, the one with the lowest number is always chosen.

	Baseline	10-fold CV
Accuracy	89.51	91.07

Table 7.3: The accuracy of the verb-temporal relationship classifier under ten-fold cross validation (%)

Overlapping semantic roles This feature is designed to capture the overlapping of semantic roles between the two verb instances. The value of this feature is of the form (*srole_1*, *srole_2*). For example, suppose that in (7.6), the A0 of *walks* and *laughs* are both *Slinky*, then the value of this feature would be (A0, A0).

Overlapping semantic roles + Levin class This feature is an expansion of the previous overlapping semantic roles feature. The value of this feature is of the form (*srole_1*, *srole_2*, *Levin class of verb 1*, *Levin class of verb 2*). For example, suppose that the verb instance *wearing* is in the AM-ADV of the verb instance *pops* in (7.7), then the value of this feature would be (A0, A0, 43.2, 41.3.1), where 43.2 is the Levin class number for *pop* and 41.3.1 is the Levin class number for *wear*.

Semantic role containment This feature only applies when both verb instances are in the same sentence. The value of this feature follows the pattern of (*verb number*, *srole*, *temporal adverb*). For example, suppose that the verb instance *wearing* is in the AM-ADV of the verb instance *pops* in (7.7), then the value of this feature would be (1, AM-ADV, None).

7.2.2 Evaluation and Results

The classifier trained with the features described in the previous section was evaluated on the same *Toy Story* data created for the other computational tasks studied in this research. The timing relationships between the pairs of verb instances are determined based on the starting time of the first action used to visualise each verb instance, and the timing of each verb instance is conditioned only on its previous verb instance. In total, 1125 training examples were extracted, and randomised ten-fold cross validation using a MaxEnt classifier was used to evaluate the effectiveness of the features. Table 7.3 shows the resultant accuracy, which is around 1% higher than the majority class baseline, or 15% error reduction.

An analysis of the classifier's outputs showed that the classifier made most of its mistakes by misclassifying data instances of type two (both verb instances start at the same time) to type one (the first verb instance starts before the second verb instance). This suggests that the features may not be very effective. However, a closer look at

Annotators	Agreement (%)
Annotator 1 and Annotator 2	81.72
Annotator 1 and the original data	60.22
Annotator 2 and the original data	69.89

Table 7.4: Inter-annotator agreement on the classification on data instance of the type two timing relationship

the misclassified data instances show that many of them are highly ambiguous. In order to determine the level of ambiguity in the data, two computational linguists were asked to examine the data instances of type two and record whether they agree with the original classification. The result of this examination is shown in Table 7.4, and indicates that there is indeed a high level of ambiguity in the data. Given this high level of ambiguity, and the high baseline, it is not surprising that the classifier is only able to marginally improve upon the majority class baseline.

7.2.3 Related Work

Chambers *et al.* (2007) reports a machine learning based system for classifying the temporal relationships between pairs of events described in documents from the TimeBank corpus (Pustejovsky *et al.* 2003) and the Opinion Corpus (Mani *et al.* 2006). Similar to the verb-timing-relationship classification task described in this section, the timing relationships of Chambers *et al.*'s system are also based on Allen's definitions.

Chambers *et al.*'s system performs the timing relation classification in two separate stages. In the first stage, it extracts the tense, aspect, event class, modality and polarity of each event. In the second stage, it combines the information extracted in the first stage with some additional features to classify the timing relations between a pair of events. These additional features include the POS tags of each event, the relative positions between the two events in a parse tree (only applicable when the two events are in the same sentence), and the head preposition of the event, if the event is in a prepositional phrase. Furthermore, Chambers *et al.* (2007)'s system partitions the dataset based on whether the pair of events are in the same sentence, and a separate classifier is trained on each partition. Chambers *et al.* (2007)'s system shows a 3% improvement in accuracy over Mani *et al.*'s system on the same data set.

The research problem investigated in Chambers *et al.*'s work is very similar to the problem of determining the verb-timing-relationships described in this section. However, there are two important difference. Firstly, the TimeBank data consists of news articles, and the events can be both nouns and verbs; in contrast, the task described in this section focuses only on verbs, and is thus less complicated. Secondly,

SRL	Precision	Recall	F-score
Gold	.508	.701	.589
Auto	.470	.490	.480

Table 7.5: Results of the automatic grounding method on the *Toy Story* data

the movie script data tends to describe actions in nearly sequential order, whereas the news articles in the TimeBank data do not. As a result, the baseline for the movie script data is much higher than that of the TimeBank data.

In terms of the features used to construct the classifier, Chambers *et al.*'s system tends to focus on the syntactic aspect of the events, whereas the features used here focus on both the syntactic and the semantic aspect of the events through the use of semantic roles. Furthermore, the verb-timing-relationship classifier described here does not explicitly split data based on whether the pair of verbs are in the same sentence. Instead, the intra-inter-sentential property of the verb pairs is represented as a feature to the classifier.

7.3 Grounding Semantic Roles

The grounding of textual semantic roles to virtual objects in a given virtual scene is an essential step in the animation cycle. If the animation cycle is to be totally automated, the grounding process must also be automated. In this research, the grounding is performed by a string matching method.

Given the textual form of a semantic role, this method first tries to match it against the user-specified names of the virtual objects, and if no virtual objects match, it then matches the string against the WordNet synsets of the virtual objects. For example, if the virtual stage contains a virtual actor named Woody, and the string *Woody* or *Woody's hand* is to be matched, the automatic grounding module will correctly return the virtual Woody. On the other hand, if the virtual stage contains multiple virtual toys, and the string *toy* is to be matched, the automatic grounding module would incorrectly return all the virtual toys. Table 7.5 shows the results of the automatic grounding method on the gold standard semantic roles and automatically-labelled semantic roles.

7.4 Actions Classifier Setup

As stated in Section 7.1.2, the basic graphics commands are simplistic, and verbs generally need to be visualised using a sequence of actions, i.e. a sequence of graphics



a b c d

Figure 7.9: Examples of the sequence of virtual scenes corresponding to *The man grabs the mug on the table*

commands that start execution from the same time frame. In this section, we will describe how machine learning algorithms can be used to train statistical models that use the linguistic and virtual scene features of the target verb instances to select the most appropriate actions for visualisation.

The task of generating sequences of actions to visualise a verb instance is treated as a Markov process and framed as a classification problem. The feature vector of the classifier denotes the verb semantic information for the target verb instance, and the current virtual scene information of all the relevant virtual objects. The class labels denote the action that should be performed in the current semantic and virtual scene context. For example, suppose the sentence to be visualised is *The man grabs the mug on the table*, and the virtual world is initially configured as in Figure 7.9a. The classifier should first select an action which will make the virtual man stand up (Figure 7.9b); after the first action has been executed, the classifier should then select a second action to move the virtual man close to the virtual mug (Figure 7.9c); once the second action has finished, the classifier should then select a third action to make the virtual man extend his hand to the mug (Figure 7.9d). Finally, the classifier should select an `end-of-sequence` action to signal the completion of the visualisation. Each of these movements needs to be executed and the virtual scene updated, and the new information obtained from analysing the updated virtual scene used to condition the selection of the next action.

It is worth emphasising that by treating the action sequence generation task as a Markov process, the animation system is only concerned with generating the next action to be performed, which means that it is less dependent on the initial configuration of the virtual scene than if the entire sequence of actions had to be generated before the target verb instance is visualised. For instance, if the virtual scene for the sentence *The man grabs the mug on the table* was initially configured as in Figure 7.9c, then the classifier should directly generate the action that extends the virtual man's hand to the mug.

The actions are not statically defined. Instead, they are extracted from the gold

standard training data. During the creation of the data, each visualised verb instance is annotated with time stamped graphics commands. For a given verb instance, all the graphics commands that begin execution at the same time stamp are treated as an action. For an `end-of-sequence` action of a verb instance, its initial execution time stamp is set to be the finish time of the last action of the same verb instance.

7.4.1 Linguistic Features

The linguistic features were designed to capture the target verb instances and their virtual-scene-independent semantics. For a given target verb instance, its linguistic features are extracted only once, just before its first action is chosen. The linguistic features can be divided into the following categories:

Verb Features These features include the original form of the target verb, and its lemmas.

Verb Generalisation Features These features include the target verb instance's Levin Verb Classes (Levin 1993), its WordNet synsets, and the hypernyms of these synsets. The features were included to provide the classifiers with the means to generalise when dealing with previously unseen verbs. Furthermore, WSD is applied to each target verb instance to see whether disambiguating the target verb instance can improve the performance of the classifier, thus removing the WordNet synsets which are not related to the chosen sense of the target verb instance.

Collocation Features These are the lemmas of all the open class words (determined by a POS tagger) that occur in the same sentence as the target verb.

Semantic Role Features These features include: the types of roles that the target verb takes (A0, AM-LOC, etc.); the WordNet synsets of the head words of the grounded constituents[8] combined with the semantic roles of the constituents; and a series of syntactic-contextual features of the constituents of the target verb. Note that we have hand-resolved all anaphors in the data, and used this oracle-style anaphora resolution for all methods presented in this chapter.

The syntactic-contextual features of the constituents were designed to capture the details missed by the grounded semantic roles. The purpose of grounding semantic roles is to find the most relevant virtual object with respect to given textual forms of constituents, which is not sufficient if the most relevant virtual object does not exactly correspond to the semantics of the constituent. For example, consider A0 and AM-LOC of the SRLed sentence in (7.10). The head of A0 is *hand* which corresponds to a part of the body of *Andy*. However, since grounding is performed

[8]Not all constituents are grounded, e.g. AM-TMP and AM-MNR are rarely grounded.

only at the level of virtual objects, A0 is grounded to the virtual *Andy*, and the head of the constituent is lost in the grounding process. Similarly for AM-LOC, the constituent is a PP, and the most relevant virtual object is the virtual *Mr. Potato Head*. Therefore, the virtual *Mr. Potato Head* is what this phrase will be grounded to, resulting in the additional locational information of *in front of* being lost. The syntactic-collocational features reinstate the linguistic content of each constituent to the feature space, providing the classifier with the means to fine-tune the grounding information and more faithfully render the virtual scene.

(7.10) Andy's hand $_{\text{A0}}$ lowers $_{\text{TARGET}}$ a ceramic piggy bank $_{\text{A1}}$ in front of Mr. Potato Head $_{\text{AM-LOC}}$.

The syntactic-collocational features are collected recursively from the root of the smallest parse subtree covering all the tokens in the corresponding constituent. Only semantic roles which are prepositional phrases (PP), noun phrases (NP), adjective phrases (ADJP) and adverb phrases (ADVP) are used to generate these features.

If the semantic role is a PP, the feature extraction process will generate a feature that includes the head preposition and the recursively retrieved features of its argument. If the constituent is an NP, the feature extraction process will generate a feature that includes the WordNet synset of the head noun and the recursively retrieved features of its possessor and modifiers. If the semantic role is an ADJP or ADVP, the feature extraction process will generate a feature that includes its head, and all the recursively retrieved features of its modifiers. Take the constituent *in front of Mr. Potato Head*, for example. *Mr. Potato Head* would generate the feature of (NP, toy_1), and *front of Mr. Potato Head* would generate the feature of (NP, front_2, (PP_MOD, PP, *of*, (A, (NP, toy_1)))).

7.4.2 Virtual Scene Features

The virtual scene features are binary features designed to capture the spatial relationships between the grounded constituents and their environments, as well as the specific properties of the virtual objects and the current status of the animation.

The general spatial relationships are denoted as the results of different proximity and orientation boolean tests between the virtual objects. For virtual objects that contain bones, the spatial relationships also include the results of the proximity tests between their bones and other virtual objects. Brief descriptions of these proximity and orientation tests are listed in Table E.1 of Appendix E. The properties of the virtual objects and the current status of the animation are also denoted by boolean tests whose brief descriptions are shown in Table E.2 of Appendix E.

These boolean tests are always performed with respect to all combinations of semantic roles. To illustrate how the semantic roles are combined with scalar parameters of the tests and virtual objects with an armature model, consider the sentence

Phrase	Semantic Role	Virtual Object
Sid	A0	$\texttt{vo}_2$
the Jenny doll	A1	$\texttt{vo}_{10}$
Hannah	A2	$\texttt{vo}_{17}$

Table 7.6: Grounded semantic roles for the sentence *Sid hands the Jenny doll to Hannah*

((A0, A1), faceTowardsNoZ($\text{vo}_2, \text{vo}_{10}$))
((A0, A2), faceTowardsNoZ($\text{vo}_2, \text{vo}_{17}$))
((A1, A2), faceTowardsNoZ($\text{vo}_{10}, \text{vo}_{17}$))
((A1, A0), faceTowardsNoZ($\text{vo}_{10}, \text{vo}_2$))
((A2, A0), faceTowardsNoZ($\text{vo}_{17}, \text{vo}_2$))
((A2, A1), faceTowardsNoZ($\text{vo}_{17}, \text{vo}_{10}$))

Table 7.7: Features generated by faceTowardsNoZ for the sentence *Sid hands the Jenny doll to Hannah*

Sid hands the Jenny doll to Hannah. If the grounded semantic roles for the verb *hands* are as shown in Table 7.6, then the test faceTowardsNoZ will generate all the features listed in Table 7.7, and the test withinBoundingCylinderPercentage would generate all the features listed in Table 7.8.[9] Furthermore, supposed that the virtual object $\texttt{vo}_2$ has an armature containing three bones (B11, B12, and B13), and the virtual object $\texttt{vo}_{17}$ has an armature containing two bones (B21 and B22), then the test boneStalkBoneCmdApply will generate all the features listed in Table 7.9.

7.4.3 Summary

In summary, we have described the commonly used features for the actions classifier and the arguments classifier in this section. In the next section, we will describe how the class labels for the argument classifiers are extracted.

[9]The Scalar parameter of the boundingCylinderPercentage test ranges from 10 to 200, with a step size of 10.

((A0, A1), 10, boundingCylinderPercentage($\text{vo}_2, \text{vo}_{10}, 10$))
((A0, A2), 10, boundingCylinderPercentage($\text{vo}_2, \text{vo}_{17}, 10$))
((A1, A2), 10, boundingCylinderPercentage($\text{vo}_{10}, \text{vo}_{17}, 10$))
((A1, A0), 10, boundingCylinderPercentage($\text{vo}_{10}, \text{vo}_2, 10$))
((A2, A0), 10, boundingCylinderPercentage($\text{vo}_{17}, \text{vo}_2, 10$))
((A2, A1), 10, boundingCylinderPercentage($\text{vo}_{17}, \text{vo}_{10}, 10$))
((A0, A1), 20, boundingCylinderPercentage($\text{vo}_2, \text{vo}_{10}, 20$))
((A0, A2), 20, boundingCylinderPercentage($\text{vo}_2, \text{vo}_{17}, 20$))
((A1, A2), 20, boundingCylinderPercentage($\text{vo}_{10}, \text{vo}_{17}, 20$))
((A1, A0), 20, boundingCylinderPercentage($\text{vo}_{10}, \text{vo}_2, 20$))
((A2, A0), 20, boundingCylinderPercentage($\text{vo}_{17}, \text{vo}_2, 20$))
((A2, A1), 20, boundingCylinderPercentage($\text{vo}_{17}, \text{vo}_{10}, 20$))
...
((A0, A1), 200, boundingCylinderPercentage($\text{vo}_2, \text{vo}_{10}, 200$))
((A0, A2), 200, boundingCylinderPercentage($\text{vo}_2, \text{vo}_{17}, 200$))
((A1, A2), 200, boundingCylinderPercentage($\text{vo}_{10}, \text{vo}_{17}, 200$))
((A1, A0), 200, boundingCylinderPercentage($\text{vo}_{10}, \text{vo}_2, 200$))
((A2, A0), 200, boundingCylinderPercentage($\text{vo}_{17}, \text{vo}_2, 200$))
((A2, A1), 200, boundingCylinderPercentage($\text{vo}_{17}, \text{vo}_{10}, 200$))

Table 7.8: Features generated by boundingCylinderPercentage for the sentence *Sid hands the Jenny doll to Hannah*

((A0, B11), (A2, B21), boneStalkBoneCmdApply(vo_2^{B11}, vo_{17}^{B21}))
((A0, B11), (A2, B22), boneStalkBoneCmdApply(vo_2^{B11}, vo_{17}^{B22}))
((A0, B12), (A2, B21), boneStalkBoneCmdApply(vo_2^{B12}, vo_{17}^{B21}))
((A0, B12), (A2, B22), boneStalkBoneCmdApply(vo_2^{B12}, vo_{17}^{B22}))
((A0, B13), (A2, B21), boneStalkBoneCmdApply(vo_2^{B13}, vo_{17}^{B21}))
((A0, B13), (A2, B22), boneStalkBoneCmdApply(vo_2^{B13}, vo_{17}^{B22}))
((A2, B21), (A0, B11), boneStalkBoneCmdApply(vo_{17}^{B21}, vo_2^{B11}))
((A2, B21), (A0, B12), boneStalkBoneCmdApply(vo_{17}^{B21}, vo_2^{B12}))
((A2, B21), (A0, B13), boneStalkBoneCmdApply(vo_{17}^{B21}, vo_2^{B13}))
((A2, B22), (A0, B11), boneStalkBoneCmdApply(vo_{17}^{B22}, vo_2^{B11}))
((A2, B22), (A0, B12), boneStalkBoneCmdApply(vo_{17}^{B22}, vo_2^{B12}))
((A2, B22), (A0, B13), boneStalkBoneCmdApply(vo_{17}^{B22}, vo_2^{B13}))

Table 7.9: Features generated by boneStalkBoneCmdApply for the sentence *Sid hands the Jenny doll to Hannah*

7.5 Argument Classifier Setup

Since the arguments to the graphics commands are semantically typed, a separate classifier was trained for each argument type. The argument classifiers share the same linguistic and virtual scene features as the commands classifier, but also have two new features: the name of the target graphics command and the name of the target argument. In the rest of this section we will discuss the class labels of the classifiers for each type of the argument.

7.5.1 Class Labels for Single Virtual Object Arguments

Arguments which are single virtual objects are always assumed to be one of the grounded semantic roles. Therefore, the class labels for the classifier of single virtual object type arguments are just the semantic role labels.

7.5.2 Class Labels for the Bone of a Virtual Object Argument

As mentioned in Chapter 4, skeleton models for 3D polyhedrons (armatures) are an important tool in creating animation. Some of the basic graphics commands have arguments that are required to refer to a bone of the skeleton model of a virtual object. The virtual objects involved in these arguments, are also assumed to be one of the grounded semantic roles. Therefore, the class labels for the classifier of single virtual object type arguments are just the semantic role labels combined with the name of the bone.[10]

7.5.3 Class Labels for Axis, Boolean, and Pre-recorded Skeleton Animation Arguments

Axis, boolean and skeleton animation arguments all have a fixed set of discrete values. The domain for axis arguments is [x-axis, y-axis, z-axis]; the domain for boolean arguments is [True, False]; and the domain for skeleton animation arguments is the list of available pre-recorded skeleton animations.

Given the fixed and discrete nature of the domains of these 3 types of arguments, the class labels for their respective classifiers are the values of their domains.

[10] Only combinations of semantic role and bone names seen in the training data are included in the semantic role labels. For the evaluation of the argument classifier, it was observed that the test data contains all the bones observed in the training data.

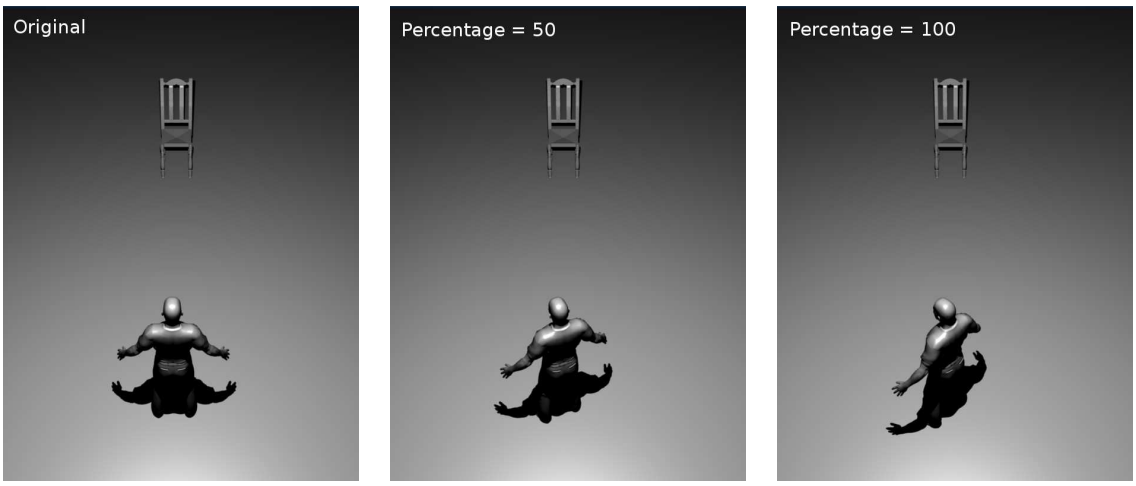


Figure 7.10: Example of the different values of the percentage parameter to `leftDblTo`

7.5.4 Class Labels for Euler, Location and Direction Arguments

Euler, location and direction arguments are all real valued, and therefore cannot be directly represented by a fixed set of class labels. As a result, some sort of discretisation method is needed to approximate the values of these arguments.

It is assumed that the values of the real valued arguments are always related to one or more grounded semantic roles. Therefore, a series of argument construction functions were developed to calculate the values of the real valued arguments on the fly. Tables F.1, F.2, and F.3 in Appendix F summarise the argument construction functions.

In general, an argument construction function requires one or two virtual object parameters, and a percentage parameter. The virtual object parameters are always specified using grounded semantic roles of the target verb instance. The percentage parameter is used to fine tune the output of the argument construction function. For example, Figure 7.10 shows results of applying the Euler construction function `leftDblTo` to a virtual man (ARG0) and a virtual chair (ARG1) with different percentage parameters, and Figure 7.11 shows results of applying the location construction function `destRadiusPer` to a virtual man (ARG0) and a virtual chair (ARG1) with different percentage parameters.

In this research, the percentage parameters are always set from zero to two hundred, with a step size of ten. The percentage parameters and the grounded semantic roles are combined to form the classification labels for the Euler angle, location and

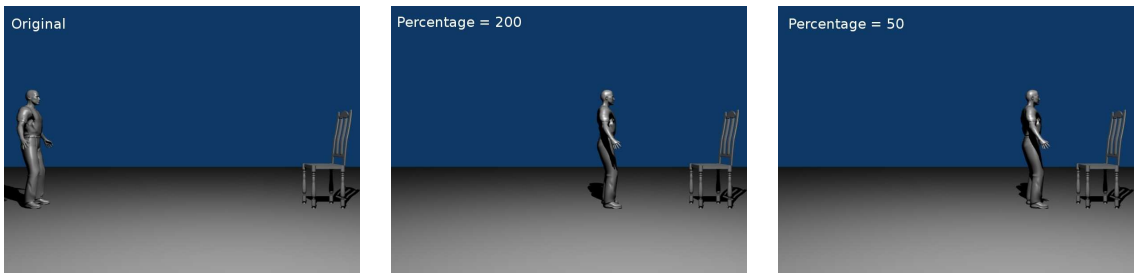


Figure 7.11: Example of the different values of the percentage parameter to `destRadiusPer`

(destRadiusPer, (arg0=A0, arg1=A1), 0)
(destRadiusPer, (arg0=A0, arg1=A1), 10)
(destRadiusPer, (arg0=A0, arg1=A1), 20)
...
(destRadiusPer, (arg0=A0, arg1=A1), 200)
(destRadiusPer, (arg0=A1, arg1=A0), 0)
(destRadiusPer, (arg0=A1, arg1=A0), 10)
(destRadiusPer, (arg0=A1, arg1=A0), 20)
...
(destRadiusPer, (arg0=A1, arg1=A0), 200)

Table 7.10: Class labels generated by the argument construction function `destRadiusPer`

direction arguments. For example, consider the sentence *Buzz gets into Woody's face*. Assuming that *Buzz* is A0 of *gets* and *Woody's face* is A1, and one of the basic graphics commands used to visualise this sentence is `TranslateObjectToPosition`, then for the location argument of `TranslateObjectToPosition`, the argument construction function `destRadiusPer` will generate the class labels shown in Table 7.10.

7.5.5 Class Labels for Scalar arguments

As shown in Tables D.1, D.2 and D.3, most of the basic graphics commands take arguments which are scalars. The scalar arguments can be divided into the following categories:

- Linear Velocity

- Angular Velocity
- Length of the IK chain
- Frequencies of update for the path planner
- Intensities of the light sources
- Playback speed for the motion capture files

Despite the different interpretations of the scalar arguments, the class labels used for their classifier are simply the values of these arguments that appeared in the training data. The justification of this configuration will be provided in the rest of this section.

In this research, the linear velocity, angular velocity, and playback speed of the motion captured files are not specified with absolute values, but as integer parameters to virtual-object-specific function. The general form of these speed functions is $f(i)$ where i is a positive integer. The implementations of these speed functions can vary with respect to different virtual objects. It could simply be a linear function that returns a speed that is multiples of a standard speed, or it could be a second degree polynomial function for a standard speed. The use of speed functions is more flexible than absolute speed because it captures the semantics of the verb instances more abstractly. For example, consider the sentence *The hunter sprinted back to its prey*, the speed of the running would be different depending on whether the species of the hunter, e.g. the sprinting of a cheetah would be much faster than the sprinting of a bear. However, the semantics of the verb instance *ran* remains the same irrespective of the species of the hunter, and this makes it difficult for a machine learner to assign the appropriate speed. On the other hand, if speed functions were used, one can simply specify in the training data that the speed of sprinting is to be calculated by $f(3)$, and let the virtual-object-specific speed function decide how to calculate the desired speed. It may be argued that the use of integer parameters to the speed functions reduces the granularity of the speed. However, since the aim of this research is to provide a tool that can generate crude animation, integer parameters to the speed functions are sufficient. The intensities of the light sources are specified in the same way.

The length of the IK chain is always a positive integer because it is determined by the number of bones in the chain. Furthermore, since the number of bones in a particular armature is always fixed, the length of the chain can only come from a fixed and finite domain, and therefore the length of the IK chain is effectively nominal.

Finally, the update frequency of the path planner also has a fixed domain with finite members. The update frequency specifies how many frames the path planner should wait before replanning the path between the source object and the destination object. Ideally, the path should be replanned at every frame to ensure the validity of

the path. However, the path planning algorithm is highly complex and would severely slow down the annotation process if performed at every single frame, especially in virtual scenes that contain many virtual objects. Furthermore, in most situations, the only moving object in the scene when the path planner is used is the source object itself, and there is no need to replan the path at every frame. As a result, only two frequencies were ever used during the annotation: five frames and infinite frames (only plan the path once). Hence, it is adequate to treat the values of the update frequency of the path planner as nominal values. It is worth noting that in theory, the update frequency of the path planner should always be one. With more efficient path planning algorithms and implementations, it may be unnecessary to keep this parameter.

7.6 Results

The focus of this chapter is on the application of machine learning algorithms to generate fully specified actions that can be used to visualise the input script. Since the process of generating actions and their parameters is framed as a classification problem, we apply metrics commonly used in statistical NLP tasks to evaluate the effectiveness of the trained classifier.

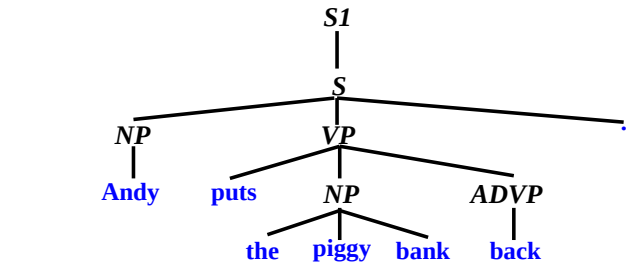
In this section, we will first discuss the various configurations of the evaluation and their purposes; we will then provide the results for the actions classifier and the argument classifier. The Student's t-test is used to perform statistical significance tests on the results ($p < 0.1$).

7.6.1 Variations of the Verb Generalisation Features

In order to test the usefulness of the verb generalisation features, the following 5 variations of these features were experimented with:

- Verb synsets + Levin classes
- Verb synsets only
- Levin classes only
- Disambiguated verb synsets only
- No verb generalisation features at all

For the disambiguated verb synsets, WSD was first performed on the target verb instance, and the synset of the chosen sense and its hypernyms are included. As stated in Chapter 6, verbs with trained supervised VSD classifiers were disambiguated by their corresponding classifiers, and the other verbs were disambiguated by SenseLearner.



Andy	puts	the piggy bank	back
-1	TARGET	+1	+2

Figure 7.12: Example of parse tree emulated semantic roles

7.6.2 Variations on the Semantic Role Features

In order to evaluate the impact of the semantic roles on the performance of the action classifier, different methods of semantic role labelling were used to label the input script: automatically labelled semantic roles, parse tree emulated semantic roles (to be discussed below), and gold standard semantic roles.

There are four types of automatically labelled semantic roles. The basic type is just the output of ASSERT (the automatic semantic role labeller described in Chapter 3). The other three types of semantic roles were obtained by first applying the PP-only semantic role labeller developed in Chapter 5 to the input script, then combining the outputs of the PP-only semantic roles with the output of ASSERT using the three different merging strategies described in Section 5.5.6.

The parse tree emulated semantic roles are essentially the relative positions of the top-level NPs, PPs, ADJPs and ADVPs with respect to the target verb instance. These phrases are extracted using the Charniak reranking parser. Figure 7.12 shows the substitute semantic roles for the sentence *Andy puts the piggy bank back*.

The gold standard semantic roles were obtained by hand correcting the output of ASSERT. Table 7.11 shows the precision, recall and F1-score of the four types of automatically obtained semantic roles with respect to the gold standard semantic roles.

7.6.3 Variations on the Grounded Semantic Roles

In order to evaluate the influence of the grounding process on the performance of the final classifier, two types of grounded semantic roles were also investigated: automatically obtained grounded verb semantic roles with the string matching method described in Section 7.3, and gold standard grounded verb semantic roles obtained by manually annotating the gold standard semantic roles.

All Verbs			
	Precision	Recall	F1-score
ASSERT Only	74.41	56.36	64.14
ASSERT+PP S1	74.37	56.38	64.14
ASSERT+PP S2	72.01	54.59	62.10
ASSERT+PP S3	71.19	53.87	61.33

Visualised Verbs Only			
	Precision	Recall	F1-score
ASSERT Only	74.79	55.93	64.00
ASSERT+PP S1	74.73	55.93	63.97
ASSERT+PP S2	72.32	54.12	61.91
ASSERT+PP S3	71.54	53.47	61.20

Table 7.11: Precision, recall and F1-score of the automatically obtained semantic roles (%)

The automatic grounding method was applied to all the different types of semantic roles. However, the gold standard grounding annotations were only applied to the gold standard semantic roles.

7.6.4 Partitioning of the Datasets

Since it was pointed out in Chapter 1 that a common weakness of animation systems driven by rule based NLP analysis is their inability to handle previously unseen verbs, we explicitly investigate the effectiveness of the machine learning based method here. Two partition methods were used to create the evaluation dataset: "stratified verbs" and "unseen verbs". For both partitioning methods, the entire dataset is partitioned into three portions: training, development and test. The training set contains roughly half of the verb instances, and the development and test sets both contain roughly a quarter of the verb instances.

For the stratified verb partition method, the partitioning of the training, development and test sets is performed such that the instances of the same verb are distributed evenly across all three datasets. In other words, this partition method is designed to minimise the impact of unseen verbs on the classifier's performance.

For the unseen verb partition method, the partitioning of the training, development and test sets is performed such that all three sets contain completely different verbs. In other words, this partition method is designed to maximise the impact of unseen verbs on the classifier's performance.

7.6.5 Action Classifiers

Table 7.12 shows the action classifier's performance under various configurations of features and partitioning methods. The most significant observation that can be made from this table is that the virtual scene features seem to have the most influence on the performance of the action classifier. Without the virtual scene features, the accuracies of the action classifier are virtually identical to the majority class baseline. The importance of the virtual scene features is not surprising, because the average number of actions per verb instance is 2.52, and the virtual scene features are the only source of information that the actions classifier can use to distinguish the different visualisation phases of the same target verb instances.

Table 7.12 also shows that the action classifier performed consistently better when the verbs are evenly distributed across the training, development and test sets than it did when the three datasets all consisted of different verbs. This is not surprising as the verb instances themselves carry a significant amount of information about how they should be visualised. It is also encouraging to see that despite the unseen verbs, the action classifier still performed better than the majority class baseline. This means that the action classifier can still be somewhat effective even when it has to handle verbs that it was not trained for. This is an improvement over rule based systems which generally cannot handle any unseen verbs at all.

The verb generalisation features seem to have some limited effectiveness. For the datasets created with the stratified verb partitioning method, overall, there is not much difference between the classifiers trained with the verb generalisation features and the classifiers trained without the verb generalisation features: when all the features were automatically obtained, the accuracy of the best performing classifier with the generalisation features (synsets and Levin Classes + Auto SRL, 54.67%) is only 2% higher than the accuracy of the best performing actions classifier without the generalisation features (Auto SRL, 52.66%); and when the gold standard features were used, the accuracy of the best performing classifier with the generalisation features (synsets and Levin Classes, 58.05%) is less than 1% higher than the accuracy of the classifier without the generalisation features (57.33%). In fact, the differences between the classifiers trained with the verb generalisation features and the classifiers trained without the verb generalisation features are not statistically significant. It is not surprising that the verb generalisation features have no significant impact for the datasets created with the stratified verb partitioning method, because in this case, the verb instances are evenly distributed across the training, development and test datasets, and there is little need for the verb generalisation features.

Accuracies (%) of the actions classifier with stratified verbs (baseline = 39.00)						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	58.05	57.97	57.73	57.57	57.33	39.05
Gold SRL + Auto. Grounding	55.55	56.04	54.67	55.07	54.35	39.29
Auto SRL	54.67	53.94	53.70	52.58	52.66	39.29
Auto SRL S1	52.74	53.14	52.90	52.49	52.49	39.29
Auto SRL S2	52.82	53.38	52.74	52.01	51.53	39.29
Auto SRL S3	53.22	53.06	52.90	52.09	51.29	39.29
Parse Tree SRL	49.44	49.03	49.27	49.76	48.55	39.29

Accuracies (%) of the actions classifier with unseen verbs (baseline = 39.00)						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	54.11	54.88	51.85	53.74	51.42	39.32
Gold SRL + Auto. Grounding	48.33	48.82	47.89	49.75	47.76	39.32
Auto SRL	45.75	45.27	44.67	48.73	45.58	39.32
Auto SRL S1	47.11	47.17	45.74	47.69	45.41	39.32
Auto SRL S2	45.36	46.80	45.70	47.50	45.41	39.32
Auto SRL S3	47.41	46.84	45.39	47.31	44.73	39.32
Parse Tree SRL	47.06	46.91	43.72	46.04	43.04	39.32

Table 7.12: Results of the Action Classifiers

The effectiveness of the verb generalisation features is slightly more pronounced with the datasets created with the unseen verb partitioning method: when all the features were automatically obtained, the accuracy of the best performing classifier with the generalisation features (Disambiguated Verb Synsets + Auto SRL, 48.73%) is 3% higher than the accuracy of the classifier without the generalisation features (Auto SRL, 45.58%); and when the gold standard features were used, the accuracy of the classifier with the generalisation features (synsets only, 54.88%) is 3.4% higher than the accuracy of the classifier without the generalisation features (51.42%). However, with the exception of the disambiguated verb synsets, there was no significant difference between all the results obtained with the verb generalisation features and the results obtained without the verb generalisation features. It is interesting that the disambiguated verb synsets performed better than the un-disambiguated verb synsets for the unseen verbs. This suggests that the excessive information in the verb synsets added more noise into the training data, and removing them through WSD could improve the performance of the classifier.

Furthermore, since some verbs occurred less than three times in *Toy Story*, there are still unseen verbs in the test sets created with the stratified verb partitioning method. More specifically, around 16.90% of the data instances in the test set were generated from verbs unseen in the training set. These instances were separated from the rest of the verbs and evaluated by themselves to see how well the classifier performed on them. Table 7.13 shows the results of both the seen and unseen verbs in the test sets created with the stratified verb partitioning method.

It can be observed from Table 7.13 that test instances with verbs seen in the training data performed significantly better than test instances without verbs seen in the training data. This is consistent with previous observations on the unseen verbs. On the other hand, the classifier trained with the stratified verb partitioning method seemed to have similar performance on unseen verbs to the classifier trained with the unseen verb partitioning method. Comparing the results of the classifier trained with the stratified partitioning method on the unseen verbs in the test set against the results of the classifier trained with the unseen verb partitioning method, the difference was once again not statistically significant. This shows that no special treatment is required for unseen verbs, underlining the robustness of the machine learning method.

The SRL features also had a positive impact on the classification task. However, it seems that they were more effective on the datasets created by the stratified verb partitioning method than they were on the datasets for the unseen verbs. Furthermore, the classifiers based on automatically obtained SRL features seem to suffer the greatest drop in performance. This is not surprising because most of the verb semantic information used by these classifiers is provided by the grounded semantic roles. When the verbs are evenly distributed in the training, development and test sets, even though the semantic role labeller didn't perform well, at least the errors were consistent between the training and test sets, and this consistency to some de-

Accuracies (%) of the actions classifier with stratified-verbs partitioning on verbs seen in the training data (baseline = 39.15)						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	59.01	59.01	58.91	58.62	57.75	39.05
Gold SRL + Auto. Grounding	56.98	57.56	56.11	55.92	55.43	39.34
Auto SRL	56.01	55.53	55.33	53.78	53.78	39.34
Auto SRL S1	53.49	54.27	54.85	53.49	53.98	39.34
Auto SRL S2	53.59	54.75	53.97	53.39	52.91	39.34
Auto SRL S3	54.46	54.17	53.98	53.30	52.33	39.34
Parse Tree SRL	50.68	50.29	50.01	50.30	49.71	39.34

Accuracies (%) of the actions classifier with stratified-verbs partitioning on verbs unseen in the training data (baseline = 39.00)						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	51.29	50.03	51.78	50.26	53.72	39.00
Gold SRL + Auto. Grounding	48.50	48.50	47.51	46.55	46.13	39.00
Auto SRL	47.89	46.02	45.56	44.08	45.99	39.00
Auto SRL S1	47.58	46.09	42.35	45.14	44.61	39.00
Auto SRL S2	48.96	46.63	46.59	45.10	44.68	39.00
Auto SRL S3	47.12	46.70	46.63	45.56	46.09	39.00
Parse Tree SRL	42.35	42.77	43.69	44.04	41.75	39.00

Table 7.13: Results of the seen and unseen verbs on the Action Classifiers with the stratified verb partitioning method

gree offsets the poor semantic role labelling performance. However, for the unseen verb datasets, the errors in automatic semantic roles are no longer consistent in the training and test sets, and only the correctly-labelled semantic roles generalise well to unseen verbs. This is why the classifier based on the gold-standard grounding annotations suffered the least performance drop among all the semantic role-based classifiers, whereas the classifier based on automatic SRL suffered the most.

On the other hand, the parser-emulated semantic roles are not greatly affected by the unseen verbs: there are no significant differences between the classifier trained with the parse-emulated semantic roles and the classifier trained with semantic roles generated by ASSERT. The relatively stable performance of the parser-emulated semantic roles is mostly due to the fact that the parser-emulated semantic roles only depend on the relative positions of the relevant phrases to the target verb. These relative positions tend to be much more verb-independent than the real semantic roles, and are therefore less affected by the variation of the target verbs.

Finally, grounding seems to have a lot of room for improvement, as the classifiers trained with gold standard grounded semantic roles generally outperformed the classifiers trained with automatically labelled gold standard semantic roles by around 2-3%. A closer look at the results showed that the gold-standard grounding annotation only identified 1266 grounded semantic roles, but the automatic grounding method came up with 1748 grounded semantic roles. This over-generation of grounded semantic roles introduced a significant amount of noise into the data, thereby lowering the performance of the classifiers trained with automatically grounded semantic roles.

7.6.6 Argument Classifiers

The evaluation of the argument classifiers is performed differently to the action classifiers. The classes of the action classifiers are all nominal, and therefore the accuracies of the classifiers can be calculated directly from how many actions were correctly chosen by the actions classifiers. On the other hand, as stated in Section 7.5, the values of some of the arguments are continuous. Therefore, the accuracies of the classifiers for these arguments cannot be calculated in the same manner as the action classifiers. Table 7.14 shows the distribution of the argument types in the annotated data and whether the value of the argument is over a continuous domain.

Furthermore, as stated in Section 7.5, the arguments to the basic graphics commands are always assumed to be related to the semantic roles of the target verb. As a result, different SRL methods will also produce different classes for the arguments. For example, consider the sentence *Sid hands the Jenny doll to Hannah*. Suppose that the phrase *Jenny doll* is labelled as A1 by the gold standard SRL, RIGHT_1 by the parse tree based SRL method, and is missed by ASSERT. If the virtual object for *Jenny doll* is used as an argument to a basic graphics command, it will generate a training example with the class label A1 for the classifier with the gold standard SRL, a training example with the class RIGHT_1 for the classifier with the parse tree

Argument Type	Percentage	Continuous
Scalar	44.55	No
Virtual Object	22.84	No
Mocap	8.30	No
Location	7.38	Yes
Euler	6.58	Yes
Axis	4.13	No
Bone	3.73	No
Bool	1.93	No
Direction	0.56	Yes

Table 7.14: Distributions of argument types in the annotated data

based SRL, and nothing for the classifier with ASSERT's SRL. Therefore, classifiers configured with different SRL methods may have different distributions of classes, and different majority class baselines.

The evaluation of the argument classifiers is separated into two groups: nominal arguments and continuous arguments. The classifiers for the nominal arguments are evaluated in the same manner as the actions classifiers. Table 7.15 shows the results of the classifiers over the nominal arguments. Table 7.16 shows the majority class baselines of these classifiers.

The classifier for the location argument is evaluated by summing up the absolute distances between the annotator assigned location and the classifier assigned location. Since the perfect classifier should assign the exact same location as the human annotator would, the smaller the distance is, the better the classifier performed. Table 7.17 shows the results of the location classifiers. Unlike the nominal arguments, the majority class baseline cannot be always applied here due to possible incompatibilities between the argument construction functions and the available semantic roles. For example, suppose the majority class baseline for the classifier of the location arguments is (destRadiusPer, (arg0=A0, arg1=A1), 20). If the only available semantic roles is A0, then it would not be possible to fully specify the parameters to the argument construction function `destRadiusPer`, and the baseline class would not generate a location for comparison. On the other hand, since the class labels for the location classifiers represent different approximations of the annotator assigned locations, it would be useful to establish an upper bound for the accuracy of these approximations using the shortest possible distance that can be calculated from the argument construction functions. Table 7.18 shows this upper bound. In order to evaluate the effectiveness of the location classifier, it is necessary to show the context of the distances with the 3D graphical models used in reanimating Toy Story. Fig-

Weighted accuracies (%) of the nominal argument classifier with stratified verbs						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	71.12	71.07	71.22	71.33	71.38	72.06
Gold SRL + Auto. Grounding	71.16	71.11	70.68	70.26	70.23	72.19
Auto SRL	72.69	72.50	72.21	71.77	71.36	72.23
Auto SRL S1	70.41	70.46	71.02	70.73	69.78	71.77
Auto SRL S2	70.13	70.28	70.12	69.70	68.47	71.05
Auto SRL S3	69.88	70.06	70.03	69.98	69.43	71.31
Parse Tree SRL	69.52	69.63	69.38	69.22	69.14	71.13

Weighted accuracies (%) of the nominal argument classifier with unseen verbs						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	67.24	67.44	67.89	67.67	68.31	68.81
Gold SRL + Auto. Grounding	67.58	67.59	68.70	68.08	68.41	67.53
Auto SRL	67.59	66.89	68.27	67.81	68.15	67.89
Auto SRL S1	65.69	65.77	67.00	66.66	67.61	66.36
Auto SRL S2	65.36	65.08	66.03	66.23	66.43	66.49
Auto SRL S3	65.81	65.63	67.05	66.71	66.81	66.41
Parse Tree SRL	64.25	64.41	65.40	64.85	65.88	64.81

Table 7.15: Results of the argument classifiers for the nominal arguments

Weighted majority class baselines (%) of nominal argument classifier with stratified verbs	
Grounding and SRL Variations	Baseline
Gold SRL + Gold Grounding	43.09
Gold SRL + Auto. Grounding	42.90
Auto SRL	40.69
Auto SRL S1	39.64
Auto SRL S2	39.64
Auto SRL S3	39.64
Parse Tree SRL	40.88

Weighted majority class baselines (%) of the nominal argument classifier with unseen verbs	
Grounding and SRL Variations	Baseline
Gold SRL + Gold Grounding	41.13
Gold SRL + Auto. Grounding	40.03
Auto SRL	37.47
Auto SRL S1	37.41
Auto SRL S2	37.41
Auto SRL S3	37.41
Parse Tree SRL	36.69

Table 7.16: Majority class baselines for the argument classifiers for the nominal arguments

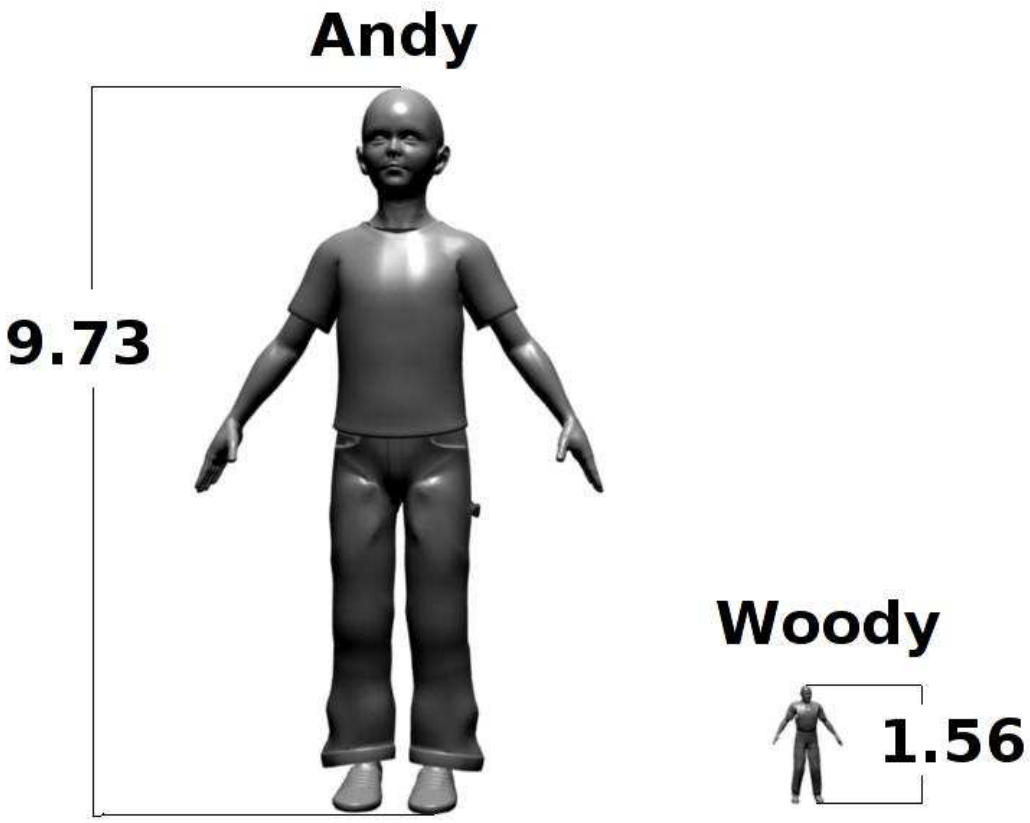


Figure 7.13: Heights of the 3D models for Andy and Woody used in reanimating Toy Story (the models are stored based on Blender's internal representation of dimensions, and therefore it is not necessary to have explicit unit for length)

ure 7.13 shows the heights of the characters Andy (an 8-year old child) and Woody (a cowboy doll), and their heights can be used to measure the effectiveness of the location classifier.

The classifiers for the Euler angle and direction arguments are evaluated by summing up the absolute differences in angles between the annotator assigned Euler angle/direction and the classifier assigned Euler angle/direction. Similar to the evaluation of the location classifier, the smaller the difference, the better the performance. Table 7.19 shows the results of the Euler angle and direction classifiers. Table 7.20 shows this upper bound of the Euler angle and direction classifiers.

One general observation that can be made from the results of the argument classifiers is that the virtual scene features are no longer essential, and the classifiers can achieve very similar results with or without them. This is not surprising because

Average of the distances for the location argument classifier with stratified verbs						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	8.96	6.59	8.70	8.48	8.76	7.45
Gold SRL + Auto. Grounding	12.12	12.64	13.46	12.60	13.50	12.10
Auto SRL	13.55	13.49	12.75	12.31	12.78	11.33
Auto SRL S1	13.40	13.47	13.64	11.76	11.60	14.71
Auto SRL S2	11.26	11.25	11.29	14.44	11.40	15.11
Auto SRL S3	11.29	11.12	11.30	11.16	11.31	15.07
Parse Tree SRL	12.25	11.94	13.03	15.96	13.03	11.08

Average of distances for the location argument classifier with unseen verbs						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	10.14	10.15	9.97	7.70	9.58	8.41
Gold SRL + Auto. Grounding	11.12	11.15	11.21	11.21	11.09	10.62
Auto SRL	13.80	13.19	13.37	13.31	13.17	13.44
Auto SRL S1	14.38	14.97	13.50	15.03	14.99	13.51
Auto SRL S2	14.52	14.53	13.51	14.50	13.50	13.53
Auto SRL S3	13.17	13.17	13.16	13.16	13.16	13.56
Parse Tree SRL	13.03	14.24	28.92	28.90	15.16	12.56

Table 7.17: Results of the argument classifiers for the location arguments

Upper bound of the location classifier with stratified verbs	
Grounding and SRL Variations	Upper bound
Gold SRL + Gold Grounding	2.83
Gold SRL + Auto. Grounding	6.33
Auto SRL	7.40
Auto SRL S1	6.68
Auto SRL S2	6.74
Auto SRL S3	6.98
Parse Tree SRL	5.10

Upper bound of the location classifier with unseen verbs	
Grounding and SRL Variations	Upper bound
Gold SRL + Gold Grounding	3.62
Gold SRL + Auto. Grounding	7.33
Auto SRL	11.04
Auto SRL S1	10.62
Auto SRL S2	10.64
Auto SRL S3	10.27
Parse Tree SRL	4.68

Table 7.18: Upper bound for the argument classifiers for the Location arguments

the values of the arguments are determined by the semantic roles of the target verb instances, and semantic roles are determined before the verb instances are visualised. Therefore, the virtual scene features provides little extra information that can be used by the classifiers and may even introduce additional noise into the data.

Another general observation is that the gold standard semantic roles no longer have any noticeable advantage over the automatically obtained semantic roles. This is somewhat surprising since the arguments to the basic graphics commands are calculated from the semantic roles. We then performed a series of additional analyses by looking at the classifier's performance on each type of argument, and identified several factors which contributed this lack of differences. Firstly, even though the automatically obtained semantic roles were less accurate than the gold standard semantic roles, the errors were consistent across the training, development and test sets, and as a result, the argument classifier was able to adapt to the mis-labelled semantic roles, and achieve similar performance to when the gold standard semantic roles were used. Secondly, the `Scalar` and `Mocap` arguments are mostly dependent on the target verbs instead of semantic roles. This is not surprising, because `Scalar` arguments generally represent the intrinsic speed of the verbs, and the `Mocap` arguments generally represent the poses depicted by the verbs, therefore, neither type of argument is strongly related to the semantic roles. Since the the `Scalar` and `Mocap` arguments together account for more than half of all the arguments, it is not surprising that the

Average of differences for the Euler angle and direction argument classifier with stratified verbs						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	39.08	38.64	39.12	37.39	37.84	43.54
Gold SRL + Auto. Grounding	44.74	45.80	43.10	42.14	42.14	46.27
Auto SRL	42.49	44.82	41.17	42.34	42.60	46.98
Auto SRL S1	47.30	44.78	45.99	46.36	46.13	48.13
Auto SRL S2	43.98	43.98	46.38	44.16	43.61	44.56
Auto SRL S3	44.37	45.31	46.71	45.73	45.74	48.26
Parse Tree SRL	51.23	45.40	48.70	50.11	49.82	55.52

Average of differences for the Euler angle and direction argument classifier with unseen verbs						
Grounding and SRL Variations	Verb Generalisation Features					No Virtual Scene Features
	Synsets and Levin Classes	Synsets Only	Levin Classes Only	Disambigauted Verb Synsets	No Generalisation Features	
Gold SRL + Gold Grounding	65.32	64.38	67.09	70.17	71.25	59.37
Gold SRL + Auto. Grounding	66.81	68.13	66.95	67.77	67.36	68.50
Auto SRL	70.62	70.75	67.89	62.02	62.09	71.50
Auto SRL S1	75.09	76.21	72.93	77.03	76.13	75.08
Auto SRL S2	78.73	77.70	80.27	76.21	80.20	74.95
Auto SRL S3	78.72	80.44	78.68	78.69	75.28	76.94
Parse Tree SRL	69.67	65.22	68.68	67.73	72.78	71.21

Table 7.19: Majority class baselines for the argument classifiers for the Euler angle and direction arguments

Upper bound of the Euler angle and direction classifier with stratified verbs	
Grounding and SRL Variations	Upper bound
Gold SRL + Gold Grounding	1.18
Gold SRL + Auto. Grounding	1.78
Auto SRL	2.34
Auto SRL S1	1.77
Auto SRL S2	1.77
Auto SRL S3	1.79
Parse Tree SRL	1.66

Upper bound of the Euler angle and direction classifier with unseen verbs	
Grounding and SRL Variations	Upper bound
Gold SRL + Gold Grounding	1.61
Gold SRL + Auto. Grounding	2.61
Auto SRL	2.66
Auto SRL S1	2.74
Auto SRL S2	2.74
Auto SRL S3	2.80
Parse Tree SRL	2.33

Table 7.20: Upper bound for the argument classifiers for the Euler angle and direction arguments

quality of the semantic roles does not have a dominant impact on the effectiveness of the argument classifier. Thirdly, some types of arguments (`Axis` and `Bool`) have dominant majority classes, and therefore are also not affected by the quality of the semantic roles.

For the continuous arguments, it seems that both the location classifier and the Euler angle classifier performed rather poorly: the average distance between the predicted locations and the gold standard locations is around 15, which is considerably greater than the height of Andy, who is involved in many verb instances; and the average difference between the predicted Euler angles and the gold standard Euler angles is around 45 degrees, which can have a noticeable visual impact. However, since the results of these two classifiers are calculated as averages, the classifiers may not have performed poorly all the time. For example, suppose that the individual distances between the predicted locations and the gold standard locations are 1, 2, 3 and 50, then the average distance will be 14, which does not accurately reflect the performance of the location classifier in the first three cases. In order to get a more accurate picture of the effectiveness of the location and Euler angle classifiers, further analysis was performed on the individual outputs of each classifier to obtain the distributions of the differences between the predicted locations and gold standard locations, and predicted Euler angles and the gold standard Euler angles. Two of these distributions are illustrated in Figures 7.14 and 7.15. It can be observed that

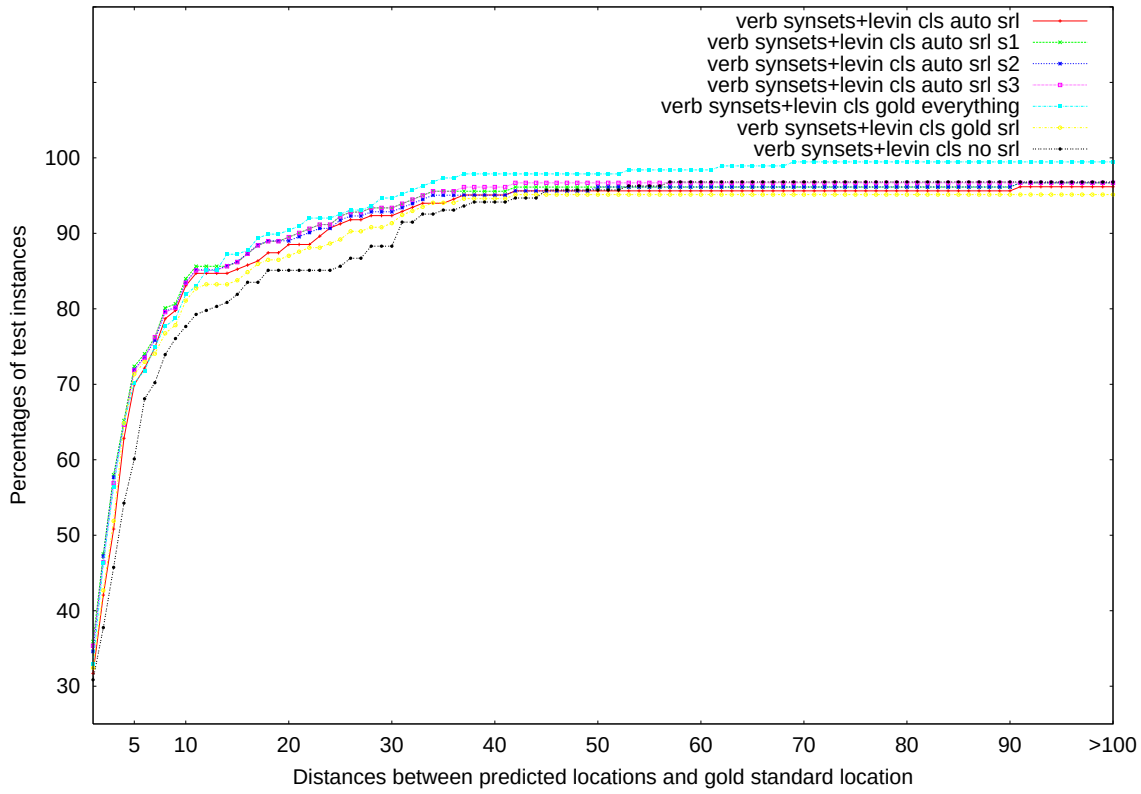


Figure 7.14: Distribution of the distances between predicted locations and gold standard locations when verb synsets and Levin classes are used as the verb generalisation features under stratified-verbs partitioning

around 70% of the predicted locations are less than 5 degrees from the gold standard locations, and 55% of predicted Euler angles are less than 10 degrees from the gold standard Euler angles. This confirms that both classifiers indeed performed reasonably well in most instances, while making some big mistakes in a small number of cases.

Some of the more significant mistakes made by the location and Euler angle classifiers come from the inconsistencies in the training data. For example, the phrase *away* in sentences such as *Woody walks away* is under-specified in terms of how it should be interpreted. As a result, the human annotator often did not interpret it consistently, which then introduced noise into the data, making the classifiers less effective.

Furthermore, the discretisation method used by the classifiers for the continuous arguments also contributed to the inaccuracy of the classifiers. Recall that the labels

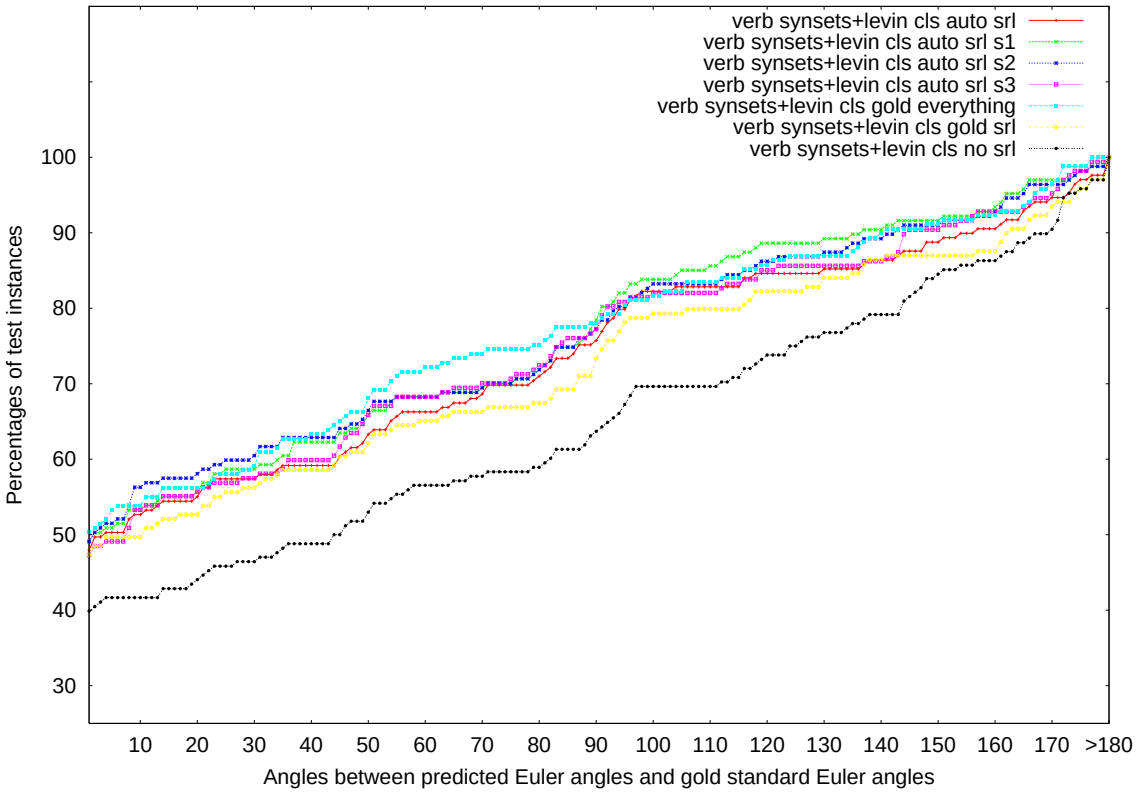


Figure 7.15: Distribution of the angles between predicted Euler angles and gold standard Euler angles when verb synsets and Levin classes are used as the verb generalisation features under stratified-verbs partitioning

used by the classifiers for the continuous arguments are in the form of (construction-function, semantic-role-list, percentage), where percentage is discretised in steps of 10 with the maximum value of 200 (i.e., [0%, 10%, 20%, 30%, ..., 200%]), and the value of the continuous argument is calculated by the construction-function with the grounded semantic roles and percentage as its arguments. The results of the location and Euler angle classifiers show that this discretisation method may be too limited to capture the distances between the target virtual objects when they are far apart. For example, if the gold standard location can be calculated with percentage being 800, and the classifier selects 200, then the distance between the predicted location and the gold standard location will be enormous. However, as the limit of the discretisation can be trivially increased, it may not be very challenging to eliminate the problems related to the discretisation method.

Finally, as with the action classifier, the argument classifier performed significantly better with the stratified-verb-partitioning than the unseen-verb partitioning. This is not surprising given the Scalar and Mocap arguments' dependencies on the verbs. Additional analysis showed that the argument classifier's performances on the Scalar and Mocap arguments each dropped around 10%, which is much worse than the other types of arguments.

7.7 Conclusion

In this chapter, I have described a machine learning framework for generating computer animation based on the linguistic analysis of natural language instructions. This framework consists of four computational tasks: determining the timing relationships between the verb instance in the same input text, grounding textual semantic roles to virtual objects, generating a sequence of actions to visualise a verb instance, and specifying the arguments to the basic commands in the chosen action.

The task of determining the timing relationships of verb instances is framed as a classification task, with the class labels denoting the type of timing relationship between two verb instances in the input text. I showed how the original thirteen types of timing relationships can be simplified to three types due to the nature of the graphics commands and inherit complexity of the task. An analysis of the data showed that in most cases, it is sufficient to visualise verb instances in their order of appearance in the text. A classifier was then trained to predict the timing relationships between pairs of verb instances, which marginally outperformed the majority class baseline for the task.

A rule based system was constructed to handle the task of grounding the textual representation of verb semantic roles to virtual objects. This system works by matching the names and WordNet synsets of the virtual objects in the scene to the string representation of the verb semantic roles. However, it tends to over-generate grounded semantic roles, which in turn introduces a significant amount of noise in

the generation of graphics commands.

The task of generating sequences of actions to visualise a verb instance is also framed as a classification task, with the class labels denoting the next action to perform. A classifier was trained with the script and a reanimated version of *Toy Story*, and multiple techniques for generalising this classifier over unseen verbs were investigated. The results showed that while the classifier's performance drops when dealing with previously unseen verbs, it is still effective. Furthermore, the virtual scene features investigated in this research are essential to the effectiveness of the action classifier.

Finally, the task of specifying the arguments to the basic commands in the chosen actions is also framed as a classification task, with the class labels denoting mechanisms to calculate the value of the target arguments. Using the same data as for training the action classifier, several argument classifiers were trained to handle the different types of arguments to the basic commands. The results showed that unlike the action classifier, the virtual scene features are no longer essential for determining the arguments. Furthermore, the results also showed that some arguments are strongly dependent on the consistency of the semantic roles, while other arguments are strongly dependent on the target verbs. Overall, the argument classifier is reasonably robust for the semantic-role-dependent arguments regardless of the quality of the semantic roles, or whether the verb has been seen in the training data before. This indicates that there is a strong correlation between these types of arguments and the semantic roles of the verbs, thus allowing the machine learner to adapt to the incorrect but consistent semantic role labelling. However, the argument classifier is less robust for the verb-dependent arguments, which indicates that the generalisation capability of the machine learning approach is still limited, and more research is needed to overcome this limitation.

Chapter 8

Conclusion

This thesis describes a machine learning framework for the task of generating computer animations based on natural language instructions in pre-constructed virtual scenes. The main contributions of this research include a novel method for performing semantic role labelling on prepositional phrases, a novel method for performing verb sense disambiguation, and most importantly, a domain-independent empirical approach for mapping verb semantics to animation procedures based on training data. With respect to the task of natural language based animation generation, the most important findings of this research are that: (1) semantic role labelling has a significantly positive contribution; (2) verb sense disambiguation can contribute positively to the task, but its contribution is marginal; and (3) virtual scene features can be used in conjunction with linguistic features to perform planning and reasoning. I will discuss each of these three findings in turn.

The positive contribution of semantic role labelling is expected. However, it is surprising that the quality of the semantic roles does not proportionally translate to performance gains in the final animation system. For example, the F-score of the chosen semantic role labeller over the movie script corpus is around 64%, but the actions classifier could only gain around 3% in accuracy using gold standard semantic roles (without gold standard grounding) over automatically obtained semantic roles. This demonstrates the robustness of the machine learning framework, which learnt to adapt to the imperfections of the semantic role labeller. Furthermore, since the machine learning framework does not have fixed rules for interpreting verb semantics, and instead creates the rules dynamically based on the training data, its robustness in verb semantic interpretation can translate to other domains.

The marginal contribution from verb sense disambiguation is disappointing. However, it does demonstrate the ineffectiveness of WSD systems when they are applied to domains which they are not designed for. The most significant problem with the VSD task is that the verb instances in the movie script corpus often do not match the senses from the chosen sense inventory (WordNet). The lack of appropriate senses caused the sense tagged training data to be noisy and inconsistent, thereby reducing

the effectiveness of the supervised VSD system. Another significant problem with the VSD task is the lack of a multiword expression recogniser, since a verb may have different sense distributions depending on whether it is part of a multiword expression.

The virtual scene features are essential to the success of the machine learning framework. The machine learning framework does not come with predefined rules to perform planning and reasoning to achieve the desired animation. However, this research has shown that the virtual scene features can be used in conjunction with the linguistic features to automatically induce these rules from the training data, thereby making the reasoning and planning process more flexible.

Overall, the research reported in this thesis offers a new alternative approach to the task of natural language based animation generation, which, up until now, has been dominated by domain-specific symbolic systems. The most significant difference between the machine learning framework proposed in this thesis and the existing symbolic systems is that the symbolic systems need human experts to handcraft rules for driving computer animation systems based on the semantics of the input instructions, while the machine learning framework needs human annotators to create training examples which are then used by empirical systems to automatically create these rules.

I began this thesis with a comprehensive overview in Chapter 1 which introduced the machine learning framework for natural language based animation generation and its pipeline architecture. This pipeline architecture consists of two main components: a linguistic analysis component which extracts semantic information from input text, and an animation generation component which maps semantic information to computer graphics commands that generate the animation. In order to put this research into perspective with current research, I reviewed the most relevant existing projects in Chapter 2, and highlighted their differences and similarities of with this research. Then, for the purpose of laying the foundation for the linguistics and virtual scene analysis to be carried out in this research, I described the most relevant natural language processing and computer graphics techniques in Chapters 3 and 4.

Descriptions of the novel research reported in this thesis start from the semantic analysis of the input text. The key linguistic analyses applied in this study are semantic role labelling and verb sense disambiguation. Semantic role labelling is used to identify the arguments and modifiers of verb instances in the input text, and verb sense disambiguation is used to identify the most appropriate sense of the verb instances. Given the importance of semantic role labelling and verb sense disambiguation, I decided that it was necessary to investigate whether the current-start-of-the-art for these two techniques could be improved. As a result, a novel method for performing semantic role labelling on prepositional phrases and a novel method for performing verb sense disambiguation were proposed.

The research on the semantic role labelling of prepositional phrases was reported in Chapter 5. The proposed method consisted of three components: verb-preposition attachment disambiguation, preposition semantic role disambiguation, and semantic

role boundary determination. It was shown that this method was able to marginally improve the performance of the top three semantic role labelling systems in CoNLL 2004, however, it fell short on the movie script data collected in this research.

The research on verb sense disambiguation was reported in Chapter 6. The proposed method combined selectional preferences extracted from automatically obtained semantic roles with commonly used n-gram features, and was evaluated on the movie script data and data from the Senseval-2 lexical samples task. The results of the evaluation were somewhat ambiguous, as the SRL-based selectional preference features made positive contributions for the Senseval-2 data, but had negative influence on the movie script data.

The animation generation component mainly focused on investigating how machine learning techniques can be applied to map verb semantic information to a sequence of graphics commands that can produce a faithful visualisation of the verb semantic information. There are four computational tasks in the animation generation component: determining the timing relationships between the verb instances in the same input text, grounding textual versions of semantic roles to objects in the virtual scene, selecting sequences of graphics commands to visualise verb instances, and specifying the arguments to the chosen graphics commands. All four tasks have been investigated and reported in Chapter 7.

The action and argument classifiers were evaluated in multiple configurations to investigate how semantic role labelling, verb sense disambiguation, verb generalisation features, grounding and the virtual scene features can influence the final performance of the framework on the individual verb instances. For the action classifier, the results show that the most influential features are the virtual scene features, followed by the grounding of the semantic roles, and the quality of the semantic role labelling. The influence of the virtual scene features are not surprising because they are the only source of information that denote the state of visualisation. Furthermore, the virtual scene features used in this research are simply binary tests of the geometric relationships between virtual objects, and can be expanded or reduced transparently with respect to the training of the classifiers. Their positive influence on the action classifier shows that the machine learning based approach may indeed be a more flexible alternative to handcrafted rules which explicitly keep track of the visualisation of verb instances.

The influence of grounding was shown by the fact that the action classifier performed significantly better with gold standard grounding than with automatically obtained grounding. This indicates that the grounding method used here has a lot of room for improvement, and I will discuss possible improvements in Section 8.1.

The influence of semantic role labelling was positive because it outperformed its parse tree alternative in most of the configurations of the action classifier, and the action classifier had consistently better results when configured with higher quality semantic roles. This result is encouraging because it shows that semantic role labelling is not an artificial NLP task and can have practical applications.

It is somewhat disappointing that verb sense disambiguation did not seem to have any significant impact on the performance of the action classifier. This ineffectiveness was caused by the lack of training data, the lack of a multiword-expression detector and the mismatch of the sense inventory (WordNet) with the senses of the verbs in the movie script corpus. I hope to address these two issues in future research. The linguistic data used in this research was the script of *Toy Story* which contained around 950 instances of around 250 unique verbs. There was just not enough data to training a supervised classifier for the verbs that have many senses in the data. The lack of a multiword-expression detector further complicated the lack of training data, because many verbs occurred as part of multiword-expressions and therefore were erroneously treated as individual verb instances. These verb instances introduced a significant amount of noise into the already small training set and further impeded the effectiveness of the supervised disambiguation system constructed in this research. Finally, WordNet often did not have the senses that accurately describe the meaning of the verb instances in the training data, causing many inconsistencies in the sense tagged corpus, and further reducing the effectiveness of the supervised verb sense disambiguation system constructed in this research.

It is also disappointing that the verb generalisation features did not make a significant contribution to the action classifier across all configurations. When the action classifier was configured with gold standard grounding and semantic role labelling data, it performed much better with the generalisation features than without. However, when it was configured with automatically obtained grounding and semantic role labelling data, the generalisation features did not make any consistently significant difference. This shows that while verb generalisation features are useful, their effectiveness still depends on the quality of the grounding and semantic role labelling process. However, even when the test data contained none of the verbs seen in the training data, the action classifier still significantly outperformed the majority class baseline. This shows that the machine learning based approach can still be somewhat useful for unseen verbs, and therefore provides a better alternative to the rule based approaches.

Finally, the argument classifier showed similar patterns to the action classifier with respect to the influence of verb generalisation features. However, the influence of semantic role labelling, grounding, and the virtual scene features is much less pronounced for the argument classifier than the action classifier.

8.1 Future Work

Some of the research questions investigated in this thesis were simplified to fit the scope of this research. In this section, I will list these tasks and describe how they can be approached in a more comprehensive way.

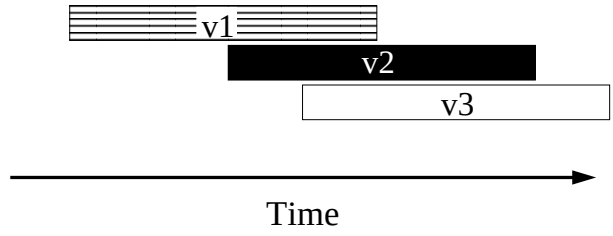


Figure 8.1: Example timing relationship between three verb instances

Verb Timing Relationships

As described in Chapter 7, the task of determining timing relationships between the verb instances in the same input text was simplified such that the original thirteen types of timing relationships were reduced to just three types. The reason for this simplification was mainly due to the fact that many of the timing relationships require a certain level of synchronisation between a pair of verb instances. Such synchronisation may not be feasible because it is not always possible to know in advance how long it will take to visualise each verb instance, as the visualisation of an earlier verb instance can be interrupted or delayed by the visualisation of a later verb instance. One way to address this problem would be to use a look-ahead-and-rewind approach in which the verb instances are visualised sequentially, and the time is rolled back after each one is visualised so that its timing information is known to the next verb instance. For example, suppose three verb instances v_1, v_2 and v_3 were to be synchronised in the way depicted in Figure 8.1, then v_1 would be visualised first so that its timing would be known to v_2; then v_2 would be visualised in a trial-and-error fashion until the timing condition between it and v_1 are satisfied; finally, v_3 would be also visualised in a trial-and-error fashion until the timing conditions between it and v_1 and v_2 are satisfied. This look-ahead-and-rewind approach would be substantially more time consuming than the simplified approach used in this research, and would also require a slightly different animation process.

Grounding

The grounding method described in Chapter 7 was based on matching the strings of the semantic roles to the WordNet synsets and names of the objects in the virtual scene. This method was shown to be inadequate and cannot take into account adjectival phrases, prepositional phrases or subordinate clauses that modify the noun phrases in the target semantic roles.

The presence of the virtual world makes it possible to ground adjectival phrases,

prepositional phrases and even subordinate clauses, and this could be done in a similar way to the argument classifiers. For example, separate classifiers could be trained to map different types of adjectival phrases, prepositional phrases and subordinate clauses to individual filter functions which can be used to eliminate virtual objects that do not fit the textual description of the semantic role. For instance, given the phrase *the biggest green ball on the floor*, the colour adjectival phrase classifier may choose a filter function `filterColorGreen` to deal with the adjective *green*, the size adjectival phrase classifier may choose a filter function `filterSizeBiggest` to deal with the adjective *biggest*, and the prepositional phrase classifier may choose a filter function `filterPositionOn` to deal with the prepositional phrase *on the floor*. Once the filters have been chosen, they can then be chained together in the order of colour-size-location to finally select the correct virtual green ball.

The principles of the filters are independent of graphics systems, and the same process of training action and argument classifiers could also be used to train the classifiers for selecting the filters.

Linguistic Analysis

A major problem with the NLP tools used in this research was that none of them were trained on the movie script data (with the exception of the supervised verb sense disambiguation system). This mismatch of genre caused these NLP tools to make more mistakes than they would on data of their target genre, and these mistakes in turn introduced a significant amount of noise into the animation generation.

One possible solution to this problem would be to retrain all these tools using the movie script data, however, this is not practical in the scope of this research due to the lack of resources. Alternatively, it has been shown by Daumé (2007) that it is possible to retrain these tools with just a small amount of data from the new genre. His method may be a more practical solution to the problem of mismatch of genres.

8.2 Summary

Generating computer animation from natural language instructions is a complex task that encompasses several key aspects of artificial intelligence including natural language understanding, computer graphics and knowledge representation. In this thesis, I have presented a machine learning framework for constructing systems to perform natural language based animation generation. This machine learning framework has the advantage of being domain-independent, more flexible and having better coverage. The work reported in this thesis is the first attempt at applying empirical methods to solve the general problem of natural language based animation generation, and the results clearly show the viability of the proposed empirical approach.

Bibliography

Agirre, Eneko, and David Martinez. 2001. Decision lists for English and Basque. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 115–118, Toulouse, France.

Allen, James F. 1983. Maintaining knowledge about temporal intervals. *Communications of the ACM* 26.832–43.

Badler, Norman I., Rama Bindiganavale, Jan Allbeck, William Schuler, Liwei Zhao, and Martha Palmer. 2000. Parameterized action representation for virtual human agents. *Embodied conversational agents* 256–284.

——, Cary B. Phillips, and Bonnie Lynn Webber. 1993. *Simulating Humans: Computer Graphics Animation and Control*. New York, USA: Oxford University Press, USA.

Baldwin, Timothy. 2006. Distributional similarity and preposition semantics. In *Syntax and Semantics of Prepositions*, ed. by Patrick Saint-Dizier, 197–210. Springer.

Berger, Adam L., Vincent J. Della Pietra, and Stephen A. Della Pietra. 1996. A maximum entropy approach to natural language processing. *Computational Linguistics* 22.39–71.

Brill, Eric. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics* 21.543–565.

Briscoe, Ted, and John Carroll. 2002. Robust accurate statistical annotation of general text. In *Proc. of the 3rd International Conference on Language Resources and Evaluation (LREC 2002)*, 1499–1504, Las Palmas, Canary Islands.

Bullon, Stephen (ed.) 2006. *Longman Dictionary of Contemporary English*. Harlow, England: Pearson ESL; 4th edition.

Cabezas, Clara, Philip Resnik, and Jessica Stevens. 2001. Supervised sense tagging using support vector machines. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 59–62, Toulouse, France.

Carreras, Xavier, and Lluís Màrquez. 2004. Introduction to the CoNLL-2004 shared task: Semantic role labeling. In *Proc. of the 8th Conference on Natural Language Learning (CoNLL-2004)*, 89–97, Boston, USA.

——, Lluís Màrquez, and Grzegorz Chrupa. 2004. Hierarchical recognition of propositional arguments with perceptrons. In *Proc. of the 8th Conference on Natural Language Learning (CoNLL-2004)*, 106–109, Boston, USA.

Chambers, Nathanael, Shan Wang, and Dan Jurafsky. 2007. Classifying temporal relations between events. In *Proc. of the 45rd Annual Meeting of the ACL Short Papers*, 173–176, Prague, Czech Republic.

Charniak, Eugene. 1997. Statistical techniques for natural language parsing. *AI Magazine* 18.33–43.

——. 2000. Maximum-entropy-inspired parser. In *Proc. of the 1st Annual Meeting of the North American Chapter of Association for Computational Linguistics (NAACL2000)*, 132–139, Seattle, USA.

——, and Mark Johnson. 2005. Coarse-to-fine n-best parsing and maxent discriminative reranking. In *Proc. of the 43rd Annual Meeting of the ACL*, 173–180, Ann Arbor, USA.

Chen, Jinying, and Martha Palmer. 2005. Towards robust high performance word sense disambiguation of english verbs using rich linguistic features. In *Proc. of the Second International Joint Conference on Natural Language Processing*, 933–944, Jeju Island, Korea.

Chen, Mao, Klaus Dorer, Ehsan Foroughi, Fredrik Heintz, ZhanXiang Huang, Spiros Kapetanakis, Kostas Kostiadis, Johan Kummeneje, Jan Murray, Itsuki Noda, Oliver Obst, Pat Riley, Timo Steffens, Yi Wang, and Xiang Yin, 2003. *RoboCup Soccer Server Users Manual, for Soccer Server Version 7.07 and later*.

Chen, Stanley, and Ronald Rosenfeld. 1999. A Gaussian prior for smoothing maximum entropy models. Technical report, School of Computer Science, Carnegie Mellon University, Pittsburgh, USA.

Church, Kenneth Ward. 1988. A stochastic parts program and noun phrase parser for unrestricted text. In *Proc. of the Second Conference on Applied Natural Language Processing*, 136–143, Austin, USA.

Ciaramita, Massimiliano, and Yasemin Altun. 2006. Broad-coverage sense disambiguation and information extraction with a supersense sequence tagger. In *Proc. of the 2006 Conference on Empirical Methods in Natural Language Processing (EMNLP 2006)*, 594–602, Sydney, Australia.

Clear, Jeremy H. 1993. The British national corpus. *The digital word: text-based computing in the humanities* 163–187.

Cohn, Trevor, Andrew Smith, and Miles Osborne. 2005. Scaling conditional random fields using error-correcting codes. In *Proc. of the 43rd Annual Meeting of the ACL*, 10–17, Ann Arbor, USA.

Collin Baker, Charles Fillmore, John Lowe. 1998. The Berkeley FrameNet Project. In *Proc. of the 36th Annual Meeting of the ACL and 17th International Conference on Computational Linguistics: COLING/ACL-98*, 86–90, Montreal, Canada.

Collins, Michael. 1997. Three generative lexicalised models for statistical parsing. In *Proc. of the 35th Annual Meeting of the ACL and 8th Conference of the EACL (ACL-EACL'97)*, 16–23, Madrid, Spain.

——, 1999. *Head-Driven Statistical Models for Natural Language Parsing*. Department of Computer and Information Science, University of Pennsylvania dissertation.

——, and James Brooks. 1995. Prepositional phrase attachment through a backed-off model. In *Proc. of the Third Workshop on Verb Large Corpora*, 27–38, Somerset, USA.

Coyne, Bob, and Richard Sproat. 2001. WordsEye: an automatic text-to-scene conversion system. In *SIGGRAPH '01: Proc. of the 28th Annual Conference on Computer Graphics and Interactive Techniques*, 487–496, New York, USA.

Crestan, Eric, Marc El-Beze, and Claude de Loupy. 2001. Improving wsd with multi-level view of context monitored by similarity measure. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 67–70, Toulouse, France.

Dale, Robert. 1989. Cooking up referring expressions. In *Proc. of the 27th Annual Meeting of the ACL*, 68–75, Vancouver, Canada.

Darroch, J.N., and Ratcliff D. 1972. Generalised iterative scaling for log-linear models. *The Annals of Mathematical Statistics* 43.1470–1480.

Daumé, Hal. 2007. Frustratingly easy domain adaptation. In *Proc. of the 45rd Annual Meeting of the ACL*, 256–263, Prague, Czech Republic.

Di Eugenio, Barbara, 1993. *Understanding natural language instructions: A computational approach to purpose clauses*. Department of Computer and Information Science, University of Pennsylvania dissertation.

Dowty, David. 1991. Thematic proto-roles and argument selection. *Language* 67.547–619.

Edmonds, Philip, 2002. Senseval: The evaluation of word sense disambiguation systems. ELRA Newsletter, Vol 7, No. 3.

——, and Scott Cotton. 2001. Senseval-2: Overview. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 21–24, Toulouse, France.

Escudero, Gerard, Lluis Márquez, and German Rigau. 2001. Using lazy boosting for word sense disambiguation. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 71–74, Toulouse, France.

Fellbaum, Christiane (ed.) 1998. *WordNet: An Electronic Lexical Database*. Cambridge, USA: The MIT Press.

Fernandez-Amoros, David, Julio Gonzalo, and Felisa Verdejo. 2001. The UNED systems and Senseval-2. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 75–78, Toulouse, France.

Foley, James D., Andries van Dam, Steven K. Feiner, John F. Hughes, and Richard L. Phillips. 1990. *Introduction to Computer Graphics*. Reading, USA: Addison-Wesley Publishing Company.

Funakoshi, Kotaro, and Tokunaga Takenobu. 2006. Identifying repair targets in action control dialogue. In *Proc. of the 11th Conference of the European Chapter of the Association of Computational Linguistics (EACL2006)*, 177–184, Trento, Italy.

——, and Takenobu Tokunaga. 2005. Controlling animated agents in natural language. In *Proc. of the 2nd International Joint Conference on Natural Language Processing (IJCNLP-05) (Poster and Demo)*, 93–98, Jeju, South Korea.

——, ——, and Hozumi Tanaka. 2002. Processing Japanese self-correction in speech dialog systems. In *Proc. of the 19th international conference on Computational linguistics*, 1–7, Taipei, Taiwan.

——, Satoru Watanabe, and Takenobu Tokunaga. 2006. Group-based generation of referring expressions. In *Proc. of the Fourth International Natural Language Generation Conference*, 73–80, Sydney, Australia.

Gale, William A., Kenneth W. Church, and David Yarowsky. 1992. One sense per discourse. In *HLT '91: Proc. of the workshop on Speech and Natural Language*, 233–237, Harriman, USA.

Giménez, Jess, and Llus Márquez. 2004. SVMTool: A general POS tagger generator based on support vector machines. In *Proc. of the 4th International Conference on Language Resources and Evaluation (LREC 2004)*, 43–46, Lisbon, Portugal.

Hacioglu, Kadri, Sameer Pradhan, Wayne Ward, James H. Martin, and Daniel Jurafsky. 2004. Semantic role labeling by tagging syntactic chunks. In *Proc. of the 8th Conference on Natural Language Learning (CoNLL-2004)*, 110–113, Boston, USA.

Haynes, Sherwood. 2001. Semantic tagging using WordNet examples. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 79–82, Toulouse, France.

Johansson, Richard, Anders Berglund, Magnus Danielsson, and Pierre Nugues. 2005. Automatic text-to-scene conversion in the traffic accident domain. In *IJCAI-05, Proc. of the Nineteenth International Joint Conference on Artificial Intelligence*, 1073–1078, Edinburgh, Scotland.

——, David Williams, Anders Berglund, and Pierre Nugues. 2004. Carsim: A System to Visualize Written Road Accident Reports as Animated 3D Scenes. In *ACL2004: Second Workshop on Text Meaning and Interpretation*, 57–64, Barcelona, Spain.

Johnson, Christopher, and Charles J. Fillmore. 2000. The FrameNet tagset for frame-semantic and syntactic coding of predicate-argument structure. In *Proc. of the 1st Annual Meeting of the North American Chapter of Association for Computational Linguistics (NAACL2000)*, 56–62, Seattle, USA.

Kate, Rohit J., and Raymond J. Mooney. 2006. Using string-kernels for learning semantic parsers. In *Proc. of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics*, 913–920, Sydney, Australia.

Kilgarriff, Adam. 2001. English lexical sample task description. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 17–20, Toulouse, France.

Kingsbury, Paul, and Martha Palmer. 2002. From treebank to PropBank. In *Proc. of the 3rd International Conference on Language Resources and Evaluation (LREC 2002)*, 1989–1993, Las Palmas, Canary Islands.

Kipper, Karin, Hoa Trang Dang, William Schuler, and Martha Palmer. 2000. Building a class-based verb lexicon using tags. In *Proc. of the Fifth International Workshop on Tree Adjoining Grammars and Related Formalisms*, 1–8, Paris, France.

Klein, Dan, Kristina Toutanova, H. Tolga Ilhan, Sepandar D. Kamvar, and Christopher D. Manning. 2001. Combining heterogeneous classifiers for word-sense disambiguation. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 87–90, Toulouse, France.

Korhonen, Anna, and Judita Preiss. 2003. Improving subcategorization acquisition using word sense disambiguation. In *Proc. of the 41st Annual Meeting of the ACL*, 48–55, Sapporo, Japan.

Lafferty, John D., Andrew McCallum, and Fernando C. N. Pereira. 2001. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *ICML '01: Proc. of the Eighteenth International Conference on Machine Learning*, 282–289, San Francisco, USA.

Leung, Ka Ming, 2003. Script visualisation. Master's thesis, Department of Computer Science and Software Engineering, University of Melbourne.

Levin, Beth. 1993. *English Verb Classes and Alternation: A Preliminary Investigation*. Chicago, USA: The University of Chicago Press.

Li, Hang, and Naoki Abe. 1998. Generalizing case frames using a thesaurus and the mdl principle. *Computational Linguistics* 24.217–244.

Litkowski, Kenneth. 2001. Using machine readable dictionaries for word-sense disambiguation in Senseval-2. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 107–110, Toulouse, France.

Litkowski, Kenneth C., and Orin Hargraves. 2007. Semeval-2007 task 06: Word-sense disambiguation of prepositions. In *Proc. of the Fourth International Workshop on Semantic Evaluations (SemEval-2007)*, 24–29, Prague, Czech Republic.

Liu, D.C., and Nocedal J. 1989. On the limited memory BFGS method for large scale optimization. *Math. Program.* 45.503–528.

Magerman, David M. 1995. Statistical decision-tree models for parsing. In *Proc. of the 33rd Annual Meeting of the ACL*, 276–283, Cambridge, USA.

Magnini, Bernardo, Carlo Strapparava, Giovanni Pezzulo, and Alfio Gliozzo. 2001. Using domain information for word sense disambiguation. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 111–114, Toulouse, France.

Mani, Inderjeet, Marc Verhagen, Ben Wellner, Chong Min Lee, and James Pustejovsky. 2006. Machine learning of temporal relations. In *Proc. of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics*, 753–760, Sydney, Australia.

Marcus, Mitchell P., Mary Ann Marcinkiewicz, and Beatrice Santorini. 1993. Building a large annotated corpus of English: the Penn Treebank. *Computational Linguistics* 19.313–330.

McCarthy, Diana, and John Carroll. 2003. Disambiguating nouns, verbs, and adjectives using automatically acquired selectional preferences. *Computational Linguistics* 29.639–654.

Mihalcea, Rada, Timothy Chklovski, and Adam Kilgarriff. 2004. The Senseval-3 English lexical sample task. In *Senseval-3: Third International Workshop on the Evaluation of Systems for the Semantic Analysis of Text*, 25–28, Barcelona, Spain.

——, and Andras Csomai. 2005. SenseLearner: Word sense disambiguation for all words in unrestricted text. In *Proc. of the 43nd Annual Meeting of the Association for Computational Linguistics, companion volume*, 53–56, Ann Arbor, USA.

——, and Dan Moldovan. 2001. Pattern learning and active feature selection for word sense disambiguation. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 127–130, Toulouse, France.

Miller, George A. 1995. WordNet: a lexical database for English. *Communications of the ACM* 38.39–41.

——, Claudia Leacock, Randee Tengi, and Ross T. Bunker. 1993. A semantic concordance. In *HLT '93: Proc. of the workshop on Human Language Technology*, 303–308, Princeton, USA.

Minnen, Guido, John Carroll, and Darren Pearce. 2001. Applied morphological processing of English. *Natural Language Engineering* 7.207–223.

Ng, Hwee Tou, and Hian Beng Lee. 1996. Integrating multiple knowledge sources to disambiguate word sense: An exemplar-based approach. In *Proc. of the 34th Annual Meeting of the ACL*, 96–1006, Santa Cruz, USA.

Ngai, Grace, and Radu Florian. 2001. Transformation-based learning in the fast lane. In *Proc. of the 2nd Annual Meeting of the North American Chapter of Association for Computational Linguistics (NAACL2001)*, 1–8, Pittsburgh, USA.

O'Hara, Tom, and Janyce Wiebe. 2003. Preposition semantic classification via treebank and FrameNet. In *Proc. of the 7th Conference on Natural Language Learning (CoNLL-2003)*, 79–86, Edmonton, Canada.

Palmer, Martha, Christiane Fellbaum, Scott Cotton, Lauren Delfs, and Hoa Trang Dang. 2001. English tasks: All-words and verb lexical sample. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 21–24, Toulouse, France.

Pedersen, Ted. 2001. Machine learning with lexical features: The Duluth approach to Senseval-2. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 139–142, Toulouse, France.

Porter, M. F. 1997. An algorithm for suffix stripping. *Readings in information retrieval* 313–316.

Punyakanok, Vasin, Dan Roth, Wen-Tau Yih, Dav Zimak, and Yuancheng Tu. 2004. Semantic role labeling via generalized inference over classifiers. In *Proc. of the 8th Conference on Natural Language Learning (CoNLL-2004)*, 130–133, Boston, USA.

Pustejovsky, James, Patrick Hanks, Roser Sauri, Andew See, David Day, Lisa Ferro, Robert Gaizauskas, Marcia Lazo, Andrea Setzer, and Beth Sundheim. 2003. The Timebank corpus. *Corpus Linguistics* 647–656.

Ratner, Peter. 1998. *3-D Human Modelling and Animation*, chapter 1, 42–44. John Wiley and Sons, Inc.

Resnik, Philip, 1993. *Selection and Information: A Class-Based Approach to Lexical Relationships*. Department of Computer and Information Science, University of Pennsylvania dissertation.

——. 1997. Selectional preference and sense disambiguation. In *Proc. of ACL SIGLEX Workshop on Tagging Text with Lexical Semantics, Why, What and How?*, 52–57, Washington, USA.

Rissanen, Joram. 1978. Modelling by shortest data description. *Automatica* 14.465–471.

Rogers, David F. 1997. *Procedural Elements for Computer Graphics*, chapter 5. New York, USA: McGraw-Hill Science/Engineering/Math, Second Edition.

——, and J. Allen Adams. 1989. *Mathematical Elements for Computer Graphics, Second Edition*, chapter 3. New York, USA: McGraw-Hill, Inc.

Sag, Ivan A., Timothy Baldwin, Francis Bond, Ann Copestake, and Dan Flickinger. 2002. Multiword expressions: A pain in the neck for NLP. In *Proc. of the 3rd International Conference on Intelligent Text Processing and Computational Linguistics (CICLing-2002)*, 1–15, Mexico City, Mexico.

Sang, Erik F. Tjong Kim, and Sabine Buchholz. 2000. Introduction to the CoNLL-2000 shared task: Chunking. In *Proc. of the 4th Conference on Computational Natural Language Learning (CoNLL-2000)*, 127–132, Lisbon, Portugal.

——, and Fien De Meulder. 2003. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proc. of the 7th Conference on Natural Language Learning (CoNLL-2003)*, 142–147, Edmonton, Canada.

——, and Jorn Veenstra. 1999. Representing text chunks. In *Proc. of the 9th Conference of the European Chapter of the Association for Computational Linguistics (EACL'99)*, 173–179, Bergen, Norway.

Sekine, S., 1998. *Corpus-based Parsing and Sublanguage Studies*. New York University dissertation.

Seo, Hee-Cheol, Sang-Zoo Lee, Hae-Chang Rim, and Ho Lee. 2001. KUNLP system using classification information model at Senseval-2. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 147–150, Toulouse, France.

Stevenson, Mark. 2003. *Word Sense Disambiguation, The Case for Combinations of Knowledge Sources*. Stanford, USA: CSLI Publications.

Tokugana, Takenoku, Kotaro Funakoshi, and Hozumi Tanaka. 2004. K2: Animated agents that understand speech commands and perform actions. In *Proc. of the 8th Pacific Rim International Conference on Artificial Intelligence: PRICAI 2004* , 635–643, Auckland, New Zealand.

Tugwell, David, and Adam Kilgarriff. 2001. Wasp-bench: a lexicographic tool supporting word sense disambiguation. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 151–154, Toulouse, France.

Vapnik, Vladimir. 1995. *The Nature of Statistical Learning Theory*. New York, USA: Springer.

Welman, Chris, 1993. Inverse kinematics and geometric constraints for articulated figure manipulation. Master's thesis, Simon Fraser University.

Wernecke, Josie. 1994. *The Inventor Mentor: Programming Object-Oriented 3D Graphics with Open Inventor, Release 2 (OpenGL)*, chapter 3. Reading, USA: Addison-Wesley Professional.

Wilks, Yorick, Brian Slator, and Louise Guthrie. 1996. *Electric Words, Dictionaries, Computers, and Meanings*, chapter 5. Cambridge, USA: The MIT Press.

Winograd, Terry. 1972. *Understanding Natural Language*. New York, USA: Academic Press.

Yarowsky, David. 1993. One sense per collocation. In *HLT '93: Proc. of the workshop on Human Language Technology*, 266–271, Princeton, USA.

——. 1995. Unsupervised word sense disambiguation rivaling supervised methods. In *Proc. of the 33rd Annual Meeting of the ACL*, 189–196, Cambridge, USA.

——. 2000. *Handbook of Natural Language Processing*, chapter 26. New York, USA: Marcel Dekker.

——, Silviu Cuzerzan, Radu Florian, Charles Schafer, and Richard Wicentowski. 2001. The John Hopkins Senseval-2 system descriptions. In *Proc. of the Second International Workshop on Evaluating Word Sense Disambiguation Systems (SENSEVAL-2)*, 163–166, Toulouse, France.

Ye, Patrick, and Timothy Baldwin. 2006a. Semantic role labeling of prepositional phrases. *ACM Transactions on Asian Language Information Processing (TALIP)* 5.228–244.

——, and ——. 2006b. Verb sense disambiguation using selectional preferences extracted with a state-of-the-art semantic role labeler. In *Proc. of the 2006 Australasian Language Technology Workshop*, 141–148, Sydney, Australia.

——, and ——. 2007. MELB-YB: Preposition sense disambiguation using rich semantic features. In *Proc. of the Fourth International Workshop on Semantic Evaluations (SemEval-2007)*, 241–244, Prague, Czech Republic.

——, and ——. 2008. Towards automatic animated storyboarding. In *Proc. of the Twenty-Third AAAI Conference on Articifical Intelligence (AAAI-08)*, 578–583, Chicago, USA.

Zelle, John M., and Raymond J. Mooney. 1996. Learning to parse database queries using inductive logic programming. In *Proc. of 13th National Conference on Artifical Intelligience (AAAI-96)*, 1050–1055, Portland, USA.

Appendix A

Accuracies of the Features Categories in Verb Sense Disambiguation

This appendix lists the accuracies of the various categories of features used in the verb sense disambiguation experiments described in Chapter 6.

Verb (Majority Class Frequency %)	Bag of Words	Bag of Synsets	Bag of POS tags	Bag of Chunk Tags	Bag of Chunk Types	Bag of NE Tags
begin(58.44)	60.05 /100.0	57.34 /100.0	*80.29* /100.0	78.58 /100.0	60.89 /100.0	57.53 /85.79
call(23.78)	30.22 /100.0	26.75 /100.0	39.01 /100.0	*44.88* /100.0	34.53 /100.0	20.14 /83.22
carry(23.13)	41.01 /100.0	*43.45* /100.0	31.02 /100.0	29.80 /100.0	29.74 /100.0	19.43 /78.36
collaborate(91.23)	*94.15* /100.0	*94.15* /100.0	91.96 /100.0	90.50 /100.0	91.23 /100.0	91.23 /82.46
develop(29.63)	39.64 /100.0	*40.91* /100.0	35.86 /100.0	37.55 /100.0	30.52 /100.0	25.89 /81.48
draw(13.25)	41.18 /100.0	*42.38* /100.0	31.67 /100.0	39.10 /100.0	36.02 /100.0	11.06 /83.13
dress(38.52)	51.45 /100.0	*53.07* /100.0	51.36 /100.0	47.46 /100.0	49.08 /100.0	31.59 /81.97
drift(29.69)	35.53 /100.0	*38.49* /100.0	29.46 /100.0	31.66 /100.0	27.93 /100.0	32.59 /75.00
drive(31.76)	58.92 /100.0	*61.27* /100.0	59.05 /100.0	60.24 /100.0	48.07 /100.0	32.55 /85.88
face(80.31)	79.10 /100.0	78.49 /100.0	76.85 /100.0	72.45 /100.0	**80.31** /100.0	75.31 /91.71
find(15.91)	*31.25* /100.0	26.61 /100.0	31.07 /100.0	25.50 /100.0	22.93 /100.0	10.87 /78.03
keep(36.30)	43.90 /100.0	40.49 /100.0	*52.03* /100.0	44.70 /100.0	39.55 /100.0	31.94 /86.67
leave(28.79)	42.23 /100.0	38.02 /100.0	*47.24* /100.0	42.84 /100.0	41.51 /100.0	22.67 /66.67
live(48.85)	55.36 /100.0	*56.16* /100.0	51.89 /100.0	46.60 /100.0	47.22 /100.0	48.67 /84.73
match(35.23)	36.52 /100.0	*37.90* /100.0	31.66 /100.0	29.53 /100.0	34.06 /100.0	19.15 /73.86
play(19.38)	45.69 /100.0	*46.40* /100.0	39.49 /100.0	25.69 /100.0	28.12 /100.0	15.07 /79.07
pull(22.13)	*38.65* /100.0	32.89 /100.0	31.34 /100.0	33.28 /100.0	30.21 /100.0	10.41 /78.69
replace(50.00)	*54.53* /100.0	48.95 /100.0	53.88 /100.0	48.48 /100.0	47.32 /100.0	46.37 /94.32
see(31.06)	49.78 /100.0	*50.58* /100.0	38.23 /100.0	36.51 /100.0	35.58 /100.0	16.27 /77.27
serve(26.00)	48.01 /100.0	45.17 /100.0	*50.56* /100.0	48.51 /100.0	40.70 /100.0	25.77 /78.00
strike(16.35)	38.79 /100.0	*43.81* /100.0	41.09 /100.0	33.13 /100.0	33.59 /100.0	5.22 /84.62
train(26.77)	*50.41* /100.0	47.54 /100.0	48.37 /100.0	36.79 /100.0	37.71 /100.0	21.17 /66.14
treat(32.97)	*53.90* /100.0	45.38 /100.0	48.68 /100.0	40.27 /100.0	40.27 /100.0	33.52 /78.02
turn(15.15)	39.92 /100.0	37.90 /100.0	49.56 /100.0	*49.67* /100.0	45.49 /100.0	12.28 /75.00
use(67.57)	67.30 /100.0	67.16 /100.0	67.18 /100.0	*68.65* /100.0	67.57 /100.0	67.57 /86.49
wander(83.00)	84.00 /100.0	84.00 /100.0	*86.20* /100.0	83.21 /100.0	83.00 /100.0	58.28 /73.00
wash(23.08)	58.59 /100.0	*60.06* /100.0	50.19 /100.0	38.78 /100.0	49.42 /100.0	17.50 /50.00
work(27.05)	30.80 /100.0	25.57 /100.0	*40.82* /100.0	33.42 /100.0	34.36 /100.0	20.26 /77.05

Table A.1: N-gram based features cross validation accuracy (and coverage) on the training set(%)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	61.55 /100.0	62.53 /100.0	62.37 /100.0	*62.77* /100.0	62.39 /100.0	62.56 /100.0
call(23.78)	34.17 /100.0	*34.98* /100.0	33.28 /100.0	32.95 /100.0	32.36 /100.0	31.72 /100.0
carry(23.13)	27.27 /100.0	26.29 /100.0	25.53 /100.0	26.15 /100.0	*30.08* /100.0	29.47 /100.0
collaborate(91.23)	91.23 /100.0	86.12 /100.0	87.95 /100.0	85.76 /100.0	92.25 /100.0	*93.42* /100.0
develop(29.63)	26.04 /100.0	24.71 /100.0	28.37 /100.0	26.59 /100.0	25.42 /100.0	**29.18** /100.0
draw(13.25)	24.77 /100.0	25.92 /100.0	29.92 /100.0	28.90 /100.0	*30.75* /100.0	30.74 /100.0
dress(38.52)	31.93 /100.0	32.02 /100.0	35.31 /100.0	37.31 /100.0	36.13 /100.0	*39.49* /100.0
drift(29.69)	27.27 /100.0	27.93 /100.0	28.83 /100.0	33.50 /100.0	*34.36* /100.0	31.09 /100.0
drive(31.76)	41.84 /100.0	47.82 /100.0	47.64 /100.0	51.05 /100.0	*51.99* /100.0	48.17 /100.0
face(80.31)	79.81 /100.0	**80.31** /100.0	73.02 /100.0	76.06 /100.0	79.81 /100.0	**80.31** /100.0
find(15.91)	18.14 /100.0	18.52 /100.0	*26.92* /100.0	23.58 /100.0	23.11 /100.0	23.31 /100.0
keep(36.30)	29.81 /100.0	33.53 /100.0	30.71 /100.0	34.82 /100.0	33.36 /100.0	*42.72* /100.0
leave(28.79)	32.49 /100.0	30.63 /100.0	*35.61* /100.0	30.98 /100.0	31.23 /100.0	29.37 /100.0
live(48.85)	42.98 /100.0	43.58 /100.0	47.10 /100.0	49.48 /100.0	*49.60* /100.0	47.65 /100.0
match(35.23)	25.74 /100.0	*43.70* /100.0	41.77 /100.0	37.24 /100.0	34.25 /100.0	35.20 /100.0
play(19.38)	15.42 /100.0	20.69 /100.0	*21.81* /100.0	21.72 /100.0	20.59 /100.0	21.79 /100.0
pull(22.13)	17.56 /100.0	24.59 /100.0	26.53 /100.0	29.03 /100.0	*31.39* /100.0	26.10 /100.0
replace(50.00)	49.23 /100.0	*52.09* /100.0	48.22 /100.0	46.95 /100.0	46.84 /100.0	48.72 /100.0
see(31.06)	**30.53** /100.0	29.25 /100.0	29.39 /100.0	29.17 /100.0	28.55 /100.0	30.37 /100.0
serve(26.00)	29.96 /100.0	28.45 /100.0	32.75 /100.0	*33.51* /100.0	32.96 /100.0	32.87 /100.0
strike(16.35)	26.98 /100.0	32.73 /100.0	34.11 /100.0	32.66 /100.0	*34.39* /100.0	32.63 /100.0
train(26.77)	*41.13* /100.0	40.77 /100.0	41.05 /100.0	40.58 /100.0	38.64 /100.0	40.67 /100.0
treat(32.97)	45.16 /100.0	42.86 /100.0	47.25 /100.0	46.87 /100.0	*47.97* /100.0	46.87 /100.0
turn(15.15)	25.69 /100.0	27.32 /100.0	29.85 /100.0	31.16 /100.0	29.91 /100.0	*32.58* /100.0
use(67.57)	66.89 /100.0	*68.78* /100.0	68.65 /100.0	67.57 /100.0	67.57 /100.0	67.84 /100.0
wander(83.00)	*84.23* /100.0	79.72 /100.0	83.27 /100.0	81.99 /100.0	80.48 /100.0	83.31 /100.0
wash(23.08)	34.10 /100.0	36.15 /100.0	*39.23* /100.0	36.92 /100.0	35.26 /100.0	34.49 /100.0
work(27.05)	24.23 /100.0	24.22 /100.0	29.70 /100.0	27.27 /100.0	29.17 /100.0	*30.94* /100.0

Table A.2: N-gram based features cross validation accuracy (and coverage) on the training set (%) (POS tags of cooccuring words to the left of the target verb and their relative positions)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	82.59 /100.0	*83.14* /100.0	82.80 /100.0	82.00 /100.0	81.57 /100.0	81.02 /100.0
call(23.78)	*40.40* /100.0	39.59 /100.0	37.78 /100.0	35.99 /100.0	30.92 /100.0	33.23 /100.0
carry(23.13)	27.41 /100.0	27.06 /100.0	25.13 /100.0	25.58 /100.0	*29.07* /100.0	26.61 /100.0
collaborate(91.23)	91.23 /100.0	91.23 /100.0	91.23 /100.0	88.60 /100.0	*93.42* /100.0	90.13 /100.0
develop(29.63)	*36.76* /100.0	32.72 /100.0	26.43 /100.0	29.28 /100.0	30.72 /100.0	28.47 /100.0
draw(13.25)	20.28 /100.0	21.96 /100.0	29.36 /100.0	26.40 /100.0	*29.92* /100.0	29.24 /100.0
dress(38.52)	49.38 /100.0	50.75 /100.0	*54.75* /100.0	52.89 /100.0	52.16 /100.0	50.79 /100.0
drift(29.69)	23.52 /100.0	*35.15* /100.0	31.00 /100.0	26.54 /100.0	26.16 /100.0	26.67 /100.0
drive(31.76)	47.88 /100.0	46.65 /100.0	*51.12* /100.0	47.17 /100.0	49.52 /100.0	48.87 /100.0
face(80.31)	78.61 /100.0	76.54 /100.0	75.13 /100.0	75.16 /100.0	76.38 /100.0	**80.31** /100.0
find(15.91)	27.36 /100.0	*28.43* /100.0	23.77 /100.0	27.77 /100.0	25.78 /100.0	26.22 /100.0
keep(36.30)	42.80 /100.0	37.66 /100.0	42.26 /100.0	45.05 /100.0	*46.48* /100.0	46.14 /100.0
leave(28.79)	33.92 /100.0	*42.73* /100.0	40.66 /100.0	39.64 /100.0	38.21 /100.0	38.86 /100.0
live(48.85)	*56.50* /100.0	55.50 /100.0	53.35 /100.0	55.37 /100.0	54.56 /100.0	53.06 /100.0
match(35.23)	31.91 /100.0	27.58 /100.0	28.86 /100.0	26.57 /100.0	31.37 /100.0	**34.56** /100.0
play(19.38)	36.54 /100.0	*40.42* /100.0	37.37 /100.0	36.95 /100.0	34.72 /100.0	34.73 /100.0
pull(22.13)	27.65 /100.0	33.78 /100.0	*36.39* /100.0	35.02 /100.0	34.13 /100.0	34.49 /100.0
replace(50.00)	*57.97* /100.0	54.59 /100.0	54.55 /100.0	57.29 /100.0	55.02 /100.0	54.98 /100.0
see(31.06)	28.33 /100.0	33.93 /100.0	*36.29* /100.0	36.20 /100.0	32.50 /100.0	32.75 /100.0
serve(26.00)	56.75 /100.0	*58.15* /100.0	55.94 /100.0	55.91 /100.0	52.34 /100.0	46.10 /100.0
strike(16.35)	*34.28* /100.0	32.90 /100.0	29.71 /100.0	32.34 /100.0	30.10 /100.0	29.16 /100.0
train(26.77)	*43.62* /100.0	37.85 /100.0	39.33 /100.0	42.30 /100.0	41.01 /100.0	40.84 /100.0
treat(32.97)	40.27 /100.0	42.58 /100.0	40.38 /100.0	45.77 /100.0	48.08 /100.0	*48.08* /100.0
turn(15.15)	41.40 /100.0	50.00 /100.0	51.89 /100.0	50.40 /100.0	*51.99* /100.0	46.89 /100.0
use(67.57)	64.90 /100.0	64.59 /100.0	65.22 /100.0	**67.03** /100.0	60.87 /100.0	66.89 /100.0
wander(83.00)	77.96 /100.0	**81.30** /100.0	77.37 /100.0	77.37 /100.0	81.14 /100.0	79.98 /100.0
wash(23.08)	*58.33* /100.0	47.37 /100.0	49.36 /100.0	44.81 /100.0	48.14 /100.0	49.81 /100.0
work(27.05)	*42.09* /100.0	35.97 /100.0	39.68 /100.0	37.85 /100.0	41.49 /100.0	39.11 /100.0

Table A.3: N-gram based features cross validation accuracy (and coverage) on the training set (%) (POS tags of cooccuring words to the right of the target verb and their relative positions)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	80.84 /100.0	81.42 /100.0	*81.59* /100.0	79.76 /100.0	79.33 /100.0	78.39 /100.0
call(23.78)	*43.04* /100.0	40.05 /100.0	41.29 /100.0	39.33 /100.0	36.80 /100.0	39.54 /100.0
carry(23.13)	*37.37* /100.0	26.11 /100.0	28.61 /100.0	28.85 /100.0	31.53 /100.0	30.67 /100.0
collaborate(91.23)	91.23 /100.0	92.33 /100.0	93.06 /100.0	90.87 /100.0	*94.15* /100.0	93.42 /100.0
develop(29.63)	31.47 /100.0	34.85 /100.0	29.36 /100.0	31.18 /100.0	31.67 /100.0	*36.84* /100.0
draw(13.25)	*33.85* /100.0	30.79 /100.0	31.68 /100.0	32.70 /100.0	28.14 /100.0	29.43 /100.0
dress(38.52)	*58.90* /100.0	54.00 /100.0	52.83 /100.0	50.11 /100.0	48.77 /100.0	47.69 /100.0
drift(29.69)	21.35 /100.0	29.11 /100.0	25.81 /100.0	*32.58* /100.0	27.94 /100.0	27.06 /100.0
drive(31.76)	59.16 /100.0	*59.35* /100.0	55.93 /100.0	56.46 /100.0	57.46 /100.0	54.69 /100.0
face(80.31)	75.47 /100.0	**80.31** /100.0	75.50 /100.0	74.70 /100.0	76.64 /100.0	76.02 /100.0
find(15.91)	*34.29* /100.0	32.70 /100.0	33.50 /100.0	31.70 /100.0	30.28 /100.0	27.39 /100.0
keep(36.30)	48.39 /100.0	47.23 /100.0	47.38 /100.0	48.31 /100.0	*52.30* /100.0	47.59 /100.0
leave(28.79)	45.85 /100.0	41.65 /100.0	*48.12* /100.0	47.28 /100.0	45.41 /100.0	45.27 /100.0
live(48.85)	53.89 /100.0	*56.59* /100.0	51.93 /100.0	53.53 /100.0	51.70 /100.0	52.54 /100.0
match(35.23)	30.48 /100.0	33.13 /100.0	33.18 /100.0	29.39 /100.0	28.71 /100.0	**35.09** /100.0
play(19.38)	37.01 /100.0	37.02 /100.0	34.83 /100.0	36.53 /100.0	36.24 /100.0	*38.48* /100.0
pull(22.13)	31.22 /100.0	33.45 /100.0	31.25 /100.0	29.81 /100.0	30.23 /100.0	*34.98* /100.0
replace(50.00)	*57.14* /100.0	54.51 /100.0	55.13 /100.0	56.38 /100.0	54.08 /100.0	50.80 /100.0
see(31.06)	*46.21* /100.0	39.40 /100.0	38.36 /100.0	38.29 /100.0	36.28 /100.0	41.11 /100.0
serve(26.00)	50.95 /100.0	48.70 /100.0	51.77 /100.0	50.90 /100.0	*52.56* /100.0	46.22 /100.0
strike(16.35)	35.40 /100.0	36.27 /100.0	37.35 /100.0	33.44 /100.0	*39.67* /100.0	35.04 /100.0
train(26.77)	*48.66* /100.0	42.98 /100.0	47.55 /100.0	44.70 /100.0	46.16 /100.0	45.42 /100.0
treat(32.97)	41.48 /100.0	48.08 /100.0	48.08 /100.0	44.78 /100.0	*54.67* /100.0	53.57 /100.0
turn(15.15)	51.24 /100.0	51.56 /100.0	*51.86* /100.0	50.75 /100.0	48.36 /100.0	46.22 /100.0
use(67.57)	69.86 /100.0	68.95 /100.0	65.48 /100.0	66.31 /100.0	*70.00* /100.0	64.55 /100.0
wander(83.00)	81.99 /100.0	82.11 /100.0	83.09 /100.0	84.06 /100.0	83.31 /100.0	*84.41* /100.0
wash(23.08)	46.03 /100.0	*46.60* /100.0	39.94 /100.0	39.23 /100.0	43.78 /100.0	42.63 /100.0
work(27.05)	36.99 /100.0	36.34 /100.0	36.79 /100.0	35.33 /100.0	35.52 /100.0	*37.03* /100.0

Table A.4: N-gram based features cross validation accuracy (and coverage) on the training set (%) (surrounding POS tags of the target verb and their relative positions)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	60.50 /100.0	62.12 /100.0	*63.82* /100.0	62.68 /100.0	60.37 /100.0	61.04 /100.0
call(23.78)	28.62 /100.0	33.46 /100.0	34.27 /100.0	33.78 /100.0	34.79 /100.0	*35.67* /100.0
carry(23.13)	24.57 /100.0	27.13 /100.0	29.64 /100.0	*31.31* /100.0	28.84 /100.0	30.41 /100.0
collaborate(91.23)	*93.42* /100.0	91.60 /100.0	91.60 /100.0	92.33 /100.0	92.33 /100.0	92.33 /100.0
develop(29.63)	28.72 /100.0	31.99 /100.0	31.21 /100.0	33.08 /100.0	36.03 /100.0	*36.95* /100.0
draw(13.25)	29.04 /100.0	26.96 /100.0	26.42 /100.0	30.37 /100.0	*30.39* /100.0	27.53 /100.0
dress(38.52)	32.47 /100.0	**37.10** /100.0	34.70 /100.0	35.97 /100.0	34.91 /100.0	34.91 /100.0
drift(29.69)	*30.91* /100.0	29.92 /100.0	28.77 /100.0	26.73 /100.0	27.02 /100.0	29.61 /100.0
drive(31.76)	26.76 /100.0	35.16 /100.0	34.62 /100.0	35.68 /100.0	42.25 /100.0	*43.84* /100.0
face(80.31)	62.92 /100.0	67.84 /100.0	72.18 /100.0	70.72 /100.0	**76.50** /100.0	76.08 /100.0
find(15.91)	18.59 /100.0	22.93 /100.0	*24.39* /100.0	20.04 /100.0	20.78 /100.0	23.40 /100.0
keep(36.30)	31.77 /100.0	34.44 /100.0	38.79 /100.0	38.64 /100.0	38.71 /100.0	*40.94* /100.0
leave(28.79)	34.27 /100.0	34.86 /100.0	*38.01* /100.0	37.37 /100.0	32.04 /100.0	31.87 /100.0
live(48.85)	42.42 /100.0	40.94 /100.0	45.24 /100.0	45.60 /100.0	*50.23* /100.0	49.60 /100.0
match(35.23)	26.70 /100.0	28.00 /100.0	30.31 /100.0	**32.46** /100.0	28.76 /100.0	29.03 /100.0
play(19.38)	20.79 /100.0	*26.71* /100.0	24.23 /100.0	21.00 /100.0	22.48 /100.0	24.37 /100.0
pull(22.13)	25.83 /100.0	28.55 /100.0	27.23 /100.0	29.52 /100.0	31.12 /100.0	*32.49* /100.0
replace(50.00)	44.75 /100.0	50.21 /100.0	*51.27* /100.0	47.86 /100.0	44.38 /100.0	44.77 /100.0
see(31.06)	*38.86* /100.0	36.93 /100.0	33.33 /100.0	36.68 /100.0	38.33 /100.0	38.41 /100.0
serve(26.00)	27.37 /100.0	32.49 /100.0	32.28 /100.0	*35.90* /100.0	34.07 /100.0	33.72 /100.0
strike(16.35)	*30.43* /100.0	26.69 /100.0	27.93 /100.0	29.67 /100.0	28.85 /100.0	26.48 /100.0
train(26.77)	40.41 /100.0	*41.80* /100.0	40.16 /100.0	37.75 /100.0	37.37 /100.0	36.31 /100.0
treat(32.97)	38.68 /100.0	38.68 /100.0	46.37 /100.0	47.47 /100.0	*48.57* /100.0	48.57 /100.0
turn(15.15)	30.24 /100.0	30.52 /100.0	30.68 /100.0	*31.28* /100.0	29.81 /100.0	30.85 /100.0
use(67.57)	60.74 /100.0	62.18 /100.0	61.07 /100.0	60.29 /100.0	**66.01** /100.0	62.64 /100.0
wander(83.00)	71.83 /100.0	72.87 /100.0	75.01 /100.0	77.08 /100.0	**78.12** /100.0	**78.12** /100.0
wash(23.08)	32.31 /100.0	30.77 /100.0	33.85 /100.0	*36.92* /100.0	35.38 /100.0	35.38 /100.0
work(27.05)	22.59 /100.0	26.03 /100.0	32.00 /100.0	31.58 /100.0	32.36 /100.0	*32.47* /100.0

Table A.5: N-gram based features cross validation accuracy (and coverage) on the training set (%) (cooccuring words to the left of the target verb and their relative positions)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	61.01 /100.0	62.39 /100.0	*63.53* /100.0	60.05 /100.0	60.55 /100.0	59.58 /100.0
call(23.78)	28.50 /100.0	33.78 /100.0	34.10 /100.0	*38.14* /100.0	36.97 /100.0	37.77 /100.0
carry(23.13)	25.02 /100.0	28.46 /100.0	37.55 /100.0	35.30 /100.0	37.24 /100.0	*37.68* /100.0
collaborate(91.23)	91.96 /100.0	89.77 /100.0	90.44 /100.0	93.06 /100.0	92.69 /100.0	*93.42* /100.0
develop(29.63)	30.90 /100.0	29.89 /100.0	31.91 /100.0	33.95 /100.0	35.27 /100.0	*36.15* /100.0
draw(13.25)	27.01 /100.0	27.91 /100.0	30.76 /100.0	29.03 /100.0	31.38 /100.0	*32.07* /100.0
dress(38.52)	36.59 /100.0	41.36 /100.0	37.84 /100.0	42.44 /100.0	*43.92* /100.0	39.20 /100.0
drift(29.69)	29.42 /100.0	28.39 /100.0	29.05 /100.0	27.27 /100.0	29.38 /100.0	*31.73* /100.0
drive(31.76)	32.99 /100.0	39.80 /100.0	37.66 /100.0	45.30 /100.0	*48.63* /100.0	41.84 /100.0
face(80.31)	66.86 /100.0	71.02 /100.0	**78.96** /100.0	74.85 /100.0	78.11 /100.0	75.23 /100.0
find(15.91)	19.13 /100.0	19.87 /100.0	*22.22* /100.0	20.10 /100.0	20.60 /100.0	20.89 /100.0
keep(36.30)	31.88 /100.0	33.00 /100.0	27.62 /100.0	32.23 /100.0	35.24 /100.0	*37.85* /100.0
leave(28.79)	38.55 /100.0	37.88 /100.0	35.19 /100.0	*40.02* /100.0	33.58 /100.0	30.79 /100.0
live(48.85)	42.31 /100.0	51.04 /100.0	47.93 /100.0	52.68 /100.0	*53.26* /100.0	52.62 /100.0
match(35.23)	30.33 /100.0	29.45 /100.0	38.08 /100.0	36.37 /100.0	*40.08* /100.0	37.25 /100.0
play(19.38)	22.48 /100.0	33.83 /100.0	36.01 /100.0	36.20 /100.0	33.67 /100.0	*36.53* /100.0
pull(22.13)	28.24 /100.0	29.08 /100.0	32.60 /100.0	33.62 /100.0	*35.58* /100.0	35.57 /100.0
replace(50.00)	**49.31** /100.0	48.21 /100.0	48.33 /100.0	45.00 /100.0	46.25 /100.0	42.70 /100.0
see(31.06)	40.40 /100.0	39.73 /100.0	35.01 /100.0	41.00 /100.0	41.04 /100.0	*44.58* /100.0
serve(26.00)	28.66 /100.0	*38.33* /100.0	37.31 /100.0	35.21 /100.0	33.20 /100.0	34.46 /100.0
strike(16.35)	*31.31* /100.0	29.24 /100.0	24.63 /100.0	25.83 /100.0	29.02 /100.0	28.40 /100.0
train(26.77)	37.93 /100.0	39.28 /100.0	42.59 /100.0	*45.20* /100.0	42.71 /100.0	38.61 /100.0
treat(32.97)	34.67 /100.0	36.59 /100.0	40.99 /100.0	45.38 /100.0	*49.78* /100.0	46.87 /100.0
turn(15.15)	31.01 /100.0	33.73 /100.0	34.67 /100.0	*35.33* /100.0	32.71 /100.0	33.56 /100.0
use(67.57)	65.61 /100.0	61.03 /100.0	*67.70* /100.0	61.03 /100.0	66.89 /100.0	63.14 /100.0
wander(83.00)	78.88 /100.0	80.89 /100.0	84.35 /100.0	*84.79* /100.0	82.96 /100.0	82.96 /100.0
wash(23.08)	31.54 /100.0	30.77 /100.0	33.85 /100.0	35.26 /100.0	35.45 /100.0	*42.82* /100.0
work(27.05)	25.62 /100.0	30.86 /100.0	*33.86* /100.0	32.63 /100.0	31.78 /100.0	33.57 /100.0

Table A.6: N-gram based features cross validation accuracy (and coverage) on the training set (%) (cooccuring words to the left of the target verb and their binary relative positions)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	*81.17* /100.0	80.31 /100.0	79.30 /100.0	75.37 /100.0	76.62 /100.0	76.23 /100.0
call(23.78)	*46.04* /100.0	45.26 /100.0	42.03 /100.0	40.99 /100.0	39.31 /100.0	39.17 /100.0
carry(23.13)	*35.79* /100.0	35.56 /100.0	34.01 /100.0	33.21 /100.0	30.93 /100.0	32.78 /100.0
collaborate(91.23)	91.23 /100.0	91.60 /100.0	90.87 /100.0	89.04 /100.0	*93.42* /100.0	91.60 /100.0
develop(29.63)	28.59 /100.0	*31.41* /100.0	29.35 /100.0	30.97 /100.0	28.09 /100.0	29.02 /100.0
draw(13.25)	*48.43* /100.0	47.98 /100.0	45.48 /100.0	42.09 /100.0	38.87 /100.0	39.80 /100.0
dress(38.52)	*50.70* /100.0	49.11 /100.0	45.69 /100.0	48.25 /100.0	47.07 /100.0	48.14 /100.0
drift(29.69)	29.85 /100.0	*35.44* /100.0	32.51 /100.0	33.54 /100.0	34.95 /100.0	30.11 /100.0
drive(31.76)	50.41 /100.0	*52.65* /100.0	51.24 /100.0	48.59 /100.0	50.89 /100.0	51.53 /100.0
face(80.31)	56.04 /100.0	67.20 /100.0	73.35 /100.0	76.55 /100.0	**78.85** /100.0	77.35 /100.0
find(15.91)	*37.96* /100.0	37.32 /100.0	33.83 /100.0	35.58 /100.0	32.92 /100.0	31.25 /100.0
keep(36.30)	41.40 /100.0	43.56 /100.0	47.30 /100.0	48.38 /100.0	*51.94* /100.0	47.59 /100.0
leave(28.79)	40.15 /100.0	42.62 /100.0	44.56 /100.0	44.68 /100.0	41.50 /100.0	*45.60* /100.0
live(48.85)	58.87 /100.0	*59.37* /100.0	54.25 /100.0	56.80 /100.0	56.39 /100.0	54.49 /100.0
match(35.23)	29.19 /100.0	30.56 /100.0	37.91 /100.0	*41.21* /100.0	38.62 /100.0	32.68 /100.0
play(19.38)	47.67 /100.0	49.00 /100.0	45.26 /100.0	48.15 /100.0	*49.32* /100.0	48.06 /100.0
pull(22.13)	43.80 /100.0	43.22 /100.0	45.34 /100.0	46.24 /100.0	*46.69* /100.0	45.45 /100.0
replace(50.00)	55.81 /100.0	59.41 /100.0	*62.58* /100.0	60.19 /100.0	59.01 /100.0	56.89 /100.0
see(31.06)	31.99 /100.0	32.12 /100.0	33.31 /100.0	32.21 /100.0	33.00 /100.0	*37.56* /100.0
serve(26.00)	44.31 /100.0	45.41 /100.0	*46.58* /100.0	45.50 /100.0	43.54 /100.0	43.64 /100.0
strike(16.35)	*39.75* /100.0	32.14 /100.0	36.01 /100.0	35.85 /100.0	34.13 /100.0	37.27 /100.0
train(26.77)	35.84 /100.0	41.66 /100.0	43.61 /100.0	*44.81* /100.0	43.90 /100.0	43.27 /100.0
treat(32.97)	33.52 /100.0	45.22 /100.0	45.11 /100.0	41.81 /100.0	49.51 /100.0	*49.51* /100.0
turn(15.15)	55.03 /100.0	55.93 /100.0	*56.37* /100.0	54.45 /100.0	53.40 /100.0	54.53 /100.0
use(67.57)	*67.96* /100.0	64.74 /100.0	59.68 /100.0	58.94 /100.0	65.26 /100.0	64.80 /100.0
wander(83.00)	69.62 /100.0	73.07 /100.0	77.98 /100.0	78.73 /100.0	82.88 /100.0	*85.71* /100.0
wash(23.08)	*52.88* /100.0	52.12 /100.0	45.64 /100.0	46.28 /100.0	49.62 /100.0	49.81 /100.0
work(27.05)	45.79 /100.0	*47.64* /100.0	45.46 /100.0	42.49 /100.0	43.74 /100.0	44.14 /100.0

Table A.7: N-gram based features cross validation accuracy (and coverage) on the training set (%) (cooccuring words to the right of the target verb and their relative positions)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	*78.80* /100.0	75.82 /100.0	72.06 /100.0	70.42 /100.0	68.62 /100.0	69.70 /100.0
call(23.78)	*43.91* /100.0	38.22 /100.0	36.79 /100.0	37.85 /100.0	36.94 /100.0	37.75 /100.0
carry(23.13)	38.39 /100.0	34.24 /100.0	*39.84* /100.0	37.06 /100.0	37.42 /100.0	37.89 /100.0
collaborate(91.23)	91.23 /100.0	90.87 /100.0	91.96 /100.0	91.60 /100.0	91.60 /100.0	*93.42* /100.0
develop(29.63)	31.96 /100.0	33.12 /100.0	*36.53* /100.0	32.20 /100.0	30.06 /100.0	30.51 /100.0
draw(13.25)	53.33 /100.0	*60.00* /100.0	58.68 /100.0	57.51 /100.0	56.67 /100.0	59.47 /100.0
dress(38.52)	50.65 /100.0	49.94 /100.0	48.72 /100.0	49.71 /100.0	*51.18* /100.0	49.25 /100.0
drift(29.69)	38.65 /100.0	*38.79* /100.0	38.12 /100.0	37.92 /100.0	36.22 /100.0	30.55 /100.0
drive(31.76)	49.55 /100.0	49.84 /100.0	50.91 /100.0	*53.72* /100.0	52.78 /100.0	49.25 /100.0
face(80.31)	62.85 /100.0	69.57 /100.0	76.80 /100.0	76.67 /100.0	78.34 /100.0	**79.34** /100.0
find(15.91)	*36.64* /100.0	32.78 /100.0	34.50 /100.0	31.79 /100.0	34.70 /100.0	27.36 /100.0
keep(36.30)	43.62 /100.0	46.57 /100.0	49.15 /100.0	*51.06* /100.0	48.94 /100.0	50.56 /100.0
leave(28.79)	39.30 /100.0	*44.62* /100.0	38.23 /100.0	40.92 /100.0	40.44 /100.0	42.11 /100.0
live(48.85)	*63.19* /100.0	59.58 /100.0	57.20 /100.0	59.45 /100.0	56.88 /100.0	56.02 /100.0
match(35.23)	35.81 /100.0	37.95 /100.0	39.64 /100.0	*45.70* /100.0	41.34 /100.0	40.72 /100.0
play(19.38)	47.14 /100.0	48.53 /100.0	54.42 /100.0	51.54 /100.0	*54.51* /100.0	53.93 /100.0
pull(22.13)	*47.91* /100.0	44.88 /100.0	44.30 /100.0	39.91 /100.0	34.59 /100.0	38.50 /100.0
replace(50.00)	60.55 /100.0	*60.62* /100.0	53.38 /100.0	52.76 /100.0	47.47 /100.0	50.71 /100.0
see(31.06)	36.03 /100.0	34.04 /100.0	33.98 /100.0	35.89 /100.0	*38.24* /100.0	35.11 /100.0
serve(26.00)	46.44 /100.0	*46.84* /100.0	46.79 /100.0	41.43 /100.0	44.61 /100.0	46.09 /100.0
strike(16.35)	37.80 /100.0	35.65 /100.0	*42.98* /100.0	40.38 /100.0	37.64 /100.0	41.75 /100.0
train(26.77)	35.80 /100.0	41.25 /100.0	43.43 /100.0	44.16 /100.0	*45.40* /100.0	44.77 /100.0
treat(32.97)	43.13 /100.0	52.09 /100.0	53.79 /100.0	58.52 /100.0	*59.51* /100.0	58.41 /100.0
turn(15.15)	54.71 /100.0	*61.64* /100.0	58.29 /100.0	57.34 /100.0	53.82 /100.0	53.56 /100.0
use(67.57)	**66.77** /100.0	64.27 /100.0	62.04 /100.0	66.35 /100.0	62.38 /100.0	65.63 /100.0
wander(83.00)	79.71 /100.0	78.73 /100.0	78.73 /100.0	83.92 /100.0	*85.58* /100.0	83.64 /100.0
wash(23.08)	58.01 /100.0	52.69 /100.0	53.27 /100.0	63.85 /100.0	*68.59* /100.0	65.26 /100.0
work(27.05)	*50.72* /100.0	47.22 /100.0	45.54 /100.0	38.17 /100.0	41.67 /100.0	37.22 /100.0

Table A.8: N-gram based features cross validation accuracy (and coverage) on the training set (%) (cooccuring words to the right of the target verb and their binary relative positions)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	79.43 /100.0	*80.02* /100.0	78.72 /100.0	77.53 /100.0	78.04 /100.0	75.59 /100.0
call(23.78)	*47.99* /100.0	46.43 /100.0	45.63 /100.0	45.28 /100.0	44.19 /100.0	42.11 /100.0
carry(23.13)	36.08 /100.0	*36.79* /100.0	34.00 /100.0	34.05 /100.0	32.33 /100.0	31.40 /100.0
collaborate(91.23)	90.50 /100.0	91.60 /100.0	93.42 /100.0	93.42 /100.0	91.96 /100.0	*94.88* /100.0
develop(29.63)	38.88 /100.0	*40.60* /100.0	35.28 /100.0	38.49 /100.0	38.21 /100.0	38.49 /100.0
draw(13.25)	*46.56* /100.0	43.00 /100.0	41.49 /100.0	41.27 /100.0	36.97 /100.0	39.76 /100.0
dress(38.52)	53.90 /100.0	*57.93* /100.0	49.14 /100.0	48.28 /100.0	49.48 /100.0	51.85 /100.0
drift(29.69)	26.66 /100.0	*36.24* /100.0	34.02 /100.0	28.65 /100.0	33.03 /100.0	23.97 /100.0
drive(31.76)	47.71 /100.0	53.64 /100.0	*56.99* /100.0	51.94 /100.0	52.12 /100.0	53.42 /100.0
face(80.31)	72.85 /100.0	74.21 /100.0	78.36 /100.0	77.56 /100.0	78.51 /100.0	**79.93** /100.0
find(15.91)	35.10 /100.0	*36.46* /100.0	33.88 /100.0	33.12 /100.0	30.20 /100.0	30.88 /100.0
keep(36.30)	45.23 /100.0	45.27 /100.0	48.69 /100.0	*49.72* /100.0	48.90 /100.0	49.58 /100.0
leave(28.79)	*48.35* /100.0	46.07 /100.0	45.22 /100.0	44.04 /100.0	40.61 /100.0	41.99 /100.0
live(48.85)	51.59 /100.0	*56.92* /100.0	52.22 /100.0	56.12 /100.0	54.44 /100.0	52.21 /100.0
match(35.23)	28.85 /100.0	32.12 /100.0	35.90 /100.0	*39.80* /100.0	32.72 /100.0	33.40 /100.0
play(19.38)	44.48 /100.0	46.20 /100.0	*47.16* /100.0	45.55 /100.0	46.47 /100.0	45.04 /100.0
pull(22.13)	43.67 /100.0	*48.57* /100.0	45.62 /100.0	48.35 /100.0	45.41 /100.0	43.72 /100.0
replace(50.00)	55.69 /100.0	55.96 /100.0	*63.31* /100.0	59.21 /100.0	52.67 /100.0	52.21 /100.0
see(31.06)	36.59 /100.0	36.97 /100.0	35.86 /100.0	40.13 /100.0	41.28 /100.0	*41.84* /100.0
serve(26.00)	*48.11* /100.0	47.89 /100.0	47.59 /100.0	44.01 /100.0	43.76 /100.0	44.19 /100.0
strike(16.35)	*41.52* /100.0	38.81 /100.0	37.79 /100.0	38.94 /100.0	36.43 /100.0	38.60 /100.0
train(26.77)	*48.19* /100.0	46.98 /100.0	46.06 /100.0	44.82 /100.0	44.50 /100.0	44.71 /100.0
treat(32.97)	41.59 /100.0	47.20 /100.0	*56.87* /100.0	53.68 /100.0	55.77 /100.0	53.68 /100.0
turn(15.15)	57.91 /100.0	*58.13* /100.0	55.42 /100.0	52.72 /100.0	50.19 /100.0	53.39 /100.0
use(67.57)	**65.56** /100.0	63.13 /100.0	63.73 /100.0	64.61 /100.0	62.65 /100.0	64.81 /100.0
wander(83.00)	80.89 /100.0	84.13 /100.0	85.23 /100.0	85.10 /100.0	*86.26* /100.0	85.10 /100.0
wash(23.08)	49.23 /100.0	*53.72* /100.0	47.88 /100.0	44.62 /100.0	39.04 /100.0	43.91 /100.0
work(27.05)	45.98 /100.0	46.12 /100.0	*49.81* /100.0	46.52 /100.0	43.38 /100.0	44.88 /100.0

Table A.9: N-gram based features cross validation accuracy (and coverage) on the training set (%) (surrounding words of the target verb and their relative positions)

Verb (Majority class Frequency %)	Window size					
	2	3	4	5	6	7
begin(58.44)	*75.32* /100.0	74.82 /100.0	70.98 /100.0	70.82 /100.0	68.29 /100.0	68.35 /100.0
call(23.78)	*45.39* /100.0	44.12 /100.0	40.72 /100.0	44.07 /100.0	42.79 /100.0	42.99 /100.0
carry(23.13)	38.19 /100.0	*41.33* /100.0	39.70 /100.0	37.37 /100.0	35.86 /100.0	40.26 /100.0
collaborate(91.23)	91.60 /100.0	90.50 /100.0	*94.88* /100.0	91.96 /100.0	93.42 /100.0	93.42 /100.0
develop(29.63)	37.33 /100.0	35.74 /100.0	37.77 /100.0	*37.92* /100.0	37.74 /100.0	37.49 /100.0
draw(13.25)	49.57 /100.0	*56.16* /100.0	51.19 /100.0	53.16 /100.0	48.79 /100.0	47.83 /100.0
dress(38.52)	50.37 /100.0	50.61 /100.0	*52.44* /100.0	50.96 /100.0	47.84 /100.0	48.65 /100.0
drift(29.69)	32.55 /100.0	34.80 /100.0	*35.34* /100.0	29.82 /100.0	23.38 /100.0	27.98 /100.0
drive(31.76)	49.24 /100.0	50.29 /100.0	*52.83* /100.0	52.29 /100.0	50.31 /100.0	48.66 /100.0
face(80.31)	74.82 /100.0	76.64 /100.0	77.66 /100.0	77.83 /100.0	**79.34** /100.0	79.29 /100.0
find(15.91)	*36.07* /100.0	32.91 /100.0	33.02 /100.0	29.71 /100.0	31.01 /100.0	22.82 /100.0
keep(36.30)	46.46 /100.0	45.51 /100.0	44.74 /100.0	47.85 /100.0	47.60 /100.0	*49.59* /100.0
leave(28.79)	*46.47* /100.0	43.76 /100.0	44.54 /100.0	39.37 /100.0	39.97 /100.0	41.56 /100.0
live(48.85)	52.41 /100.0	54.34 /100.0	52.77 /100.0	56.36 /100.0	*58.98* /100.0	56.13 /100.0
match(35.23)	33.88 /100.0	34.31 /100.0	*43.85* /100.0	39.92 /100.0	40.41 /100.0	42.79 /100.0
play(19.38)	49.10 /100.0	53.78 /100.0	*55.47* /100.0	51.77 /100.0	53.23 /100.0	49.74 /100.0
pull(22.13)	45.98 /100.0	*48.18* /100.0	44.42 /100.0	39.04 /100.0	37.58 /100.0	37.37 /100.0
replace(50.00)	*57.99* /100.0	56.89 /100.0	57.91 /100.0	49.04 /100.0	45.00 /100.0	46.65 /100.0
see(31.06)	*45.02* /100.0	42.83 /100.0	38.47 /100.0	42.54 /100.0	39.13 /100.0	39.23 /100.0
serve(26.00)	49.96 /100.0	*50.71* /100.0	47.70 /100.0	41.11 /100.0	40.84 /100.0	45.50 /100.0
strike(16.35)	45.27 /100.0	40.91 /100.0	39.67 /100.0	*46.41* /100.0	43.19 /100.0	43.71 /100.0
train(26.77)	43.06 /100.0	43.54 /100.0	47.13 /100.0	47.15 /100.0	*49.54* /100.0	43.56 /100.0
treat(32.97)	46.32 /100.0	53.79 /100.0	60.38 /100.0	65.99 /100.0	*68.19* /100.0	66.10 /100.0
turn(15.15)	50.87 /100.0	*56.10* /100.0	54.98 /100.0	52.17 /100.0	55.11 /100.0	52.87 /100.0
use(67.57)	**66.79** /100.0	63.11 /100.0	60.73 /100.0	63.58 /100.0	63.94 /100.0	65.81 /100.0
wander(83.00)	85.10 /100.0	85.65 /100.0	84.19 /100.0	85.16 /100.0	*86.20* /100.0	*86.20* /100.0
wash(23.08)	48.33 /100.0	52.82 /100.0	47.69 /100.0	*66.67* /100.0	56.41 /100.0	58.91 /100.0
work(27.05)	45.99 /100.0	45.40 /100.0	*48.22* /100.0	43.84 /100.0	44.41 /100.0	47.03 /100.0

Table A.10: N-gram based features cross validation accuracy (and coverage) on the training set (%) (surrounding words of the target verb and their relative binary positions)

Verb (Majority class Frequency %)	Window size							
	3	4	5	6	7	8	9	10
begin(58.44)	54.00 /100.0	53.33 /100.0	54.54 /100.0	56.00 /100.0	56.11 /100.0	57.14 /100.0	57.52 /100.0	**57.69** /100.0
call(23.78)	*29.43* /100.0	16.31 /100.0	11.54 /100.0	5.42 /100.0	5.21 /100.0	7.31 /100.0	7.03 /100.0	7.03 /100.0
carry(23.13)	*28.78* /100.0	17.20 /100.0	15.21 /100.0	14.31 /100.0	16.70 /100.0	13.85 /100.0	11.62 /100.0	12.36 /100.0
collaborate(91.23)	91.23 /100.0	91.23 /100.0	91.23 /100.0	89.84 /100.0	93.06 /100.0	92.33 /100.0	*93.42* /100.0	*93.42* /100.0
develop(29.63)	**27.33** /100.0	26.26 /100.0	18.81 /100.0	18.09 /100.0	16.78 /100.0	15.61 /100.0	14.09 /100.0	15.28 /100.0
draw(13.25)	*28.15* /100.0	22.71 /100.0	22.35 /100.0	24.76 /100.0	25.72 /100.0	24.76 /100.0	25.24 /100.0	24.28 /100.0
dress(38.52)	*41.58* /100.0	30.07 /100.0	30.61 /100.0	28.10 /100.0	21.95 /100.0	18.87 /100.0	19.69 /100.0	21.33 /100.0
drift(29.69)	**27.01** /100.0	23.65 /100.0	27.00 /100.0	20.69 /100.0	23.64 /100.0	26.59 /100.0	26.59 /100.0	26.59 /100.0
drive(31.76)	*48.41* /100.0	31.26 /100.0	19.03 /100.0	12.93 /100.0	13.99 /100.0	11.63 /100.0	11.63 /100.0	11.63 /100.0
face(80.31)	**69.54** /100.0	62.53 /100.0	51.26 /100.0	41.89 /100.0	37.56 /100.0	33.31 /100.0	30.91 /100.0	30.70 /100.0
find(15.91)	*25.07* /100.0	20.49 /100.0	23.16 /100.0	16.79 /100.0	12.87 /100.0	12.72 /100.0	12.57 /100.0	12.87 /100.0
keep(36.30)	**30.96** /100.0	27.46 /100.0	21.63 /100.0	17.99 /100.0	10.75 /100.0	11.18 /100.0	8.96 /100.0	8.67 /100.0
leave(28.79)	*35.45* /100.0	30.46 /100.0	29.13 /100.0	25.72 /100.0	24.11 /100.0	24.32 /100.0	24.11 /100.0	24.32 /100.0
live(48.85)	44.21 /100.0	47.98 /100.0	46.10 /100.0	48.71 /100.0	47.75 /100.0	**48.72** /100.0	48.57 /100.0	**48.72** /100.0
match(35.23)	**28.50** /100.0	19.06 /100.0	22.40 /100.0	22.53 /100.0	25.88 /100.0	23.15 /100.0	24.40 /100.0	24.40 /100.0
play(19.38)	*24.54* /100.0	24.01 /100.0	16.63 /100.0	13.51 /100.0	14.64 /100.0	15.02 /100.0	13.55 /100.0	13.55 /100.0
pull(22.13)	*30.92* /100.0	24.26 /100.0	21.13 /100.0	18.67 /100.0	14.23 /100.0	13.55 /100.0	13.22 /100.0	13.22 /100.0
replace(50.00)	**47.29** /100.0	39.76 /100.0	29.87 /100.0	32.03 /100.0	32.03 /100.0	32.03 /100.0	32.03 /100.0	32.03 /100.0
see(31.06)	**20.81** /100.0	16.03 /100.0	11.42 /100.0	10.63 /100.0	7.35 /100.0	7.35 /100.0	7.35 /100.0	7.35 /100.0
serve(26.00)	*38.19* /100.0	31.89 /100.0	31.37 /100.0	22.85 /100.0	21.85 /100.0	20.05 /100.0	20.05 /100.0	20.18 /100.0
strike(16.35)	*28.58* /100.0	28.18 /100.0	22.96 /100.0	18.42 /100.0	19.34 /100.0	19.53 /100.0	19.99 /100.0	19.94 /100.0
train(26.77)	30.47 /100.0	29.53 /100.0	29.71 /100.0	33.27 /100.0	33.88 /100.0	*34.83* /100.0	*34.83* /100.0	*34.83* /100.0
treat(32.97)	*36.98* /100.0	34.01 /100.0	28.63 /100.0	25.33 /100.0	19.67 /100.0	20.55 /100.0	20.55 /100.0	18.35 /100.0
turn(15.15)	*36.95* /100.0	36.85 /100.0	27.59 /100.0	27.42 /100.0	27.22 /100.0	26.85 /100.0	23.98 /100.0	26.00 /100.0
use(67.57)	66.62 /100.0	65.27 /100.0	65.95 /100.0	65.95 /100.0	63.92 /100.0	**67.30** /100.0	66.62 /100.0	**67.30** /100.0
wander(83.00)	**55.84** /100.0	39.40 /100.0	25.52 /100.0	19.58 /100.0	9.77 /100.0	5.90 /100.0	5.90 /100.0	5.90 /100.0
wash(23.08)	35.77 /100.0	*36.73* /100.0	*36.73* /100.0	31.54 /100.0	31.54 /100.0	32.31 /100.0	31.54 /100.0	30.77 /100.0
work(27.05)	*29.01* /100.0	25.39 /100.0	19.39 /100.0	18.04 /100.0	16.16 /100.0	14.84 /100.0	14.84 /100.0	14.84 /100.0

Table A.11: Non-parse-tree based syntactic features (Additional chunk based features: Chunk-type-sequence after the target verb) cross validation accuracy (and coverage) on the training set (%)

Verb (Majority class Frequency %)	Window size							
	3	4	5	6	7	8	9	10
begin(58.44)	54.00 /100.0	53.33 /100.0	54.54 /100.0	56.00 /100.0	56.29 /100.0	57.14 /100.0	57.52 /100.0	**57.69** /100.0
call(23.78)	*28.66* /100.0	16.01 /100.0	10.91 /100.0	6.05 /100.0	5.14 /100.0	7.24 /100.0	7.24 /100.0	7.38 /100.0
carry(23.13)	*24.53* /100.0	18.10 /100.0	16.57 /100.0	16.55 /100.0	16.87 /100.0	13.72 /100.0	12.36 /100.0	13.11 /100.0
collaborate(91.23)	91.23 /100.0	91.23 /100.0	91.23 /100.0	89.40 /100.0	*93.42* /100.0	92.69 /100.0	92.69 /100.0	*93.42* /100.0
develop(29.63)	**28.53** /100.0	27.09 /100.0	20.02 /100.0	19.04 /100.0	17.74 /100.0	16.57 /100.0	13.96 /100.0	15.17 /100.0
draw(13.25)	*27.15* /100.0	23.73 /100.0	23.31 /100.0	26.20 /100.0	25.24 /100.0	25.72 /100.0	24.76 /100.0	24.76 /100.0
dress(38.52)	*41.50* /100.0	30.07 /100.0	31.43 /100.0	28.10 /100.0	22.92 /100.0	20.51 /100.0	19.69 /100.0	20.51 /100.0
drift(29.69)	27.01 /100.0	23.65 /100.0	**27.62** /100.0	22.16 /100.0	23.64 /100.0	26.59 /100.0	26.59 /100.0	26.59 /100.0
drive(31.76)	*45.82* /100.0	31.26 /100.0	19.03 /100.0	12.93 /100.0	13.99 /100.0	11.63 /100.0	11.63 /100.0	11.63 /100.0
face(80.31)	**69.04** /100.0	62.94 /100.0	51.57 /100.0	42.10 /100.0	37.87 /100.0	33.52 /100.0	30.70 /100.0	30.91 /100.0
find(15.91)	*25.37* /100.0	20.46 /100.0	24.75 /100.0	17.14 /100.0	12.87 /100.0	13.17 /100.0	12.87 /100.0	12.72 /100.0
keep(36.30)	**30.96** /100.0	28.33 /100.0	23.15 /100.0	18.89 /100.0	11.74 /100.0	10.20 /100.0	8.48 /100.0	8.92 /100.0
leave(28.79)	*35.75* /100.0	30.33 /100.0	30.27 /100.0	26.24 /100.0	24.32 /100.0	24.11 /100.0	24.32 /100.0	24.32 /100.0
live(48.85)	42.51 /100.0	47.98 /100.0	46.57 /100.0	48.54 /100.0	47.91 /100.0	48.57 /100.0	48.57 /100.0	**48.72** /100.0
match(35.23)	**28.50** /100.0	19.52 /100.0	22.40 /100.0	23.78 /100.0	25.88 /100.0	23.15 /100.0	23.15 /100.0	24.40 /100.0
play(19.38)	24.22 /100.0	*24.33* /100.0	16.63 /100.0	13.67 /100.0	14.64 /100.0	15.02 /100.0	13.55 /100.0	13.55 /100.0
pull(22.13)	*29.31* /100.0	24.81 /100.0	20.53 /100.0	18.68 /100.0	13.90 /100.0	13.22 /100.0	12.88 /100.0	13.22 /100.0
replace(50.00)	**48.43** /100.0	39.76 /100.0	29.87 /100.0	32.03 /100.0	32.03 /100.0	32.03 /100.0	32.03 /100.0	32.03 /100.0
see(31.06)	**20.63** /100.0	16.03 /100.0	10.72 /100.0	9.93 /100.0	7.35 /100.0	7.35 /100.0	7.35 /100.0	7.35 /100.0
serve(26.00)	*37.71* /100.0	33.44 /100.0	30.63 /100.0	22.33 /100.0	21.34 /100.0	20.15 /100.0	20.33 /100.0	19.64 /100.0
strike(16.35)	*29.68* /100.0	28.94 /100.0	23.84 /100.0	20.16 /100.0	18.73 /100.0	19.20 /100.0	18.81 /100.0	19.72 /100.0
train(26.77)	30.47 /100.0	29.53 /100.0	29.71 /100.0	33.27 /100.0	33.88 /100.0	*34.83* /100.0	*34.83* /100.0	*34.83* /100.0
treat(32.97)	*36.98* /100.0	34.01 /100.0	28.63 /100.0	25.33 /100.0	19.67 /100.0	20.55 /100.0	20.55 /100.0	18.35 /100.0
turn(15.15)	*39.67* /100.0	37.87 /100.0	27.94 /100.0	27.20 /100.0	27.59 /100.0	27.45 /100.0	24.58 /100.0	24.69 /100.0
use(67.57)	67.30 /100.0	66.62 /100.0	65.27 /100.0	65.95 /100.0	65.95 /100.0	66.62 /100.0	*67.97* /100.0	*67.97* /100.0
wander(83.00)	**55.84** /100.0	39.40 /100.0	25.52 /100.0	19.58 /100.0	9.77 /100.0	5.90 /100.0	5.90 /100.0	5.90 /100.0
wash(23.08)	35.00 /100.0	35.96 /100.0	*40.00* /100.0	33.08 /100.0	30.77 /100.0	33.08 /100.0	32.31 /100.0	32.31 /100.0
work(27.05)	*29.19* /100.0	25.39 /100.0	19.39 /100.0	18.04 /100.0	16.16 /100.0	14.84 /100.0	14.84 /100.0	14.84 /100.0

Table A.12: Non-parse-tree based syntactic features (Additional chunk based features: Regular expression representation of the chunks after the target verb) cross validation accuracy (and coverage) on the training set (%)

Verb (Majority class Frequency %)	Window size							
	3	4	5	6	7	8	9	10
begin(58.44)	*60.71* /100.0	59.97 /100.0	59.12 /100.0	59.12 /100.0	59.12 /100.0	59.12 /100.0	59.12 /100.0	59.12 /100.0
call(23.78)	**7.77** /100.0	6.44 /100.0	5.59 /100.0	5.38 /100.0	5.45 /100.0	5.31 /100.0	5.31 /100.0	5.45 /100.0
carry(23.13)	12.45 /100.0	**12.75** /100.0	11.85 /100.0	12.15 /100.0	11.10 /100.0	11.40 /100.0	11.40 /100.0	9.91 /100.0
collaborate(91.23)	91.23 /100.0	92.69 /100.0	93.42 /100.0	92.69 /100.0	*94.15* /100.0	93.42 /100.0	92.69 /100.0	*94.15* /100.0
develop(29.63)	**9.77** /100.0	8.37 /100.0	8.37 /100.0	8.37 /100.0	8.37 /100.0	8.37 /100.0	8.37 /100.0	8.37 /100.0
draw(13.25)	*24.22* /100.0	23.13 /100.0	23.61 /100.0	23.13 /100.0	22.65 /100.0	22.17 /100.0	22.65 /100.0	23.13 /100.0
dress(38.52)	**20.44** /100.0	18.75 /100.0	17.93 /100.0	18.75 /100.0	17.93 /100.0	17.11 /100.0	17.93 /100.0	17.93 /100.0
drift(29.69)	**26.56** /100.0	**26.56** /100.0	**26.56** /100.0	**26.56** /100.0	**26.56** /100.0	**26.56** /100.0	**26.56** /100.0	**26.56** /100.0
drive(31.76)	9.53 /100.0	**10.59** /100.0	**10.59** /100.0	**10.59** /100.0	**10.59** /100.0	**10.59** /100.0	**10.59** /100.0	**10.59** /100.0
face(80.31)	**15.18** /100.0	7.23 /100.0	7.44 /100.0	7.54 /100.0	7.44 /100.0	7.54 /100.0	7.34 /100.0	7.54 /100.0
find(15.91)	**11.80** /100.0	10.59 /100.0	8.41 /100.0	8.56 /100.0	9.32 /100.0	8.11 /100.0	9.47 /100.0	9.17 /100.0
keep(36.30)	10.99 /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0
leave(28.79)	**22.67** /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0
live(48.85)	47.78 /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0
match(35.23)	**21.42** /100.0	18.23 /100.0	15.62 /100.0	15.62 /100.0	15.62 /100.0	15.62 /100.0	15.62 /100.0	16.87 /100.0
play(19.38)	**17.51** /100.0	8.53 /100.0	8.53 /100.0	8.53 /100.0	8.53 /100.0	8.53 /100.0	8.53 /100.0	8.53 /100.0
pull(22.13)	**17.21** /100.0	14.71 /100.0	12.48 /100.0	12.48 /100.0	12.14 /100.0	13.15 /100.0	12.48 /100.0	12.81 /100.0
replace(50.00)	**24.08** /100.0	**24.08** /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0
see(31.06)	**6.06** /100.0	**6.06** /100.0	**6.06** /100.0	**6.06** /100.0	**6.06** /100.0	**6.06** /100.0	**6.06** /100.0	**6.06** /100.0
serve(26.00)	**18.16** /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0
strike(16.35)	*24.43* /100.0	15.89 /100.0	13.54 /100.0	14.69 /100.0	13.92 /100.0	14.31 /100.0	15.08 /100.0	14.31 /100.0
train(26.77)	32.91 /100.0	*34.70* /100.0	*34.70* /100.0	*34.70* /100.0	*34.70* /100.0	*34.70* /100.0	*34.70* /100.0	*34.70* /100.0
treat(32.97)	**15.16** /100.0	12.97 /100.0	12.97 /100.0	12.97 /100.0	12.97 /100.0	12.97 /100.0	12.97 /100.0	12.97 /100.0
turn(15.15)	*27.17* /100.0	21.39 /100.0	21.44 /100.0	21.78 /100.0	21.10 /100.0	21.78 /100.0	21.44 /100.0	21.44 /100.0
use(67.57)	*68.65* /100.0	*68.65* /100.0	*68.65* /100.0	*68.65* /100.0	67.97 /100.0	*68.65* /100.0	67.97 /100.0	*68.65* /100.0
wander(83.00)	**9.35** /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0
wash(23.08)	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0
work(27.05)	**7.88** /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0

Table A.13: Non-parse-tree based syntactic features (Additional chunk based features: the first word of each chunk after the target verb) cross validation accuracy (and coverage) on the training set (%)

Verb (Majority class Frequency %)	Window size							
	3	4	5	6	7	8	9	10
begin(58.44)	*62.58* /100.0	60.80 /100.0	60.50 /100.0	60.50 /100.0	60.50 /100.0	60.50 /100.0	60.50 /100.0	60.50 /100.0
call(23.78)	**7.62** /100.0	7.55 /100.0	7.55 /100.0	7.41 /100.0	**7.62** /100.0	**7.62** /100.0	7.48 /100.0	7.48 /100.0
carry(23.13)	**11.24** /100.0	**11.24** /100.0	**11.24** /100.0	10.50 /100.0	10.50 /100.0	10.50 /100.0	10.50 /100.0	9.00 /100.0
collaborate(91.23)	91.96 /100.0	94.15 /100.0	92.69 /100.0	92.69 /100.0	*94.88* /100.0	92.69 /100.0	94.15 /100.0	94.15 /100.0
develop(29.63)	**6.89** /100.0	**6.89** /100.0	**6.89** /100.0	**6.89** /100.0	**6.89** /100.0	**6.89** /100.0	**6.89** /100.0	**6.89** /100.0
draw(13.25)	*24.34* /100.0	22.17 /100.0	23.61 /100.0	24.10 /100.0	22.65 /100.0	23.13 /100.0	23.61 /100.0	22.65 /100.0
dress(38.52)	**19.48** /100.0	17.11 /100.0	17.11 /100.0	17.93 /100.0	17.93 /100.0	17.11 /100.0	17.93 /100.0	18.75 /100.0
drift(29.69)	**28.44** /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0
drive(31.76)	**13.92** /100.0	**13.92** /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0
face(80.31)	**5.34** /100.0	4.44 /100.0	4.34 /100.0	4.44 /100.0	4.13 /100.0	4.55 /100.0	4.55 /100.0	4.44 /100.0
find(15.91)	**11.80** /100.0	**11.80** /100.0	10.53 /100.0	10.53 /100.0	10.38 /100.0	10.53 /100.0	10.23 /100.0	10.38 /100.0
keep(36.30)	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0	**12.22** /100.0
leave(28.79)	**21.36** /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0	19.13 /100.0
live(48.85)	48.75 /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0	*49.57* /100.0
match(35.23)	12.39 /100.0	12.39 /100.0	12.39 /100.0	12.39 /100.0	12.39 /100.0	**13.64** /100.0	12.39 /100.0	**13.64** /100.0
play(19.38)	**15.70** /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0
pull(22.13)	**17.29** /100.0	13.17 /100.0	12.48 /100.0	12.81 /100.0	12.81 /100.0	12.81 /100.0	12.81 /100.0	12.14 /100.0
replace(50.00)	**24.08** /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0	22.94 /100.0
see(31.06)	**6.44** /100.0	**6.44** /100.0	**6.44** /100.0	**6.44** /100.0	**6.44** /100.0	**6.44** /100.0	**6.44** /100.0	**6.44** /100.0
serve(26.00)	**18.39** /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0	16.29 /100.0
strike(16.35)	*20.45* /100.0	14.34 /100.0	14.31 /100.0	13.54 /100.0	15.08 /100.0	14.69 /100.0	14.31 /100.0	14.69 /100.0
train(26.77)	33.67 /100.0	*35.46* /100.0	*35.46* /100.0	*35.46* /100.0	*35.46* /100.0	*35.46* /100.0	*35.46* /100.0	*35.46* /100.0
treat(32.97)	**12.97** /100.0	**12.97** /100.0	**12.97** /100.0	**12.97** /100.0	**12.97** /100.0	**12.97** /100.0	**12.97** /100.0	**12.97** /100.0
turn(15.15)	*27.16* /100.0	21.78 /100.0	21.10 /100.0	21.44 /100.0	21.44 /100.0	21.10 /100.0	21.44 /100.0	21.44 /100.0
use(67.57)	66.89 /100.0	**67.57** /100.0	**67.57** /100.0	66.89 /100.0	**67.57** /100.0	**67.57** /100.0	**67.57** /100.0	**67.57** /100.0
wander(83.00)	**7.55** /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0
wash(23.08)	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0
work(27.05)	**9.23** /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0	6.56 /100.0

Table A.14: Non-parse-tree based syntactic features (Additional chunk based features: the last word of each chunk after the target verb) cross validation accuracy (and coverage) on the training set (%)

Verb (Majority class Frequency %)	Window size							
	3	4	5	6	7	8	9	10
begin(58.44)	*65.91* /100.0	64.69 /100.0	63.09 /100.0	62.46 /100.0	61.64 /100.0	61.64 /100.0	61.64 /100.0	61.64 /100.0
call(23.78)	**19.60** /100.0	9.76 /100.0	5.21 /100.0	5.28 /100.0	5.28 /100.0	5.49 /100.0	5.21 /100.0	5.35 /100.0
carry(23.13)	13.72 /100.0	**14.32** /100.0	12.67 /100.0	13.58 /100.0	12.53 /100.0	11.92 /100.0	11.32 /100.0	12.23 /100.0
collaborate(91.23)	91.23 /100.0	86.05 /100.0	*93.42* /100.0	91.96 /100.0	91.23 /100.0	89.77 /100.0	89.77 /100.0	*93.42* /100.0
develop(29.63)	**15.95** /100.0	9.48 /100.0	10.37 /100.0	10.37 /100.0	11.04 /100.0	11.04 /100.0	11.04 /100.0	11.04 /100.0
draw(13.25)	21.45 /100.0	23.01 /100.0	*24.10* /100.0	*24.10* /100.0	23.13 /100.0	22.17 /100.0	23.13 /100.0	22.65 /100.0
dress(38.52)	**33.69** /100.0	25.47 /100.0	20.20 /100.0	18.66 /100.0	18.66 /100.0	19.48 /100.0	20.30 /100.0	20.30 /100.0
drift(29.69)	*31.02* /100.0	25.09 /100.0	25.09 /100.0	25.09 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0
drive(31.76)	**26.97** /100.0	16.27 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0
face(80.31)	**47.79** /100.0	31.79 /100.0	24.16 /100.0	16.37 /100.0	16.18 /100.0	16.18 /100.0	15.87 /100.0	16.28 /100.0
find(15.91)	*19.63* /100.0	11.79 /100.0	10.52 /100.0	10.74 /100.0	10.59 /100.0	10.59 /100.0	10.74 /100.0	10.59 /100.0
keep(36.30)	**24.46** /100.0	14.18 /100.0	12.86 /100.0	11.41 /100.0	11.26 /100.0	11.48 /100.0	11.48 /100.0	11.41 /100.0
leave(28.79)	**27.40** /100.0	20.73 /100.0	20.95 /100.0	19.51 /100.0	19.51 /100.0	19.51 /100.0	19.51 /100.0	19.51 /100.0
live(48.85)	46.94 /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0
match(35.23)	**23.19** /100.0	14.05 /100.0	15.30 /100.0	17.73 /100.0	17.36 /100.0	16.55 /100.0	17.43 /100.0	18.61 /100.0
play(19.38)	*21.14* /100.0	14.97 /100.0	12.95 /100.0	11.51 /100.0	11.51 /100.0	11.51 /100.0	11.51 /100.0	11.51 /100.0
pull(22.13)	**20.93** /100.0	14.92 /100.0	11.86 /100.0	12.40 /100.0	11.40 /100.0	12.40 /100.0	12.07 /100.0	11.73 /100.0
replace(50.00)	**30.36** /100.0	23.06 /100.0	21.92 /100.0	21.92 /100.0	21.92 /100.0	21.92 /100.0	21.92 /100.0	21.92 /100.0
see(31.06)	**10.81** /100.0	7.84 /100.0	7.84 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0
serve(26.00)	*33.71* /100.0	26.41 /100.0	22.32 /100.0	18.18 /100.0	18.18 /100.0	18.18 /100.0	18.12 /100.0	18.18 /100.0
strike(16.35)	*26.26* /100.0	21.91 /100.0	18.73 /100.0	16.53 /100.0	17.69 /100.0	17.30 /100.0	18.07 /100.0	16.92 /100.0
train(26.77)	*40.82* /100.0	37.83 /100.0	37.95 /100.0	37.95 /100.0	37.95 /100.0	37.95 /100.0	37.95 /100.0	37.95 /100.0
treat(32.97)	**28.13** /100.0	25.05 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0
turn(15.15)	*31.81* /100.0	27.58 /100.0	21.89 /100.0	22.20 /100.0	21.82 /100.0	22.46 /100.0	22.84 /100.0	22.84 /100.0
use(67.57)	67.30 /100.0	**67.57** /100.0	66.22 /100.0	66.89 /100.0	66.22 /100.0	66.22 /100.0	66.22 /100.0	66.22 /100.0
wander(83.00)	**29.11** /100.0	7.41 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0
wash(23.08)	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0
work(27.05)	**14.28** /100.0	8.85 /100.0	10.74 /100.0	10.74 /100.0	8.85 /100.0	10.74 /100.0	9.80 /100.0	10.74 /100.0

Table A.15: Non-parse-tree based syntactic features (Additional chunk based features: the POS tag of the first word of each chunk after the target verb) cross validation accuracy (and coverage) on the training set (%)

Verb (Majority class Frequency %)	Window size							
	3	4	5	6	7	8	9	10
begin(58.44)	*65.91* /100.0	64.69 /100.0	63.09 /100.0	62.46 /100.0	61.64 /100.0	61.64 /100.0	61.64 /100.0	61.64 /100.0
call(23.78)	**20.40** /100.0	10.54 /100.0	5.28 /100.0	5.42 /100.0	5.49 /100.0	5.21 /100.0	5.28 /100.0	5.35 /100.0
carry(23.13)	14.32 /100.0	**14.83** /100.0	12.67 /100.0	13.58 /100.0	11.62 /100.0	11.92 /100.0	12.23 /100.0	12.23 /100.0
collaborate(91.23)	87.58 /100.0	86.05 /100.0	93.42 /100.0	93.42 /100.0	*94.88* /100.0	93.42 /100.0	90.50 /100.0	93.42 /100.0
develop(29.63)	**16.33** /100.0	9.70 /100.0	10.37 /100.0	10.26 /100.0	11.04 /100.0	11.04 /100.0	11.04 /100.0	11.04 /100.0
draw(13.25)	20.84 /100.0	22.77 /100.0	*23.13* /100.0	*23.13* /100.0	*23.13* /100.0	22.65 /100.0	*23.13* /100.0	22.65 /100.0
dress(38.52)	**33.69** /100.0	25.61 /100.0	20.20 /100.0	20.30 /100.0	19.48 /100.0	19.48 /100.0	19.48 /100.0	20.30 /100.0
drift(29.69)	*31.02* /100.0	25.09 /100.0	22.14 /100.0	25.09 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0
drive(31.76)	**26.97** /100.0	16.27 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0
face(80.31)	**47.18** /100.0	32.90 /100.0	24.97 /100.0	15.97 /100.0	16.07 /100.0	16.68 /100.0	16.28 /100.0	16.38 /100.0
find(15.91)	*20.50* /100.0	12.56 /100.0	10.22 /100.0	10.74 /100.0	10.74 /100.0	10.74 /100.0	10.74 /100.0	10.89 /100.0
keep(36.30)	**24.57** /100.0	14.03 /100.0	12.93 /100.0	11.41 /100.0	11.33 /100.0	11.41 /100.0	11.56 /100.0	11.41 /100.0
leave(28.79)	**27.40** /100.0	20.73 /100.0	20.95 /100.0	19.51 /100.0	19.51 /100.0	19.51 /100.0	19.51 /100.0	19.51 /100.0
live(48.85)	46.12 /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0	**47.12** /100.0
match(35.23)	**21.94** /100.0	13.68 /100.0	17.73 /100.0	16.48 /100.0	17.43 /100.0	16.55 /100.0	17.36 /100.0	16.18 /100.0
play(19.38)	*22.94* /100.0	14.97 /100.0	12.95 /100.0	11.51 /100.0	11.51 /100.0	11.51 /100.0	11.51 /100.0	11.51 /100.0
pull(22.13)	**20.51** /100.0	15.92 /100.0	12.53 /100.0	12.07 /100.0	11.40 /100.0	11.73 /100.0	12.07 /100.0	11.73 /100.0
replace(50.00)	**30.36** /100.0	23.06 /100.0	21.92 /100.0	21.92 /100.0	21.92 /100.0	21.92 /100.0	21.92 /100.0	21.92 /100.0
see(31.06)	**8.78** /100.0	7.84 /100.0	7.84 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0
serve(26.00)	*33.94* /100.0	26.58 /100.0	22.38 /100.0	18.07 /100.0	18.24 /100.0	18.12 /100.0	18.12 /100.0	18.30 /100.0
strike(16.35)	*26.45* /100.0	21.51 /100.0	19.90 /100.0	18.46 /100.0	17.29 /100.0	16.52 /100.0	16.13 /100.0	17.69 /100.0
train(26.77)	*40.82* /100.0	37.83 /100.0	37.95 /100.0	37.95 /100.0	37.95 /100.0	37.95 /100.0	37.95 /100.0	37.95 /100.0
treat(32.97)	**28.13** /100.0	25.05 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0
turn(15.15)	*32.18* /100.0	26.75 /100.0	21.67 /100.0	22.46 /100.0	21.06 /100.0	22.84 /100.0	22.80 /100.0	21.74 /100.0
use(67.57)	65.27 /100.0	**67.57** /100.0	66.89 /100.0	66.89 /100.0	66.22 /100.0	65.54 /100.0	66.89 /100.0	66.89 /100.0
wander(83.00)	**29.11** /100.0	7.41 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0
wash(23.08)	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0
work(27.05)	**15.22** /100.0	10.74 /100.0	10.74 /100.0	10.74 /100.0	9.80 /100.0	7.91 /100.0	10.74 /100.0	10.74 /100.0

Table A.16: Non-parse-tree based syntactic features (Additional chunk based features: the POS tag of the last word of each chunk after the target verb) cross validation accuracy (and coverage) on the training set (%)

Verb (Majority class Frequency %)	Window size							
	3	4	5	6	7	8	9	10
begin(58.44)	*70.01* /100.0	64.13 /100.0	62.76 /100.0	61.99 /100.0	61.69 /100.0	61.69 /100.0	61.69 /100.0	61.69 /100.0
call(23.78)	**18.83** /100.0	8.69 /100.0	5.21 /100.0	5.56 /100.0	5.28 /100.0	5.35 /100.0	5.28 /100.0	5.49 /100.0
carry(23.13)	**15.88** /100.0	15.27 /100.0	13.27 /100.0	13.27 /100.0	12.53 /100.0	11.92 /100.0	11.92 /100.0	12.23 /100.0
collaborate(91.23)	89.40 /100.0	*94.15* /100.0	*94.15* /100.0	*94.15* /100.0	93.42 /100.0	*94.15* /100.0	91.96 /100.0	*94.15* /100.0
develop(29.63)	**13.47** /100.0	9.11 /100.0	9.11 /100.0	9.11 /100.0	9.11 /100.0	9.11 /100.0	9.11 /100.0	9.11 /100.0
draw(13.25)	20.72 /100.0	*24.10* /100.0	22.65 /100.0	22.65 /100.0	23.61 /100.0	22.65 /100.0	23.61 /100.0	22.17 /100.0
dress(38.52)	**29.86** /100.0	24.17 /100.0	20.30 /100.0	17.93 /100.0	17.11 /100.0	17.11 /100.0	17.93 /100.0	18.75 /100.0
drift(29.69)	**28.05** /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0	26.56 /100.0
drive(31.76)	**18.86** /100.0	13.92 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0	10.59 /100.0
face(80.31)	**33.42** /100.0	22.03 /100.0	15.88 /100.0	14.77 /100.0	15.38 /100.0	15.38 /100.0	14.77 /100.0	15.19 /100.0
find(15.91)	*19.86* /100.0	12.02 /100.0	10.74 /100.0	10.59 /100.0	10.44 /100.0	10.59 /100.0	10.89 /100.0	10.59 /100.0
keep(36.30)	**20.12** /100.0	12.65 /100.0	11.48 /100.0	11.33 /100.0	11.48 /100.0	11.33 /100.0	11.41 /100.0	11.33 /100.0
leave(28.79)	**23.25** /100.0	19.30 /100.0	19.51 /100.0	18.07 /100.0	18.07 /100.0	18.07 /100.0	18.07 /100.0	18.07 /100.0
live(48.85)	46.31 /100.0	**48.75** /100.0	**48.75** /100.0	**48.75** /100.0	**48.75** /100.0	**48.75** /100.0	**48.75** /100.0	**48.75** /100.0
match(35.23)	**20.11** /100.0	13.48 /100.0	13.27 /100.0	12.02 /100.0	11.14 /100.0	12.02 /100.0	12.39 /100.0	13.19 /100.0
play(19.38)	*24.07* /100.0	12.83 /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0	9.96 /100.0
pull(22.13)	**18.25** /100.0	14.34 /100.0	11.73 /100.0	12.40 /100.0	12.40 /100.0	12.07 /100.0	11.73 /100.0	11.73 /100.0
replace(50.00)	**26.69** /100.0	24.19 /100.0	23.06 /100.0	23.06 /100.0	23.06 /100.0	23.06 /100.0	23.06 /100.0	23.06 /100.0
see(31.06)	**10.09** /100.0	7.84 /100.0	7.84 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0	6.44 /100.0
serve(26.00)	*32.02* /100.0	20.17 /100.0	20.28 /100.0	18.12 /100.0	18.12 /100.0	18.18 /100.0	18.07 /100.0	18.12 /100.0
strike(16.35)	*24.98* /100.0	18.53 /100.0	16.51 /100.0	16.12 /100.0	16.89 /100.0	16.51 /100.0	16.51 /100.0	16.12 /100.0
train(26.77)	36.09 /100.0	36.09 /100.0	*36.22* /100.0	*36.22* /100.0	*36.22* /100.0	*36.22* /100.0	*36.22* /100.0	*36.22* /100.0
treat(32.97)	**28.35** /100.0	23.96 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0	19.56 /100.0
turn(15.15)	*31.99* /100.0	25.63 /100.0	22.50 /100.0	22.01 /100.0	21.70 /100.0	22.16 /100.0	21.70 /100.0	22.20 /100.0
use(67.57)	66.22 /100.0	**67.57** /100.0	66.89 /100.0	66.89 /100.0	**67.57** /100.0	66.22 /100.0	66.22 /100.0	66.22 /100.0
wander(83.00)	**9.63** /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0	3.83 /100.0
wash(23.08)	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0	*30.77* /100.0
work(27.05)	**12.63** /100.0	9.04 /100.0	8.10 /100.0	9.04 /100.0	8.29 /100.0	7.91 /100.0	9.04 /100.0	8.29 /100.0

Table A.17: Non-parse-tree based syntactic features (Additional chunk based features: Chunk-type-sensitive combinations) cross validation accuracy (and coverage) on the training set (%)

Verb (Majority Class Frequency %)	Voice of the verb	Whether the verb is part of a direct quotation	The relative positions of direct quotations to the target verb
begin(58.44)	**58.44** /100.0	**58.44** /100.0	**58.44** /100.0
call(23.78)	21.51 /100.0	28.62 /100.0	*29.74* /100.0
carry(23.13)	**23.13** /100.0	**23.13** /100.0	**23.13** /100.0
collaborate(91.23)	**91.23** /100.0	**91.23** /100.0	**91.23** /100.0
develop(29.63)	**29.63** /100.0	**29.63** /100.0	27.70 /100.0
draw(13.25)	13.84 /100.0	13.01 /100.0	*15.18* /100.0
dress(38.52)	**36.98** /100.0	36.21 /100.0	**36.98** /100.0
drift(29.69)	*32.50* /100.0	31.15 /100.0	28.20 /100.0
drive(31.76)	*32.65* /100.0	28.24 /100.0	29.53 /100.0
face(80.31)	**80.31** /100.0	**80.31** /100.0	**80.31** /100.0
find(15.91)	*18.14* /100.0	16.72 /100.0	17.27 /100.0
keep(36.30)	**36.30** /100.0	**36.30** /100.0	**36.30** /100.0
leave(28.79)	27.35 /100.0	**28.79** /100.0	28.07 /100.0
live(48.85)	**48.85** /100.0	**48.85** /100.0	**48.85** /100.0
match(35.23)	**34.05** /100.0	33.88 /100.0	33.14 /100.0
play(19.38)	*22.05* /100.0	19.72 /100.0	19.69 /100.0
pull(22.13)	**22.13** /100.0	**22.13** /100.0	20.49 /100.0
replace(50.00)	**50.00** /100.0	**50.00** /100.0	49.38 /100.0
see(31.06)	30.91 /100.0	*31.71* /100.0	31.01 /100.0
serve(26.00)	**25.01** /100.0	22.60 /100.0	24.05 /100.0
strike(16.35)	15.44 /100.0	14.53 /100.0	*16.99* /100.0
train(26.77)	*29.33* /100.0	26.09 /100.0	26.09 /100.0
treat(32.97)	26.37 /100.0	35.16 /100.0	*36.26* /100.0
turn(15.15)	*17.28* /100.0	16.48 /100.0	16.63 /100.0
use(67.57)	**67.57** /100.0	**67.57** /100.0	**67.57** /100.0
wander(83.00)	**83.00** /100.0	**83.00** /100.0	**83.00** /100.0
wash(23.08)	*25.00* /100.0	25.00 /100.0	25.00 /100.0
work(27.05)	25.54 /100.0	**26.07** /100.0	22.83 /100.0

Table A.18: Non-parse-tree based syntactic features (Verb voice and quotation) cross validation accuracy (and coverage) on the training set (%)

Verb (Majority Class Frequency %)	Verb Argument Structure	Type 1	Type 2	Type 3	Type 4	Type 5	Type 6
begin(58.44)	*86.84* /89.34	77.64 /92.36	73.15 /92.36	77.89 /92.36	75.06 /92.36	76.94 /92.36	75.72 /92.36
call(23.78)	32.33 /86.71	33.29 /93.01	30.05 /93.01	31.74 /93.01	29.49 /93.01	*41.11* /93.01	38.97 /93.01
carry(23.13)	*33.22* /76.12	25.42 /94.78	21.87 /94.78	23.03 /94.78	21.05 /94.78	24.81 /94.78	17.45 /94.78
collaborate(91.23)	*93.42* /87.72	91.23 /96.49	91.23 /96.49	91.23 /96.49	91.23 /96.49	89.40 /96.49	91.23 /96.49
develop(29.63)	37.91 /77.78	33.44 /93.33	35.02 /93.33	33.44 /93.33	35.02 /93.33	35.22 /93.33	*39.07* /93.33
draw(13.25)	*36.43* /85.54	28.74 /98.80	24.58 /98.80	27.46 /98.80	23.13 /98.80	28.41 /98.80	21.08 /98.80
dress(38.52)	44.88 /68.03	49.54 /91.80	47.25 /91.80	*50.73* /91.80	47.54 /91.80	49.54 /91.80	47.05 /91.80
drift(29.69)	26.63 /71.88	25.05 /98.44	22.38 /98.44	22.09 /98.44	23.03 /98.44	31.17 /98.44	*32.16* /98.44
drive(31.76)	37.14 /75.29	42.92 /95.29	41.63 /95.29	*43.67* /95.29	38.81 /95.29	40.48 /95.29	29.49 /95.29
face(80.31)	73.57 /95.85	**76.30** /96.37	75.79 /96.37	**76.30** /96.37	75.48 /96.37	75.29 /96.37	75.29 /96.37
find(15.91)	28.85 /89.39	30.03 /97.73	21.20 /97.73	30.53 /97.73	21.85 /97.73	*31.37* /97.73	28.02 /97.73
keep(36.30)	*37.00* /88.89	35.42 /96.30	34.26 /96.30	35.14 /96.30	34.61 /96.30	35.54 /96.30	33.43 /96.30
leave(28.79)	*35.05* /87.12	33.24 /91.67	31.88 /91.67	34.30 /91.67	32.79 /91.67	28.60 /91.67	29.38 /91.67
live(48.85)	*58.12* /85.50	53.08 /93.13	46.71 /93.13	53.21 /93.13	46.73 /93.13	49.36 /93.13	51.66 /93.13
match(35.23)	36.62 /88.64	36.70 /95.45	29.01 /95.45	35.71 /95.45	29.95 /95.45	*37.36* /95.45	23.68 /95.45
play(19.38)	*48.52* /84.50	30.41 /93.80	27.52 /93.80	27.71 /93.80	28.01 /93.80	27.09 /93.80	24.23 /93.80
pull(22.13)	28.31 /74.59	28.12 /99.18	21.62 /99.18	27.47 /99.18	22.08 /99.18	*28.86* /99.18	24.18 /99.18
replace(50.00)	44.19 /89.77	**49.46** /95.45	47.05 /95.45	**49.46** /95.45	47.05 /95.45	49.19 /95.45	43.26 /95.45
see(31.06)	33.77 /97.73	36.89 /99.24	37.44 /99.24	36.96 /99.24	*37.87* /99.24	32.06 /99.24	29.34 /99.24
serve(26.00)	*56.14* /93.00	50.24 /97.00	47.67 /97.00	50.32 /97.00	48.78 /97.00	42.16 /97.00	45.48 /97.00
strike(16.35)	19.92 /75.96	23.35 /88.46	24.19 /88.46	22.35 /88.46	*25.83* /88.46	24.03 /88.46	20.43 /88.46
train(26.77)	40.38 /88.19	37.88 /95.28	36.58 /95.28	38.82 /95.28	36.58 /95.28	44.15 /95.28	*48.41* /95.28
treat(32.97)	*50.88* /94.51	49.56 /97.80	46.26 /97.80	46.26 /97.80	42.97 /97.80	40.38 /97.80	41.87 /97.80
turn(15.15)	32.01 /59.09	41.32 /94.70	39.48 /94.70	*43.61* /94.70	41.19 /94.70	41.50 /94.70	38.95 /94.70
use(67.57)	67.84 /92.57	65.74 /93.24	63.60 /93.24	65.74 /93.24	62.92 /93.24	*73.45* /93.24	68.73 /93.24
wander(83.00)	47.39 /78.00	73.63 /93.00	75.74 /93.00	73.63 /93.00	75.74 /93.00	74.75 /93.00	**75.80** /93.00
wash(23.08)	39.87 /65.38	38.85 /96.15	43.27 /96.15	43.14 /96.15	*46.73* /96.15	35.58 /96.15	31.28 /96.15
work(27.05)	*39.81* /75.41	35.93 /91.80	32.35 /91.80	35.26 /91.80	32.54 /91.80	34.57 /91.80	32.96 /91.80

Table A.19: Parse tree based features cross validation accuracy (and coverage) on the training set (%)

Verb (Majority Class Frequency %)	WordNet Synsets of SRole Headnoun	Relationships between the SRole Headnouns	SRole's relative positions to target verb	SRole head noun (without AR)	SRole head noun (with AR)	Preposition of adjunct SRoles
begin(58.44)	72.33 /99.82	60.16 /44.94	*75.91* /99.47	68.48 /98.22	68.37 /98.22	60.58 /61.81
call(23.78)	*35.05* /99.30	23.28 /67.13	32.32 /99.30	26.10 /97.90	25.41 /97.90	5.33 /32.87
carry(23.13)	37.31 /100.0	12.39 /71.64	31.59 /98.51	41.34 /97.76	*42.73* /97.76	13.18 /50.75
collaborate(91.23)	*91.96* /100.0	91.23 /63.16	91.23 /96.49	90.87 /96.49	91.60 /96.49	91.23 /47.37
develop(29.63)	39.77 /97.04	22.96 /56.30	33.22 /97.04	*44.43* /94.81	42.86 /94.81	8.01 /34.81
draw(13.25)	44.86 /100.0	13.12 /63.86	27.31 /100.0	39.35 /100.0	*45.54* /100.0	23.03 /57.83
dress(38.52)	*45.54* /95.08	14.29 /36.07	41.40 /89.34	43.25 /86.89	41.46 /86.07	29.06 /41.80
drift(29.69)	26.17 /100.0	29.53 /10.94	*38.65* /98.44	25.24 /98.44	31.18 /98.44	30.51 /70.31
drive(31.76)	*67.93* /98.82	20.05 /56.47	50.41 /97.65	60.76 /97.65	56.89 /97.65	9.35 /50.59
face(80.31)	**79.32** /99.48	69.22 /87.05	78.82 /99.48	74.85 /99.48	77.54 /99.48	36.55 /46.63
find(15.91)	23.12 /100.0	17.17 /73.48	19.69 /99.24	*27.45* /97.73	24.95 /97.73	10.63 /50.00
keep(36.30)	*41.87* /100.0	20.38 /58.52	23.64 /95.56	33.52 /94.07	29.30 /94.07	16.83 /43.70
leave(28.79)	*36.93* /98.48	18.39 /43.18	33.49 /94.70	27.86 /87.88	30.98 /87.88	12.43 /46.21
live(48.85)	49.38 /99.24	50.36 /16.03	*54.95* /98.47	51.91 /97.71	52.40 /97.71	53.81 /80.15
match(35.23)	32.29 /97.73	24.69 /48.86	*38.36* /96.59	29.95 /93.18	34.73 /93.18	28.38 /52.27
play(19.38)	48.28 /98.45	11.60 /48.84	31.85 /97.67	47.21 /97.67	*51.78* /97.67	16.95 /60.47
pull(22.13)	*36.15* /99.18	18.64 /68.03	24.00 /99.18	26.95 /97.54	28.38 /97.54	10.62 /53.28
replace(50.00)	55.28 /100.0	52.41 /77.27	59.40 /100.0	57.74 /100.0	*60.01* /100.0	28.98 /53.41
see(31.06)	*34.99* /100.0	25.90 /73.48	33.41 /99.24	25.20 /99.24	27.90 /99.24	21.43 /58.33
serve(26.00)	*46.23* /100.0	28.55 /54.00	41.36 /98.00	43.97 /95.00	45.75 /95.00	15.33 /54.00
strike(16.35)	*38.21* /95.19	12.74 /50.96	22.71 /92.31	34.47 /92.31	35.70 /92.31	14.53 /45.19
train(26.77)	41.33 /96.85	24.93 /63.78	36.07 /95.28	41.04 /95.28	*47.09* /95.28	29.23 /37.80
treat(32.97)	61.26 /100.0	44.62 /73.63	36.26 /98.90	*66.59* /98.90	62.09 /98.90	23.96 /41.76
turn(15.15)	*48.92* /99.24	17.01 /53.03	37.43 /98.48	38.08 /96.97	42.02 /96.97	17.00 /56.06
use(67.57)	66.62 /100.0	66.42 /69.59	*70.00* /99.32	62.62 /98.65	63.16 /98.65	66.89 /38.51
wander(83.00)	*84.35* /100.0	40.24 /47.00	74.95 /98.00	69.76 /96.00	71.07 /96.00	45.71 /70.00
wash(23.08)	51.67 /100.0	17.69 /42.31	*52.18* /96.15	46.79 /92.31	48.59 /92.31	21.15 /57.69
work(27.05)	39.38 /98.36	16.82 /51.64	35.89 /96.72	38.11 /93.44	*39.40* /93.44	20.66 /59.02

Table A.20: Selectional preferences based features cross validation accuracy (and coverage) on the training set (%)

Appendix B

Impact of Feature Selection and the Tuning of the Gaussian Smoothing Parameter in Verb Sense Disambiguation

The figures listed in this appendix show the influence of applying feature selections and tuning of the MaxEnt classifier's Gaussian smoothing parameters on the 29 verbs of the lexical sample task of Senseval-2.

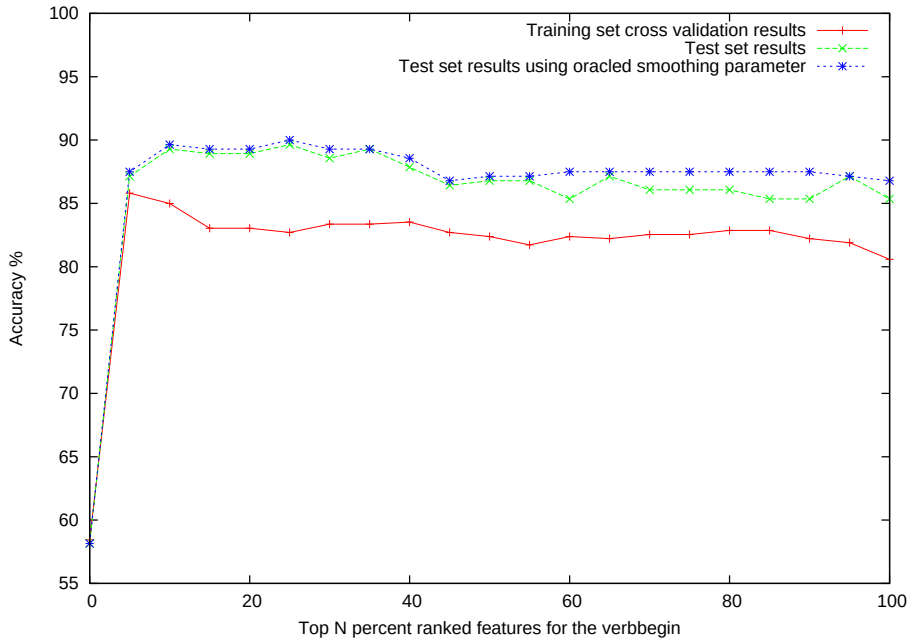


Figure B.1: Results of the ranking-percentage based feature selection for the verb *begin*

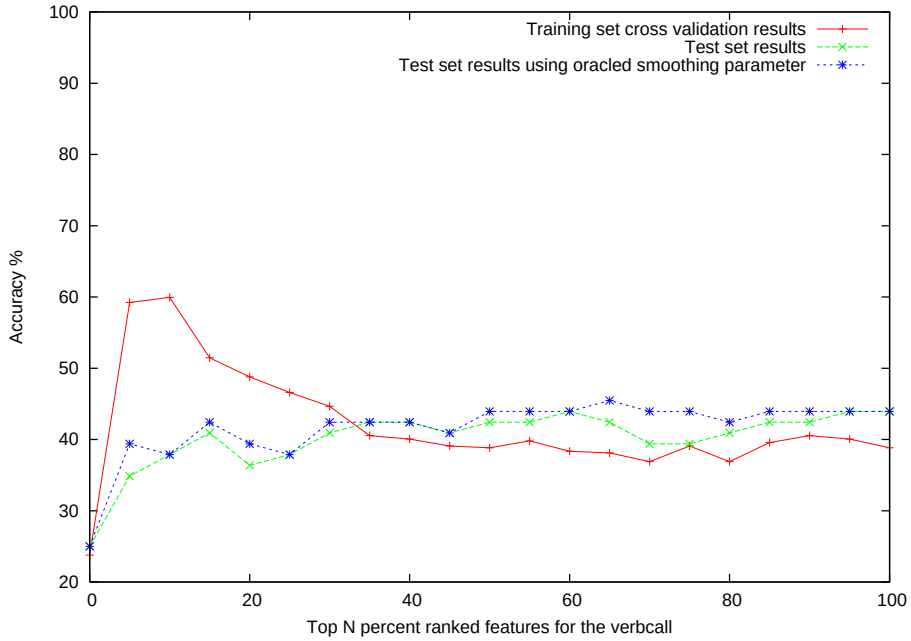


Figure B.2: Results of the ranking-percentage based feature selection for the verb *call*

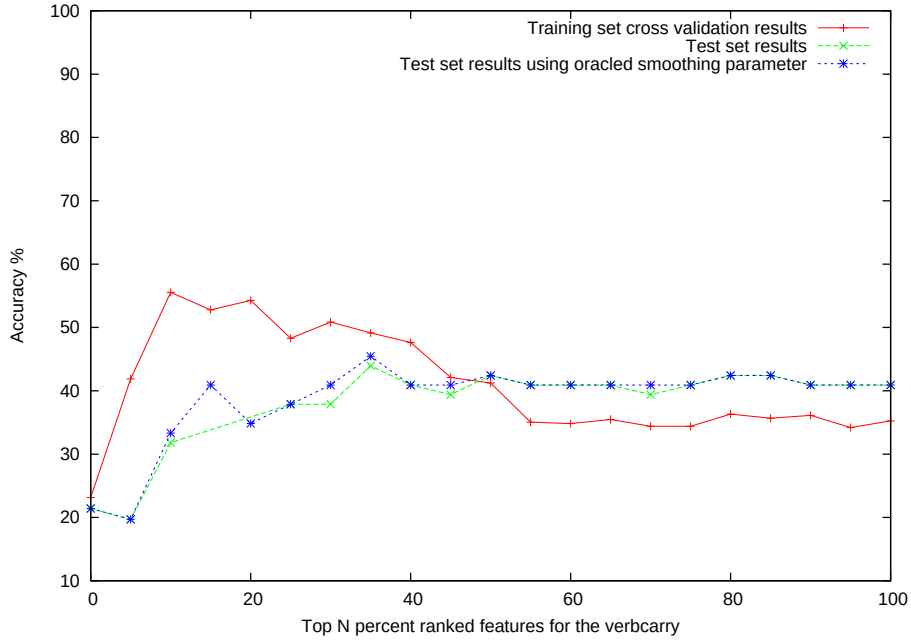


Figure B.3: Results of the ranking-percentage based feature selection for the verb *carry*

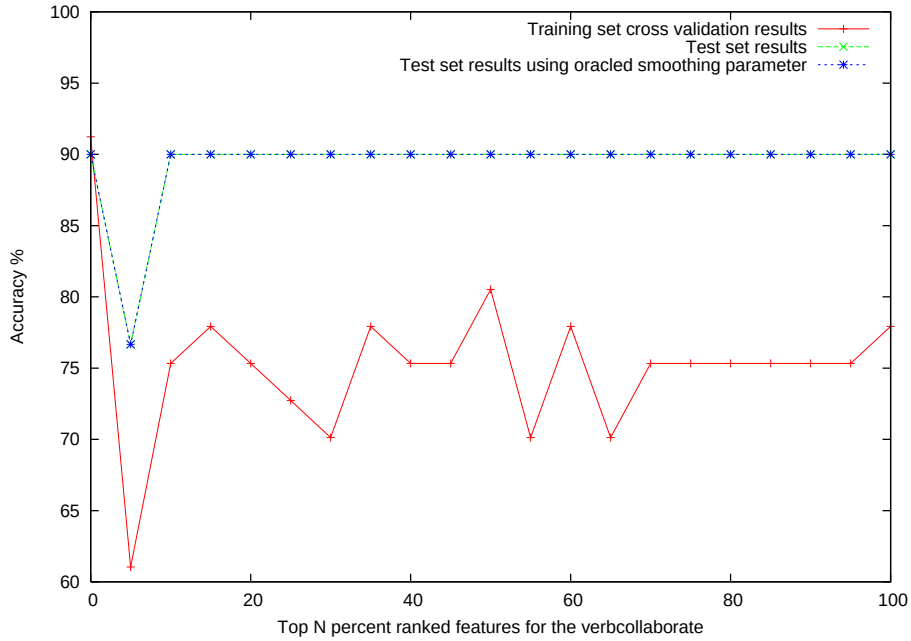


Figure B.4: Results of the ranking-percentage based feature selection for the verb *collaborate*

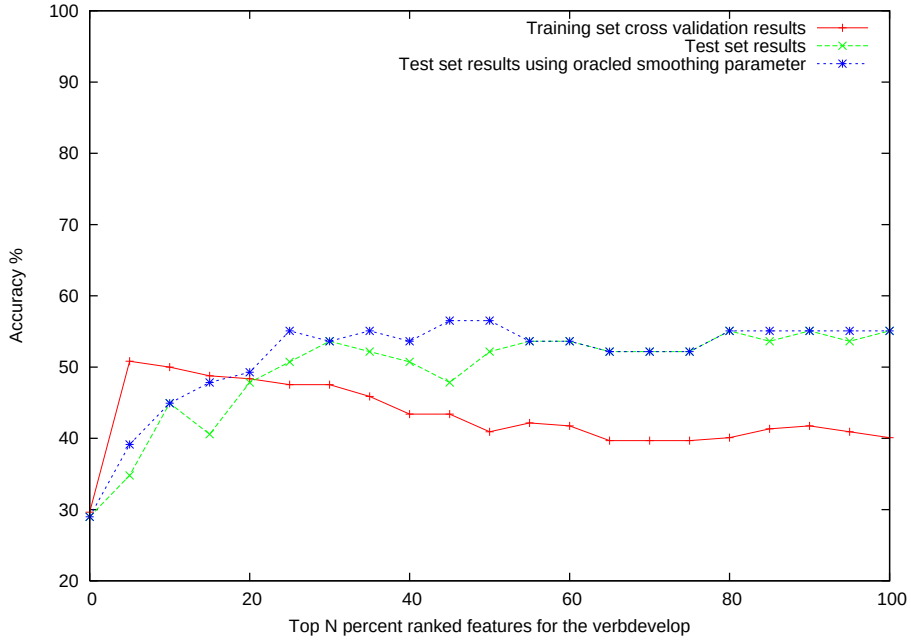


Figure B.5: Results of the ranking-percentage based feature selection for the verb *develop*

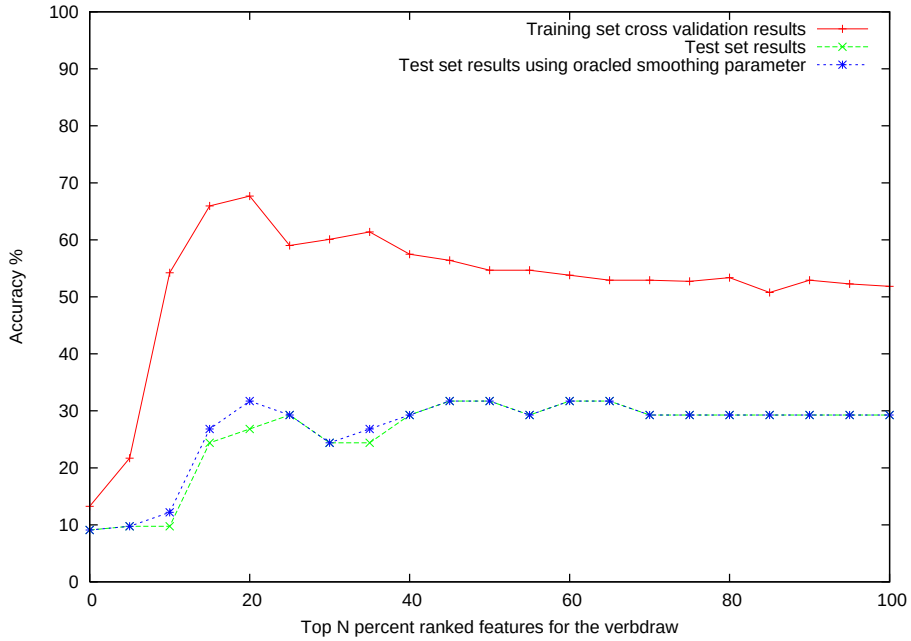


Figure B.6: Results of the ranking-percentage based feature selection for the verb *draw*

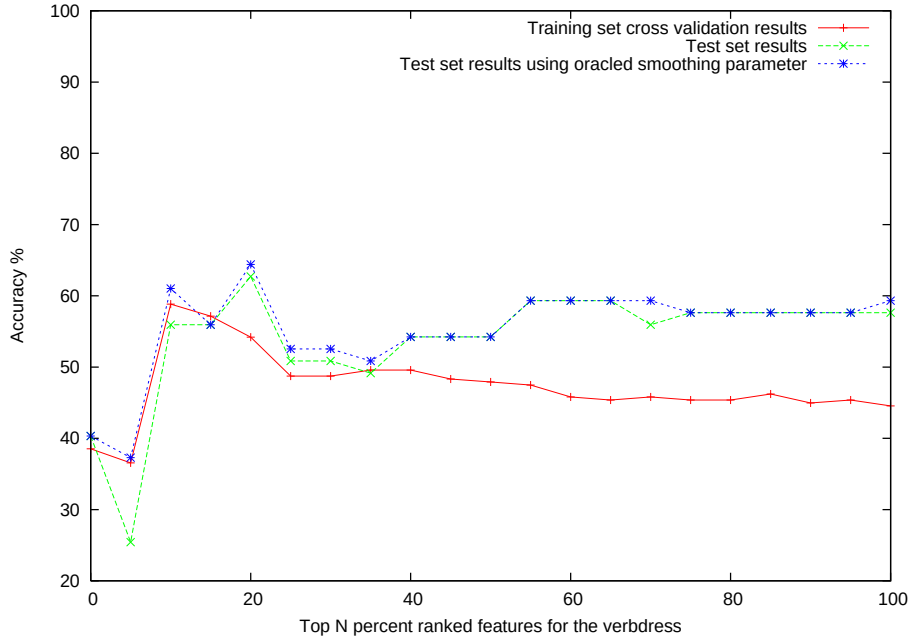


Figure B.7: Results of the ranking-percentage based feature selection for the verb *dress*

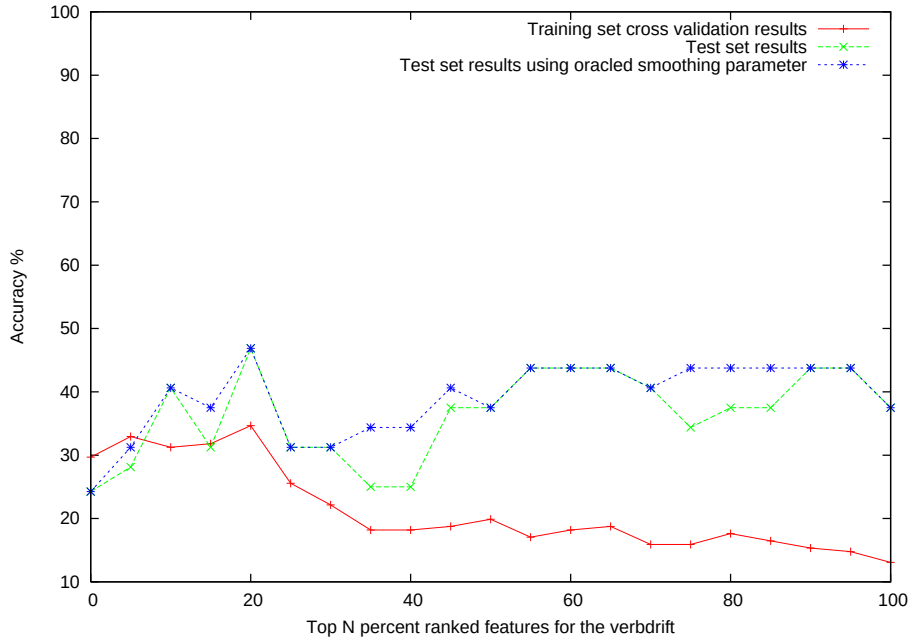


Figure B.8: Results of the ranking-percentage based feature selection for the verb *drift*

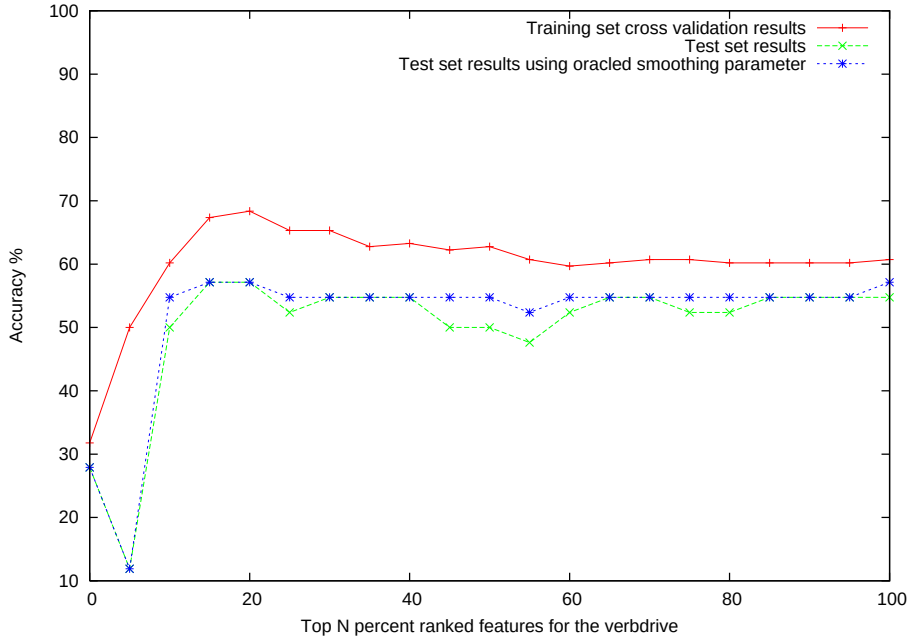


Figure B.9: Results of the ranking-percentage based feature selection for the verb *drive*

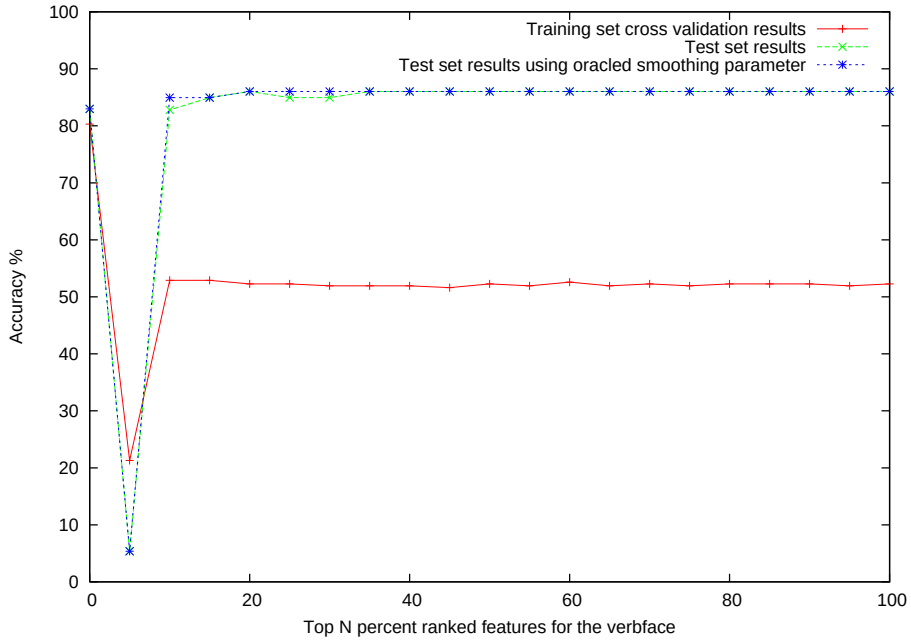


Figure B.10: Results of the ranking-percentage based feature selection for the verb *face*

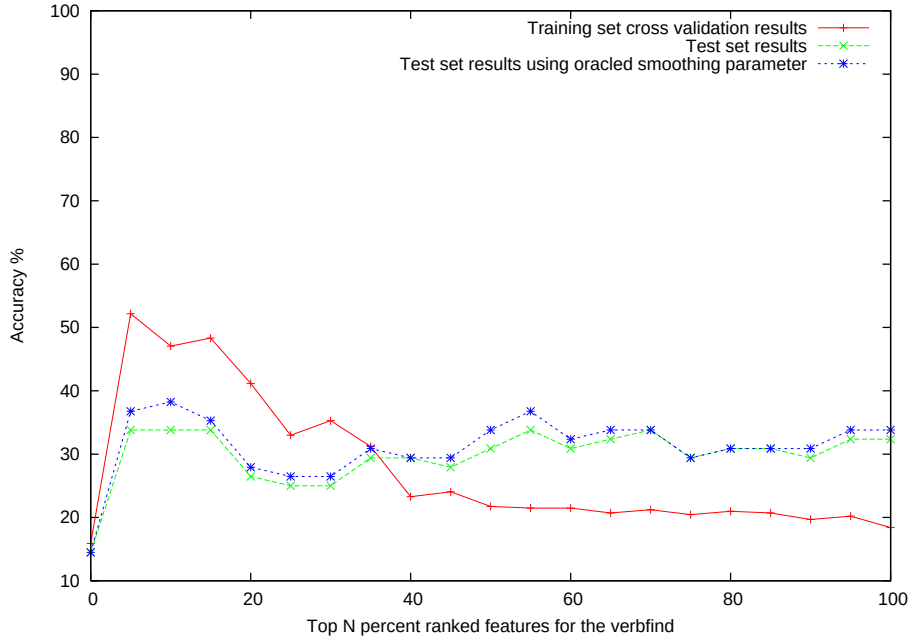


Figure B.11: Results of the ranking-percentage based feature selection for the verb *find*

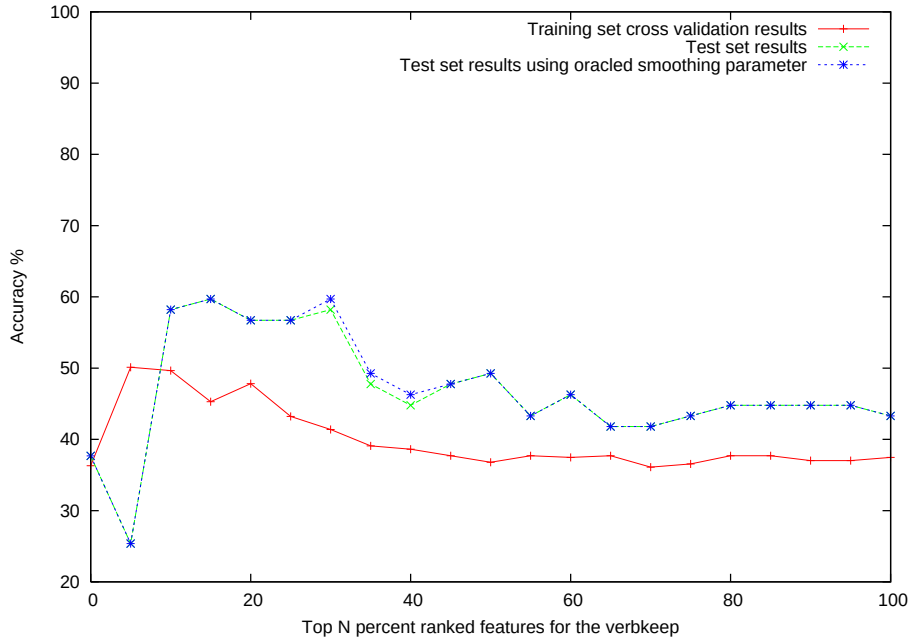


Figure B.12: Results of the ranking-percentage based feature selection for the verb *keep*

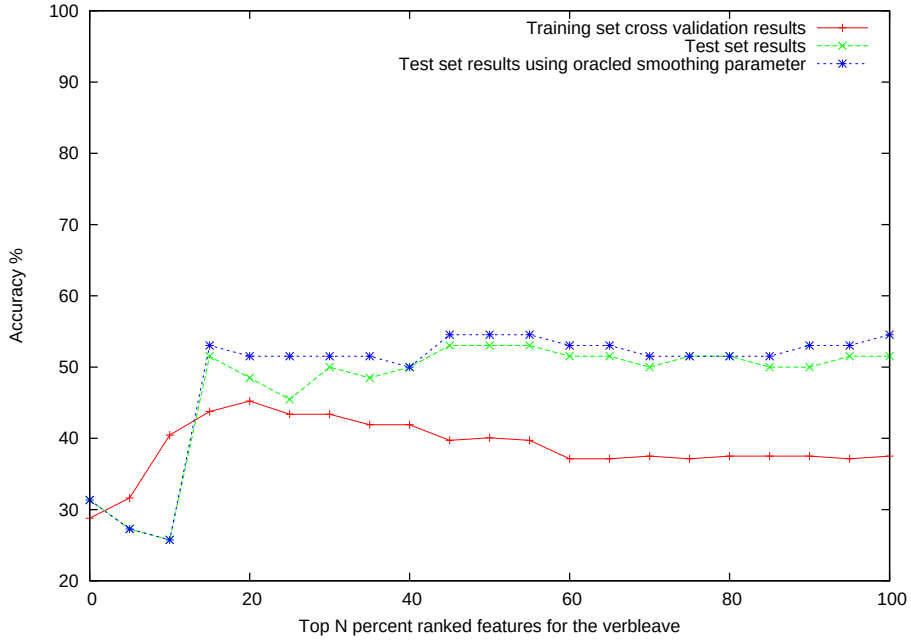


Figure B.13: Results of the ranking-percentage based feature selection for the verb *leave*

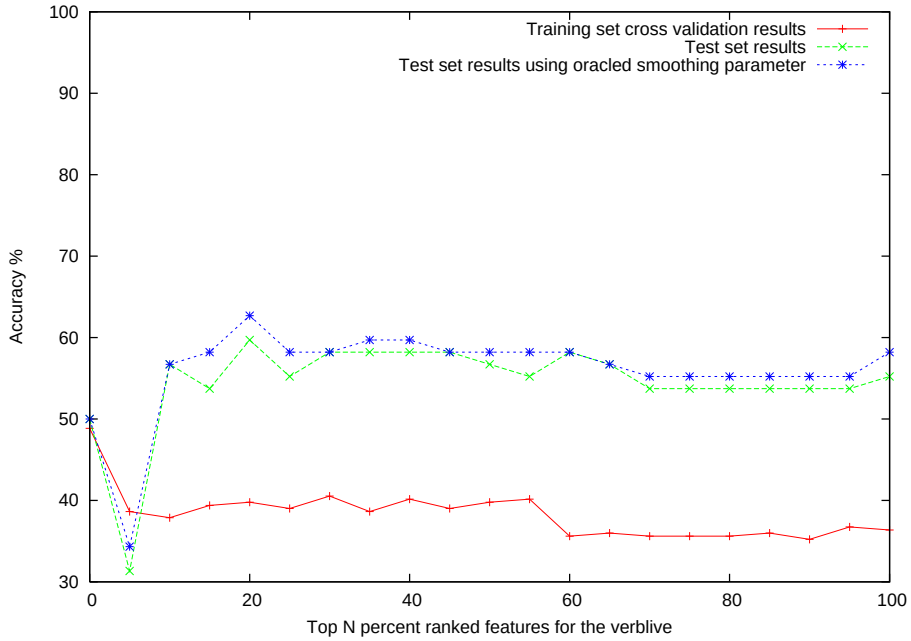


Figure B.14: Results of the ranking-percentage based feature selection for the verb *live*

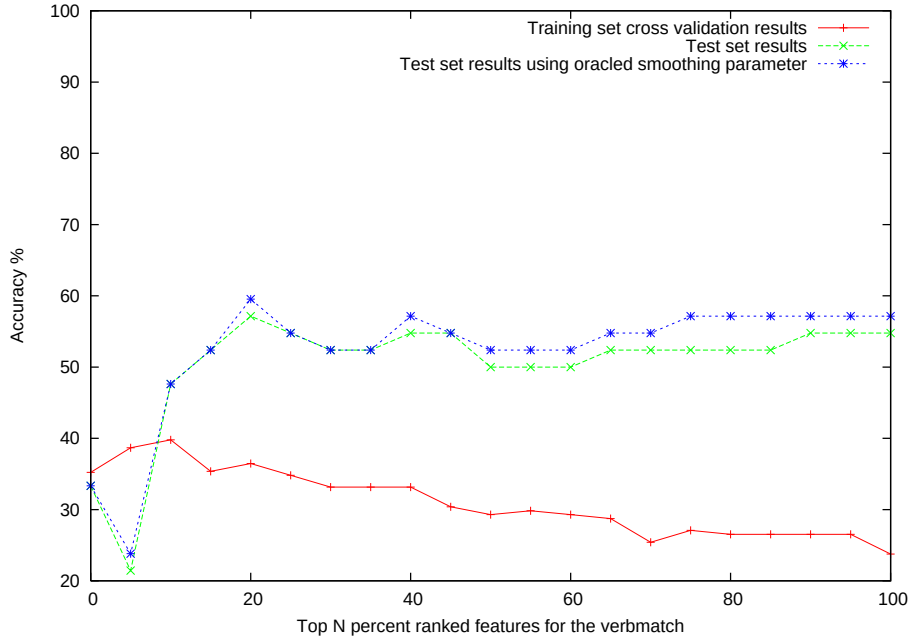


Figure B.15: Results of the ranking-percentage based feature selection for the verb *match*

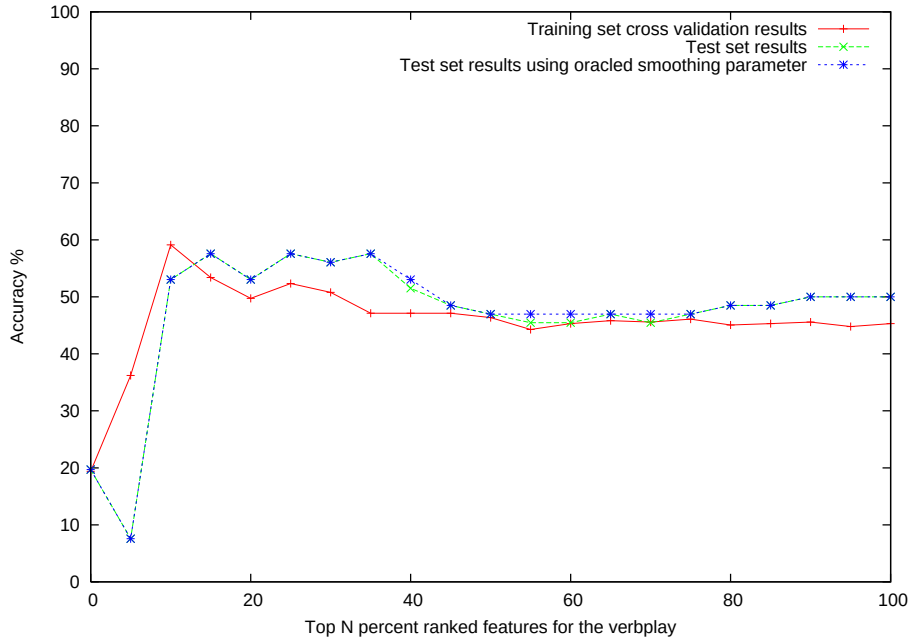


Figure B.16: Results of the ranking-percentage based feature selection for the verb *play*

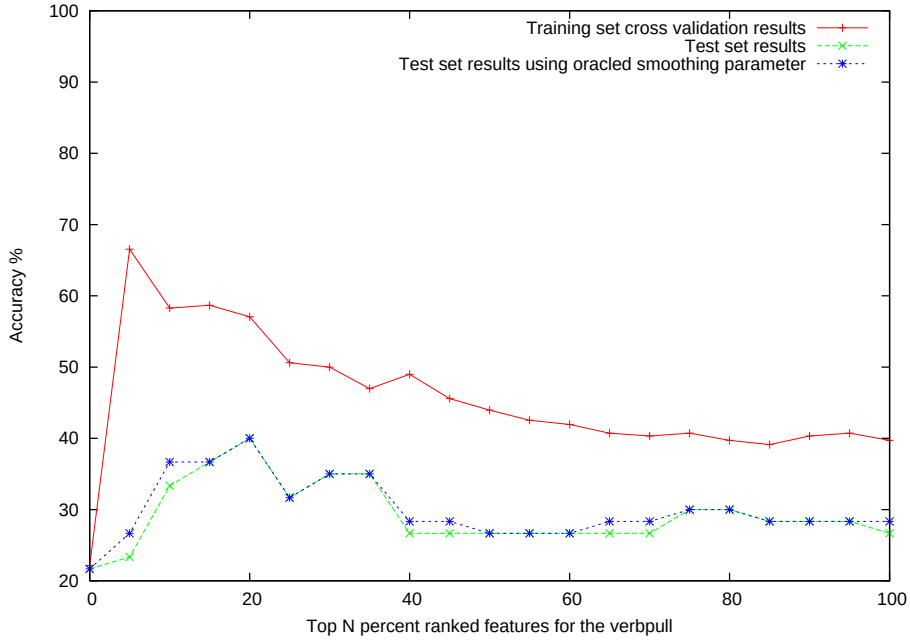


Figure B.17: Results of the ranking-percentage based feature selection for the verb *pull*

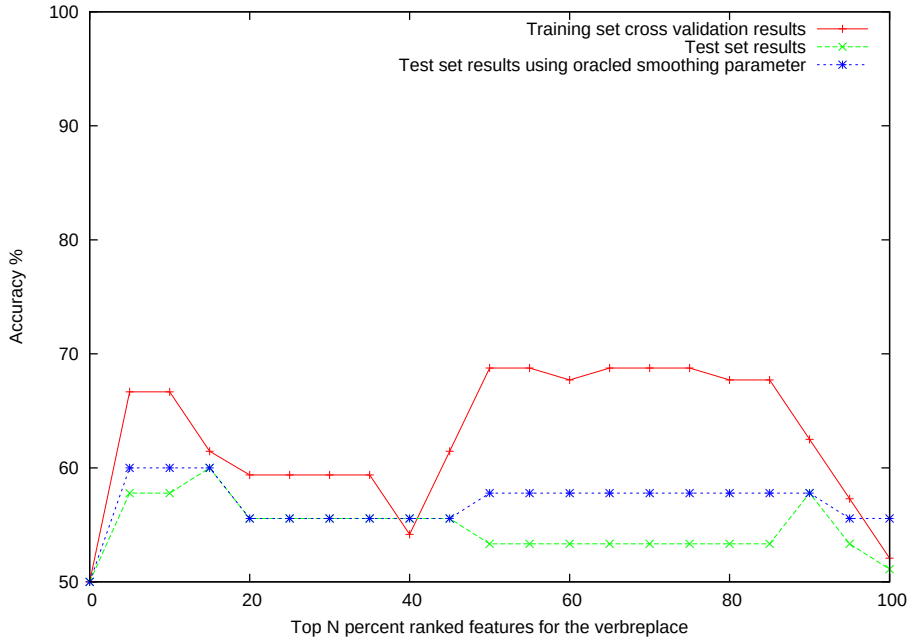


Figure B.18: Results of the ranking-percentage based feature selection for the verb *replace*

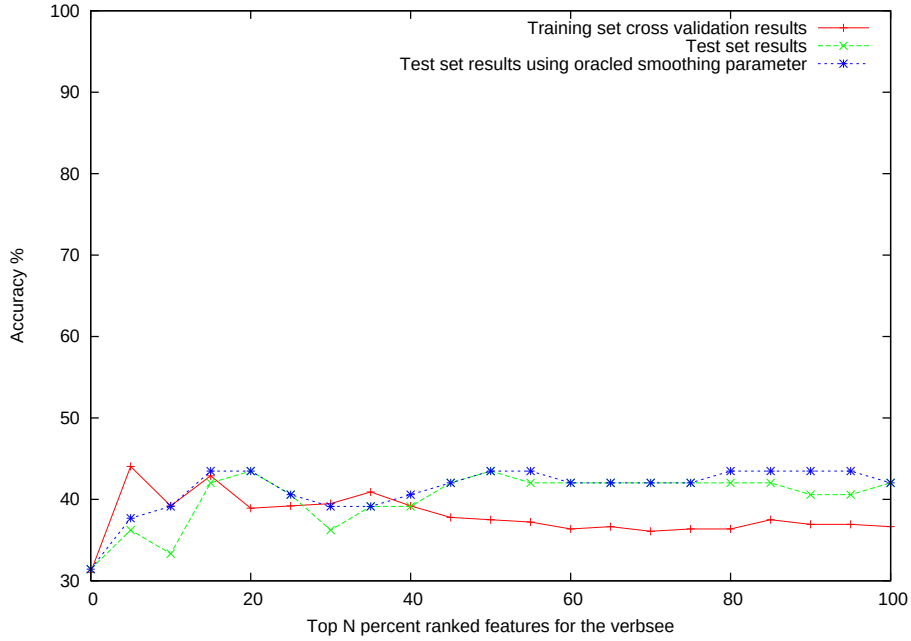


Figure B.19: Results of the ranking-percentage based feature selection for the verb *see*

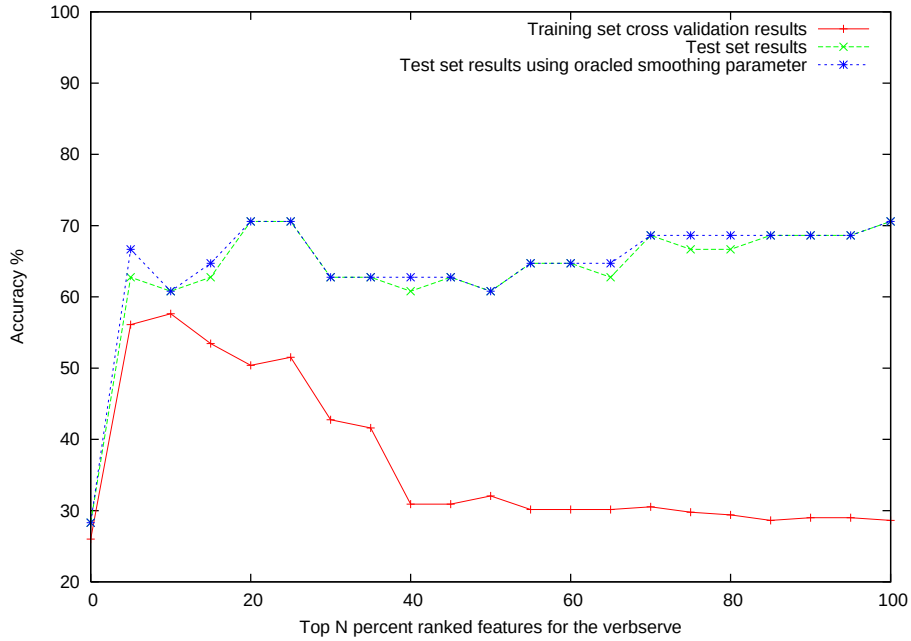


Figure B.20: Results of the ranking-percentage based feature selection for the verb *serve*

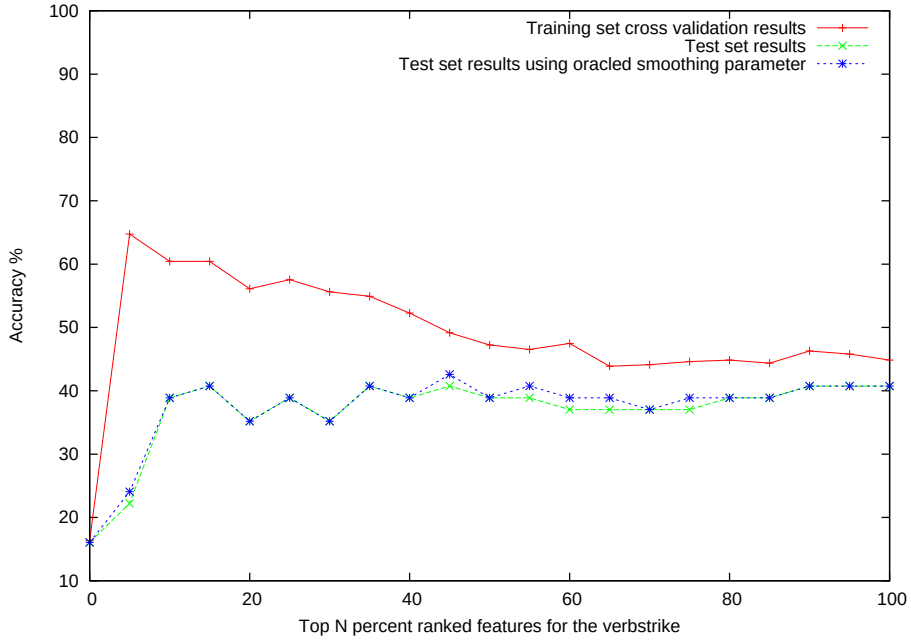


Figure B.21: Results of the ranking-percentage based feature selection for the verb *strike*

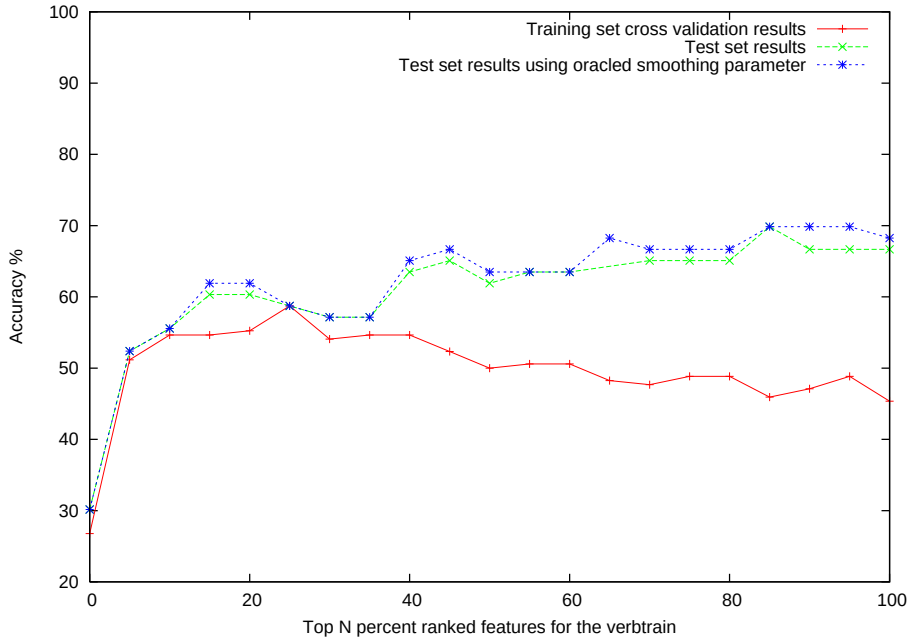


Figure B.22: Results of the ranking-percentage based feature selection for the verb *train*

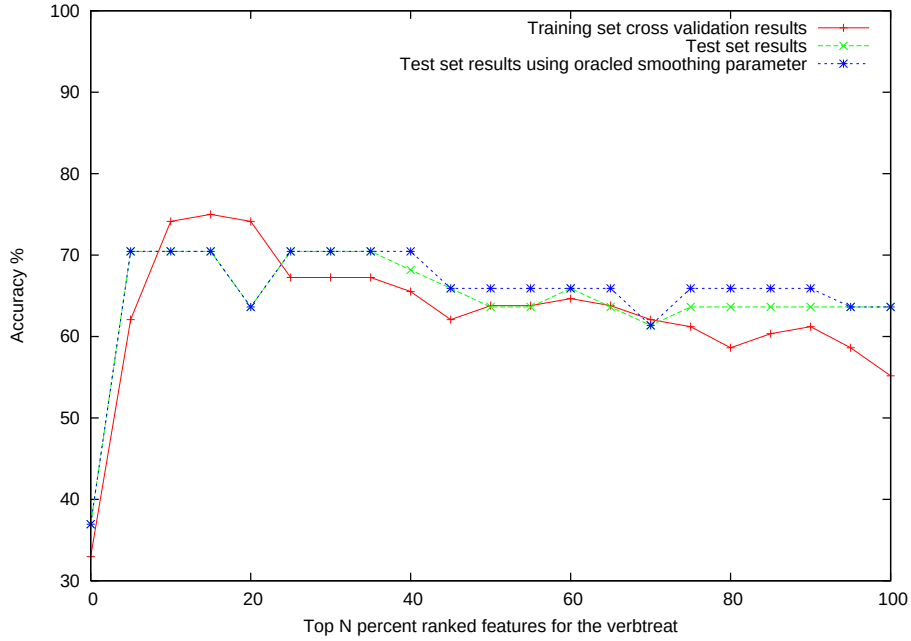


Figure B.23: Results of the ranking-percentage based feature selection for the verb *treat*

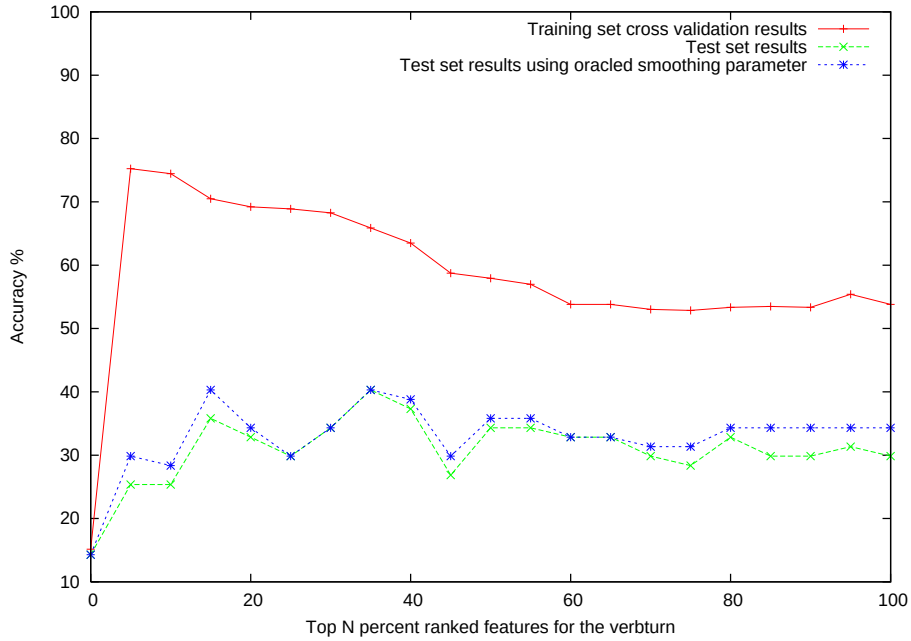


Figure B.24: Results of the ranking-percentage based feature selection for the verb *turn*

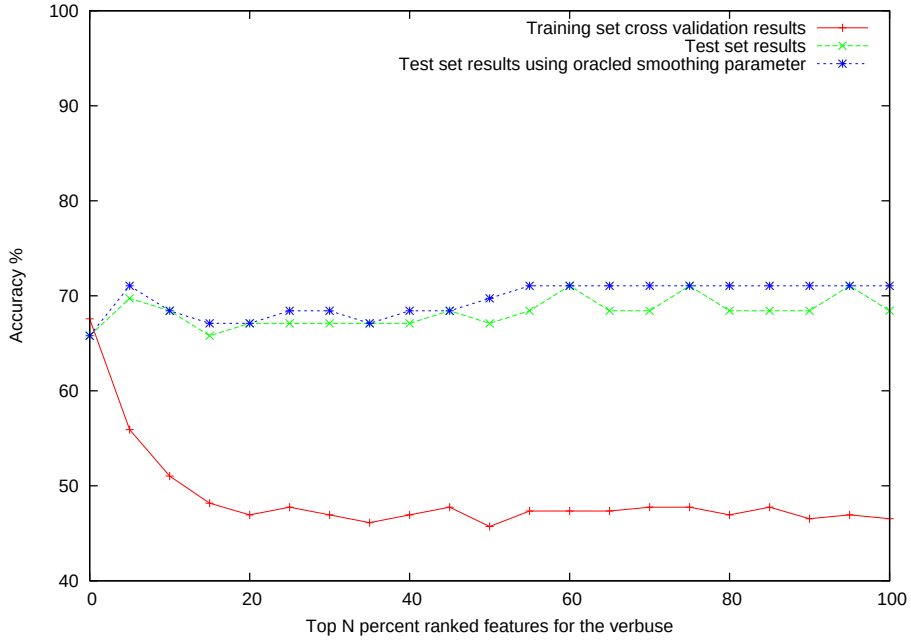


Figure B.25: Results of the ranking-percentage based feature selection for the verb *use*

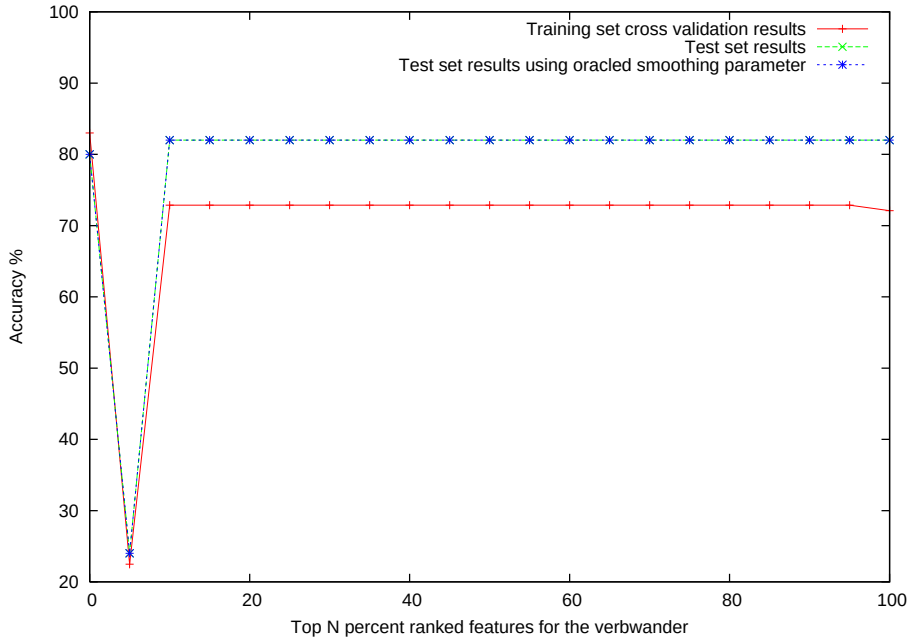


Figure B.26: Results of the ranking-percentage based feature selection for the verb *wander*

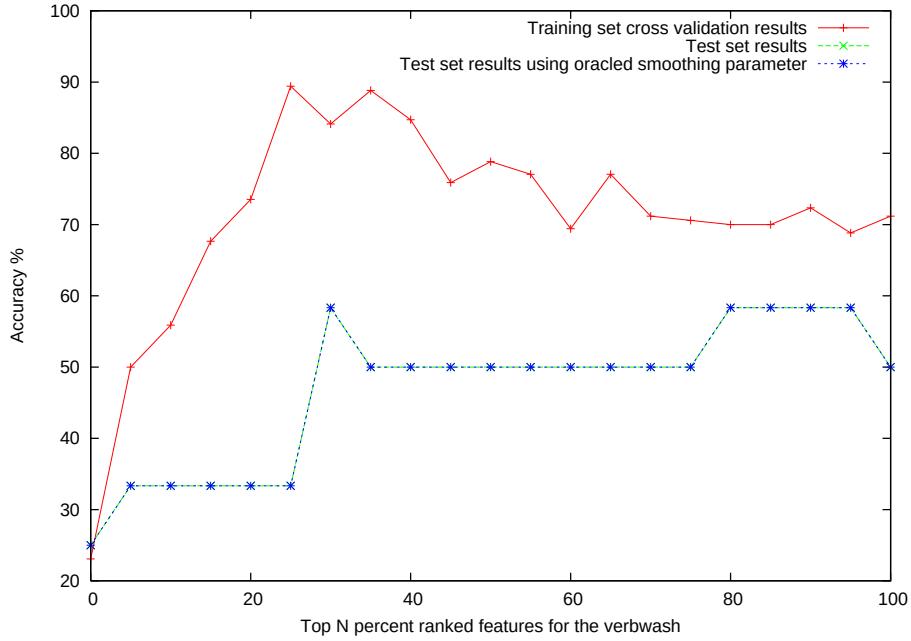


Figure B.27: Results of the ranking-percentage based feature selection for the verb *wash*

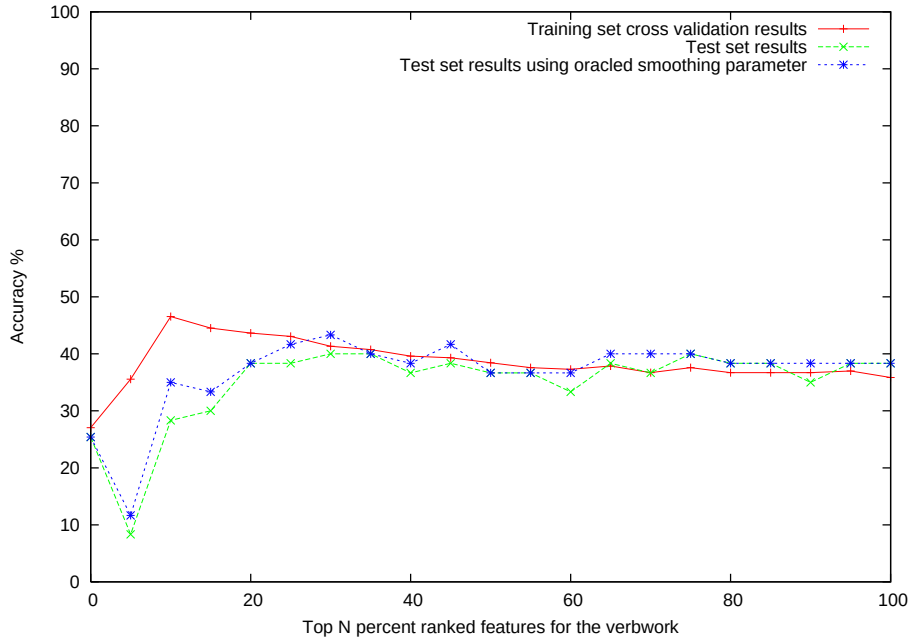


Figure B.28: Results of the ranking-percentage based feature selection for the verb *work*

Appendix C

Sense Tagging the Verbs in the Movie Script Corpus

This appendix summarises the process for sense tagging the eight chosen verbs in the movie script corpus.

C.1 Graphical User Interface

Figure C.1 shows an example of the Graphical User Interface (GUI) of the sense tagging tool. This GUI consists of 3 major sections:

1. Toolbar: this tool bar contains the following four components:
   (a) **Previous** button: this button will cause the previous sentence to be shown.
   (b) **Next** button: this button will cause the next sentence to be shown.
   (c) **Sentence progress** label: this label will appear in the format of <current sentence>/<total sentence>.
   (d) **One click tagging** checkbox: by selecting this option, each tagging action will automatically lead to the displaying of the next sentence.
2. Sense buttons: this area contains an array of buttons. Each button corresponds to a particular sense of the target word. By moving the mouse over a particular button, a tool tip containing the definition of the senses will pop up. The user can sense-tag a target verb by clicking the buttons, however, clicking a button which represents a sense that the target verbs has already been tagged with will cause that sense to be untagged. It is also possible to tag the same instance of a particular target verb with multiple senses.

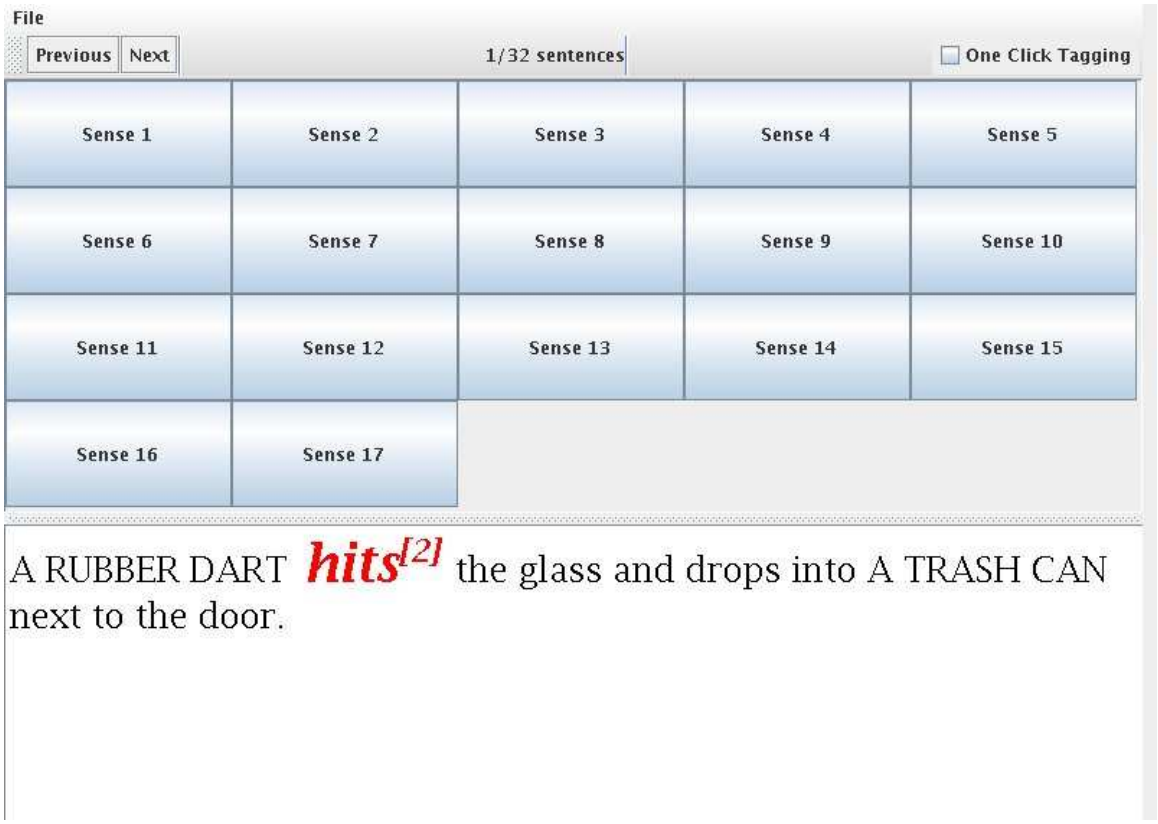


Figure C.1: Interface of the sense tagger

3. Sentence displaying area: this area will be used to display the sentences to be tagged. Only a single sentence will be displayed at any given time. The font of the target word will be red and bigger than all the non-target words which are black. The senses of the target word will appear as superscripts of the target word.

C.2 Sense Tagging

When starting to senses of a new target word, the annotator should first familiarize himself/herself with all the senses of that word. When tagging a new sentence, the annotator should **always** choose at least one sense that best describes the target word in the given sentence, if there are more than one sense that can best describe the target word in the given sense, the annotator should select **all** of them.

Appendix D

Base Graphics Commands

This section lists the 21 base graphics commands implemented in this research.

Name	Arguments and Type	Brief Description
TranslateObjectToObject	`srcObj`: Virtual Object `refObj`: Virtual Object `speed`: Scalar	Moves `srcObj` towards `refObj` at the given `speed`
TranslateObjectToPosition	`srcObj`: Virtual Object `refPos`: Location `speed`: Scalar	Moves `srcObj` to `refPos` at the given `speed`
TranslateObjectToBone	`srcObj`: Virtual Object `refBone`: Bone `speed`: Scalar	Move `srcObj` towards `refBone` at the given `speed`
TranslateObjectInDirection	`srcObj`: Virtual Object `refDir`: Direction `speed`: Scalar	Move `srcObj` in `refDir` at the given `speed`
RotateObjectTowardsObject	`srcObj`: Virtual Object `refObj`: Virtual Object `speed`: Scalar	Rotate `srcObj` at the given `speed` so that it faces `refObj`
RotateObjectToAngle	`srcObj`: Virtual Object `refAng`: Euler Angle `speed`: Scalar	Rotate `srcObj` at the given `speed` to make its angle is `refAng`
RotateObjectInDirection	`srcObj`: Virtual Object `speedX`: Scalar `speedY`: Scalar `speedZ`: Scalar	Rotate `srcObj` around x-axis at `speedX`, y-axis at `speedY` and z-axis at `speedZ`
NavigateObjectToObject	`srcObj`: Virtual Object `refObj`: Virtual Object `speed`: Scalar `update`: Scalar	Plan an obstacle-free path between `srcObj` and `refObj`, and move `srcObj` towards `refObj` at the given `speed`, re-planning the path every `update` frames
NavigateObjectToPosition	`srcObj`: Virtual Object `refPos`: Location `speed`: Scalar `update`: Scalar	Plan an obstacle-free path from `srcObj` to `refPos`, and move `srcObj` to `refPos` at the given `speed`, replanning the path path every `update` frames
DropObjectToTheGround	`srcObj`: Virtual Object `refDir`: Direction `speed`: Scalar	Move `srcObj` in the direction of `refDir` at the given `speed` until `srcObj` hits another object

Table D.1: Basic graphics commands for rigid body transformation

Name	Arguments and Type	Brief Description
MocapCommand	`srcObj`: Virtual Object `speed`: Scalar `mocapName`: Mocap	Use `srcObj` to play back the skeleton animation specified by `mocapName` at the given `speed`
PointArmatureBoneAtObject	`srcBone`: Bone `refObj`: Virtual Object `speed`: Scalar `chainLength`: Scalar	Form an IK chain whose head bone is specified by `srcBone`, and whose length is specified by `chainLength`, then move the `srcBone` as close to `refObj` as possible at the given `speed`
PointArmatureBoneAtBone	`srcBone`: Bone `refBone`: Bone `speed`: Scalar `chainLength`: Scalar	Form an IK chain whose head bone is specified by `srcBone`, and whose length is specified by `chainLength`, then move the `srcBone` as close to `refBone` as possible at the given `speed`
PointArmatureBoneAtPosition	`srcBone`: Bone `refPos`: Location `speed`: Scalar `chainLength`: Scalar	Form an IK chain whose head bone is specified by `srcBone`, and whose length is specified by `chainLength`, then move the `srcBone` to `refPos` at the given `speed`
PointArmatureBoneAtDirection	`srcBone`: Bone `refDir`: Direction `speed`: Scalar `chainLength`: Scalar	Form an IK chain whose head bone is specified by `srcBone`, and whose length is specified by `chainLength`, then move the `srcBone` in the direction of `refDir` at the given `speed`

Table D.2: Basic graphics commands for forward and inverse kinematics based skeleton animation

Name	Arguments and Type	Brief Description
ArmatureStalkObject	`srcBone`: Bone `refObj`: Virtual Object `chainLength`: Scalar	Form an IK chain whose head bone is specified by `srcBone`, and and whose length is specified by `chainLength`, then make `srcBone` follow `refObj`
ArmatureStalkBone	`srcBone`: Bone `refBone`: Bone `chainLength`: Scalar	Form an IK chain whose head bone is specified by `srcBone`, and whose length is specified by `chainLength`, then make `srcBone` follow `refBone`
ObjectStalkArmatureBone	`srcObj`: Virtual Object `refBone`: Bone `copyTranslation`: Bool `copyRotation`: Bool	Make `refBone` the parent of `srcObj`. If `copyTranslation` is False, `srcObj` does not translate with `refBone`. If `copyRotation` is False, `srcObj` does not rotate with `refBone`
ObjectStalkObject	`srcObj`: Virtual Object `refObj`: Virtual Object `axis`: Axis	Make `srcObj` move in the same direction as `refObj`, rotating `srcObj` around the given `axis` when `refObj` changes its direction
ObjectParentObject	`srcObj`: Virtual Object `refObj`: Virtual Object	Make `srcObj` the transform parent of `refObj`
ManipulateLight	`srcObj`: Virtual Object `intensity`: Scalar	Change the intensity of the light specified by `srcObj` to the value specified by `intensity`

Table D.3: Basic graphics commands for transform scene graph transformation and lighting

Appendix E

Virtual Scene Feature Functions

This appendix lists the virtual scene feature functions mentioned in Section 7.4.2 of Chapter 7.

Test Name	Arguments and Type	Brief Description
boundingCylinderPercentage	arg0: Virtual Object arg1: Virtual Object arg2: Scalar	Returns True if arg1 is within X percent of the radius of the bounding cylinder of arg0 where X is specified by arg2
sameZLevel	arg0: Virtual Object arg1: Virtual Object	Returns True if the bottoms of arg0 and arg1 are about the same
higher	arg0: Virtual Object arg1: Virtual Object	Returns True if the top of arg0 is higher than the top of arg1
lower	arg0: Virtual Object arg1: Virtual Object	Returns True if the top of arg0 is lower than the top of arg1
intersectZ	arg0: Virtual Object arg1: Virtual Object	Returns True if the top of arg0 is in between the top and bottom of arg1, or the top of arg1 is in between the top and bottom of arg0
faceTowardsNoZ	arg0: Virtual Object arg1: Virtual Object	Returns True if the direction in which arg0 is facing is about the same as vector from the centre of arg0 to the centre of arg1 on the XY plane
canArmatureReachObj	arg0: Virtual object arg1: Bone arg2: Scalar	Returns True if the IK chain whose head bone and length are specified by arg1 and arg2 respectively can reach the virtual object specified by arg0
inVerticalPose	arg0: Virtual Object	Returns True if arg0 is roughly vertical

Table E.1: Spatial relationship tests for the virtual scene features

Test Name	Arguments and Type	Brief Description
objectHasBone	`arg0`: Virtual Object `arg1`: String	Returns True if `arg0` has an armature containing a bone whose name is given by `arg1`
parentCommandApply	`arg0`: Virtual Object `arg1`: Virtual Object	Returns True if `arg0` is the transform parent of `arg1` at the current time
objStalkObjectCommandApply	`arg0`: Virtual Object `arg1`: Virtual Object	Returns True if `arg0` is set to stalk `arg1` at the current time
objStalkArmatureCommandApply	`arg0`: Virtual Object `arg1`: Bone	Returns True if `arg0` is set to stalk `arg1` at the current time
boneStalkBoneCmdApply	`arg0`: Bone `arg1`: Bone	Returns True if `arg0` is set to stalk `arg1` at the current time
involvedInCommand	`arg0`: Virtual Object `arg1`: String	Returns True if `arg0` is involved in the command whose name is given by `arg1`

Table E.2: Tests for the properties of the virtual objects and the status of the animation

Appendix F

Construction Functions for Location, Euler and Direction Arguments

This section lists the construction functions for Location, Euler and Direction Arguments.

Function Name	Arguments and Type	Brief Description
left	arg0: Virtual Object arg1: percentage	Select an Euler angle such that rotating arg0 to this angle would be considered as rotating arg0 to its left
right	arg0: Virtual Object arg1: percentage	Select an Euler angle such that rotating arg0 to this angle would be considered as rotating arg0 to its right
down	arg0: Virtual Object arg1: percentage	Select an Euler angle such that rotating arg0 to this angle would be considered as rotating arg0 downwards
up	arg0: Virtual Object arg1: percentage	Select an Euler angle such that rotating arg0 to this angle would be considered as rotating arg0 upwards
leftDblTo	arg0: Virtual object arg1: Virtual Object arg2: percentage	Select an Euler angle such that rotating arg0 to this angle would be considered as rotating arg0 towards the left of arg1 with arg0 facing towards arg1
rightDblTo	arg0: Virtual object arg1: Virtual Object arg2: percentage	Select an Euler angle such that rotating arg0 to this angle would be considered as rotating arg0 towards the right of arg1 with arg0 facing towards arg1
leftDblAw	arg0: Virtual object arg1: Virtual Object arg2: percentage	Select an Euler angle such that rotating arg0 to this angle would be considered as rotating arg0 towards the left of arg1 with arg0 facing away from arg1
rightDblAw	arg0: Virtual object arg1: Virtual Object arg2: percentage	Select an Euler angle such that rotating arg0 to this angle would be considered as rotating arg0 towards the right of arg1 with arg0 facing away from arg1

Table F.1: Argument construction functions for Euler arguments

Function Name	Arguments and Type	Brief Description
inside	arg0: Virtual Object	Select a location that can be considered as inside arg0
fallToGround	arg0: Virtual Object	Calculate the location that arg0 would be if it fell directly downwards
destRadiusPer	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location between arg0 and arg1 such that its distance to arg1 is arg2% of the radius of arg1's bounding cylinder
srcRadiusPer	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location between arg0 and arg1 such that its distance to arg1 is arg2% of the radius of arg0's bounding cylinder
insideDbl	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location inside arg1 such that if arg0 is moved to there, it would appear to be inside of arg1
outsideGrd	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location outside arg1 such that if arg0 is moved to there, it would appear to be outside of arg1, and on even ground as arg1
outsideAir	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location outside arg1 such that if arg0 is moved there it would appear to be outside of arg1, and just on top of arg1
onEdge	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location on top of arg1 such that if arg0 is moved to this location, it would appear to be on the edge of arg1
under	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location outside arg1 such that if arg0 is moved to this location, it would appear to be under arg1
left	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location outside arg1 such that if arg0 is moved to this location, it would appear to be to the left of arg1
right	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location outside arg1 such that if arg0 is moved to this location, it would appear to be to the right of arg1
front	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location outside arg1 such that if arg0 is moved to this location, it would appear to be in front of arg1
back	arg0: Virtual Object arg1: Virtual Object arg2: percentage	Select a location outside arg1 such that if arg0 is moved to this location, it would appear to be behind arg1

Table F.2: Argument construction functions for Location arguments

Function Name	Arguments and Type	Brief Description
front	`arg0`: Virtual Object	Select a direction that points to the front of `arg0`
back	`arg0`: Virtual Object	Select a direction that points to the back of `arg0`
left	`arg0`: Virtual Object	Select a direction that points to the left of `arg0`
right	`arg0`: Virtual Object	Select a direction that points to the right of `arg0`
up	`arg0`: Virtual Object	Select a direction that points 45 degrees upwards from `arg0`
down	`arg0`: Virtual Object	Select a direction that points to 45 degrees downwards from `arg0`
top	`arg0`: Virtual Object	Select a direction that points directly upwards from `arg0`
bottom	`arg0`: Virtual Object	Select a direction that points directly downwards from `arg0`

Table F.3: Argument construction functions for Direction arguments